

© 2009 Neera Jain. All rights reserved.

DYNAMIC MODELING AND VALIDATION OF A COMMERCIAL
TRANSPORT REFRIGERATION SYSTEM

BY

NEERA JAIN

B.S., Massachusetts Institute of Technology, 2006

THESIS

Submitted in partial fulfillment of the requirements
for the degree of Master of Science in Mechanical Engineering
in the Graduate College of the
University of Illinois at Urbana-Champaign, 2009

Urbana, Illinois

Advisor:

Professor Andrew Alleyne

Abstract

The invention of refrigeration, and then its application to the transportation of foods and other goods, has had a profound impact on the lifestyle of people all around the world. Unfortunately, the widespread use of refrigeration, and shortly thereafter, air-conditioning, has had its negative impacts as well, most notably on the environment. Refrigeration equipment for road transport is required to operate over a larger envelope of operation conditions than stationary refrigeration equipment. Moreover, strict temperature control standards for transported food products make it difficult to balance the tradeoffs between performance and efficiency. Although changes in hardware and system design can lead to improvements in efficiency, the use of advanced control techniques is another avenue through which efficiency gains can be achieved. In this thesis, a control-oriented dynamic model of a commercially available transport refrigeration unit is developed and validated with test data collected on a fully instrumented unit. Such a model enables the use of advanced control strategies to be developed and tested in simulation, ultimately resulting in a better performing, and more efficient, refrigeration system. A major contribution of this thesis is the development of a control-oriented model of the heat cycle, or Heat Mode, operation of the transport refrigerant unit. The development of both the Cool Mode (i.e. cool cycle) and Heat Mode operation models resulted in the addition of new component models for the Thermosys Toolbox.

To engineers, past and present, who inspire me to use my skills to impact our society in a positive way.

Acknowledgements

First and foremost, I would like to thank my advisor, Dr. Andrew Alleyne, for all of his guidance and mentorship over the last 2 ½ years. He has helped me to develop and mature as a researcher and to grow confidence in my abilities as an engineer. I want to thank Bill Mohs for not only supervising my summer internship at Thermo King, but for all of his expertise on refrigeration systems which guided us through the various modeling and validation challenges that we encountered. I also want to thank Jeff Berge, Alan Gustafson, KC Kolstad, and Robert Srichai (and many others!) at Thermo King Corporation who, in addition to being wonderful coworkers during my summer internship in Minnesota, provided valuable insight throughout the development of the models presented in this thesis.

I have been extremely fortunate to work in a lab as supportive, both socially and academically, as the Alleyne Research Group. My peers in the ARG are among the most intelligent and kind people I have ever known. I want to thank Tom for his much-needed expertise in thermal systems when I was first beginning my modeling work. I want to thank Brandon for his help with all technical matters that arose in the lab as well with my many questions about Matlab. Many thanks also go to Bin and Vikas for engaging discussions and useful suggestions which have helped to move my research forward. I am very excited to continue to work closely with them both over the next few years.

I must also thank Doug for introducing me to Meet the Press and for many hours of healthy political discussion; Brian, Scott, and Dave for many hilarious distractions in the office; and Tim for his help in classes we have shared. Special thanks go to Serena and Kira for keeping me sane, and for offering both personal and academic support when I needed it most.

I am learning (first from Doug, and now from my own experience) that when you stay in the ARG long enough, you will meet many fellow ARGers; some will stay with you along the entire journey, and others will join after, and leave before, you do. To those who joined our lab most recently – Rich, Yangmin, Nanjun, and Sandipan – it has been a pleasure getting to know each of you over the last few months, and I am excited for the coming year(s) of having you in our lab.

I would also like to thank two people outside of the ARG who are very special to me. Cathy, thank you for being a fantastic roommate and for making me dinner when you know I was stressed out. Shreyas, thank you for all of your encouragement and for always helping me to put things in perspective. Last but not least, I must thank my family – my parents, my sister, Bhavana, and my brother, Ankush – who, despite not understanding why I need to be at the lab at 8pm on a Sunday night, have continued to love and support me.

Finally, I would like to acknowledge the organizations that have funded my research: Thermo King Corporation and the Air-Conditioning and Refrigeration Center at the University of Illinois at Urbana-Champaign.

Table of Contents

CHAPTER	Page
LIST OF TABLES	x
LIST OF FIGURES	xiii
NOMENCLATURE	xviii
1 INTRODUCTION	1
1.1 Transport Refrigeration.....	1
1.2 Vapor Compression Systems	3
1.3 Control-Oriented Modeling Framework	5
1.3.1 Model-based Control.....	5
1.3.2 Thermosys Toolbox	6
1.4 Organization of Thesis.....	8
2 EXPERIMENTAL SYSTEM	10
2.1 System Description	10
2.1.1 Cool Mode.....	11
2.1.2 Heat Mode.....	13
2.2 Data Acquisition	15
2.2.1 Test Unit Instrumentation	15
2.2.2 Steady State Testing.....	18
2.2.3 Transient Testing.....	19
3 COOL MODE MODELING.....	24
3.1 Dynamic Model Components	24

3.1.1	Heat Exchangers	24
3.1.2	Accumulator.....	32
3.1.3	Distributor	36
3.2	Static Model Components.....	39
3.2.1	Scroll Compressor.....	39
3.2.2	Suction Line Heat Exchanger.....	41
3.2.3	Electronic Expansion Valve.....	43
3.2.4	Throttle Valve	50
3.2.5	Heat Exchanger Fan	50
4	HEAT MODE MODELING.....	54
4.1	Dynamic Model Components	54
4.1.1	Distributor	54
4.1.2	Evaporator.....	56
4.1.3	Accumulator.....	59
4.2	Static Model Components.....	61
4.2.1	DPR Valve	62
4.2.2	Hot Gas Line	63
4.2.3	Hydraulic Resistance Element	67
5	COOL MODE MODEL VALIDATION.....	68
5.1	Individual Component Validation.....	68
5.1.1	Compressor	69
5.1.2	Condenser with Receiver	72
5.1.3	Electronic Expansion Valve.....	77
5.1.4	Distributor	81
5.1.5	Evaporator.....	84
5.1.6	Suction Line Heat Exchanger and Accumulator	89
5.1.7	Throttle Valve	96
5.2	Complete System Validation	99
6	HEAT MODE MODEL VALIDATION	106
6.1	Individual Component Validation.....	106

6.1.1	Compressor	106
6.1.2	Discharge Pressure Regulator Valve.....	110
6.1.3	Hot Gas Line	114
6.1.4	Distributor	118
6.1.5	Evaporator and Accumulator	120
6.1.6	Throttle Valve	127
6.2	Complete System Validation	131
7	CONCLUSIONS AND FUTURE WORK	142
7.1	Summary of Research Contributions	142
7.1.1	Model Developments	142
7.1.2	Model Validation	143
7.2	Future Work.....	143
7.2.1	Model Development.....	144
7.2.2	Model Validation	145
	LIST OF REFERENCES.....	146
	APPENDIX A INPUT SIGNAL GENERATION.....	149
	APPENDIX B DISTRIBUTOR SYSTEM ID.....	150
B.1	Cool Mode	150
B.2	Heat Mode.....	151
	APPENDIX C MAP GENERATING FILES	153
C.1	Mean Void Fraction Lookup Table.....	153
C.2	EEV Flow Coefficient Map	154
C.3	Moist Air Fluid Properties Table	155
C.4	Condenser J Factor.....	157
C.5	Evaporator J Factor	158
	APPENDIX D COOL MODE MODEL FILES	160
D.1	Compressor	160
D.2	Condenser with Receiver	163
D.3	Suction Line Heat Exchanger (SLHX)	168

D.4	Electronic Expansion Valve (EEV)	171
D.5	Distributor – Cool Mode.....	174
D.6	Evaporator – Cool Mode.....	175
D.7	Accumulator – Cool Mode.....	179
D.8	Throttle Valve	182
D.9	Evaporator Fan.....	184
D.10	Condenser Fan	187
APPENDIX E HEAT MODE MODEL FILES		191
E.1	DPR Valve	191
E.2	Hot Gas Line	193
E.3	Distributor – Heat Mode	195
E.4	Evaporator – Heat Mode.....	196
E.5	Hydraulic Resistance	201
E.6	Accumulator – Heat Mode.....	204
APPENDIX F ADDITIONAL FIGURES		208

List of Tables

Table 2.1	Summary of Steady State Data Collected During Cool Mode Operation	18
Table 2.2	Summary of Steady State Data Collected During Heat Mode Operation	19
Table 3.1	Matrix Elements of $Z(x,u)$ for the Nonlinear Evaporator	25
Table 3.2	Vector Elements of $f(x,u)$ for the Nonlinear Evaporator	26
Table 3.3	Matrix Elements of $Z(x,u)$ for the Nonlinear Condenser with Receiver	26
Table 3.4	Vector Elements of $f(x,u)$ for the Nonlinear Condenser with Receiver	27
Table 3.5	Mach Number of Refrigerant Flow through Accumulator	33
Table 4.1	Comparison of Data Measurements used for Distributor Identification	55
Table 4.2	Matrix Elements of $Z(x,u)$ for the Heat Mode Evaporator Model	58
Table 4.3	Vector Elements of $f(x,u)$ for the Heat Mode Evaporator Model	58
Table 4.4	Mach Number of Refrigerant Flow through Hot Gas Line	65
Table 5.1	Compressor Model Inputs	69
Table 5.2	Compressor Model Outputs	69
Table 5.3	Steady State Validation Results for Compressor Model	70
Table 5.4	Normalized RMS Error for Compressor Transient Validation	72
Table 5.5	Condenser with Receiver Model Inputs	73
Table 5.6	Condenser with Receiver Model Outputs	73
Table 5.7	Steady State Validation Results for Condenser Pressure	74
Table 5.8	Steady State Validation Results for Condenser Outlet Enthalpy	74
Table 5.9	Steady State Validation Results for Condenser Air Outlet Temperature	74
Table 5.10	Normalized RMS Error for Condenser with Receiver Transient Validation	77
Table 5.11	EEV Model Inputs	77
Table 5.12	EEV Model Outputs	78
Table 5.13	Steady State Validation Results for EEV	78

Table 5.14	Normalized RMS Error for EEV Transient Validation.....	81
Table 5.15	Distributor Model Inputs.....	82
Table 5.16	Distributor Model Outputs	82
Table 5.17	Steady State Validation Results for Distributor	82
Table 5.18	Normalized RMS Error for Distributor Transient Validation	83
Table 5.19	Evaporator Model Inputs.....	84
Table 5.20	Evaporator Model Outputs	84
Table 5.21	Steady State Validation Results for Evaporator Pressure	85
Table 5.22	Steady State Validation Results for Evaporator Outlet Enthalpy.....	85
Table 5.23	Steady State Validation Results for Evaporator Outlet Air Temperature	86
Table 5.24	SLHX Model Inputs.....	89
Table 5.25	SLHX Model Outputs	89
Table 5.26	Accumulator Model Inputs	89
Table 5.27	Accumulator Model Outputs.....	90
Table 5.28	Steady State Validation Results for SLHX Liquid Side Pressure	92
Table 5.29	Steady State Validation Results for SLHX Liquid Side Enthalpy	92
Table 5.30	Steady State Validation Results for SLHX Vapor Side Pressure.....	92
Table 5.31	Steady State Validation Results for SLHX Vapor Side Enthalpy.....	93
Table 5.32	Normalized RMS Error for SLHX Transient Validation	95
Table 5.33	Throttle Valve Model Inputs.....	96
Table 5.34	Throttle Valve Model Outputs	96
Table 5.35	Steady State Validation Results for Throttle Valve Outlet Pressure	97
Table 5.36	Steady State Validation Results for Throttle Valve Outlet Enthalpy	97
Table 5.37	Normalized RMS Error for Throttle Valve Transient Validation	99
Table 5.38	Complete System Model Inputs	99
Table 5.39	Normalized RMS Error for Complete Cool Mode Cycle System.....	105
Table 6.1	Steady State Validation Results for Compressor Mass Flow Rate	107
Table 6.2	Steady State Validation Results for Compressor Outlet Enthalpy	107
Table 6.3	Normalized RMS Error for Compressor Transient Validation	110
Table 6.4	DPR Valve Model Inputs	110
Table 6.5	DPR Valve Model Outputs	110
Table 6.6	Steady State Validation Results for DPR Valve Inlet Pressure.....	111
Table 6.7	Steady State Validation Results for DPR Valve Outlet Enthalpy	111
Table 6.8	Normalized RMS Error for DPR Transient Validation.....	113

Table 6.9	Hot Gas Line Model Inputs	114
Table 6.10	Hot Gas Line Model Outputs	114
Table 6.11	Steady State Validation Results for Hot Gas Line Inlet Pressure	115
Table 6.12	Steady State Validation Results for Hot Gas Line Outlet Enthalpy	115
Table 6.13	Normalized RMS Error for Hot Gas Line Transient Validation	118
Table 6.14	Steady State Validation Results for Distributor	118
Table 6.15	Normalized RMS Error for Distributor Transient Validation	120
Table 6.16	Heat Mode Accumulator Model Inputs	121
Table 6.17	Heat Mode Accumulator Model Outputs	121
Table 6.18	Steady State Validation Results for Evaporator Pressure	122
Table 6.19	Steady State Validation Results for Evaporator Outlet Air Temperature	123
Table 6.20	Steady State Validation Results for Accumulator Pressure	123
Table 6.21	Steady State Validation Results for Refrigerant Temperature	123
Table 6.22	Normalized RMS Error for Evaporator and Accumulator Transient Validation ...	127
Table 6.23	Steady State Validation Results of Throttle Valve Outlet Pressure	128
Table 6.24	Steady State Validation Results of Throttle Valve Outlet Enthalpy	128
Table 6.25	Normalized RMS Error for Throttle Valve Transient Validation	131
Table 6.26	Complete System Model Inputs	131
Table 6.27	Normalized RMS Error for Complete Heat Mode Cycle System	140

List of Figures

Figure 1.1: Schematic of ideal subcritical vapor compression system.	4
Figure 1.2: P-h diagram for ideal subcritical vapor compression cycle.....	4
Figure 1.3: Simulink block diagram for static modeling framework.....	7
Figure 1.4: Simulink block diagram for dynamic modeling framework.	8
Figure 2.1: Thermo King transport refrigeration unit mounted on a truck body.	10
Figure 2.2: System schematic during Cool Mode operation.....	12
Figure 2.3: Pressure-enthalpy diagram for Cool Mode cycle.	12
Figure 2.4: System schematic during Heat Mode operation.....	14
Figure 2.5: Pressure-enthalpy diagram for Heat Mode cycle.	14
Figure 2.6: Schematic of test unit instrumentation during Cool Mode testing.	17
Figure 2.7: Schematic of test unit instrumentation during Heat Mode testing	17
Figure 2.8: PRBS input signal for compressor speed.	21
Figure 2.9: RGS input signal for EEV opening at 0 100 °F operating condition.....	22
Figure 2.10: PRBS input signal for compressor speed during Heat Mode testing.....	23
Figure 3.1: Cross flow configuration for air to refrigerant heat exchanger.	29
Figure 3.2: Counter-cross flow configuration for air to refrigerant heat exchanger.....	29
Figure 3.3: Evaporator j factor data.	32
Figure 3.4: Condenser j factor data.....	32
Figure 3.5: Accumulator refrigerant heat transfer coefficient as a function of refrigerant mass flow rate.....	36
Figure 3.6: Detrended system identification results for the distributor model.....	38
Figure 3.7: EEV input signal for data set (used for flow coefficient map generation) collected at -20 50 °F box ambient for compressor operating at low speed.	44
Figure 3.8: Comparison between predicted and measured valve flow coefficients using valve stroke and pressure differential as predictors.	45

Figure 3.9: Comparison between predicted and measured valve flow coefficients using valve stroke, inlet pressure, and outlet pressure as predictors.	45
Figure 3.10: Flow coefficient map for an EEV. Black points represent experimental data.....	47
Figure 3.11: Figure 3.10 rotated to better show experimental data relative to surface mapping... ..	48
Figure 3.12: Comparison between predicted and measured valve flow coefficients using valve stroke and valve inlet subcooling as predictors.....	49
Figure 3.13: Three dimensional map for determining outlet pressure of the throttle valve.....	50
Figure 3.14: Example of intersection between fan and resistance curves for evaporator fan.....	52
Figure 3.15: Example of intersection between fan and resistance curves for condenser fan.....	53
Figure 4.1: Detrended system identification results for distributor model.	55
Figure 4.2: Accumulator refrigerant heat transfer coefficient as a function of refrigerant mass flow rate during Heat Mode operation.	61
Figure 4.3: DPR valve flow coefficient as a function of refrigerant mass flow rate. Note that the flow coefficient is in English units.....	62
Figure 4.4: Empirically derived refrigerant-side heat transfer coefficient as a function of refrigerant mass flow rate.....	64
Figure 4.5: Sample code showing pipe loss coefficients hard coded into Thermosys.....	66
Figure 5.1: Thermosys Simulink model for compressor validation.....	70
Figure 5.2: Compressor transient validation results at 0 100 °F for perturbations in EEV stroke.	71
Figure 5.3: Compressor transient validation results at -20 50 °F for perturbations in compressor speed.....	71
Figure 5.4: Thermosys Simulink model for condenser with receiver validation.	73
Figure 5.5: Condenser with receiver transient validation results at 0 100 °F for perturbations in EEV stroke.	75
Figure 5.6: Condenser with receiver transient validation results at the -20 50 °F condition for perturbations in compressor speed.	76
Figure 5.7: Thermosys Simulink model for EEV validation.	78
Figure 5.8: EEV transient validation results at 0 100 °F for perturbations in EEV stroke.	79
Figure 5.9: EEV transient validation results at -20 50 °F for perturbations in EEV stroke.	79
Figure 5.10: EEV transient validation results at 0 100 °F, adjusted by $K_v = 0.0158$	80
Figure 5.11: EEV transient validation results at 0 100 °F, adjusted by $K_v = 0.008$	80
Figure 5.12: EEV transient validation results at -20 50 °F for perturbations in compressor speed.....	81
Figure 5.13: Thermosys Simulink model for distributor validation.	82

Figure 5.14: Distributor transient validation results at 0 100 °F for perturbations in EEV stroke.	83
Figure 5.15: Distributor transient validation results at -20 50 °F for perturbations in compressor speed.	83
Figure 5.16: Thermosys Simulink model for evaporator validation.	85
Figure 5.17: Evaporator transient validation results at 0 100 °F for perturbations in EEV stroke.	87
Figure 5.18: Evaporator transient validation results at -20 50 °F for perturbations in compressor speed.	88
Figure 5.19: Thermosys Simulink model for SLHX and accumulator validation.	91
Figure 5.20: Liquid side SLHX transient validation results at 0 100 °F for perturbations in EEV stroke.	93
Figure 5.21: Vapor side SLHX transient validation results at 0 100 °F for perturbations in EEV stroke.	94
Figure 5.22: Liquid side SLHX transient validation results at -20 50 °F for perturbations in compressor speed.	94
Figure 5.23: Vapor side SLHX transient validation results at -20 50 °F for perturbations in compressor speed.	95
Figure 5.24: Thermosys Simulink model for throttle valve validation.	96
Figure 5.25: Throttle valve transient validation results at 0 100 °F for perturbations in EEV stroke.	98
Figure 5.26: Throttle valve transient validation results at -20 50 °F for perturbations in compressor speed.	98
Figure 5.27: Complete cycle system transient inputs at 0 100 °F.	100
Figure 5.28: Transient validation results for compressor outlet enthalpy.	100
Figure 5.29: Transient validation results for EEV mass flow rate.	101
Figure 5.30: Transient validation results for condenser model.	101
Figure 5.31: Transient validation results for liquid pass of SLHX.	102
Figure 5.32: Transient validation results for distributor model.	102
Figure 5.33: Transient validation results for evaporator model.	103
Figure 5.34: Transient validation results for vapor pass of SLHX and accumulator.	103
Figure 5.35: Transient validation results for throttle valve model.	104
Figure 6.1: Thermosys Simulink model for compressor validation.	107
Figure 6.2: Compressor transient validation results at 70 35 °F.	108
Figure 6.3: Compressor transient validation results at 70 0 °F.	108
Figure 6.4: Compressor transient validation results at 35 0 °F.	109

Figure 6.5: Thermosys Simulink model for DPR valve validation.....	111
Figure 6.6: DPR valve transient validation results at 70 35 °F.....	112
Figure 6.7: DPR valve transient validation results at 70 0 °F.....	112
Figure 6.8: DPR valve transient validation results at 35 0 °F.....	113
Figure 6.9: Thermosys Simulink model for hot gas line validation.....	114
Figure 6.10: Hot Gas Line transient validation results at 70 35 °F.....	116
Figure 6.11: Hot Gas Line transient validation results at 70 0 °F.....	116
Figure 6.12: Hot Gas Line transient validation results at 35 0 °F.....	117
Figure 6.13: Thermosys Simulink model for distributor validation.	118
Figure 6.14: Distributor transient validation results at 70 35 °F.....	119
Figure 6.15: Distributor transient validation results at 70 0 °F with detrend constants based on the 70 35 °F operating condition.	119
Figure 6.16: Distributor transient validation results at 35 0 °F.....	119
Figure 6.17: Thermosys Simulink model for evaporator and accumulator validation.....	122
Figure 6.18: Evaporator transient validation results at 70 35 °F.....	124
Figure 6.19: Accumulator transient validation results at 70 35 °F.....	125
Figure 6.20: Evaporator transient validation results at 35 0 °F.....	125
Figure 6.21: Accumulator transient validation results at 35 0 °F.....	126
Figure 6.22: Thermosys Simulink model for throttle valve validation.....	127
Figure 6.23: Throttle Valve transient validation results at 70 35 °F.....	129
Figure 6.24: Throttle Valve transient validation results at 70 0 °F.....	129
Figure 6.25: Throttle Valve transient validation results at 35 0 °F.....	130
Figure 6.26: Complete cycle system transient inputs at 70 35 °F.....	132
Figure 6.27: Compressor transient validation results at 70 35 °F.....	132
Figure 6.28: DPR valve transient validation results at 70 35 °F.....	133
Figure 6.29: Hot gas line transient validation results at 70 35 °F.....	133
Figure 6.30: Distributor transient validation results at 70 35 °F.....	134
Figure 6.31: Evaporator transient validation results at 70 35 °F.....	134
Figure 6.32: Accumulator transient validation results at 70 35 °F.....	135
Figure 6.33: Throttle valve transient validation results at 70 35 °F.....	135
Figure 6.34: Complete cycle system transient inputs at 35 0 °F.....	136
Figure 6.35: Compressor transient validation results at 35 0 °F.....	136
Figure 6.36: DPR valve transient validation results at 35 0 °F.....	137
Figure 6.37: Hot gas line transient validation results at 35 0 °F.....	137

Figure 6.38: Distributor transient validation results at 35 0 °F.....	138
Figure 6.39: Evaporator transient validation results at 35 0 °F.....	138
Figure 6.40: Accumulator transient validation results at 35 0 °F.....	139
Figure 6.41: Throttle valve transient validation results at 35 0 °F.....	139
Figure F.1: Cool Mode complete system Simulink diagram.	208
Figure F.2: Heat Mode complete system Simulink diagram.....	209

NOMENCLATURE

Variable

α	Heat Transfer Coefficient
η	Efficiency
$\bar{\gamma}$	Mean Void Fraction
$\dot{m}$	Mass Flow Rate
ρ	Density
A	Area
C_p	Specific Heat
h	Specific Enthalpy
j	Colburn j factor
k	Conductivity
L	Length
m	Mass
P	Pressure
Pr	Prandtl Number
$\dot{Q}$	Heat Transfer Rate
Re	Reynold's Number
S	Slip Ratio
St	Stanton Number
T	Temperature
t	Time
u	Specific Internal Energy
UA	Lumped Heat Transfer Coefficient
V	Volume, Volts
x	Quality, Dynamic States
y	Outputs
Z	Descriptor Form Matrix
Δ	Change
μ	Viscosity
ω	Angular Velocity

Subscript

1,2,3	1 st , 2 nd , 3 rd Region
acc	Accumulator

air, a	Air
amb	Ambient
ave	Average
c	Condenser
cs	Cross-Sectional
e	Evaporator
f	Liquid
g	Vapor
h	Hydraulic
i	Inner
in	In
int	Intermediate
k	Compressor
o	Outer
out	Out
r	Refrigerant
rec	Receiver
sat	Saturation
T	Total
total	Total
v	Valve, Vapor
w	Wall

Chapter 1

INTRODUCTION

1.1 Transport Refrigeration

The invention of refrigeration, and then its application to the transportation of foods such as produce, dairy, meat, etc., has had a profound impact on the lifestyle of people, particularly in the developed world. Although ancient civilizations found ways of preserving foods as well as cooling food, drink, and themselves, refrigeration as it is known today first appeared in the early 20th century. Its application to food transport came in 1938 when Joseph Numero and Frederick Jones patented the “Model A”, the first transport refrigeration unit. Soon after, the two men founded Thermo King Corporation, now a unit of Ingersoll Rand Company Limited [30].

The use of refrigeration in trucks and railway cars enabled the growth of cities, as produce could now be transported from rural farms to grocers in the heart of a city. Fruits and vegetables were no longer seasonal but instead available all year-round, and the United States saw the birth of regional produce specialization: Florida oranges, Washington apples, etc. [16]. Today, frozen vegetables and prepared frozen meals are sure to be found in every home’s freezer; not only can these be kept for several months in residential freezers, but they are transported to the local grocery store from a given manufacturing facility in refrigerated trucks and/or railway cars. Refrigeration has also enabled progress with respect to women and their role in the workplace; the flexibilities afforded by refrigerators, such as keeping leftovers in the refrigerator (or freezer), allows families to more easily manage a household with two working adults.

Unfortunately, the widespread use of refrigeration, and shortly thereafter, air-conditioning, has had its negative impacts as well, most notably on the environment. According to the U.S. Department of Energy (<http://www.energy.gov/>), nearly 20% of the energy consumed in a typical home is for air-conditioning and refrigeration alone. In the United Kingdom, food transport using heavy goods vehicles (HGVs) is responsible for 8300 kton of CO₂ emissions. Although only approximately one-third of the HGVs carrying food products are refrigerated, this emissions statistic does not include emissions from the smaller diesel engine used to drive the refrigeration unit itself, nor the refrigerant leakage from the unit [29]. Refrigeration equipment for road transport is required to operate over a larger envelope of operation conditions than stationary refrigeration equipment. Moreover, strict temperature control standards for transported food products make it difficult to balance the tradeoffs between performance and efficiency. As a result, a standard residential refrigerator operates with a coefficient of performance (COP) of 3 [28], whereas the COP of a refrigeration unit used for transportation applications is likely to be between 0.5 and 1.5 [29]. A larger COP indicates a more efficient refrigeration system.

Although changes in hardware and system design can lead to improvements in efficiency, the use of advanced control techniques is another avenue through which efficiency gains can be achieved. In the HVAC industry, multiple single-input-single-output (SISO) control loops, whose gains are tuned online, are generally used to control the system. However, more advanced actuators, such as variable speed compressors and electronic expansion valves, allow the control engineer to achieve desired performance objectives without having to cycle the compressor on and off, which has been shown to improve the overall efficiency of the system [14]. Although these actuators are still costly in comparison to traditional actuators (i.e. fixed-speed compressors and thermostatic expansion valves) the opportunities they afford are encouraging their use in transport refrigeration units.

In this thesis, an accurate, control-oriented dynamic model of a commercially available transport refrigeration unit is developed and validated with test data collected on a fully instrumented unit. A major contribution of this thesis is the development of a control-oriented model of the heat cycle, or Heat Mode, operation of the transport

refrigerant unit. The development of both the Cool Mode (i.e. cool cycle) and Heat Mode operation models resulted in the addition of new component models for the Thermosys Toolbox, given in the list below.

- Suction Line Heat Exchanger
- Refrigerant Distributor
- Throttle Valve
- Discharge Pressure Regulator (DPR) Valve
- Hot Gas Line

Matlab Simulink is used to connect the individual component models and simulate the complete refrigeration cycle. Such a model enables the use of advanced control strategies to be developed and tested in simulation, ultimately resulting in a better performing and more efficient refrigeration system.

The remainder of this chapter is organized as follows. Section 1.2 provides a background discussion of vapor compression systems. Section 1.3 describes the control-oriented modeling framework that guides the development of the models presented in this thesis, and introduces Thermosys, the Matlab Toolbox in which the models are developed and simulated. Finally, Section 1.4 gives an outline of the thesis.

1.2 Vapor Compression Systems

Vapor compression systems, such as refrigeration and air-conditioning systems, are used to move energy from one location to another. A basic vapor compression cycle (VCC) system contains four components: a compressor, a condenser, an expansion device, and an evaporator. However, most industrial VCC systems contain additional components, such as receivers, accumulators, suction line heat exchangers, etc. Although these additional components are often used to maximize the capacity and/or efficiency of the system, the basic operation of any VCC system is the same.

An idealized subcritical VCC system, as shown in Figure 1.1, is driven by the phase characteristics of the refrigerant that is flowing through it. Therefore it is useful to

describe the system in terms of the state of its refrigerant, as shown on a pressure-enthalpy (P-h) diagram (see Figure 1.2).

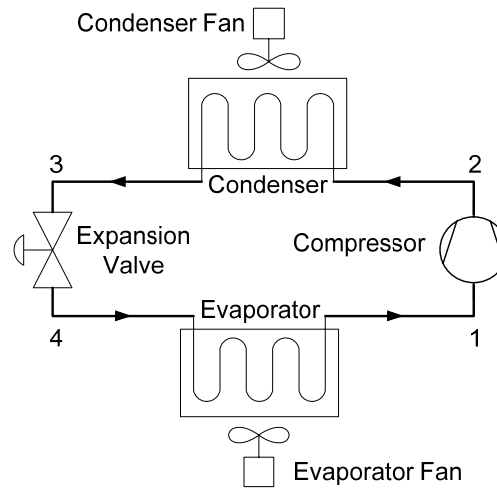


Figure 1.1: Schematic of ideal subcritical vapor compression system.

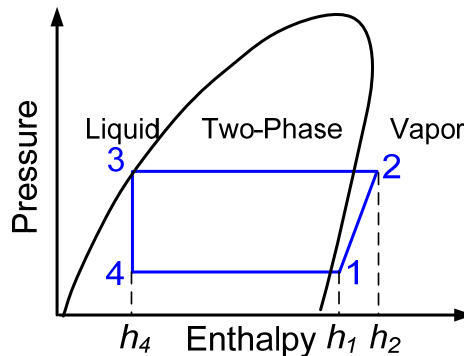


Figure 1.2: P-h diagram for ideal subcritical vapor compression cycle.

Beginning at 1 in Figure 1.2, superheated vapor refrigerant is compressed to a higher temperature and pressure. The still superheated vapor refrigerant enters the condenser at 2 where it loses energy to ambient air passing over the condenser coil. The refrigerant leaves as a saturated liquid and then enters an expansion device at 3 (e.g. thermal expansion valve, electronic expansion valve, capillary tube, etc.). The refrigerant undergoes a rapid drop in pressure and temperature. At 4 the refrigerant enters the evaporator as a two-phase fluid and begins absorbing energy from air passing across the evaporator coil. The refrigerant leaves the evaporator as a superheated vapor and the cycle begins again at the inlet of the compressor. An ideal VCC system assumes isentropic compression, isenthalpic expansion, and isobaric condensation and

evaporation. Although the system model presented in this thesis contains numerous components in addition the basic components described here, these assumptions will still be applied.

1.3 Control-Oriented Modeling Framework

1.3.1 Model-based Control

As mentioned in Section 1.1, the use of a model in control design is generally more efficient than a more heuristic approach of tuning gains online. When using advanced control techniques, such as multi-input-multi-output (MIMO) control strategies, it is actually necessary to have a dynamic model of the system to be controlled. Although commercial modeling packages such as e-Thermal [3], Modelica [31], EASY5 [2], and SINDA/FLUENT [5] utilize a discretized modeling approach for detailed and accurate models, this is done at the expense of model complexity, resulting in models that are not suitable for control design. Instead, a lumped parameter, moving boundary modeling approach will be used in this thesis.

1.3.1.1 Moving Boundary Models

In the lumped parameter, moving boundary modeling approach, the heat exchanger is modeled with a fixed number of fluid regions (defined by fluid phase), and the boundary between each fluid region is a dynamic variable, allowing the length of the fluid regions to vary. Fluid properties such as temperature, density, etc., are lumped in each region, and an average is used for model computations. Although this approach results in some loss in accuracy, the resulting models are of low dynamic order, making them very well suited for control design.

Since 1979, models have been developed in the moving boundary framework. A review of the literature shows that they have been applied to a variety of VCC systems, often with variations in the details of the modeling approach [4]. Kapadia (1984) presented a moving-boundary condenser model in the Rankine cycle but with limited validation results [13]. Grald and MacArthur (1992) later developed a moving-boundary evaporator model, and showed that a spatially dependent model and a moving-boundary

model for an evaporator have very similar dynamic responses. However, limited validation results were provided [8].

He (1997) was the first author to present linearized moving-boundary models for both an evaporator and a condenser for the purpose of multivariable control design. Adequate model validation results were included [9],[10]. Many authors have produced similar models to those discussed but with some extensions to the modeling framework [12],[22],[35]. In 2004, Rasmussen derived moving-boundary models for transcritical cycles [25]. Eldredge (2006) presented heat exchanger with receiver/accumulator models with adequate validation results [6]. Also in 2006, Keir presented a lumped-parameter moving-boundary evaporator frost growth model used to improve fault detection and diagnosis of vapor compression systems [15].

1.3.1.2 Model Validation

Model validation is a critical part of the modeling process. An overview of various model validation approaches can be found in [31]. Technically speaking, a model cannot be validated by a finite set of data, but instead, only invalidated [23]. However, each successive successful validation of the model should increase confidence in the model's predictive capability.

In this thesis, each model will be validated for both its steady state performance as well as its transient performance. At steady state, the level of agreement between model predictions and the actual system can be quantified with a simple percentage difference error. During transients, the model validation is first a qualitative one. That is, the question is asked, "Does the model accurately characterize the dominant dynamics of the system?" If this is true, then a more quantitative metric, such as RMS error, can be used to compare the accuracy of different models, or the same model at different operating conditions.

1.3.2 Thermosys Toolbox

The Thermosys 2007a Toolbox for Matlab is a suite of simulation tools for vapor compression systems developed at the University of Illinois at Urbana-Champaign through sponsorship by the Air Conditioning and Refrigeration Center. Until now,

Thermosys has contained mostly basic VCC system components (i.e. evaporator, condenser, expansion valve, compressor). This thesis will expand the existing toolbox to contain components such as a suction line heat exchanger, a refrigerant distributor, etc., that are often included in industrial refrigeration units.

The Thermosys 2007a modeling framework requires that each model be designated as either a static model or a dynamic model. Static models are of the form shown in Equation 1.1, where the outputs are solely a function of exogenous inputs collected in a vector u .

$$y = g(u) \quad (1.1)$$

Components which are modeled with static equations often have dynamics which are much faster as compared to those of other components. The transient response to a perturbation in this type of component occurs so quickly that it inherently does not have a large effect on the overall system dynamics, allowing for its dynamics to be ignored. Figure 1.3 shows the basic Simulink block diagram for static models. The actual model equations are contained within a Matlab .m file that is called at each time step by the *Matlab Function* block. An example of such a .m file is shown in Appendix D.1.

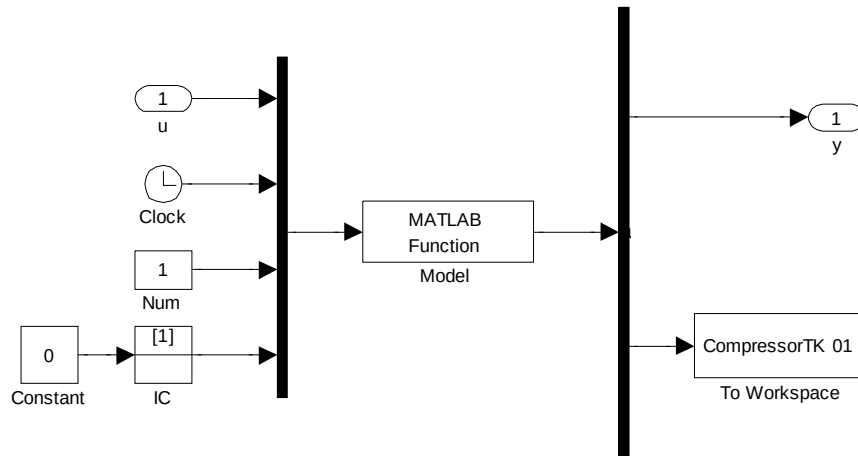


Figure 1.3: Simulink block diagram for static modeling framework.

Dynamic models are of the form shown in Equation 1.2 and Equation 1.3, where both the state derivative vector, $\dot{x}$, and the output vector, y , are each a function of the states of the model, x , and the exogenous inputs, u .

$$\dot{x} = f(x, u) \quad (1.2)$$

$$y = g(x, u) \quad (1.3)$$

Examples of components with slower, and therefore more dominant, dynamics, include heat exchangers and mass storage devices (such as accumulators and receivers). Figure 1.4 shows the basic Simulink diagram for a dynamic model. Again, the model equations are contained within a Matlab .m file; an example of a .m file for a dynamic model is shown in Appendix D.2.

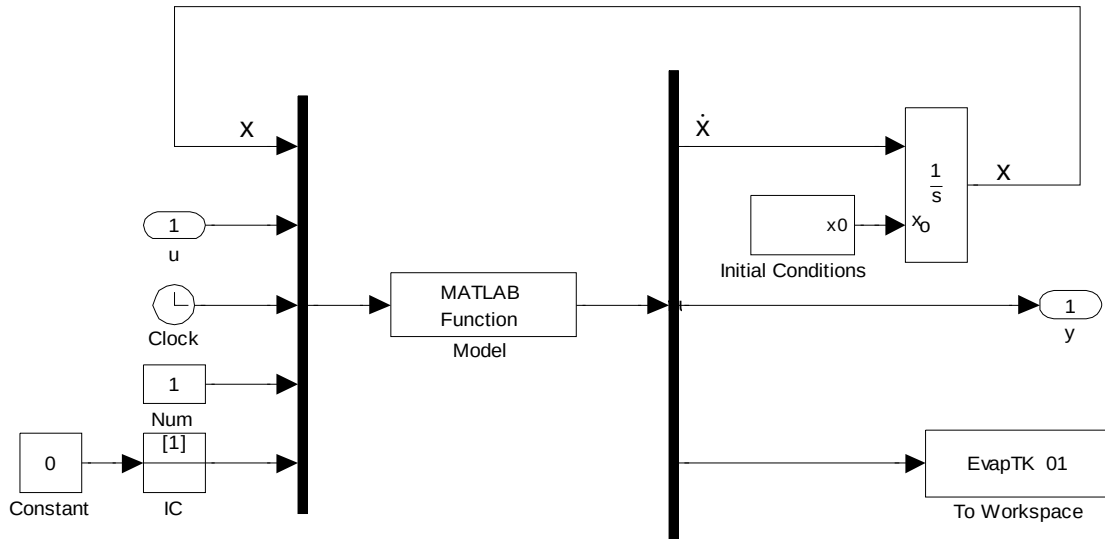


Figure 1.4: Simulink block diagram for dynamic modeling framework.

Additional information regarding the development of Thermosys 2007a can be found in help files contained within the Thermosys Toolbox itself.

1.4 Organization of Thesis

The remainder of this thesis is organized as follows. Chapter 2 describes the two modes of operation of the experimental system – *Cool Mode* and *Heat Mode* – as well as the process through which data was acquired for both semi-empirical models and the ultimate validation of the system model. Chapter 3 describes the modeling of the individual components which are active during Cool Mode operation. Chapter 4 describes the modeling of the individual components which are active during Heat Mode operation. Chapter 5 and Chapter 6 present the model validation of the Cool Mode and

Heat Mode models, respectively. The thesis concludes in Chapter 7 with a summary of the modeling and validation results as well as directions for future work.

Chapter 2

EXPERIMENTAL SYSTEM

2.1 System Description

Previous modeling efforts [6],[24] have focused on an idealized vapor compression cycle (VCC) system, i.e. one containing the four basic components of a VCC system: compressor, expansion valve, condenser, and evaporator. However, in order to model a commercial transport refrigeration system developed and used in industry, many additional components need to be considered, such as refrigerant distributors and secondary valves. Figure 2.1 shows a photo of a Thermo King transport refrigeration unit mounted on a truck body.



Figure 2.1: Thermo King transport refrigeration unit mounted on a truck body.

Specifically for transport refrigeration units, such as the Thermo King TS-500, there is a need for the capability to both cool and heat the attached environment depending on the ambient conditions and the desired temperature set point for the environment. For this reason, the refrigeration unit described here is a dual mode system. Section 2.1.1 describes the primary mode of operation, Cool Mode, in which the refrigeration unit is used to cool (i.e. extract heat from) the attached environment. Section 2.1.2 describes the second mode of operation, Heat Mode, in which the refrigeration unit is used to heat the attached environment.

2.1.1 Cool Mode

The primary mode of operation for the refrigerant unit is Cool Mode, and consists of the following components:

- Compressor
- 3-Way Valve
- Condenser
- Receiver
- Suction Line Heat Exchanger
- Thermostatic Expansion Valve
- Distributor
- Evaporator
- Accumulator
- Throttle Valve.

Figure 2.2 shows a schematic of the system in Cool Mode operation. Figure 2.3 shows the Cool Mode cycle plotted on a pressure-enthalpy diagram for refrigerant R404a at a given operating condition.

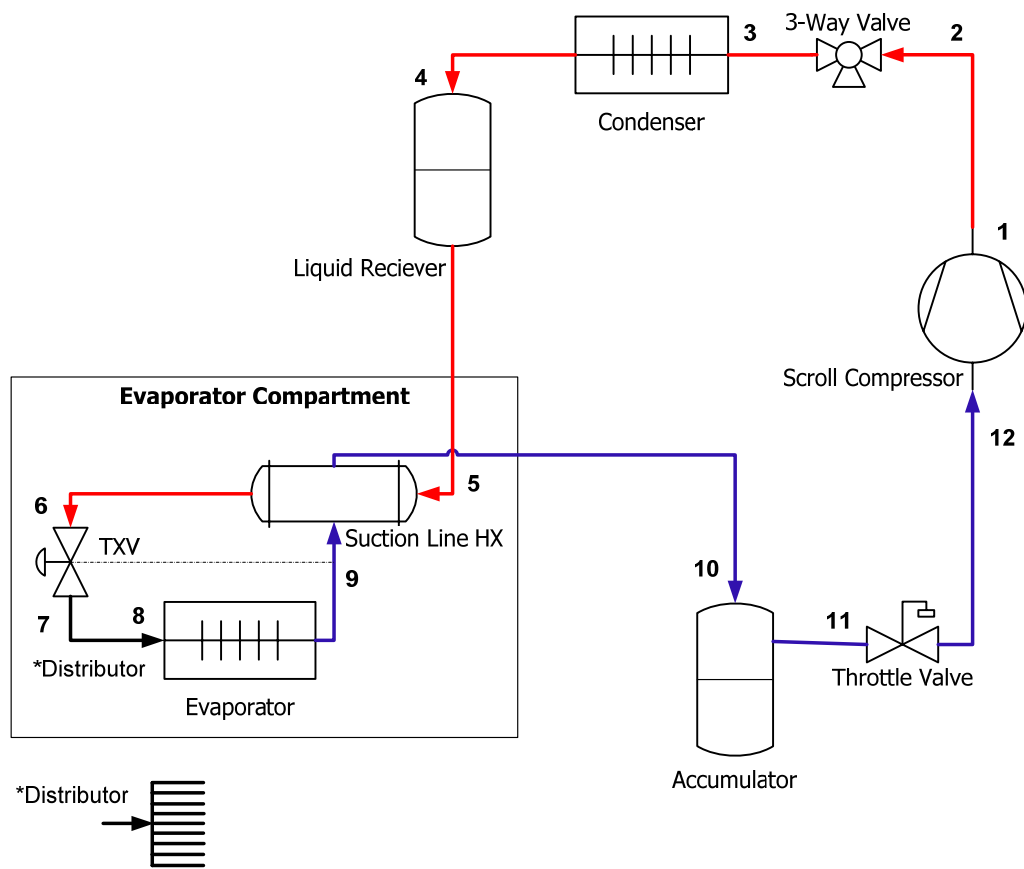


Figure 2.2: System schematic during Cool Mode operation.

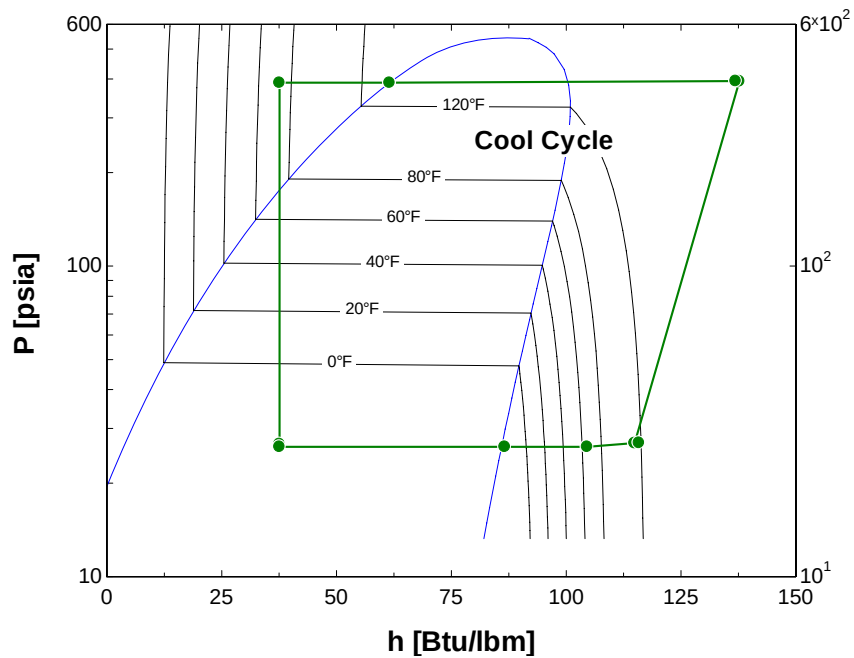


Figure 2.3: Pressure-enthalpy diagram for Cool Mode cycle.

The refrigerant exits the compressor at (1) and is directed by a 3-way valve (2) to the condenser coil. The superheated vapor enters the condenser coil (3) where it is condensed into a two-phase fluid. The two-phase refrigerant is then separated by the receiver tank (4) so that only saturated liquid continues through the system. The liquid refrigerant flows through the inner coil of a suction line heat exchanger (SLHX) (5) where it loses heat to the cooler refrigerant flowing through the outer shell of the SLHX. The now subcooled liquid is expanded as it passes through the thermostatic expansion valve (TXV) (6) and then enters the distributor (7) as a two-phase refrigerant at a lower pressure. The distributor splits the refrigerant flow among the parallel circuits of the evaporator coil. The refrigerant then absorbs heat as it passes through the evaporator coil (8) and exits as a superheated vapor. It is directed back towards the SLHX and flows through the outer shell (9). Recall that this is the refrigerant that absorbed the heat lost by the liquid refrigerant exiting the condenser. This now superheated vapor passes through the accumulator (10) to a throttle valve (11) which is used to control the pressure entering the compressor. Finally, the refrigerant returns to the compressor (12).

2.1.2 Heat Mode

The second mode of operation for most transport refrigeration units is Heat Mode. The transition from Cool Mode to Heat Mode is completed using a 3-way valve which redirects the refrigerant exiting the compressor to a discharge pressure regulator (DPR) valve rather than to the condenser coil. Figure 2.4 shows a schematic of the system in Heat Mode operation. Figure 2.5 shows the Heat Mode cycle plotted on a pressure-enthalpy diagram for refrigerant R404a at a given operating condition.

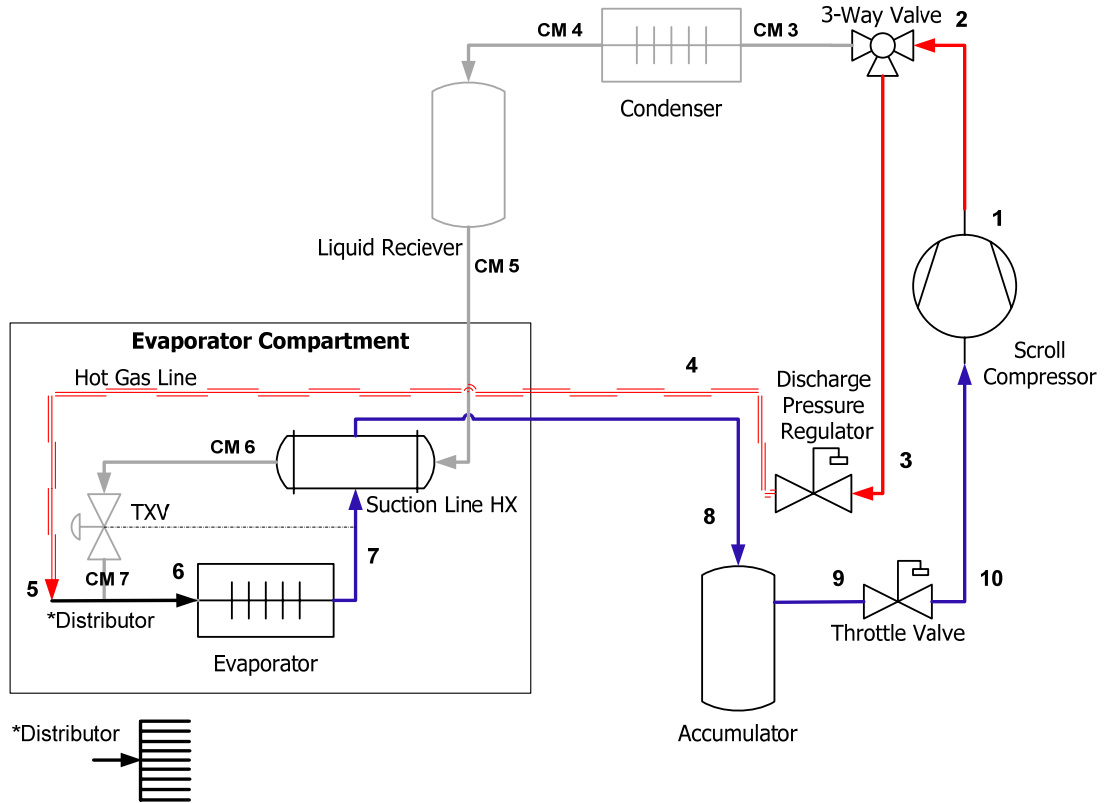


Figure 2.4: System schematic during Heat Mode operation.

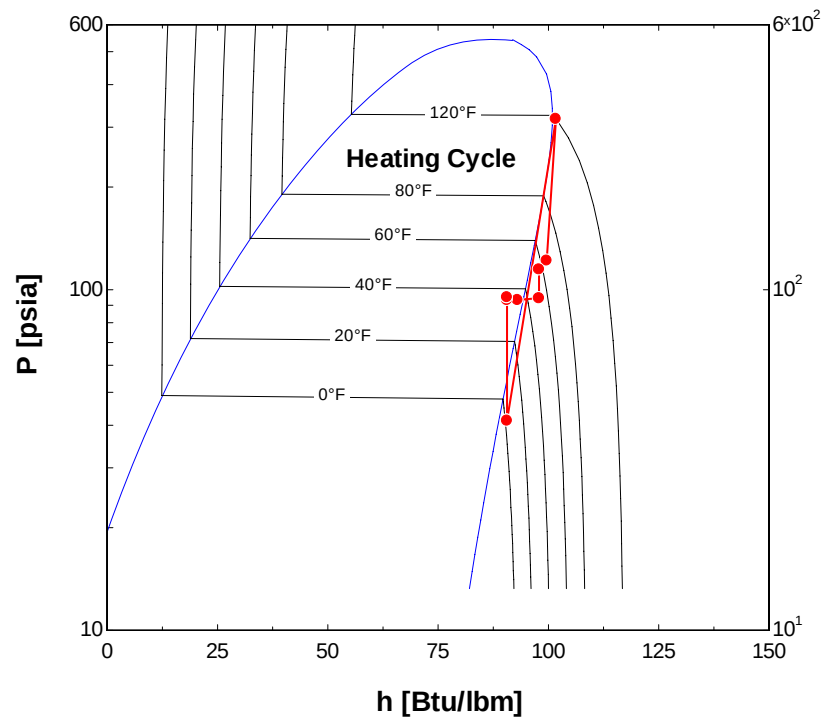


Figure 2.5: Pressure-enthalpy diagram for Heat Mode cycle.

The 3-way valve and a check valve (located immediately downstream of the receiver tank) are closed so as to trap the refrigerant sitting between the inlet of the condenser (CM 3) and the outlet of the expansion valve (CM 7) when the unit switches from Cool Mode to Heat Mode (see Figure 2.4). In this way, the condenser, receiver, and TXV are no longer a part of the refrigerant unit. The remaining refrigerant in the system exits the compressor (1), passes through the DPR valve via the 3-way valve (3), and then flows through the hot gas line (4) to the distributor. The distributor (5) splits the flow among the parallel circuits of the evaporator coil which now behaves like a condenser, taking superheated vapor in and condensing it into a two-phase fluid (6). The two-phase flow recombines at the outlet of the evaporator (7) before it continues to the accumulator. Because refrigerant has been trapped in the inner coil of the SLHX during Heat Mode, the two-phase refrigerant exiting the evaporator passes through the outer shell of the SLHX but does not undergo significant heat transfer (7). For this reason the SLHX will be modeled as a simple pipe element in the Heat Mode configuration. The two-phase refrigerant is separated by the accumulator (8). Lastly, saturated vapor exits the accumulator (9), passes through the throttle valve, and finally returns to the compressor (10).

2.2 Data Acquisition

All of the data used for the validation of the Cool Mode and Heat Mode models was collected at Thermo King Corporation in Bloomington, Minnesota. The tests were conducted in an ARI-1110 approved Calibrated-Box test facility.

2.2.1 Test Unit Instrumentation

The refrigeration unit used for the data collection was heavily instrumented with thermocouples and pressure transducers. The refrigerant temperatures were measured using immersion-type thermocouples. Air temperatures were measured using thermocouples with 8-point measurements at the condenser inlet, evaporator inlet, and evaporator outlet. The outlet of the condenser was measured by an array of 4 thermocouples. Pressure transducers were located at the following locations:

- compressor inlet
- compressor outlet
- liquid injection valve
- receiver outlet
- EEV inlet
- EEV outlet
- evaporator outlet
- throttle valve inlet.

Additional instrumentation allowed for the following measurements to be collected on the system:

- compressor speed
- motor/condenser fan speed
- box relative humidity
- refrigerant mass flow rate
 - at the inlet of the EEV during Cool Mode
 - at the outlet of the compressor during Heat Mode
- standby motor power (Volts, Amps, Watts)
- fuel flow rate
- tube wall temperature at condenser coil mid-point.

Figure 2.6 and Figure 2.7 provide schematics of the test unit instrumentation during Cool Mode and Heat Mode testing, respectively.

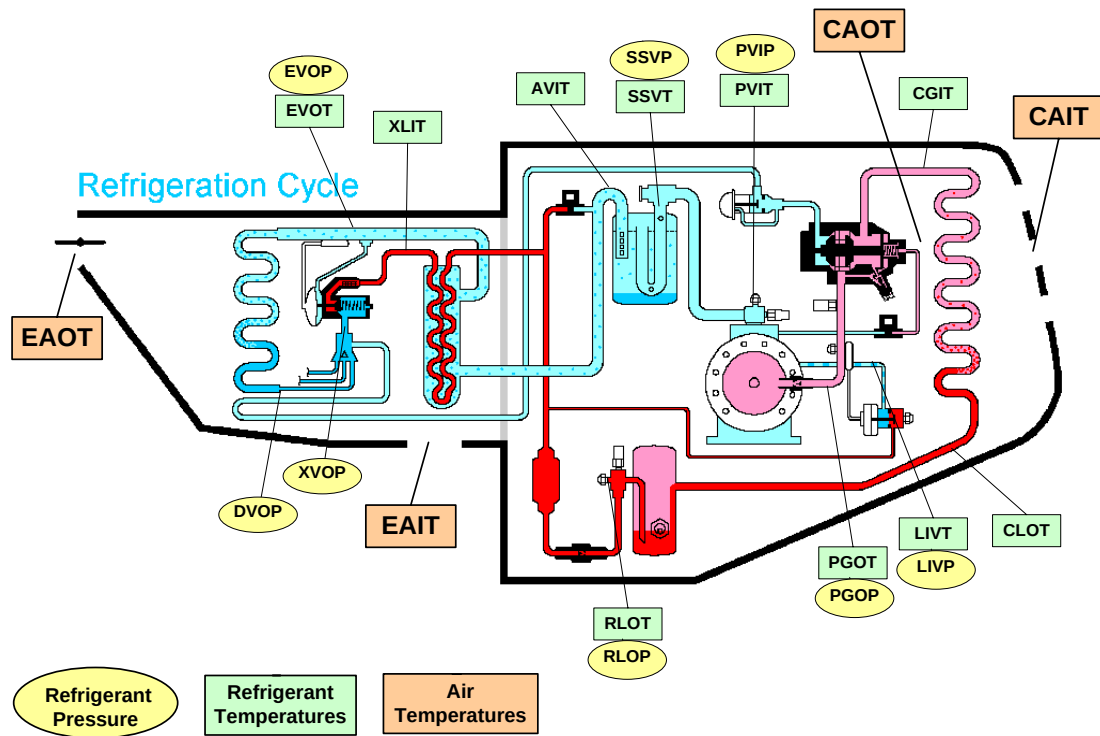


Figure 2.6: Schematic of test unit instrumentation during Cool Mode testing.

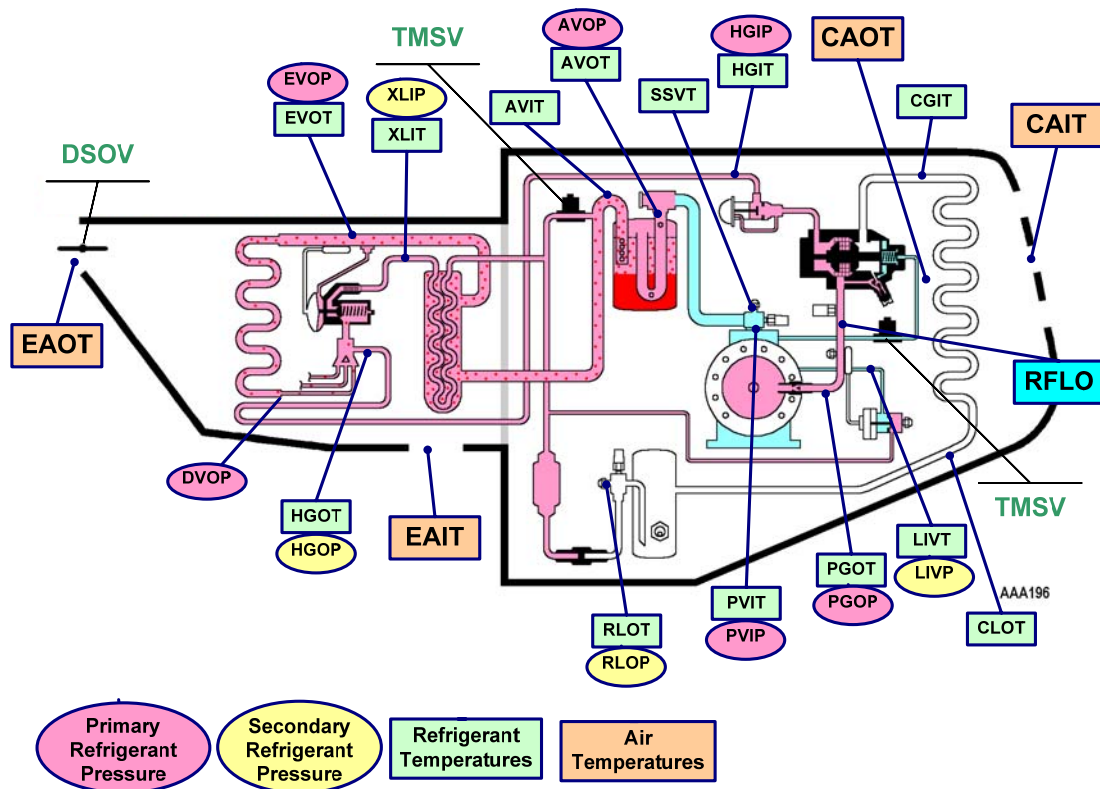


Figure 2.7: Schematic of test unit instrumentation during Heat Mode testing

2.2.2 Steady State Testing

The refrigeration unit is designed to operate at a wide range of conditions during both Cool Mode and Heat Mode. Here, an operating condition is defined as “*box|ambient*”, where the *box* temperature represents the temperature set point within the truck compartment, and the *ambient* temperature represents the temperature of the air surrounding the truck. During Cool Mode operation, the system typically maintains box temperatures of -20 °F, 0 °F, 35 °F, and 70 °F for all of the following ambient temperatures: 50 °F, 75 °F, 100 °F, and 120 °F.

The compressor is operated using a diesel engine and can run at two speeds, denoted high speed and low speed (approximately 4000 rpm and 2500 rpm, respectively). The compressor can also be operated using an electric motor. For completeness, steady-state data, sampled at 0.0667 Hz (i.e. every 15 seconds), was collected at every combination of box and ambient temperature for Cool Mode (given that the ambient temperature was higher than the box temperature). Moreover, for each *box|ambient* condition, the compressor was operated at low speed using the diesel engine (LSD), high speed using the diesel engine (HSD), and with a fixed speed using the electric motor (ELEC). Data was captured using the electric motor to validate the system without the additional influences of the engine (i.e. an additional condenser load due to the radiator and extra heat in the ambient space around the individual components). Table 2.1 summarizes the steady state testing envelope for Cool Mode operation.

Table 2.1 Summary of Steady State Data Collected During Cool Mode Operation

Box Temperature, °F	Ambient Temperature, °F			
	50	75	100	120
70		HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC
35	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC
0	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC
-20	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC

During Heat Mode operation, a similar approach was used but with a different set of *box|ambient* conditions. Table 2.2 summarizes the steady state testing envelope for Heat Mode operation.

Table 2.2 Summary of Steady State Data Collected During Heat Mode Operation

Box Temperature, °F	Ambient Temperature, °F		
	-20	0	35
70		HSD, LSD, ELEC	HSD, LSD, ELEC
50	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC
35	HSD, LSD, ELEC	HSD, LSD, ELEC	HSD, LSD, ELEC

2.2.3 Transient Testing

While the steady state data provides information about the system after it has reached equilibrium, it is valuable to have data which accurately captures transients in the system. This type of data can be collected by perturbing the system with step changes in various inputs simultaneously, effectively exciting all modes, or dynamics, of the system. It must also be collected at a higher frequency to sufficiently capture these dynamics; in this case, data was collected at 1 Hz.

This data is useful for two reasons. First, it can be used to perform system identification on the system, or on specific components within the system. Second, it can be used to validate the capability of a given model to predict transient behavior. The latter is something that is of particular interest because for model-based control design, an accurate representation of system dynamics is essential.

Two types of signals were used to perturb the system: pseudo-random binary sequence (PRBS) signals and random Gaussian sequence (RGS) signals. It is preferable to use a RGS signal when possible simply because it allows you to perturb the system at multiple points around a nominal operating condition. A PRBS signal is a binary signal and therefore can take on only two values. Nevertheless, both types of signals can sufficiently excite the system. In this case, because the compressor is not operated using a variable speed drive, it was necessary to use a PRBS signal for the compressor speed

input. Both types of signals are generated using the System Identification Toolbox in Matlab with the function *idinput*.

2.2.3.1 Cool Mode

The first set of transient data was collected on the instrumented unit with a TXV as the expansion device. However, because the TXV is a thermo-mechanical control valve, it could not be perturbed using an RGS signal. Moreover, both the condenser and evaporator fans are powered by the same engine which powers the compressor through a belt drive system, and their speed could not be individually controlled. The only input that could be perturbed was the compressor speed. A second controllable input was created by electronically opening and closing the dampers in front of the evaporator coil; when they were closed, the airflow rate across the evaporator was effectively zero.

After examining the data collected this way, we determined it was not an ideal set of data for the transient validation of the Cool Mode model. Because all of the collected data captured the system response to perturbations in both compressor speed and the opening/closing of evaporator dampers, the same input signals used on the experimental system would need to be used in the model simulation. However, in order to mimic the closing of dampers in simulation, the evaporator airflow rate would need to be set to zero at certain intervals, and the evaporator model is currently not able to function with a zero airflow rate input.

This led to a second set of transient data collection. The TXV was replaced with a comparable electronic expansion valve (EEV) to allow for the valve flow opening to be controlled to specific values around a nominal operating condition. Ideally, the system should be perturbed with simultaneous step changes in all controllable inputs (in this case, EEV stroke and compressor speed). However, problems related to the loss of evaporator superheat prevented such data from being collected. To be more specific, for large changes in compressor speed (e.g. while changing from high speed to low speed, or vice versa), the value of EEV stroke needed to maintain a satisfactory level of superheat (i.e. a minimum of 15 °F) changed dramatically. The input signal for the EEV was designed to perturb the valve no more than 10% about a given nominal operating condition which was not large enough to maintain 15 °F of superheat when the

compressor speed changed. Consequently, the system was perturbed with only one input at a time.

To perturb the compressor, a pseudo-random binary sequence of 1's and 0's was generated using the Matlab function *idinput* and then translated into high speed and low speed operating commands for the compressor. Figure 2.8 shows the PRBS input signal that was generated for compressor speed. Note that the signal shown is the actual recorded signal and not the signal as it was generated by Matlab. For a more complete description of *idinput* and the generated signal, see Appendix A.

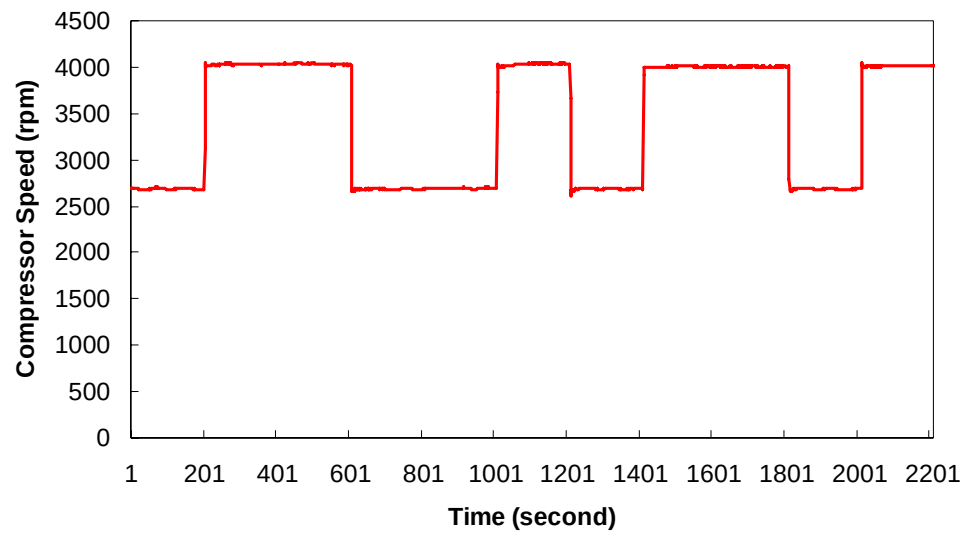


Figure 2.8: PRBS input signal for compressor speed.

The EEV is operated using a stepper motor, and therefore it was possible to choose a small range of values to step between using a random Gaussian sequence of ± 1 's, ± 2 's, and ± 3 's, again generated using Matlab. With 0 representing some nominal operating condition, the sequence of ± 1 's, ± 2 's, and ± 3 's was scaled and then added to that nominal operating condition. For the EEV, the nominal operating condition, at a given box|ambient condition, was chosen by running the system at steady-state and determining the command needed to maintain at least fifteen degrees of evaporator superheat. Figure 2.9 shows the RGS input signal that was used to perturb the EEV at the 0|100 °F operating condition. Again, note that the signal shown is the actual recorded signal and not the signal as it was generated by Matlab.

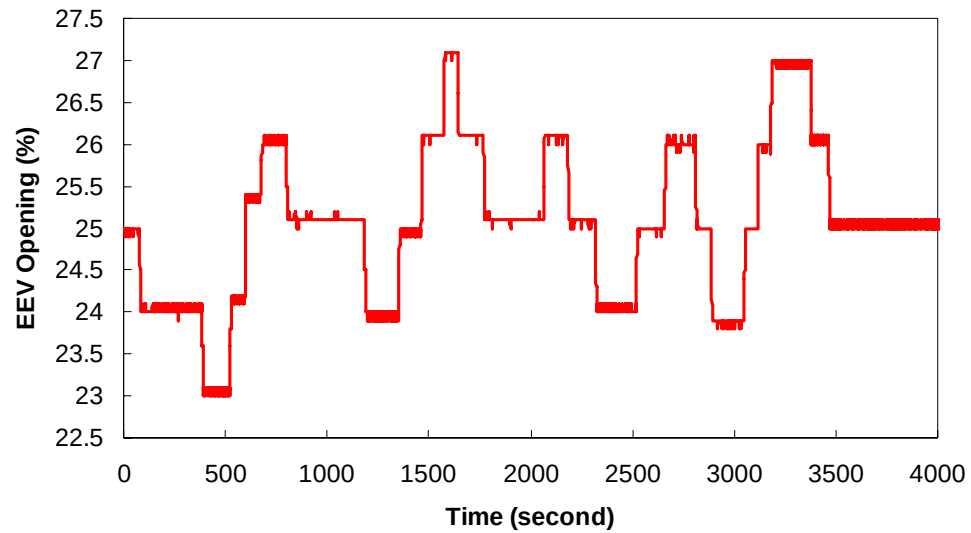


Figure 2.9: RGS input signal for EEV opening at 0|100 °F operating condition.

Transient data collection is time consuming, and consequently the data was only collected at a small subset of the box|ambient conditions that were tested at steady state (specifically 0|100 °F and -20|50 °F box|ambient).

2.2.3.2 Heat Mode

In Heat Mode operation, the expansion valve is isolated from the system. Therefore, only the compressor speed can be perturbed. Again, because the compressor can only be operated at two speeds, a PRBS signal, rather than a RGS signal, was generated using the Matlab command *idinput* to be applied to the compressor during the Heat Mode transient testing (see Figure 2.10). Since the high and low speeds of the compressor are fixed values, the same signal was applied to the compressor at multiple operating conditions.

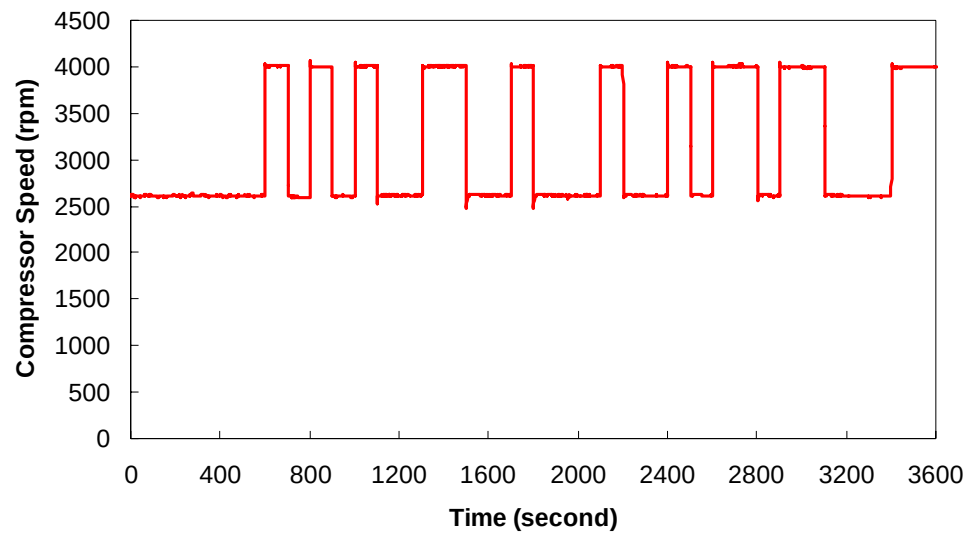


Figure 2.10: PRBS input signal for compressor speed during Heat Mode testing.

Chapter 3

COOL MODE MODELING

This chapter describes the model development of the Cool Mode Thermosys model.

3.1 Dynamic Model Components

As described in Chapter 1, those components whose dynamics are slow, and therefore dominant in the system, are modeled using dynamic equations. During Cool Mode operation, this includes the two heat exchangers, the receiver and accumulator, as well as the distributor.

3.1.1 Heat Exchangers

The nonlinear evaporator and condenser (with receiver) models developed for the Cool Mode model are adaptations of heat exchanger models previously developed in [24]. These models were developed in the lumped-parameter moving boundary framework. The governing equations are derived by integrating the governing partial differential equations (PDEs) for conservation of mass, and conservation of refrigerant energy and tube wall energy, respectively (Equations 4.2, 4.3, and 4.4) along the length of the heat exchanger. The assumption of lumped parameters associated with each fluid region is applied.

The resulting nonlinear evaporator and condenser (with receiver) models are presented in Equation 3.1 and Equation 3.2, respectively, and are of the form $Z(x, u) \cdot \dot{x} = f(x, u)$, with the elements of the Z matrix given in Table 3.1 and Table 3.3,

respectively, and the elements of the f vector given in Table 3.2 and Table 3.4, respectively. Detailed derivations of each of these models can be found in [24].

$$\begin{bmatrix} z_{11} & z_{12} & 0 & 0 & 0 \\ z_{21} & z_{22} & z_{23} & 0 & 0 \\ z_{31} & z_{32} & z_{33} & 0 & 0 \\ 0 & 0 & 0 & z_{44} & 0 \\ z_{51} & 0 & 0 & 0 & z_{55} \end{bmatrix} \begin{bmatrix} \dot{L}_1 \\ \dot{P}_c \\ \dot{h}_{out} \\ \dot{T}_{w,1} \\ \dot{T}_{w,2} \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \end{bmatrix} \quad (3.1)$$

Table 3.1 Matrix Elements of $Z(x, u)$ for the Nonlinear Evaporator

z_{11}	$\left[\rho_f (h_f - h_g) \right] (1 - \bar{\gamma}) A_{cs}$
z_{12}	$\left[\left(\frac{d(\rho_f h_f)}{dP_c} - \frac{d\rho_f}{dP_c} h_g \right) (1 - \bar{\gamma}) + \left(\frac{d(\rho_g h_g)}{dP_c} - \frac{d\rho_g}{dP_c} h_g \right) \bar{\gamma} - 1 \right] A_{cs} L_1$
z_{21}	$\rho_2 (h_g - h_2) A_{cs}$
z_{22}	$\left[\left[\left(\frac{\partial \rho_2}{\partial P_e} \right)_{h_2} + \left(\frac{1}{2} \right) \left(\frac{\partial \rho_2}{\partial h_2} \right)_{P_e} \right] \left(\frac{dh_g}{dP_e} \right) \right] (h_2 - h_g) + \left(\frac{\rho_2}{2} \right) \left(\frac{dh_g}{dP_e} \right) - 1 \right] A_{cs} L_2$
z_{23}	$\left[\frac{1}{2} \left(\frac{\partial \rho_2}{\partial h_2} \right)_{P_e} \right] (h_2 - h_g) + \left(\frac{\rho_2}{2} \right) A_{cs} L_2$
z_{31}	$\left[(\rho_g - \rho_2) + (\rho_f - \rho_g) (1 - \bar{\gamma}) \right] A_{cs}$
z_{32}	$\left[\left[\left(\frac{\partial \rho_2}{\partial P_e} \right)_{h_2} + \frac{1}{2} \left(\frac{\partial \rho_2}{\partial h_2} \right)_{P_e} \right] \left(\frac{dh_g}{dP_e} \right) \right] L_2 + \left[\left(\frac{d\rho_f}{dP_e} \right) (1 - \bar{\gamma}) + \left(\frac{d\rho_g}{dP_e} \right) \bar{\gamma} \right] L_1 \right] A_{cs}$
z_{33}	$\frac{1}{2} \left(\frac{\partial \rho_2}{\partial h_2} \right)_{P_e} A_{cs} L_2$
z_{44}	$(C_p \rho V)_w$
z_{51}	$(C_p \rho V)_w \left(\frac{T_{w1} - T_{w2}}{L_2} \right)$

Table 3.1 (continued)

z_{55}	$(C_p \rho V)_w$
----------	------------------

Table 3.2 Vector Elements of $f(x, u)$ for the Nonlinear Evaporator

f_1	$\dot{m}_i (h_i - h_g) + \alpha_{i,1} A_i \left(\frac{L_1}{L_{total}} \right) (T_{w,1} - T_{r,1})$
f_2	$\dot{m}_o (h_g - h_o) + \alpha_{i,2} A_i \left(\frac{L_2}{L_{total}} \right) (T_{w,2} - T_{r,2})$
f_3	$\dot{m}_i - \dot{m}_o$
f_4	$\alpha_{i,1} A_i (T_{r,1} - T_{w,1}) + \alpha_o A_o (T_{a,1} - T_{w,1})$
f_5	$\alpha_{i,2} A_i (T_{r,2} - T_{w,2}) + \alpha_o A_o (T_{a,2} - T_{w,2})$

$$\begin{bmatrix} z_{11} & z_{12} & 0 & 0 & 0 & 0 \\ z_{21} & z_{22} & z_{23} & z_{24} & 0 & 0 \\ z_{31} & z_{32} & z_{33} & z_{34} & 0 & 0 \\ 0 & z_{42} & 0 & z_{44} & 0 & 0 \\ z_{51} & 0 & 0 & 0 & z_{55} & 0 \\ 0 & 0 & 0 & 0 & 0 & z_{66} \end{bmatrix} \begin{bmatrix} \dot{L}_1 \\ \dot{P} \\ \dot{\gamma} \\ \dot{m}_{rec} \\ \dot{T}_{w1} \\ \dot{T}_{w2} \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \end{bmatrix} \quad (3.2)$$

Table 3.3 Matrix Elements of $Z(x, u)$ for the Nonlinear Condenser with Receiver

z_{11}	$(\rho_1 h_1 - \rho_1 h_g) A_{cs}$
z_{12}	$\left[(h_1 - h_g) \left(\frac{\partial \rho_1}{\partial P_c} + \frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \frac{dh_g}{dP_c} \right) + \left(\frac{1}{2} \frac{dh_g}{dP_c} \right) \rho_1 - 1 \right] A_{cs} L_1$
z_{21}	$(\rho_f h_g - \rho_f h_f) (1 - \bar{\gamma}) A_{cs}$

Table 3.3 (continued)

z_{22}	$\left(\frac{d(\rho_f h_f)}{dP_c} (1 - \bar{\gamma}) + \frac{d(\rho_g h_g)}{dP_c} \bar{\gamma} - \frac{d\rho_f}{dP_c} h_g (1 - \bar{\gamma}) - \frac{d\rho_g}{dP_c} h_g \bar{\gamma} - 1 \right) A_{cs} L_2$
z_{23}	$(\rho_f h_g - \rho_f h_f) A_{cs} L_2$
z_{24}	$(h_{\text{int}2} - h_g)$
z_{31}	$\left[(\rho_g - \rho_f)(1 - \bar{\gamma}) + (\rho_1 - \rho_g) \right] A_{cs}$
z_{32}	$\left[\left(\frac{d\rho_f}{dP_c} (1 - \bar{\gamma}) + \frac{d\rho_g}{dP_c} \bar{\gamma} \right) L_2 + \left(\frac{\partial \rho_1}{\partial P_c} + \frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \frac{dh_g}{dP_c} \right) L_1 \right] A_{cs}$
z_{33}	$(\rho_g - \rho_f) A_{cs} L_2$
z_{34}	1
z_{42}	$\left[\frac{d\rho_g}{dP_c} V_g u_g + \frac{d\rho_f}{dP_c} V_f u_f + m_g \frac{du_g}{dP_c} + m_f \frac{du_f}{dP_c} - \left(\frac{\rho_g u_g - \rho_f u_f}{\rho_g - \rho_f} \right) \left(\frac{d\rho_g}{dP_c} V_g + \frac{d\rho_f}{dP_c} V_f \right) \right]$
z_{44}	$\left(\frac{\rho_g u_g - \rho_f u_f}{\rho_g - \rho_f} - h_{\text{int}2} \right)$
z_{51}	$(C_p \rho V)_w \left(\frac{T_{w1} - T_{w2}}{L_1} \right)$
z_{55}	$(C_p \rho V)_w$
z_{66}	$(C_p \rho V)_w$

Table 3.4 Vector Elements of $f(x, u)$ for the Nonlinear Condenser with Receiver

f_1	$\dot{m}_i (h_i - h_g) + \alpha_{i,1} A_i \left(\frac{L_1}{L_{\text{total}}} \right) (T_{w,1} - T_{r,1})$
f_2	$\dot{m}_o (h_g - h_{\text{int}2}) + \alpha_{i,2} A_i \left(\frac{L_2}{L_{\text{total}}} \right) (T_{w,2} - T_{r,2})$

Table 3.4 (continued)

f_3	$\dot{m}_i - \dot{m}_o$
f_4	$\dot{m}_o (h_{\text{int}2} - h_f) - UA_{\text{rec}} (T_{\text{rec}} - T_{\text{amb}})$
f_5	$\alpha_{i,1} A_i (T_{r,1} - T_{w,1}) + \alpha_o A_o (T_{a,1} - T_{w,1})$
f_6	$\alpha_{i,2} A_i (T_{r,2} - T_{w,2}) + \alpha_o A_o (T_{a,2} - T_{w,2})$

The evaporator and condenser coils used in the TS-500 have been designed with a counter-crossflow configuration for refrigerant and air flow as well as a multi-circuit geometry, both of which were not considered in the original models. Moreover, the air-side heat transfer coefficients for the original heat exchanger models were based on the Colburn j -factor correlation [6]. The correlation requires a set of parameters to be determined offline based on the physical geometry of a given heat exchanger. Consequently, modifications were made such that these differences were taken into account while maintaining the basic structure of the original models. These modifications are described in more detail in the following sections.

3.1.1.1 Multi-Circuit Heat Exchangers

In order to compensate for a multi-circuit evaporator, the evaporator was modeled as a single-circuit heat exchanger with a reduced mass flow rate

$$\dot{m}_{\text{single}} = \frac{1}{n} \dot{m}_{\text{total}} \quad (3.3)$$

where n is the number of circuits. Furthermore, relevant physical parameters, i.e. total heat exchanger mass, coil length, external surface area, internal surface area, and air flow cross-sectional area, were also reduced by a factor of $1/n$. The underlying assumption is that heat transfer characteristics of each coil are uniform across all n circuits. This assumption is later validated in Chapter 5.

In the condenser coil, the number of circuits is not uniform across the length of the coil. Given that there is a higher heat transfer rate across the two-phase region, and assuming that the dynamics in that region would consequently be dominant, the number

of circuits in the two-phase fluid region is assumed for the entire condenser coil. This assumption is also validated in Chapter 5.

3.1.1.2 Counter-Cross Air Flow Configuration

A cross flow heat exchanger is one in which the two flow paths are orthogonal to one another (see Figure 3.1). In a counter-cross flow heat exchanger, not only are the flow paths orthogonal, but they enter and exit the heat exchanger at opposing ends (see Figure 3.2).

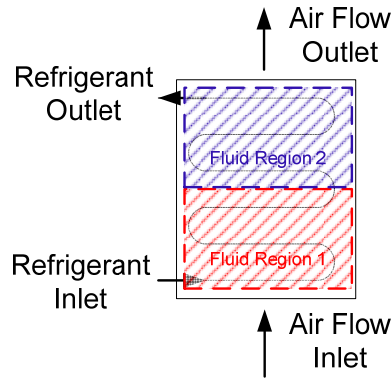


Figure 3.1: Cross flow configuration for air to refrigerant heat exchanger.

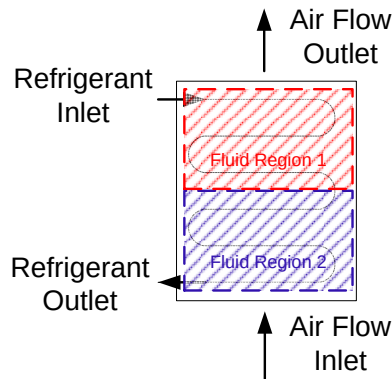


Figure 3.2: Counter-cross flow configuration for air to refrigerant heat exchanger.

For the evaporator and condenser coils in the TS-500, this implies that the *inlet* air passes over the *outlet* refrigerant. Similarly, the *outlet* air passes over the *inlet* refrigerant. In the original modeling framework, a single average air temperature was assumed across all fluid regions. Assuming two fluid regions in each heat exchanger, the following steady state energy balances can then be written for the coil walls in the first and second fluid regions, respectively,

$$\alpha_{i,1}A_i(T_{r,1}-T_{w,1})=\alpha_oA_o(T_a-T_{w,1}) \quad (3.4)$$

$$\alpha_{i,2}A_i(T_{r,2}-T_{w,2})=\alpha_oA_o(T_a-T_{w,2}) \quad (3.5)$$

where T_a is a weighted average of the inlet and outlet air temperatures. However, in a counter-cross flow configuration, it is important to note that the air passing over the first fluid region has already undergone heat transfer with the refrigerant in the second fluid region. For this reason, a separate lumped air temperature was assumed for each region. With this assumption, Equation 3.4 and Equation 3.5 become

$$\alpha_{i,1}A_i(T_{r,1}-T_{w,1})=\alpha_oA_o(T_{a,1}-T_{w,1}) \quad (3.6)$$

$$\alpha_{i,2}A_i(T_{r,2}-T_{w,2})=\alpha_oA_o(T_{a,2}-T_{w,2}) \quad (3.7)$$

where $T_{a,1}$ and $T_{a,2}$ are defined in Equation 3.8 and Equation 3.9.

$$T_{a,1}=\frac{1}{2}(T_{a,m}+T_{ao}) \quad (3.8)$$

$$T_{a,2}=\frac{1}{2}(T_{ai}+T_{a,m}) \quad (3.9)$$

The middle air temperature, T_{am} , is a weighted average of the inlet and outlet air temperatures. The only input temperature for the evaporator model is the air inlet temperature, so two additional equations are needed to solve the remaining temperatures of interest: $T_{a,1}$, $T_{a,2}$, $T_{a,m}$, and $T_{a,o}$. The energy balances given in Equation 3.10 and Equation 3.11 [26] complete the system of equations so that these temperatures can be determined.

$$T_{a,2}=\frac{2\dot{m}c_pT_{a,i}+\alpha_oA_o\frac{L_2}{L_{total}}T_{w,2}}{2\dot{m}c_p+\alpha_oA_o\frac{L_2}{L_{total}}} \quad (3.10)$$

$$T_{a,1} = \frac{2\dot{m}c_p T_{a,m} + \alpha_o A_o \frac{L_1}{L_{total}} T_{w,1}}{2\dot{m}c_p + \alpha_o A_o \frac{L_1}{L_{total}}} \quad (3.11)$$

3.1.1.3 Air-Side Heat Transfer Coefficient

In [6] a more accurate prediction of the air-side heat transfer coefficient was developed for the heat exchanger models in the Thermosys Toolbox. The semi-empirical Colburn j factor model was introduced, where the j factor is a parameter that is used to describe experimentally determined heat transfer characteristics of a heat exchanger and correlate them to the Reynolds number of air flowing through the heat exchanger. The j factor correlation takes the form given in Equation 3.12.

$$j = \text{St} \text{Pr}^{2/3} \quad (3.12)$$

The Stanton number, denoted St , is based on the air-side heat transfer coefficient, α_o , as shown in Equation 3.13.

$$\text{St} = \frac{\alpha_o}{G \cdot c_{p,air}} \quad (3.13)$$

The maximum mass velocity, G , is a function of the mass flow rate of air through the heat exchanger, and $c_{p,air}$ is the specific heat of air. By combining Equation 3.12 and Equation 3.13, we arrive at an expression for the external heat transfer coefficient, as given in Equation 3.14 [6].

$$\alpha_o = \frac{j \cdot G \cdot c_{p,air}}{\text{Pr}^{2/3}} \quad (3.14)$$

Using the plain-fin heat exchanger correlation proposed by [32], a new set of j factor data was obtained for each of the two heat exchangers (i.e. the evaporator and the condenser). The Matlab code used to implement the correlation can be found in Appendix C.4 and Appendix C.5 for the condenser and evaporator, respectively.

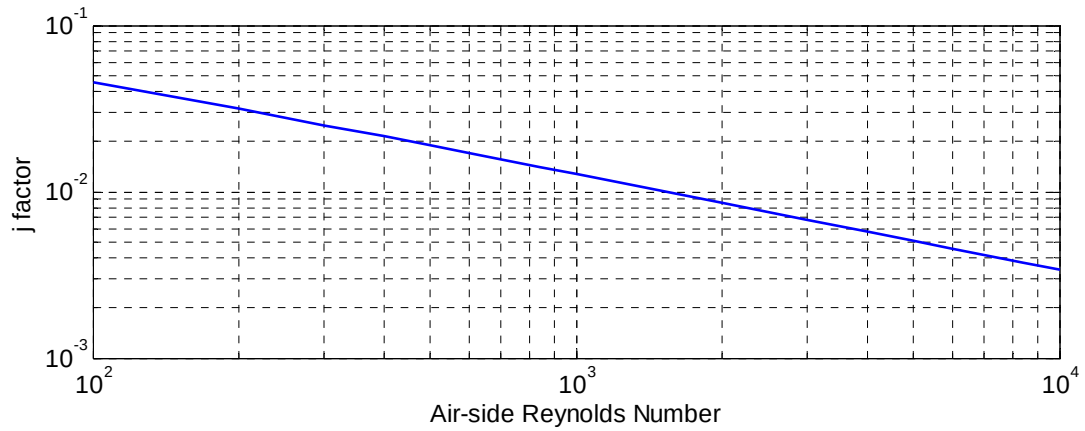


Figure 3.3: Evaporator j factor data.

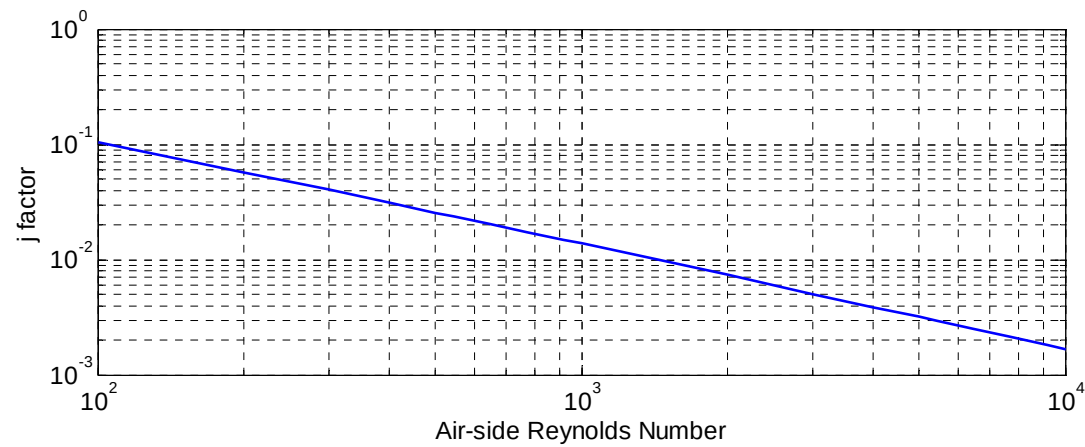


Figure 3.4: Condenser j factor data.

3.1.2 Accumulator

An accumulator is a device used to separate vapor from liquid in two-phase flow. Specifically for VCC systems, an accumulator is often used downstream of the evaporator to ensure that only refrigerant vapor enters the compressor. In Cool Mode operation, however, the accumulator plays a different role. The accumulator is not located directly downstream of the evaporator. Instead, the refrigerant exiting the evaporator first passes through the SLHX where it absorbs heat from the liquid refrigerant passing through the inner coil of the heat exchanger. The refrigerant then

exits the SLHX and enters the accumulator as a superheated vapor. Due to the addition of the SLHX in this refrigeration system, the refrigerant is guaranteed to be a superheated vapor when it enters the accumulator. For this reason, the accumulator does not have to act as a separation device but instead as a pressure drop element.

A second unique aspect of the accumulator is that in the TS-500 refrigeration unit, the engine delivering power to the compressor sits adjacent to the accumulator. Given the configuration of the condenser and the engine with respect to the accumulator, the air exiting the condenser first blows across the engine (where it absorbs heat) and then blows across the accumulator, where it loses heat to the refrigerant inside the tank. In this way, the accumulator acts as a simple pressure drop element with heat transfer characteristics.

3.1.2.1 Pressure Drop

The pressure drop in the accumulator is primarily a function of the sudden expansion of the refrigerant as it enters the accumulator tank and the sudden contraction of the fluid as it exits. The pressure drop correlations for incompressible flow can only be applied here if the Mach number of the flow is sufficiently small, i.e. less than 0.3. The Mach number is defined in Equation 3.15

$$Ma = \frac{v}{a} \quad (3.15)$$

where v is the flow velocity of the refrigerant and a is the speed of sound through the refrigerant [34]. Equation 3.15 was evaluated using steady state test data collected at a range of Cool Mode operating conditions.

Table 3.5 Mach Number of Refrigerant Flow through Accumulator

Operating Condition	Compressor Speed	Mach Number (inlet/outlet diameter)	Mach Number (tank diameter)
35 50	High	0.0633	0.0017
35 70	Electric	0.0576	0.0016
35 120	High	0.0501	0.0014
70 120	Low	0.0185	0.0005
-20 50	High	0.0806	0.0022
0 100	Low	0.0501	0.0014

The average Mach numbers for refrigerant flow through the inlet/outlet pipes, and through the accumulator tank, are 0.0534 and 0.0015, respectively, across a range of Cool Mode operating conditions. Therefore, incompressibility will be assumed for the refrigerant flow through this device.

For incompressible flow, pressure losses due to sudden expansions or contractions of fluid can be calculated using Equation 3.16 and Equation 3.17, respectively [34].

$$\Delta P_{SE} = \frac{1}{2} \rho v^2 K_{SE} \quad (3.16)$$

$$\Delta P_{SC} = \frac{1}{2} \rho v^2 K_{SC} \quad (3.17)$$

The dimensionless loss coefficients K_{SE} and K_{SC} are given in Equation 3.18 and Equation 3.19 [34].

$$K_{SE} = \left(1 - \frac{D_i^2}{D_{tank}^2} \right)^2 \quad (3.18)$$

$$K_{SC} \approx 0.42 \left(1 - \frac{D_o^2}{D_{tank}^2} \right) \quad (3.19)$$

The total pressure drop can be expressed as the sum of Equation 3.16 and Equation 3.17.

$$\Delta P_{total} = \frac{1}{2} \rho v^2 (K_{SE} + K_{SC}) \quad (3.20)$$

3.1.2.2 Heat Transfer

During Cool Mode operation, the refrigerant inside the accumulator tank absorbs heat from the air surrounding the tank. Moreover, because the accumulator tank is made entirely of steel and is not covered with any insulating material, there is a large thermal capacitance dynamic that must be modeled. This can be done using the simple energy balance presented in Equation 3.21.

$$\frac{dE_{tank}}{dt} = Q_{air} - Q_{ref} \quad (3.21)$$

The terms in Equation 3.21 are expanded, resulting in Equation 3.22.

$$(mc)_{tank} \frac{dT_{tank}}{dt} = \alpha_{air} A_{tank} (T_{engine} - T_{tank}) - \alpha_{ref} A_{tank} (T_{tank} - T_{ref}) \quad (3.22)$$

The temperature of the accumulator tank wall is assumed to be uniform. The coefficient of convection between the air and outer surface of the accumulator tank, α_{air} , is determined using the Nusselt number, given in Equation 3.23, where the expression for the Nusselt number given flow across a cylinder [20], is described in Equation 3.24.

$$Nu = \frac{\alpha_{air} d_{tank}}{k_{tank}} \quad (3.23)$$

$$Nu = 0.3 + \left(\frac{0.62 Re^{1/2} Pr^{1/3}}{\left[1 + \left(\frac{0.4}{Pr} \right)^{2/3} \right]^{1/4}} \right) \left[1 + \left(\frac{Re}{282,000} \right)^{1/2} \right]; \quad 4 \times 10^5 < Re < 5 \times 10^6 \quad (3.24)$$

A second energy balance was used to represent the thermal capacitance dynamic of the refrigerant inside the accumulator tank.

$$\frac{dE_{ref}}{dt} = (\dot{m}h)_{r,i} - (\dot{m}h)_{r,o} - Q_{tank} \quad (3.25)$$

The terms in Equation 3.25 are expanded, resulting in Equation 3.26.

$$(mc_p)_{ref} \frac{dT_{ref}}{dt} = (\dot{m}_{r,i} h_{a,ri}) - (\dot{m}_{r,o} h_{a,ro}) + \alpha_{ref} A_{tank} (T_{tank} - T_{ref}) \quad (3.26)$$

Using steady state data collected at a range of Cool Mode operating conditions, a refrigerant heat transfer coefficient, α_{ref} , was calculated using Equation 3.27 and plotted as a function of mass flow rate (see Figure 3.5).

$$\alpha_{ref} = \frac{\dot{m}_{r,o} (h_{a,ro} - h_{a,ri})}{A_{tank} (T_{a,ro} - T_{a,ri})} \quad (3.27)$$

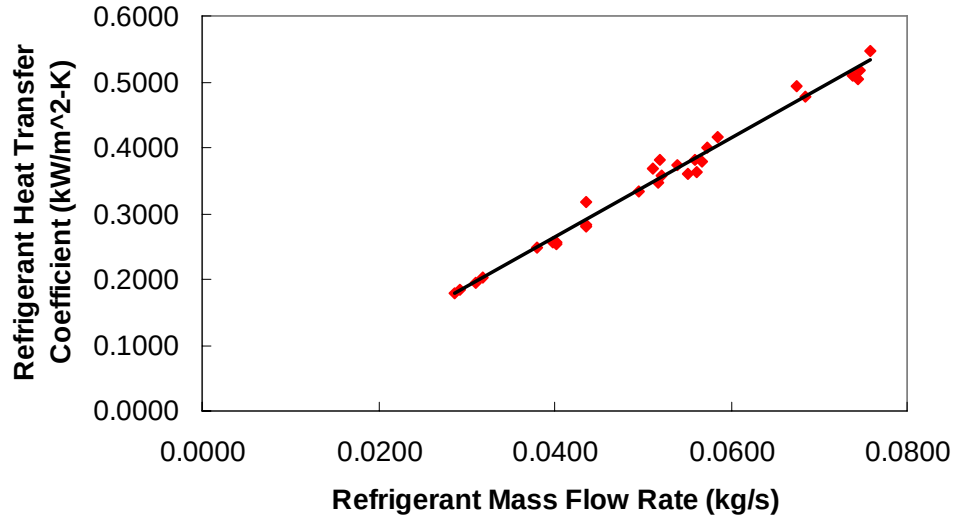


Figure 3.5: Accumulator refrigerant heat transfer coefficient as a function of refrigerant mass flow rate.

A linear regression was performed on the data to produce a linear function relating refrigerant mass flow rate to the refrigerant heat transfer coefficient. The linear regression was evaluated using the square of the Pearson product moment correlation coefficient [21], given in Equation 3.28.

$$r = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 \sum (y - \bar{y})^2}} \quad (3.28)$$

The r^2 value provides a measure of the proportion of the variance in y that is attributable to the variance in x . For the regression shown in Figure 3.5, $r^2 = 0.9849$.

3.1.3 Distributor

Many evaporators, including that in the TS-500 refrigeration unit, utilize a parallel coil circuitry to maximize their capacity while limiting the overall refrigerant-side pressure drop. With this type of evaporator design, a distributor is needed to evenly distribute the refrigerant entering the evaporator among the various circuits. It is difficult to accurately model the pressure drop across this component using first principles for two reasons. First, the distributor used in the TS-500 has a very complex geometry. Second, the refrigerant flowing through the distributor is two-phase (i.e. a mix of vapor and liquid

refrigerant), and even the best two-phase pressure drop correlations predict pressure drop within a 40% error margin [33]. For these reasons, system identification was considered as an alternative to first-principles modeling of this component.

The distributor component is situated immediately downstream of the EEV and immediately before the evaporator model. The evaporator pressure is a state of the evaporator model and therefore, the value is always available to the distributor model. Therefore, the distributor model is constructed such that its inputs are the mass flow rate through the distributor and its *outlet* pressure, and the output is its predicted *inlet* pressure. As described in Chapter 2, the TS-500 test unit was excited with a random Gaussian sequence (RGS) signal in order to generate a set of data which captured all of the dynamics of the refrigeration system. The test unit was instrumented with a pressure transducer at the evaporator outlet and at the outlet of the EEV, as well as a mass flow meter at the inlet of the EEV. The pressure at the outlet of the EEV was assumed to be equal to the pressure at the inlet of the distributor. Similarly, the pressure at the outlet of the evaporator was assumed to be equal to the pressure at the inlet of the evaporator (from experimental data it has been observed that there is little pressure drop across the evaporator). The test data used for the system identification was collected at the 0|100°F box|ambient operating condition with a sampling rate of 1 Hz.

A standard prediction error/maximum likelihood system identification (ID) algorithm in the System Identification Toolbox [1] was used to identify a second-order dynamic model predicting the pressure drop across the distributor. The data used for the identification was preprocessed by subtracting the zeroth order trend, i.e. the sample mean, from each signal. An example is shown in Equation 3.29, where $\bar{u}_1$ is the sample mean of the first input signal.

$$u_{1,detrend} = u_1 - \bar{u}_1 \quad (3.29)$$

The identification results are shown in Figure 3.6. It is important to note that the identified model maps $u_{1,detrend}$ and $u_{2,detrend}$ to $y_{1,detrend}$ and $y_{2,detrend}$. Therefore, when running the model in simulation, it is necessary to take into account the values that were subtracted from the various signals during the identification. These values will be referred to as detrend constants.

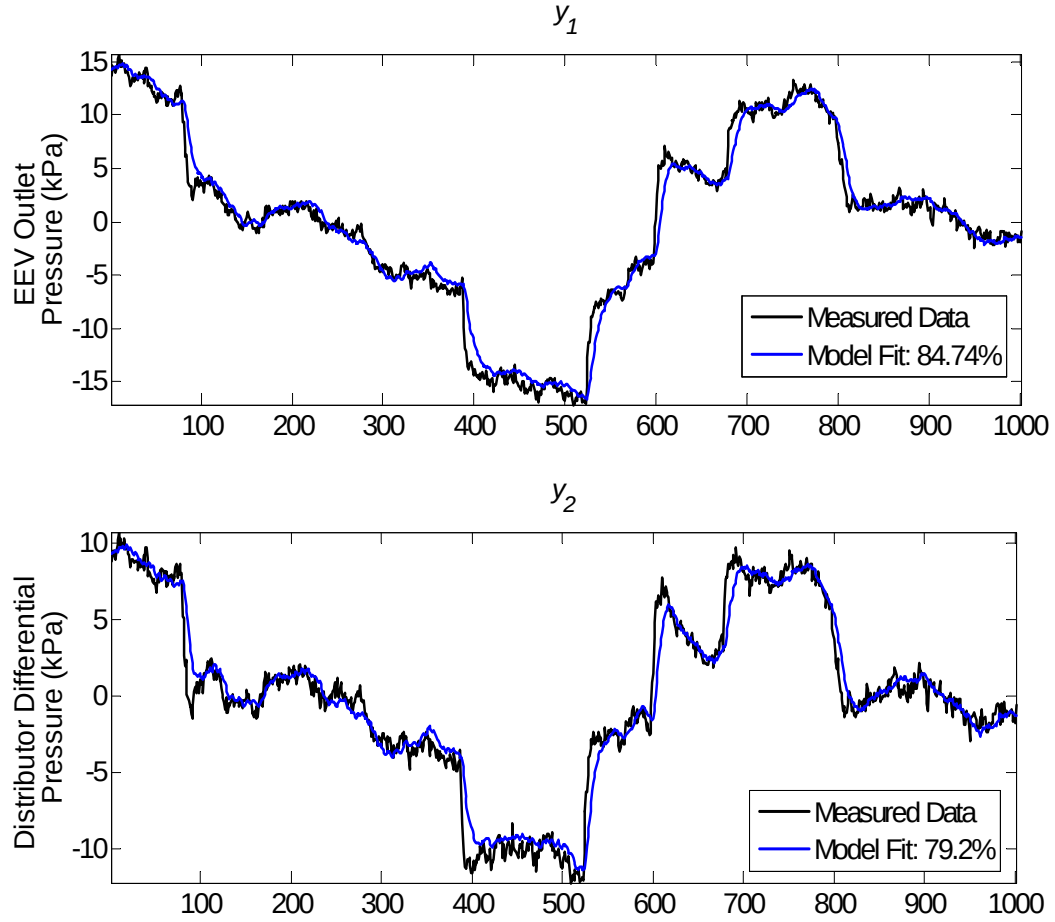


Figure 3.6: Detrended system identification results for the distributor model.

The measure of predictive capability used by the System Identification Toolbox is the percentage of the output variation given in Equation 3.30, where y is the measured output, $\bar{y}$ is the average of the measured output, and $\hat{y}$ is the predicted output [1]. For the identified distributor model presented here, the fit percentage was 84.74%.

$$fit = 100 \left(1 - \frac{\|\hat{y} - y\|}{\|y - \bar{y}\|} \right) \quad (3.30)$$

The complete state space representation of the identified model is given in Equation 3.31. The Matlab code used to run the identification algorithm can be found in Appendix B.1.

$$\begin{aligned}
A &= \begin{bmatrix} -0.105 & -0.0556 \\ 0.0183 & -0.248 \end{bmatrix} \\
B &= \begin{bmatrix} 91.5 & 0.0175 \\ -76.9 & 0.0386 \end{bmatrix} \\
C &= \begin{bmatrix} 10.5 & -4.19 \\ 8.56 & -8.35 \end{bmatrix} \quad D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \\
\mathbf{u} &= [\dot{m} \quad P_e]^T \quad \mathbf{y} = [P_{v,ro} \quad dP]
\end{aligned} \tag{3.31}$$

Although the distributor model was identified using data collected at a specific operating condition, the dynamics are indeed accurate across a range of operating conditions. The behavior of a given fluid, or refrigerant in this case, changes significantly with changes in phase, but assuming a saturated refrigerant is flowing through the distributor at most operating conditions in the Cool Mode operating envelope, it is not surprising that the dynamics are consistent across operating conditions. Recall that the data was detrended before it was used for the identification, so while the dynamics are accurate, the detrend constants for each input and output signal need to be adjusted for different operating conditions. Steady state data collected across the Cool Mode operating envelope is used to estimate the detrend constants for operating conditions aside from the one at which the model was identified. This approach is validated in Chapter 5.

3.2 Static Model Components

The remaining components in the Cool Mode system configuration are modeled using static equations under the assumption that the dynamics of these components are fast as compared to those of the heat exchangers, accumulator, and distributor, and therefore less dominant in the overall system. The components are presented in an order that is consistent with refrigerant flow through the cycle.

3.2.1 Scroll Compressor

The TS-500 refrigeration system contains an open-drive scroll compressor. Thermo King has developed semi-empirical models for each of its compressors. These semi-empirical models consist of three compressor performance curves which are used to

evaluate the main volumetric efficiency, auxiliary port volumetric efficiency, and the isentropic (adiabatic) efficiency of the compressor. Given these efficiencies, the mass flow rate at the outlet of the compressor can be easily determined. The particular compressor used on the TS-500 does not contain an auxiliary port, but the associated parameters are included in the model for completeness [7].

The auxiliary port variable, φ_{ax} , quantifies the theoretical volume available for auxiliary gas as a ratio to the total volumetric displacement. The expression for φ_{ax} is given in Equation 3.32. The main pressure ratio φ_{mn} is given in Equation 3.33.

$$\varphi_{ax} = \frac{V_{ax}}{V} - \left(\frac{P_{k,ri}}{P_{ax}} \right)^{\frac{1}{k_{ax}}} \quad (3.32)$$

$$\varphi_{mn} = \left(\frac{P_{k,ro}}{P_{k,ri}} \right)^{\frac{1}{k_{mn}}} = \frac{v_s}{v_d} \quad (3.33)$$

The dependent variables of the model are η_{mn} , the main suction volumetric efficiency, η_{ax} , the auxiliary suction volumetric efficiency, and η_{is} , the isentropic (adiabatic) efficiency. The performance curve for each of these efficiencies is given by Equations 3.34, 3.35, and 3.36, respectively.

$$\begin{aligned} \eta_{mn} = & A_0 + A_1\omega_k + A_2\omega_k^2 + A_3\varphi_{mn} + A_4\varphi_{mn}^2 + A_5\omega_k\varphi_{mn} + A_6\varphi_{ax} + A_7\varphi_{mn}\varphi_{ax} \\ & + A_8\omega_k\varphi_{ax} + A_9\omega_k\varphi_{ax}^2 + A_{10}\omega_k^2\varphi_{ax} + A_{11}P_{k,ro} \end{aligned} \quad (3.34)$$

$$\begin{aligned} \eta_{ax} = & B_0 + B_1\omega_k + B_2\omega_k^2 + B_3\varphi_{mn} + B_4\varphi_{mn}^2 + B_5\omega_k\varphi_{mn} + B_6\varphi_{ax} + B_7\varphi_{mn}\varphi_{ax} \\ & + B_8\omega_k\varphi_{ax} + B_9\omega_k\varphi_{ax}^2 + B_{10}\omega_k^2\varphi_{ax} + B_{11}P_{k,ro} \end{aligned} \quad (3.35)$$

$$\begin{aligned} \eta_{is} = & C_0 + C_1\omega_k + C_2\omega_k^2 + C_3\varphi_{mn} + C_4\varphi_{mn}^2 + C_5\omega_k\varphi_{mn} + C_6\varphi_{ax} + C_7\varphi_{mn}\varphi_{ax} \\ & + C_8\omega_k\varphi_{ax} + C_9\omega_k\varphi_{ax}^2 + C_{10}\omega_k^2\varphi_{ax} + C_{11}P_{k,ro} \end{aligned} \quad (3.36)$$

Each performance curve is characterized by a set of coefficients, A_0 through A_{11} , B_0 through B_{11} , and C_0 through C_{11} , respectively. These coefficients are unique to a given class of compressors. Using test data the independent and dependent variables of the performance curves were plotted, and a multilinear least squares regression was used to

determine the coefficient values. After determining the compressor efficiencies, the total outlet mass flow rate can be obtained by evaluating Equation 3.37.

$$\dot{m}_{total} = \dot{m}_{mn} + \dot{m}_{ax} \quad (3.37)$$

The auxiliary mass flow rate, $\dot{m}_{ax}$, is equal to zero for this compressor because there is no auxiliary port. The expression for $\dot{m}_{mn}$ is given in Equation 3.38

$$\dot{m}_{mn} = \frac{\eta_{mn} \delta_k \omega_k}{\rho} \quad (3.38)$$

where δ_k is the compressor displacement. Finally, the isentropic efficiency is used to calculate the total power used by the compressor, p_k .

$$p_k = \frac{\dot{m}_{total} (h_{k,ro} - h_{k,ri})}{\eta_{is} / 100} \quad (3.39)$$

It is important to note that the performance curves were generated using data in English units. While most of the coefficients are non-dimensional, the coefficients A_{11} and C_{11} are only accurate when the function argument (i.e. $P_{k_{ro}}$) is given in English units.

3.2.2 Suction Line Heat Exchanger

The suction line heat exchanger (SLHX) is a refrigerant liquid to refrigerant vapor heat exchanger. The thermal capacitance of the SLHX is assumed to be small in comparison to that of the evaporator and condenser, and therefore, the associated dynamics can be treated as static. That is to say, an instantaneous transfer of energy is assumed between the two fluid passes albeit there is some energy storage inside the coils of the heat exchanger itself.

The model for the SLHX is derived from a model previously developed at Thermo King [27]. The expression for pressure loss through a straight pipe is given in Section 4.2.2. While the friction factor, f , is normally a function of both the Reynolds number and surface roughness, it was modeled only as a function of Reynolds number (Equation 3.40) because the nature of the experimental test used to derive the correlation made it impossible to include surface roughness. To compensate for this, an

experimental variable was introduced into Equation 4.22 such that the pressure loss in each of the fluid passes can be expressed using Equation 3.41 and Equation 3.42.

$$f = \gamma (\text{Re})^{-\frac{1}{5}} \quad (3.40)$$

$$\Delta P_{liq} = 4A (\text{Re}_{liq})^{-\frac{1}{5}} \frac{L_{liq}}{D_{liq}} \left(\frac{\rho_{liq} V_{liq}^2}{2} \right) \quad (3.41)$$

$$\Delta P_{vap} = 4B (\text{Re}_{vap})^{-\frac{1}{5}} \frac{L_{vap}}{D_{vap}} \left(\frac{\rho_{vap} V_{vap}^2}{2} \right) \quad (3.42)$$

A least squares regression was used to estimate the experimental variables A and B by minimizing the squared error between the measured and predicted pressure drop across each fluid pass (Equation 3.43).

$$\gamma = \min \left[\left(\Delta P_{measured} - \Delta P_{predicted} \right)^2 \right] \quad (3.43)$$

The overall heat transfer coefficient, UA , for the heat exchanger can also be derived using a semi-empirical approach using the Nusselt number [11] given in Equation 3.44.

$$Nu = \frac{hD}{k} \quad (3.44)$$

For turbulent flow through smooth circular tubes, the Nusselt number can be determined using the Colburn equation, given in 3.45.

$$Nu = 0.023 \text{Re}_D^{\frac{4}{5}} \text{Pr}^{\frac{1}{3}} \quad (3.45)$$

For this derivation, a constant, C , will be used in place of the constant derived for the Colburn equation. The new expression for the Nusselt number is given in Equation 3.46.

$$Nu = C \text{Re}_D^{\frac{4}{5}} \text{Pr}^{\frac{1}{3}} \quad (3.46)$$

Setting Equation 3.44 and Equation 3.46 equal and solving for h gives the expression in Equation 3.47.

$$h = \frac{C \text{Re}_D^{4/5} \text{Pr}^{1/3} k}{D} \quad (3.47)$$

An expression for the overall heat transfer coefficient, UA , can be obtained by considering a summation of thermal resistances.

$$\frac{1}{R_{total}} = \frac{1}{R_{liquid}} + \frac{1}{R_{vapor}} \quad (3.48)$$

$$\frac{1}{UA} = \frac{1}{h_{liq} A_{liq}} + \frac{1}{h_{vap} A_{vap}} \quad (3.49)$$

Solving for UA results in Equation 3.50.

$$UA = \frac{h_{vap} A_{vap} h_{liq} A_{liq}}{h_{vap} A_{vap} + h_{liq} A_{liq}} \quad (3.50)$$

By substituting Equation 3.47 into Equation 3.50, a semi-empirical expression for UA can be obtained, where C is an experimental variable that can be estimated by minimizing the squared error between the estimated and measured values for UA .

$$UA = C \frac{\text{Re}_{D,vap}^{4/5} \text{Pr}_{vap}^{1/3} k_{vap} A_{vap} \text{Re}_{D,liq}^{4/5} \text{Pr}_{liq}^{1/3} k_{liq} A_{liq}}{\text{Re}_{D,vap}^{4/5} \text{Pr}_{vap}^{1/3} k_{vap} A_{vap} D_{liq} + \text{Re}_{D,liq}^{4/5} \text{Pr}_{liq}^{1/3} k_{liq} A_{liq} D_{vap}} \quad (3.51)$$

3.2.3 Electronic Expansion Valve

The electronic expansion valve (EEV) is an electronic control valve that regulates the pressure at the inlet of the evaporator by changing the valve opening area. In the system model, the purpose of the EEV model is to provide a mass flow rate value as an input to both the condenser and evaporator models. A basic expression for the mass flow rate of a valve is given in Equation 3.52,

$$\dot{m}_v = A_v C_v \left[\rho_v (P_{v,i} - P_{v,o}) \right]^{1/n} \quad (3.52)$$

assuming a sharp orifice type valve, $n = 2$ in Equation 3.52. For valves with a variable valve opening area [6], the valve area is lumped together with the discharge coefficient into a single parameter called the valve flow coefficient, C_f .

$$\dot{m}_v = C_f \sqrt{\rho_v (P_{v,i} - P_{v,o})} \quad (3.53)$$

The valve flow coefficient is a difficult parameter to derive using first principles, and therefore it is most often derived using experimental data. An expression for C_f can be obtained by solving for the flow coefficient in Equation 3.53.

Using experimental measurements for the mass flow rate, inlet and outlet refrigerant pressure, and the inlet refrigerant density, the flow coefficient was calculated across a range of operating conditions for the valve. Specifically, the EEV stroke input was decreased and then increased incrementally, and the resulting mass flow rate was measured and recorded. This was done at multiple operating conditions at both the high and low compressor speeds.

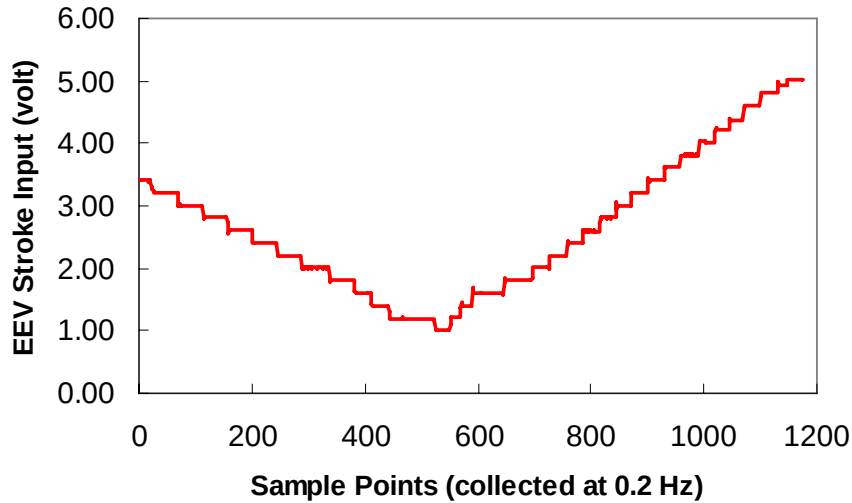


Figure 3.7: EEV input signal for data set (used for flow coefficient map generation) collected at -20/50 °F box|ambient for compressor operating at low speed.

Previous modeling efforts have considered both two input and three input models for an EEV. In [6], valve stroke, and valve pressure differential were used in a two input model. For the three input model, the inputs were modified such that the inlet pressure and outlet pressure were considered individually. Figure 3.8 and Figure 3.9 show the experimentally calculated valve flow coefficient plotted against the flow coefficient predicted by the map for the case of two inputs and three inputs, respectively.

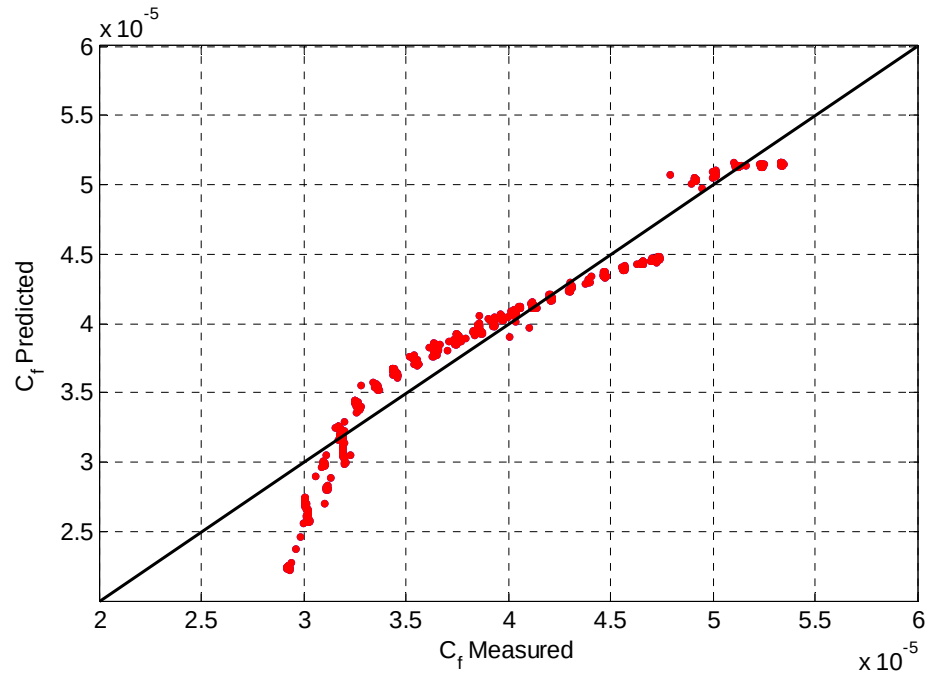


Figure 3.8: Comparison between predicted and measured valve flow coefficients using valve stroke and pressure differential as predictors.

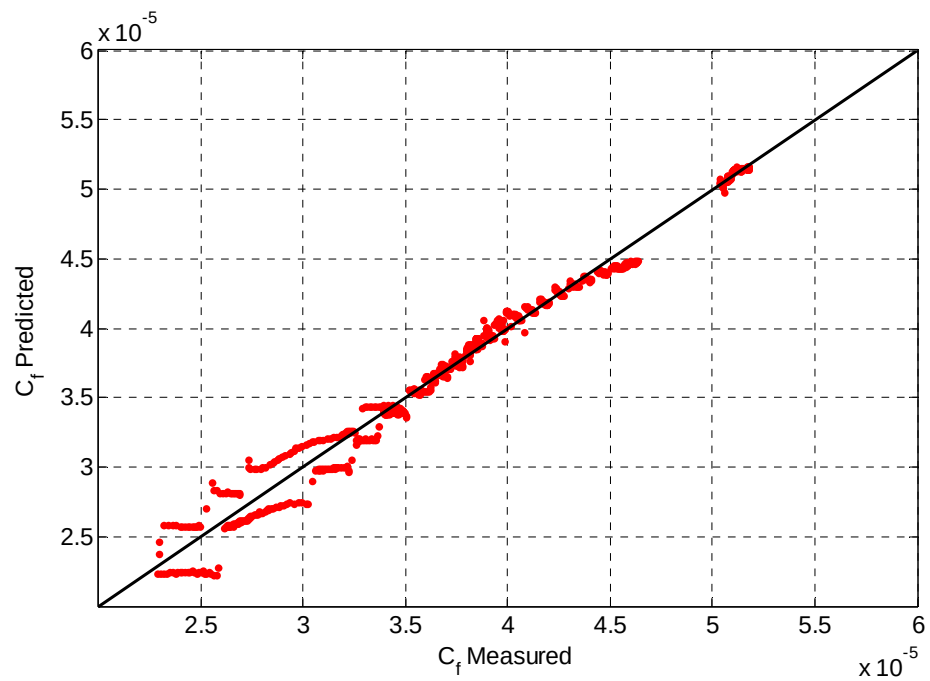


Figure 3.9: Comparison between predicted and measured valve flow coefficients using valve stroke, inlet pressure, and outlet pressure as predictors.

When two inputs are used to generate the flow coefficient map for the EEV in the TS-500, the map is a poor predictor of flow coefficient. The map is more accurate when three inputs are used but at the cost of added complexity. One major difference between the EEV being modeled here and that modeled in [6] was the amount of refrigerant subcooling at the inlet of the EEV. If the refrigerant is a two-phase fluid, both the temperature and pressure of the fluid do not characterize it any more than does the pressure alone. However, if the refrigerant is a subcooled liquid, it is important to characterize the fluid by both its temperature and pressure. With this intuition, a natural input choice for the map is the valve inlet subcooling, defined in Equation 3.54.

$$T_{sc} = T_{c,sat} - T_{v,i} \quad (3.54)$$

The amount of subcooling of the refrigerant at the inlet of the EEV is unique in that it characterizes both the temperature and pressure of refrigerant. Since T_{sc} does not carry any information about the outlet pressure of the valve, P_{out} can be considered as the third input for the valve map. By performing a multilinear regression, cross terms and squared terms of the input parameters can be considered as possible predictors of the output, resulting in a polynomial of the form

$$C_f = a_0 + a_1(X_v) + a_2(T_{sc}) + a_3(P_{out}) + a_4(X_v T_{sc}) + a_5(X_v P_{out}) + a_6(T_{sc} P_{out}) + a_7(X_v)^2 + a_8(T_{sc})^2 + a_9(P_{out})^2 \quad (3.55)$$

With the Matlab command *stepwisefit*, a stepwise method is used to estimate each coefficient, a_i , for each of the predictors. This specific command produces a final model which includes only those predictors which improve the regression. All of the terms containing P_{out} were omitted with the exception of one of the cross terms containing it. This provided a strong suggestion that P_{out} was not needed as an input parameter. Therefore, the model was reduced to two inputs: valve stroke and refrigerant subcooling. Equation 3.56 shows the simplified expression for the valve flow coefficient, C_f .

$$C_f = a_0 + a_1(X_v) + a_2(T_{sc}) + a_3(X_v T_{sc}) + a_4(X_v)^2 + a_5(T_{sc})^2 \quad (3.56)$$

The Matlab command *regress* was used to perform a multilinear regression considering all five predictors shown in Equation 3.56. The complete expression is given in Equation 3.57.

$$C_f = 1e^{-4} \left[-0.210 + 0.0119(X_v) + 0.0211(T_{sc}) - 1.02e^{-4}(X_v T_{sc}) - 5.59e^{-5}(X_v)^2 - 2.75e^{-4}(T_{sc})^2 \right] \quad (3.57)$$

Figure 3.10 shows a surface plot of Equation 3.57 evaluated at values of stroke and refrigerant subcooling within the operating range of the TS-500. The data used for the multilinear regression is plotted against the surface for comparison.

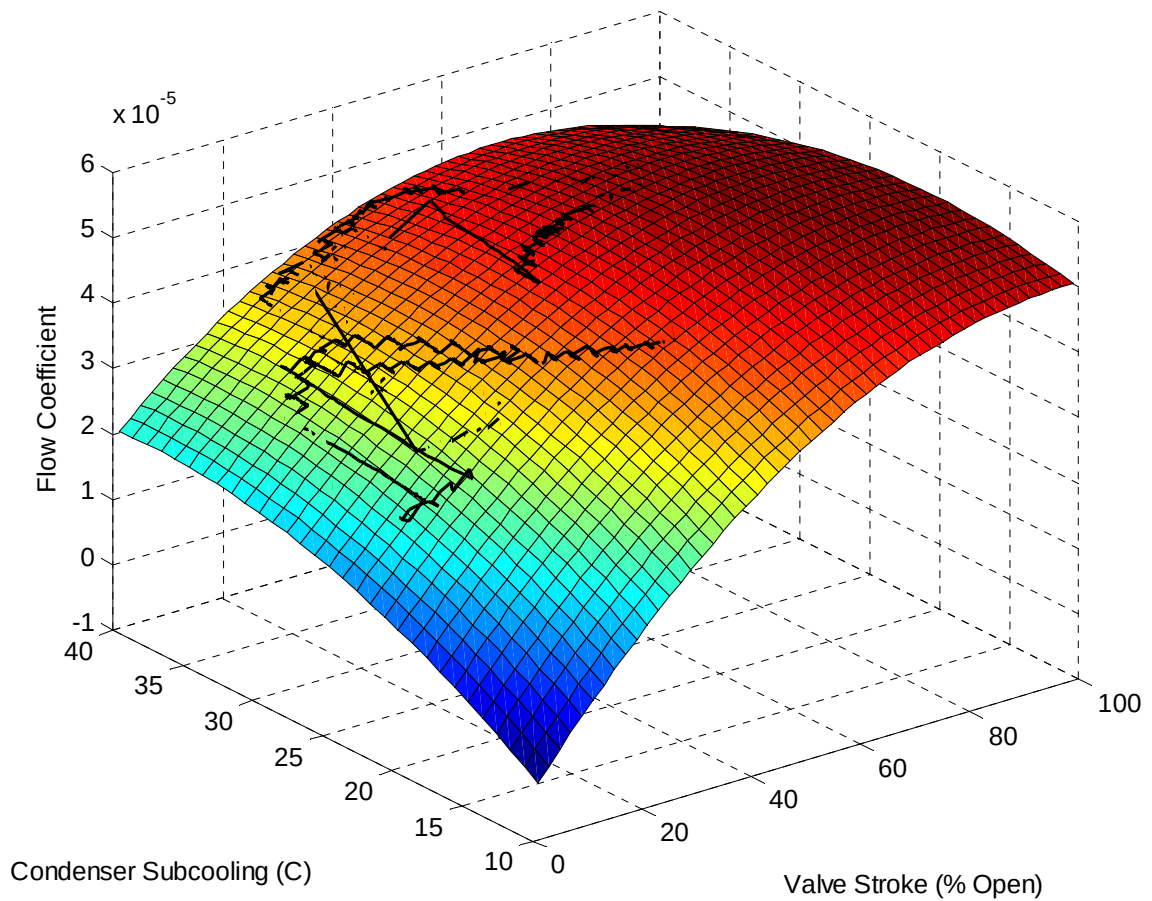


Figure 3.10: Flow coefficient map for an EEV. Black points represent experimental data.

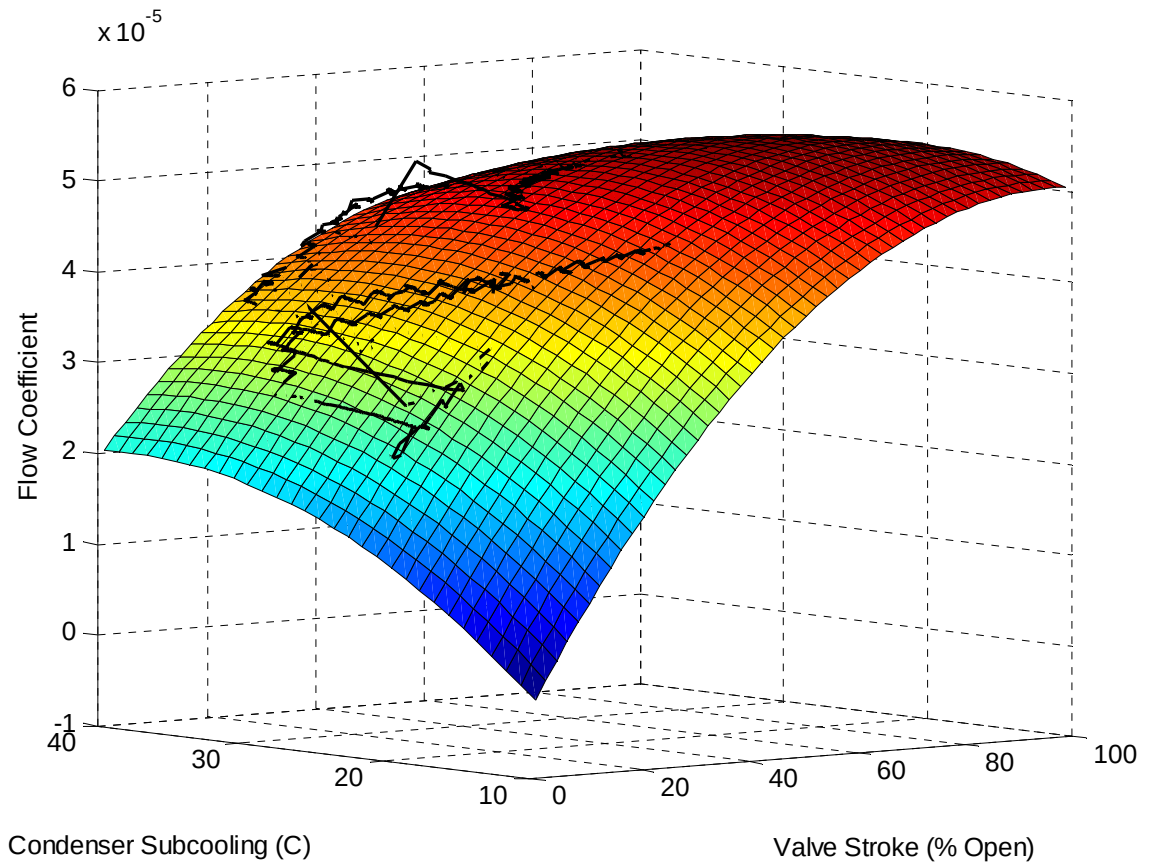


Figure 3.11: Figure 3.10 rotated to better show experimental data relative to surface mapping.

Figure 3.12 provides another qualitative measure of the predictive capability of the map by plotting the experimentally calculated valve flow coefficient against that predicted by the map for given values of refrigerant subcooling and valve stroke.

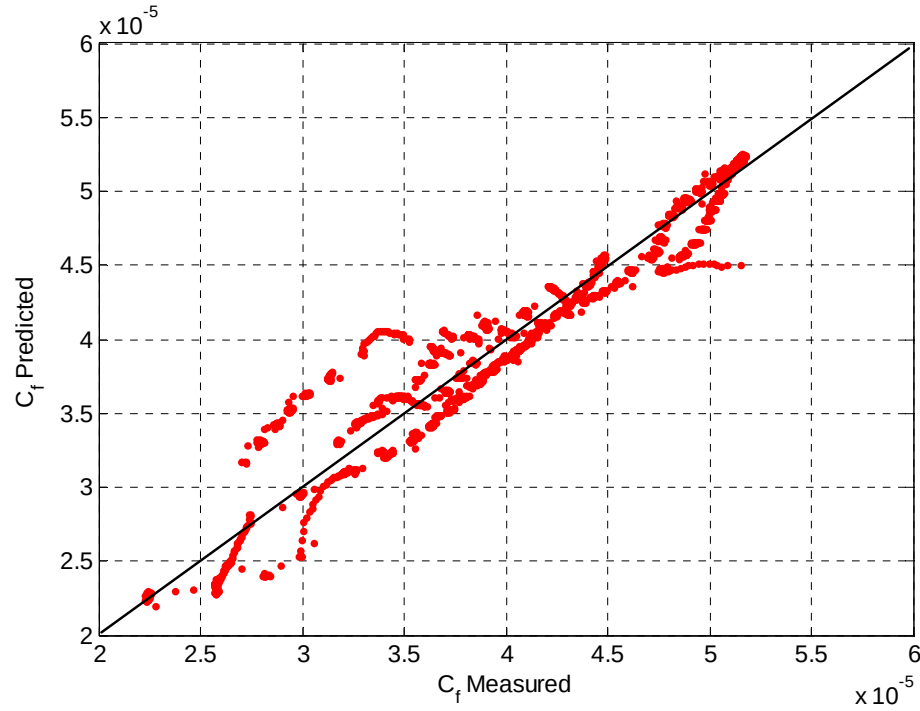


Figure 3.12: Comparison between predicted and measured valve flow coefficients using valve stroke and valve inlet subcooling as predictors.

Comparing Figure 3.9 against Figure 3.12 shows that the choice of three predictors (valve stroke, inlet pressure, and outlet pressure) results in a map that is a slightly better predictor of valve flow coefficient than that using two predictors (valve stroke and valve inlet subcooling). The trade-off, however, is the dimensionality of the map; that is, a three-dimensional map will be more computationally efficient than a four-dimensional map. The three-dimensional map is validated in Chapter 5 where it is shown to be a good predictor of the EEV dynamics.

Future work will include the development of a thermostatic expansion valve (TXV) model. The main challenge in developing a flow coefficient map for the TXV is that the valve stroke cannot be manually controlled. However, a multilinear regression approach can still be used, and valve inlet subcooling should again be considered as a predictor for flow coefficient. Steady state data collected on the TXV system (during Cool Mode operation) can be utilized to generate this map.

3.2.4 Throttle Valve

A throttle valve, also known as a suction pressure regulator valve, is a mechanical control valve that is used to regulate the pressure of the refrigerant vapor at the compressor inlet. Based on the calibration of the valve, it either opens or closes to decrease or increase, respectively, the pressure of the refrigerant entering the compressor.

The throttle valve is modeled using a three dimensional (3D) map which was generated and provided by Thermo King. The map takes the inlet throttle valve pressure ($P_{tv,ri}$) and refrigerant mass flow rate through the valve ($\dot{m}_{r,tv}$) as inputs and interpolates to produce the outlet throttle valve pressure ($P_{tv,ro}$).

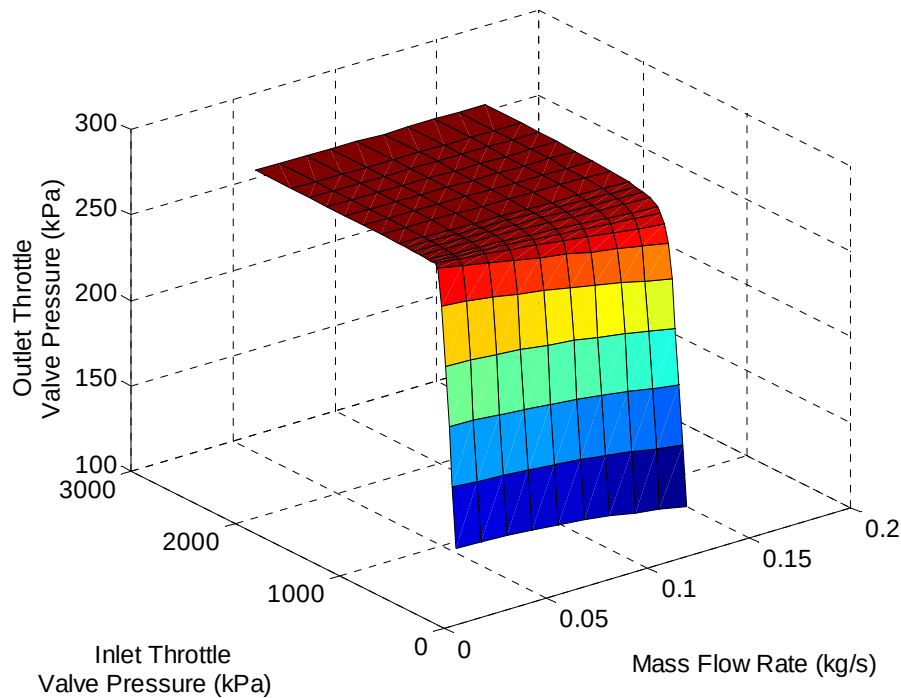


Figure 3.13: Three dimensional map for determining outlet pressure of the throttle valve.

3.2.5 Heat Exchanger Fan

For each of the two heat exchangers, respectively, there is a fan which drives air across the heat exchanger coil. By increasing or decreasing the speed of each fan, the airflow rate across each heat exchanger increases or decreases, directly increasing or

decreasing the rate of heat transfer between the air and the refrigerant in that heat exchanger. However, each fan, as well as the compressor, are driven by a single engine; as a result, the speed of each fan cannot be controlled individually.

Since airflow rate has a direct impact on the capacity of each heat exchanger, it is important to have an accurate fan model. Static pressure and resistance curves were empirically derived for both the evaporator and condenser fans (used in the TS-500 refrigeration unit) by Thermo King. The testing of each fan was completed per AMCA Standard 210.

3.2.5.1 Evaporator Fan

At a fixed fan speed of 3800 rpm, the change in static pressure for different airflow rates through the fan was measured and recorded. The resistance through the refrigerant unit for different airflow rates was also measured and recorded. A polynomial mapping airflow rate (x) with static pressure (SP) was then derived using this data (see Equation 3.58). A second polynomial, mapping airflow rate (x) with resistance (R) was also derived (see Equation 3.59). Note that the airflow rate, x , is the airflow rate generated by a secondary fan which was used in the experimental setup.

$$SP_{evaporator} = (4.74e-12)x^4 - (1.79e-8)x^3 + (2.37e-5)x^2 - (1.41e-2)x + 4.31 \quad (3.58)$$

$$R_{evaporator} = (2.38e-7)x^2 + (3.08e-5)x \quad (3.59)$$

Operating the fan at a different speed will produce a static pressure (i.e. fan) curve that is parallel to the original fan curve which was derived. Using a second relationship, given in Equation 3.60, the original fan curve can be scaled such that the appropriate parallel fan curve is determined.

$$SP_{e,adjusted} = SP_{evaporator} \cdot \left(\frac{rpm_{fan}}{3800} \right)^2 \quad (3.60)$$

The intersection of this new fan curve and the resistance curve determines the actual airflow rate produced by the fan (for a given fan speed).

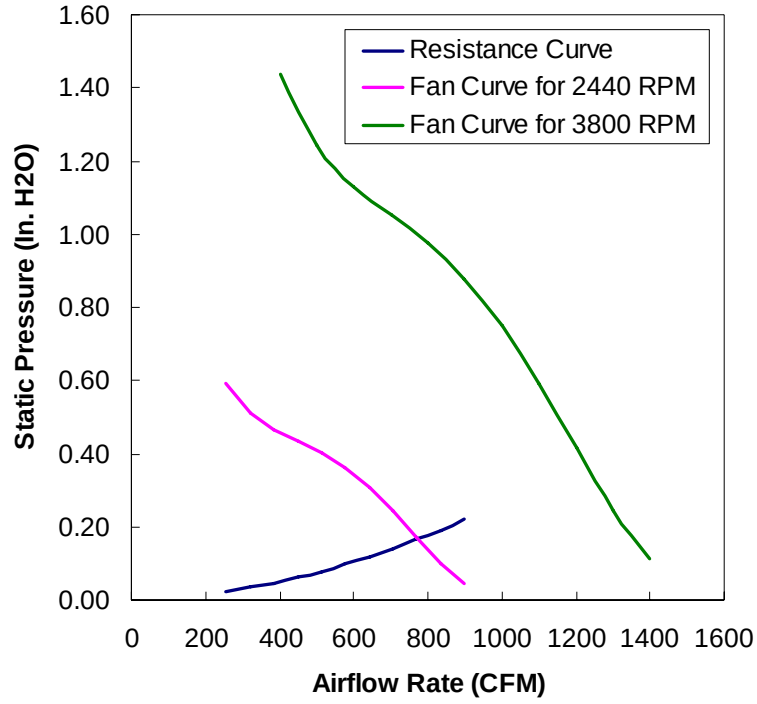


Figure 3.14: Example of intersection between fan and resistance curves for evaporator fan.

3.2.5.2 Condenser Fan

For the condenser fan, a very similar procedure was used to derive static pressure and resistance curves. At a fixed fan speed of 2700 rpm, the change in static pressure for different airflow rates through the fan was measured and recorded. The resistance through the refrigerant unit for different airflow rates was also measured and recorded. A polynomial mapping airflow rate (x) with static pressure (SP) was then derived using this data (see Equation 3.61). A second polynomial, mapping airflow rate (x) with resistance (R) was also derived (see Equation 3.62). Again, note that the airflow rate, x , is the airflow rate generated by a secondary fan which was used in the experimental setup.

$$SP_{condenser} = (-1.32e-14)x^4 + (1.45e-10)x^3 - (5.45e-7)x^2 + (3.21e-5)x + 3.28 \quad (3.61)$$

$$R_{condenser} = (2.28e-7)x^2 + (1.42e-4)x \quad (3.62)$$

Using the relationship given in Equation 3.63, the original fan curve can be scaled such that the appropriate parallel fan curve is determined.

$$SP_{c,adjusted} = SP_{condenser} \cdot \left(\frac{rpm_{fan}}{2700} \right)^2 \quad (3.63)$$

The intersection of this new fan curve and the resistance curve determines the actual airflow rate produced by the fan (for a given fan speed).

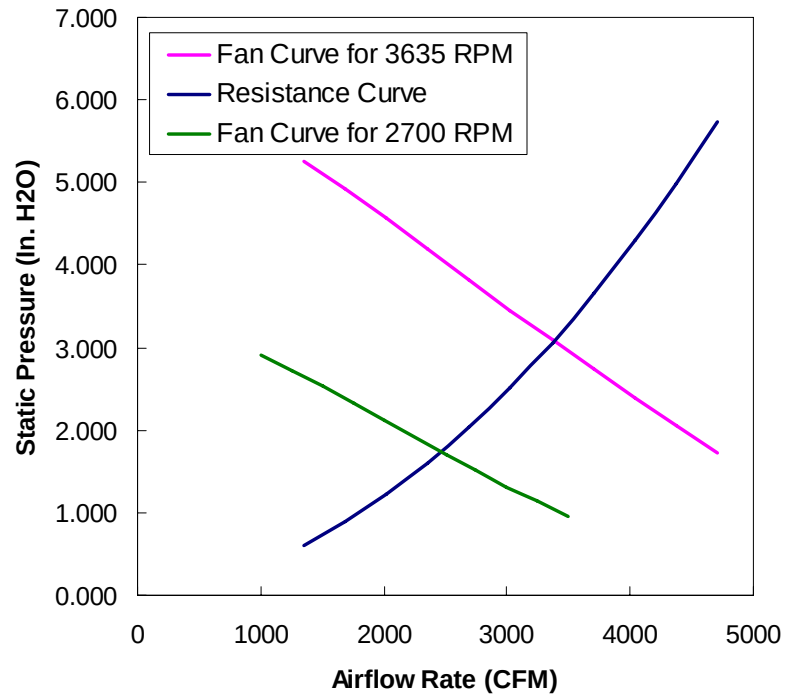


Figure 3.15: Example of intersection between fan and resistance curves for condenser fan.

Chapter 4

HEAT MODE MODELING

This chapter describes the development of the Heat Mode Thermosys model.

4.1 Dynamic Model Components

As described in Chapter 1, those components whose dynamics are slow, and therefore dominant, in the system, are modeled using dynamic equations. During Heat Mode operation, this includes the distributor, the evaporator, and the accumulator components. The components are presented in an order that is consistent with refrigerant flow through the Heat Mode cycle.

4.1.1 Distributor

While the distributor component itself does not change between Cool Mode and Heat Mode, the fluid flowing through it is no longer a saturated liquid. In Heat Mode operation, the refrigerant flowing through the distributor is a superheated vapor and consequently exhibits different pressure drop characteristics than a liquid or two-phase fluid. For this reason, a second model was identified for the distributor using data collected while operating the system in Heat Mode.

For the Heat Mode distributor model, one output, rather than two, was used for the identification. Moreover, given the system configuration of Heat Mode (i.e. the EEV is isolated) and the change in instrumentation between the Cool Mode and Heat Mode testing, a different set of data measurements were used for the identification.

Table 4.1 Comparison of Data Measurements used for Distributor Identification

ID Model Parameter	Cool Mode Measurement	Heat Mode Measurement
u_1 , Refrigerant mass flow rate	EEV mass flow rate	Compressor mass flow rate
u_2 , Distributor outlet pressure	Evaporator outlet pressure	Distributor outlet pressure
y_1 , Distributor inlet pressure	EEV outlet pressure	Hot Gas Line outlet pressure
y_2 , Distributor pressure differential	$y_1 - u_2$	N/A

The identified model was evaluated using Equation 3.30 and resulted in a fit percentage of 72.6%. Figure 4.1 shows the results of the identification.

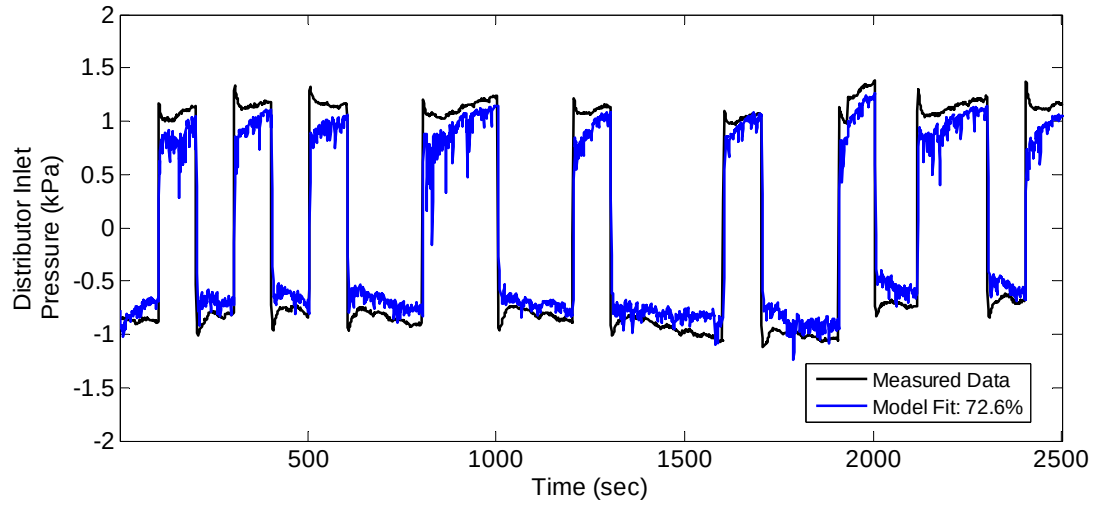


Figure 4.1: Detrended system identification results for distributor model.

The complete state space representation of the identified model is given in Equation 4.1. The Matlab code used to run the identification algorithm can be found in Appendix B.2.

$$\begin{aligned}
 A &= \begin{bmatrix} -0.555 & 0.987 \\ 0.141 & -0.299 \end{bmatrix} \\
 B &= \begin{bmatrix} 5.64 & 0.00219 \\ -1.48 & -0.000633 \end{bmatrix} \\
 C &= [272 \quad -19.3] \quad D = [0 \quad 0] \\
 \mathbf{u} &= [\dot{m} \quad P_{d,ro}]^T \quad \mathbf{y} = [P_{hg,ro}]
 \end{aligned} \tag{4.1}$$

The main challenge in developing a first-principles distributor model stemmed from the fact that the refrigerant flow is two-phase during Cool Mode operation. During Heat Mode, however, the flow is single-phase which alleviates some of the challenge in accurately predicting pressure drop. Therefore, the development of a first-principles distributor model for Heat Mode operation should be considered in the future.

4.1.2 Evaporator

As described in Chapter 2, the evaporator model acts like a condenser during Heat Mode operation. Specifically, superheated vapor enters the evaporator coil and exits as a two-phase fluid. Previously developed condenser models include a nonlinear condenser with receiver model as well as a nonlinear three-zone (i.e. superheat, two-phase, and subcooled fluid regions) condenser model [24]. Therefore, a two-zone (i.e. superheat and two-phase fluid regions) condenser model was derived in the lumped-parameter moving boundary framework following the method presented in [24].

The governing ordinary differential equations (ODEs) are obtained by integrating the governing partial differential equations (PDEs) for conservation of mass (Equation 4.2), and conservation of refrigerant energy and tube wall energy, respectively (Equation 4.3 and Equation 4.4) along the length of the heat exchanger. The assumption of lumped parameters associated with each fluid region is applied.

$$\frac{\partial(\rho A_{cs})}{\partial t} + \frac{\partial(\dot{m})}{\partial z} = 0 \quad (4.2)$$

$$\frac{\partial(\rho A_{cs} h - A_{cs} P)}{\partial t} + \frac{\partial(\dot{m} h)}{\partial z} = p_i \alpha_i (T_w - T_r) \quad (4.3)$$

$$(C_p \rho A)_w \frac{\partial(T_w)}{\partial t} = p_i \alpha_i (T_r - T_w) + p_o \alpha_o (T_a - T_w) \quad (4.4)$$

Equation 4.5 and Equation 4.6 give the ODEs for conservation of mass of the superheated and two-phase fluid regions, respectively.

$$\left(\frac{\partial \rho_1}{\partial P_c} + \frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \frac{dh_g}{dP_c} \right) A_{cs} L_1 \dot{P} + \left(\frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \right) A_{cs} L_1 \dot{h}_i + (\rho_1 - \rho_g) A_{cs} \dot{L}_1 = \dot{m}_i - \dot{m}_{int1} \quad (4.5)$$

$$(\rho_g - \rho_f)(1 - \bar{\gamma}) A_{cs} \dot{L}_1 + \left(\frac{d\rho_f}{dP_c}(1 - \bar{\gamma}) + \frac{d\rho_g}{dP_c} \bar{\gamma} \right) A_{cs} L_2 \dot{P}_c + (\rho_g - \rho_f) A_{cs} L_2 \dot{\bar{\gamma}} = \dot{m}_{int1} - \dot{m}_o \quad (4.6)$$

Equation 4.7 and Equation 4.8 give the ODEs for conservation of refrigerant energy of the superheated and two-phase fluid regions, respectively.

$$\left[\left(\frac{\partial \rho_1}{\partial P_c} + \frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \frac{dh_g}{dP_c} \right) h_1 + \left(\frac{1}{2} \frac{dh_g}{dP_c} \right) \rho_1 - 1 \right] A_{cs} L_1 \dot{P}_c + \left[\left(\frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \right) h_1 + \frac{1}{2} \rho_1 \right] A_{cs} L_1 \dot{h}_i + (\rho_1 h_1 - \rho_g h_g) A_{cs} \dot{L}_1 = \dot{m}_i h_i - \dot{m}_{int1} h_g + \alpha_{i,1} A_i \left(\frac{L_1}{L_{total}} \right) (T_{w,1} - T_{r,1}) \quad (4.7)$$

$$(\rho_g h_g - \rho_f h_f)(1 - \bar{\gamma}) A_{cs} \dot{L}_1 + \left(\frac{d(\rho_f h_f)}{dP_c}(1 - \bar{\gamma}) + \frac{d(\rho_g h_g)}{dP_c} \bar{\gamma} - 1 \right) A_{cs} L_2 \dot{P}_c + (\rho_g h_g - \rho_f h_f) A_{cs} L_2 \dot{\bar{\gamma}} = \dot{m}_{int1} h_g - \dot{m}_o h_o + \alpha_{i,2} A_i \left(\frac{L_2}{L_{total}} \right) (T_{w,2} - T_{r,2}) \quad (4.8)$$

Equation 4.9 and Equation 4.10 give the ODEs for conservation of tube wall energy of the superheated and two-phase fluid regions, respectively.

$$(C_p \rho V)_w \left(\dot{T}_{w,1} + \frac{T_{w,1} - T_{w,2}}{L_1} \dot{L}_1 \right) = \alpha_{i,1} A_i (T_{r,1} - T_{w,1}) + \alpha_o A_o (T_{a,1} - T_{w,1}) \quad (4.9)$$

$$(C_p \rho V)_w \dot{T}_{w,2} = \alpha_{i,2} A_i (T_{r,2} - T_{w,2}) + \alpha_o A_o (T_{a,2} - T_{w,2}) \quad (4.10)$$

These six differential equations are combined to eliminate $\dot{m}_{int1}$. Through simulation, the parameter $\dot{h}_i$ was determined to be sufficiently small such that it can be assumed to equal zero. This significantly simplifies Equation 4.5 and Equation 4.7. Combining the equations in this way results in a model with five states: $L_1, P_c, \bar{\gamma}, T_{w,1}$, and $T_{w,2}$. The resulting model is given in Equation 4.11 and is of the form $Z(x, u) \cdot \dot{x} = f(x, u)$, with the elements of the Z matrix given in Table 4.2, and the elements of the f vector given in Table 4.3.

$$\begin{bmatrix} z_{11} & z_{12} & 0 & 0 & 0 \\ z_{21} & z_{22} & z_{23} & 0 & 0 \\ z_{31} & z_{32} & z_{33} & 0 & 0 \\ z_{41} & 0 & 0 & z_{44} & 0 \\ 0 & 0 & 0 & 0 & z_{55} \end{bmatrix} \begin{bmatrix} \dot{L}_1 \\ \dot{P}_c \\ \dot{\gamma} \\ \dot{T}_{w,1} \\ \dot{T}_{w,2} \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \end{bmatrix} \quad (4.11)$$

Table 4.2 Matrix Elements of $Z(x, u)$ for the Heat Mode Evaporator Model

z_{11}	$(\rho_1 h_1 - \rho_1 h_g) A_{cs}$
z_{12}	$\left[(h_1 - h_g) \left(\frac{\partial \rho_1}{\partial P_c} + \frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \frac{dh_g}{dP_c} \right) h_1 + \left(\frac{1}{2} \frac{dh_g}{dP_c} \right) \rho_1 - 1 \right] A_{cs} L_1$
z_{21}	$(\rho_f h_g - \rho_f h_f) (1 - \bar{\gamma}) A_{cs}$
z_{22}	$\left(\frac{d(\rho_f h_f)}{dP_c} (1 - \bar{\gamma}) + \frac{d(\rho_g h_g)}{dP_c} \bar{\gamma} - \frac{d\rho_f}{dP_c} h_g (1 - \bar{\gamma}) - \frac{d\rho_g}{dP_c} h_g \bar{\gamma} - 1 \right) A_{cs} L_2$
z_{23}	$(\rho_f h_g - \rho_f h_f) A_{cs} L_2$
z_{31}	$\left[(\rho_g - \rho_f) (1 - \bar{\gamma}) + (\rho_1 - \rho_g) \right] A_{cs}$
z_{32}	$\left[\left(\frac{d\rho_f}{dP_c} (1 - \bar{\gamma}) + \frac{d\rho_g}{dP_c} \bar{\gamma} \right) L_2 + \left(\frac{\partial \rho_1}{\partial P_c} + \frac{1}{2} \frac{\partial \rho_1}{\partial h_1} \frac{dh_g}{dP_c} \right) L_1 \right] A_{cs}$
z_{33}	$(\rho_g - \rho_f) A_{cs} L_2$
z_{41}	$(C_p \rho V)_w \frac{T_{w,1} - T_{w,2}}{L_1}$
z_{44}	$(C_p \rho V)_w$
z_{55}	$(C_p \rho V)_w$

Table 4.3 Vector Elements of $f(x, u)$ for the Heat Mode Evaporator Model

f_1	$\dot{m}_i (h_i - h_g) + \alpha_{i,1} A_i \left(\frac{L_1}{L_{total}} \right) (T_{w,1} - T_{r,1})$
-------	---

Table 4.3 (continued)

f_2	$\dot{m}_o (h_g - h_o) + \alpha_{i,2} A_i \left(\frac{L_2}{L_{total}} \right) (T_{w,2} - T_{r,2})$
f_3	$\dot{m}_i - \dot{m}_o$
f_4	$\alpha_{i,1} A_i (T_{r,1} - T_{w,1}) + \alpha_o A_o (T_{a,1} - T_{w,1})$
f_5	$\alpha_{i,2} A_i (T_{r,2} - T_{w,2}) + \alpha_o A_o (T_{a,2} - T_{w,2})$

The modifications for multi-circuit geometry, counter-cross air flow configuration, and the air-side heat transfer coefficient discussed in Section 3.1.1 are applied to the Heat Mode evaporator model as well. Finally, the mean void fraction is used to determine the quality of the refrigerant exiting the evaporator using a method developed by McKinley [18] which relates mean void fraction, refrigerant pressure, and outlet vapor quality using look up tables. The Matlab code used to generate these tables is provided in Appendix C.1.

4.1.3 Accumulator

An accumulator is a device used to separate vapor from liquid in two-phase flow. Specifically for VCC systems, an accumulator is used downstream of the evaporator to ensure that only refrigerant vapor enters the compressor. During Heat Mode operation, the evaporator behaves like a condenser; two-phase refrigerant, rather than superheated vapor, exits the evaporator. Therefore, the accumulator is needed to separate the vapor from the liquid before the refrigerant reaches the compressor inlet.

As is the case during Cool Mode operation, the engine delivering power to the compressor sits adjacent to the accumulator. Given the configuration of the condenser and the engine with respect to the accumulator, the air exiting the condenser first blows across the engine (where it absorbs heat) and then blows across the accumulator, where it loses heat to the refrigerant inside the tank. In this way, the accumulator acts as a separation device with heat transfer characteristics.

The governing ODEs of the accumulator are obtained by applying conservation of mass and energy to the accumulator. For this model, the pressure inside the tank is

assumed to be uniform. Moreover, the outlet refrigerant enthalpy of the accumulator is not assumed to be saturated vapor but instead some quality close to, but not equal to, one. This is because the separation efficiency of the accumulator is known to be less than one. For a given separation efficiency, η_{sep} , the outlet enthalpy can be described as

$$h_{r,o} = \eta_{sep} h_g + (1 - \eta_{sep}) h_f . \quad (4.12)$$

The mass balance across the accumulator is given in Equation 4.13.

$$\dot{m}_{r,i} - \dot{m}_{r,o} = \frac{dm}{dt} = V \frac{d\rho}{dt} \quad (4.13)$$

By treating the density as a function of pressure and enthalpy, Equation 4.13 can be rearranged as shown in Equation 4.14.

$$\frac{\dot{m}_{r,i} - \dot{m}_{r,o}}{\rho V} = \frac{1}{\rho} \left[\frac{\partial \rho}{\partial P} \frac{dP}{dt} + \frac{\partial \rho}{\partial h} \frac{dh}{dt} \right] \quad (4.14)$$

Conservation of energy for the accumulator is given by Equation 4.15.

$$\dot{Q}_{in} + (\dot{m}h)_{r,i} - (\dot{m}h)_{r,o} = \frac{d(mu)}{dt} = u \frac{dm}{dt} + m \frac{du}{dt} \quad (4.15)$$

Rewriting the right-hand side of Equation 4.15 and applying the chain rule gives Equation 4.16.

$$\dot{Q}_{in} + (\dot{m}h)_{r,i} - (\dot{m}h)_{r,o} = u \frac{dm}{dt} + m \frac{d}{dt} \left(h - \frac{P}{\rho} \right) = u \frac{dm}{dt} + m \frac{dh}{dt} - V \frac{dP}{dt} + \frac{P}{\rho} V \frac{d\rho}{dt} \quad (4.16)$$

Combining Equation 4.16 with Equation 4.13 and rearranging the terms gives Equation 4.17.

$$\frac{\dot{Q}_{in} + \dot{m}(h_{r,i} - h) - \dot{m}(h_{r,o} - h)}{\rho V} + \frac{1}{\rho} \frac{dP}{dt} = \frac{dh}{dt} \quad (4.17)$$

The heat transfer rate into the accumulator is determined using Equation 4.18,

$$\dot{Q}_{in} = \alpha_{ref} A_{tank} (T_{tank} - T_{ref}) \quad (4.18)$$

where T_{tank} is a dynamic variable described by the expression used in the Cool Mode accumulator model (Equation 3.22). However, during heat Mode operation, the accumulator is filled with two-phase refrigerant, and so the refrigerant temperature, T_{ref} , is taken to be the saturated refrigerant temperature at the pressure inside the accumulator. This is different than the Cool Mode accumulator model in which T_{ref} is treated as a dynamic variable.

In order to determine the refrigerant heat transfer coefficient, α_{ref} , steady state data collected at a range of Heat Mode operating conditions was used to evaluate Equation 3.27. The result was then plotted as a function of mass flow rate as shown in Figure 4.2.

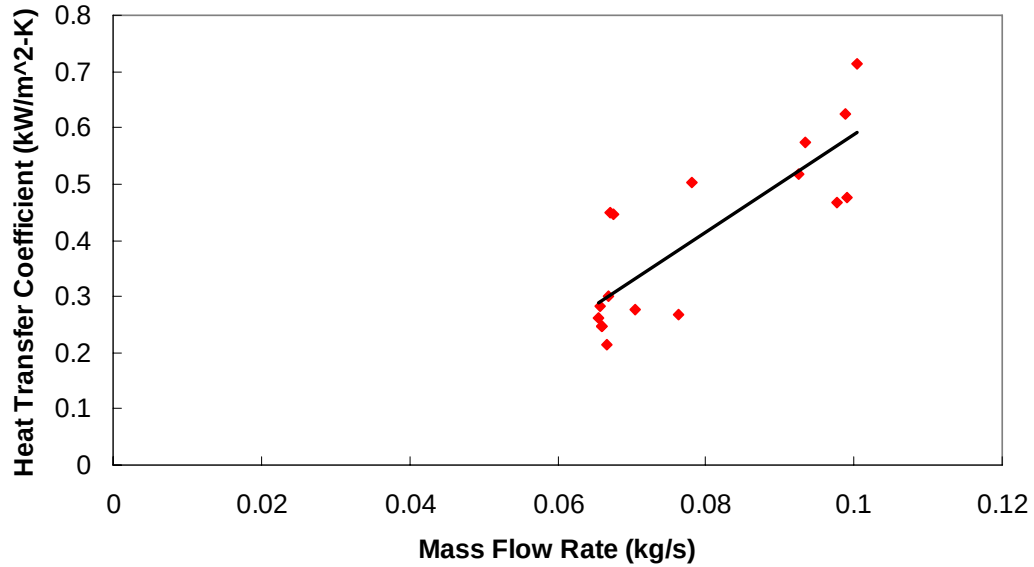


Figure 4.2: Accumulator refrigerant heat transfer coefficient as a function of refrigerant mass flow rate during Heat Mode operation.

For the regression shown in Figure 4.2, $r^2 = 0.6797$.

4.2 Static Model Components

The remaining components in the Heat Mode system configuration are modeled using static equations under the assumption that the dynamics of these components are faster as compared to those of the distributor, evaporator, and accumulator, and therefore less dominant in the overall system.

4.2.1 DPR Valve

In Heat Mode operation, the heat of compression generated by the compressor is used to heat the supply air to the truck box. However, the compressor does not have variable speed capability, and therefore the pressure ratio across the compressor cannot be varied using the compressor speed. Instead, the compressor speed is fixed to meet the desired heating capacity, and a discharge pressure regulator (DPR) valve is used to control the pressure ratio across the compressor. By adjusting the valve opening, a backpressure is created which causes the compressor to work harder and as a result, generate more heat.

Like the EEV, a map or correlation for the flow coefficient is needed in order to accurately model the valve in Thermosys. Recall that when generating a flow coefficient map for the EEV, a special set of data was collected where the EEV stroke was increased and decreased across its entire range. The DPR valve is a mechanical valve and therefore no such input signal could be applied. The data used to generate a flow coefficient map was limited to steady state data collected across the Heat Mode operating envelope. Using this data, the flow coefficient was calculated using Equation 4.19 and plotted as a function of mass flow rate (see Figure 4.3).

$$\dot{m}_v = C_f \sqrt{\rho_v (P_{v,i} - P_{v,o})} \quad (4.19)$$

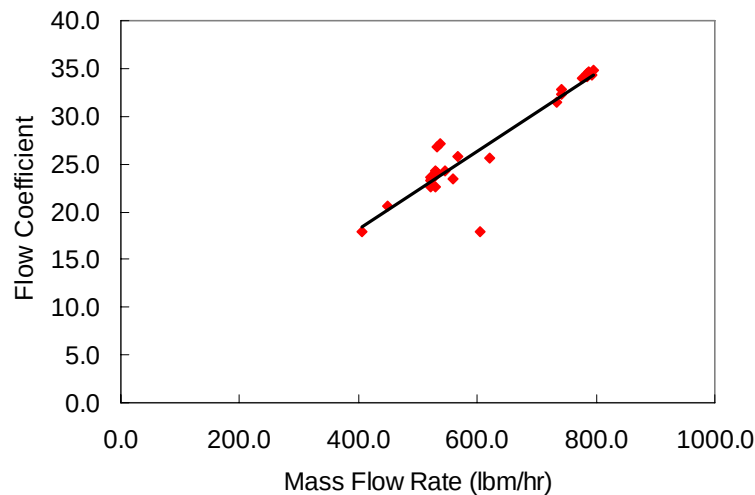


Figure 4.3: DPR valve flow coefficient as a function of refrigerant mass flow rate.

Note that the flow coefficient is in English units.

A linear regression was performed on the data to produce a linear function relating refrigerant mass flow rate to flow coefficient. For the curve shown in Figure 4.3, $r^2 = 0.859$.

4.2.2 Hot Gas Line

The hot gas line is a series of pipes that are engaged only in the Heat Mode configuration to connect the outlet of the DPR valve to the inlet of the distributor. Due to the state of the refrigerant flowing through the hot gas line (superheated vapor) and the geometry of the various pipes, there is a significant amount of heat loss and pressure drop across the length of the hot gas line.

4.2.2.1 Heat Transfer

A refrigerant-side heat transfer coefficient, UA_{ref} , correlation was derived empirically from steady state test data taken at a range of Heat Mode operating conditions. A simple energy balance was used to calculate the heat transfer coefficient. The refrigerant-side heat transfer coefficient is in units of kilowatt per degree Kelvin and is therefore independent of area.

$$\dot{Q}_{hg,r} = \dot{m}_{ref} (h_{hg,ri} - h_{hg,ro}) = (UA_{ref}) (T_{hg,ri} - T_{hg,ro}) \quad (4.20)$$

Solving for UA_{ref} in Equation 4.20 yields Equation 4.21.

$$(UA_{ref}) = \frac{\dot{m}_{ref} (h_{hg,ri} - h_{hg,ro})}{(T_{hg,ri} - T_{hg,ro})} \quad (4.21)$$

Using steady state test data, Equation 4.21 was evaluated, and the resulting UA_{ref} value was correlated against refrigerant mass flow rate as shown in Figure 4.4.

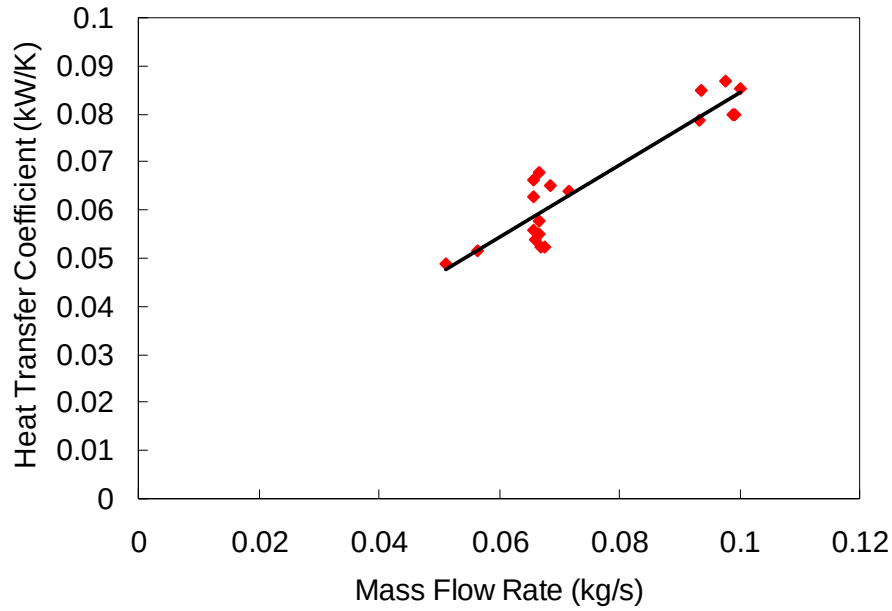


Figure 4.4: Empirically derived refrigerant-side heat transfer coefficient as a function of refrigerant mass flow rate.

A linear regression was performed on the data to derive an expression relating the heat transfer coefficient to refrigerant mass flow rate. The linear regression was evaluated using the square of the Pearson product moment correlation coefficient (Equation 3.28) resulting in a r^2 value of 0.868.

4.2.2.2 Pressure Drop

The refrigerant flowing through the Hot Gas Line is superheated vapor. The pressure drop correlations for incompressible flow can only be applied here if the Mach number of the flow is sufficiently small, i.e. less than 0.3. Equation 3.15 was evaluated using steady state test data collected at a range of Heat Mode operating conditions. The results are shown in Table 4.4.

Table 4.4 Mach Number of Refrigerant Flow through Hot Gas Line

Operating Condition	Compressor Speed	Mach Number
70 35	High	0.0800
50 0	Electric	0.1136
35 0	High	0.1452
35 0	Low	0.1192
35 0	Electric	0.1173
35 -20	High	0.1485
35 -20	Low	0.1270
35 -20	Electric	0.1606
50 -20	High	0.1344
50 -20	Low	0.1162
50 -20	Electric	0.1579
70 -20	High	0.1261
50 0	High	0.1346
50 0	Low	0.1062
35 35	Low	0.1011
35 35	Electric	0.1242
50 35	High	0.1243
50 35	Low	0.0772
50 35	Electric	0.1040
70 35	Electric	0.0805

The average Mach number across Heat Mode operating regime is 0.1199, and therefore, incompressibility will be assumed for the refrigerant flow through the hot gas line. For incompressible flow, the pressure drop due to friction for straight flow through a pipe is given by Equation 4.22,

$$\Delta P = \frac{1}{2} \rho v^2 f \frac{L}{D} \quad (4.22)$$

where f is the dimensionless friction factor [34]. Moreover, there are additional pressure losses that result because of specific geometric pipe characteristics such as bends and elbows. Such losses are characterized by a dimensionless loss coefficient, K , defined in Equation 4.23.

$$K = 2 \frac{\Delta P}{\rho v^2} \quad (4.23)$$

These losses are not well understood and therefore, the loss coefficients are determined primarily through experimentation and then correlated with pipe geometry. A

loss coefficient, K_f , can be defined for Equation 4.22, such that all of the individual losses in the hot gas line pipe system can be summed as expressed in Equation 4.25.

$$K_f = f \frac{L}{D} \quad (4.24)$$

$$\Delta P = \frac{1}{2} \rho v^2 (K_f + \sum K) \quad (4.25)$$

Equation 4.25 assumes a constant flow velocity through all pipes in the system. The loss coefficients for bends in the hot gas line pipe system were determined using [19]. Currently, many pipe loss coefficients are available as a series of plotted curves rather than mathematical correlations or simple look-up tables. For this reason, the specific values needed for the hot gas line model are hard coded into the Thermosys model (see Figure 4.5). For future development of the model, a look-up table or library of pipe loss coefficients should be developed and added to Thermosys. This will enable the use of interpolation schemes for determining any loss coefficient within the scope of the tables included.

```

239
240 |*****
241 * 9. HotGasLineTK 1
242 *****
243
244 - Physical_Parameters.HotGasLineTK1.a1          = 0.753;
245 - Physical_Parameters.HotGasLineTK1.a0          = 0.0091;
246 - Physical_Parameters.HotGasLineTK1.Kb_1        = 0.33;
247 - Physical_Parameters.HotGasLineTK1.Kb_2        = 0.484;
248 - Physical_Parameters.HotGasLineTK1.Kb_3        = 0.429;
249 - Physical_Parameters.HotGasLineTK1.Kb_4        = 1.155;
250 - Physical_Parameters.HotGasLineTK1.Kb_5        = 0.88;
251 - Physical_Parameters.HotGasLineTK1.Kb_6        = 0.627;
252 - Physical_Parameters.HotGasLineTK1.Kb_7        = 0.528;
253 - Physical_Parameters.HotGasLineTK1.Diameter    = 0.0111;
254 - Physical_Parameters.HotGasLineTK1.L_total     = 7.4874;
255

```

Figure 4.5: Sample code showing pipe loss coefficients hard coded into Thermosys.

4.2.3 Hydraulic Resistance Element

The hydraulic resistance element is a pipe block element adapted from [17]. This component model is used to characterize the pressure drop in pipe connections between components specifically when the inlet and outlet pressure of the pipe are known. For known inlet and outlet pressure conditions, the model uses an iterative procedure to

determine the mass flow rate through the pipe. This situation arises between the evaporator and accumulator models in the Heat Mode system model.

The pressure drop at any instant in time is related to the mass flow by Equation 4.25 described in Section 4.2.2.2. This expression includes pressure drop due to friction for straight flow through a pipe as well as pressure drop due to head losses associated with pipe bends. An empirical hydraulic resistance term, ΔP_R , is added as a third source of pressure drop so that the total pressure drop across the hydraulic resistance element is expressed by Equation 4.26,

$$\Delta P_{total} = \left[\frac{1}{2} \rho v^2 (K_f + \sum K) \right] + \Delta P_R \quad (4.26)$$

where

$$\Delta P_R = R \cdot Q^a \quad (4.27)$$

and Q is the volumetric flow rate of refrigerant through the hydraulic resistance element. The values for R and the exponent, a , are chosen heuristically by the user. Based on the pressure drop characteristics of the hydraulic resistance element and the inlet and outlet pressure at a given time instant, the corresponding mass flow rate through the hydraulic resistance element can be determined.

Chapter 5

COOL MODE MODEL VALIDATION

The Cool Mode model was validated using data collected on an instrumented test unit per the data collection procedure described in Chapter 2. Section 5.1 describes the validation of each individual component. The models were then connected to form the complete refrigeration cycle system; the validation of this complete system is described in Section 5.2.

5.1 Individual Component Validation

Each component in the Cool Mode model was validated individually using both steady state and transient data. For the steady state case, each model was tested with data collected at a range of operating conditions. As described in Chapter 2, steady state data was collected with two different test units; the first set was collected with the test unit containing a thermostatic expansion valve (TR15640), and the second set was collected on the same unit but with the TXV replaced by an EEV (TR16172). Although only an EEV model was created for the Cool Mode system model, each individual model can still be validated using steady state data collected on the TXV system. The EEV model, of course, is validated using steady state data collected on the EEV system.

Each model was tested with two different sets of transient data. The first transient data set was collected while perturbing the experimental unit with step changes in EEV stroke at 0|100 °F box|ambient. The second transient data set was collected while perturbing the system with step changes in compressor speed at -20|50 °F box|ambient.

5.1.1 Compressor

The compressor model predicts its outlet mass flow rate and outlet refrigerant enthalpy. Table 5.1 and Table 5.2 describe the inputs and outputs, respectively, of this model.

Table 5.1 Compressor Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{k,ri}$	Pk_ri	Pressure at compressor inlet	kPa
$P_{k,ro}$	Pk_ro	Pressure at compressor outlet	kPa
$H_{k,ri}$	Hk_ri	Enthalpy at compressor inlet	kJ/kg
rpm	rpm	Compressor rotational speed	rev/min

Table 5.2 Compressor Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_k$	mdot_k	Refrigerant mass flow rate exiting compressor	kg/s
$H_{k,ro}$	Hk_ro	Enthalpy at compressor outlet	kJ/kg

Figure 5.1 shows the Thermosys representation of the compressor model in Matlab Simulink. The yellow tags represent input parameters that were supplied to the model from the validation data sets. For steady state validation, a constant value was supplied to each tag. For transient validation, a dynamic signal was sent to the model via each tag. The dynamic signal was collected with the data acquisition equipment at Thermo King Corporation and was not filtered or scaled. The only change that was made to each dynamic signal was a change of units from the English unit system to the SI unit system.

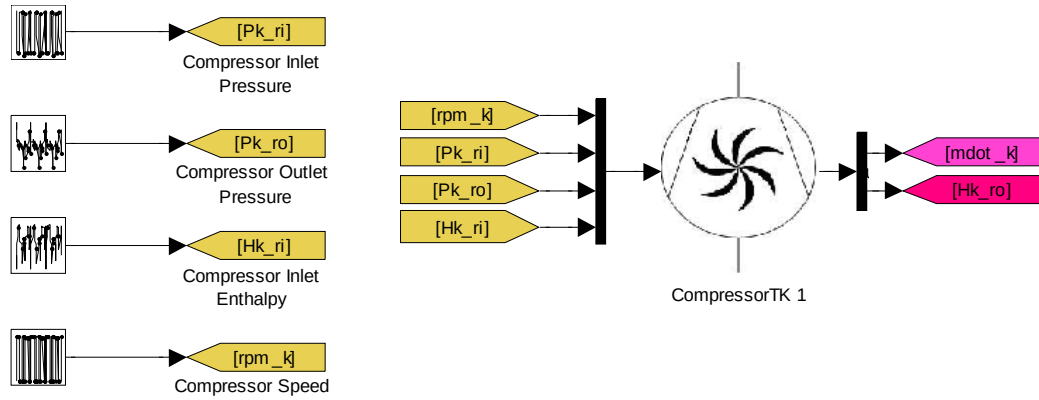


Figure 5.1: Thermosys Simulink model for compressor validation.

On the instrumented unit, a mass flow meter was located at the inlet of the EEV but not at the outlet of the compressor. When the system is operating at steady state, this is sufficient; that is, the mass flow rate at the inlet of the EEV should be equal to the mass flow rate at the compressor outlet. However, during system transients, there will necessarily be a difference in mass flow rate at each of these two locations. Therefore, a transient validation of the compressor was performed and the results were compared to the measured mass flow rate at the EEV inlet with the understanding that the two signals should not be identical.

Table 5.3 shows the steady state validation results for the compressor model.

Table 5.3 Steady State Validation Results for Compressor Model

Operating Condition Box/Amb (°F)	Test Number	Liquid Injection Valve	Engine Speed	Actual mass flow rate (kg/s)	Simulated mass flow rate (kg/s)	Percent Error (%)
35/100	TR16172-001-B	ON	high speed	0.0699	0.0726	3.91
35/100	TR16172-002-A	OFF	high speed	0.0696	0.0721	3.59
35/100	TR16172-003-A	OFF	low speed	0.0457	0.0478	4.65
0/100	TR16172-007-B	OFF	high speed	0.0566	0.0585	3.31
-20/50	TR16172-009-A	ON	high speed	0.0409	0.0402	1.83

Figure 5.2 and Figure 5.3 compare the measured transient outputs with the predicted transient outputs for the compressor model.

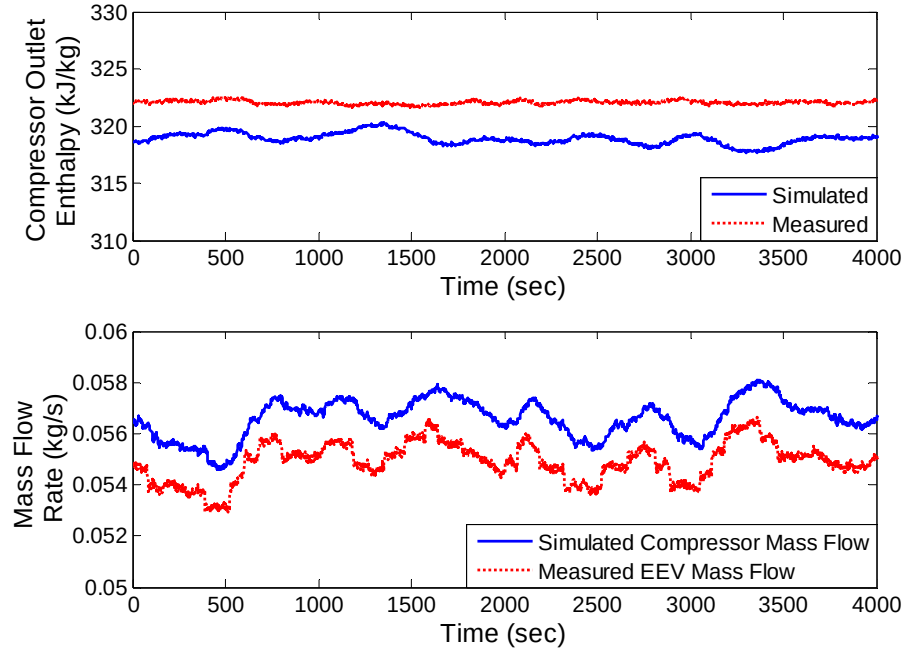


Figure 5.2: Compressor transient validation results at 0|100 °F for perturbations in EEV stroke.

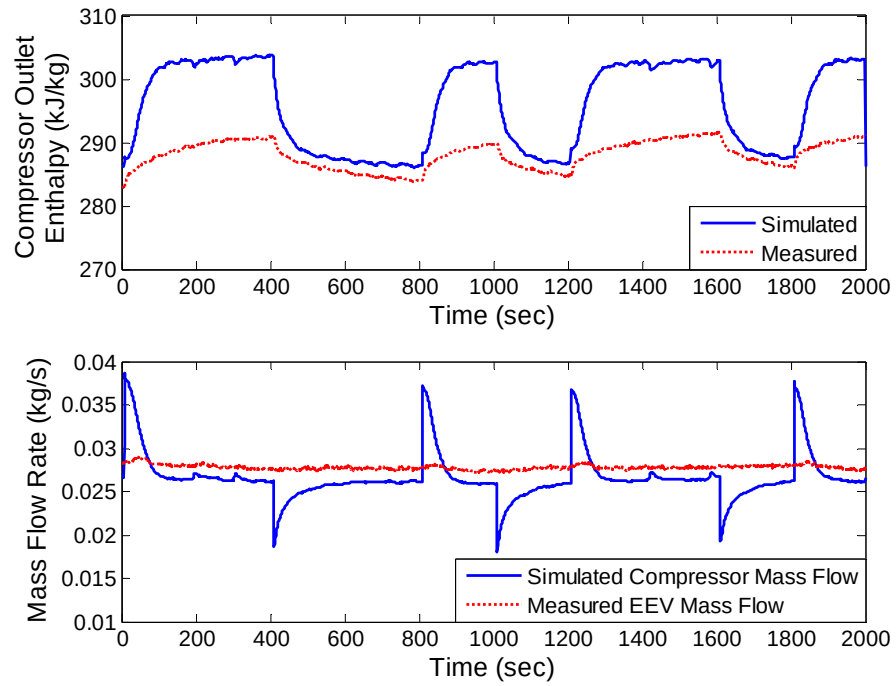


Figure 5.3: Compressor transient validation results at -20|50 °F for perturbations in compressor speed.

The compressor model predicts spikes in mass flow rate when the compressor speed undergoes a step change from high to low speed, or vice versa. It is unclear whether this is accurate because the refrigerant mass flow rate was not actually measured at the outlet of the compressor. However, it does seem reasonable that the measured EEV mass flow rate should look like a filtered version of the simulated compressor mass flow rate signal given that the EEV is downstream of the compressor.

A normalized root mean square (RMS) error, defined in Equation 5.1, is used to characterize the effectiveness of the model in predicting the measured system dynamics.

$$e_{rms} = \frac{1}{\bar{y}_{measured}} \left(\frac{\|y_{measured} - y_{predicted}\|}{\sqrt{n}} \right) \quad (5.1)$$

Table 5.4 shows the normalized RMS error between the simulated and measured compressor outlet enthalpy signal at each of the transient testing conditions. The normalized RMS error was not computed for the compressor mass flow rate signal because only the EEV mass flow rate was measured on the experimental system.

Table 5.4 Normalized RMS Error for Compressor Transient Validation

Operating Condition Box/Amb (°F)	Normalized RMS Error
	<i>Hk_ro</i>
0 100	0.0099
-20 50	0.0322

Qualitatively, the compressor model accurately predicts the transients of the outlet enthalpy signal. The prediction of *Hk_ro* is more accurate at 0|100 °F than at -20|50 °F. The actual compressor loses heat to ambient air through its shell, but the model neglects this heat transfer. Therefore it is not surprising that at -20|50 °F, for step increases in compressor speed, the model predicts a larger increase in outlet enthalpy than was actually measured.

5.1.2 Condenser with Receiver

The condenser with receiver model predicts the pressure inside the condenser (assumed to be uniform across the coil), the outlet refrigerant enthalpy, as well as the

temperature of the air exiting the condenser. Table 5.5 and Table 5.6 list the inputs and outputs, respectively, of the condenser with receiver model.

Table 5.5 Condenser with Receiver Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_{r,i}$	mdot_i	Refrigerant mass flow rate entering condenser	kg/s
$\dot{m}_{r,o}$	mdot_o	Refrigerant mass flow rate exiting condenser	kg/s
$H_{c,ri}$	Hc_ri	Enthalpy at condenser inlet	kJ/kg
$T_{c,ai}$	Tc_ai	Temperature of air entering condenser	°C
$\dot{m}_{a,i}$	mc_air	Air mass flow rate through condenser	kg/s
T_{amb}	T_amb	Temperature of ambient air	°C

Table 5.6 Condenser with Receiver Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
P_c	Pc	Pressure inside condenser	kPa
$H_{c,ro}$	Hc_ro	Enthalpy at condenser outlet	kJ/kg
$T_{c,ao}$	Tc_ao	Temperature of air exiting condenser	°C

Figure 5.4 shows the Thermosys representation of the condenser with receiver model in Matlab Simulink. As shown in Table 5.5, the condenser with receiver model takes both its inlet and outlet mass flow rates as *inputs*. At steady state, these two mass flow rates are assumed to be equal.

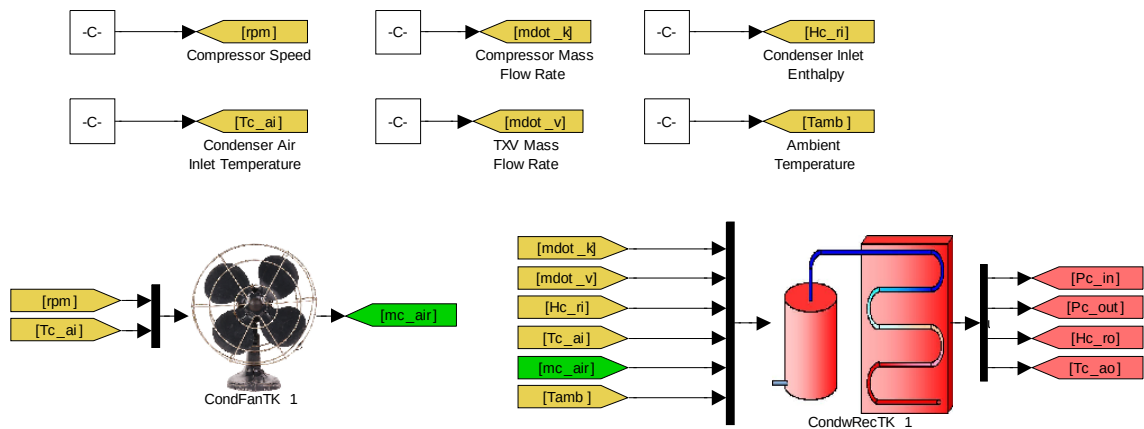


Figure 5.4: Thermosys Simulink model for condenser with receiver validation.

Table 5.7, Table 5.8, and Table 5.9 show the steady state validation results for the condenser with receiver model.

Table 5.7 Steady State Validation Results for Condenser Pressure

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual P_c (kPa)	Simulated P_c (kPa)	Percent Error (%)
35/100	TR16172-001-B	0.0699	2167.7	2456.1	-13.30
0/100	TR16172-007-B	0.0566	2130.5	2310.8	-8.46
-20/50	TR16172-009-A	0.0409	987.3	1126.8	-14.13

Table 5.8 Steady State Validation Results for Condenser Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual H_{c_ro} (kPa)	Simulated H_{c_ro} (kPa)	Percent Error (%)
35/100	TR16172-001-B	0.0699	125.6	135.6	-7.96
0/100	TR16172-007-B	0.0566	124.3	130.6	-5.07
-20/50	TR16172-009-A	0.0409	76.2	83.4	-9.45

Table 5.9 Steady State Validation Results for Condenser Air Outlet Temperature

Operating Condition Box/Amb (°F)	Test Number	Mass flow rate (kg/s)	Actual T_{c_ao} (kPa)	Simulated T_{c_ao} (kPa)	Percent Error (%)
35/100	TR16172-001-B	0.0699	55.4	59.9	-8.12
0/100	TR16172-007-B	0.0566	53.3	56.5	-6.00
-20/50	TR16172-009-A	0.0409	21.9	22.5	-2.74

Figure 5.5 compares the measured transient outputs with the predicted transient outputs for the condenser with receiver model. As mentioned earlier, the condenser model requires both its inlet and outlet mass flow rate signals as inputs. However, only one measured mass flow rate signal is available. Consequently, the compressor model was used to provide a *predicted* transient compressor mass flow rate signal for the simulation. This predicted signal is necessarily different than the actual signal. Therefore, in order to effectively conduct a transient validation of the condenser model,

the compressor mass flow rate signal was scaled by a factor of 0.97. This value was chosen through trial and error.

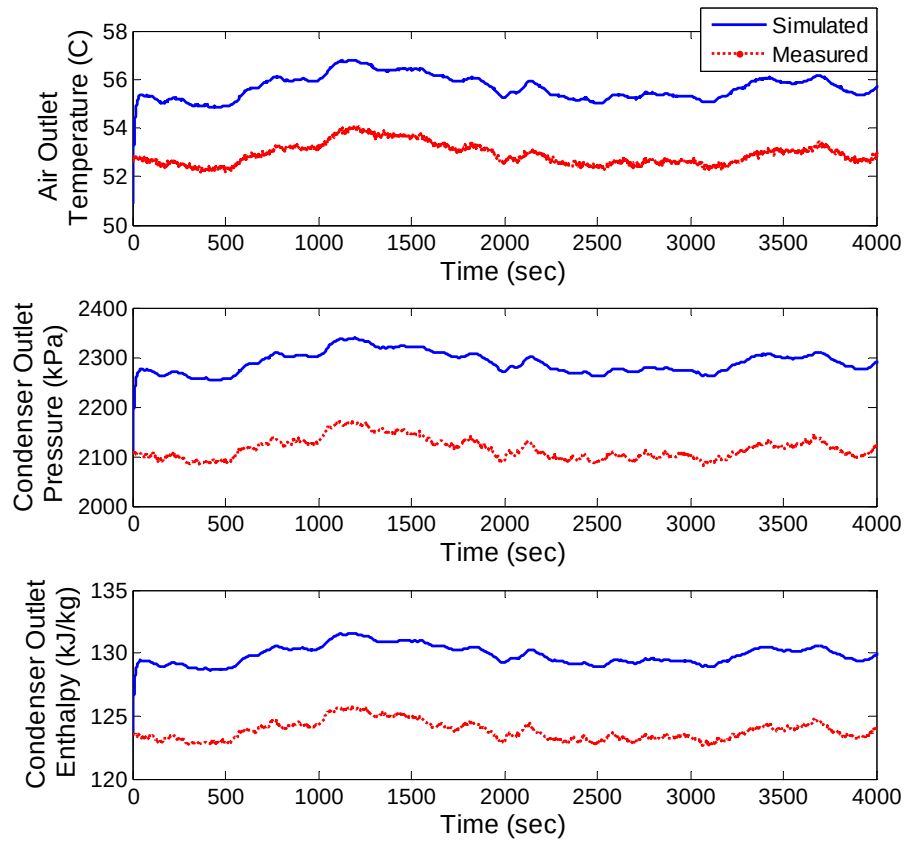


Figure 5.5: Condenser with receiver transient validation results at 0|100 °F for perturbations in EEV stroke.

At -20|50 °F, the simulated compressor mass flow rate contains large peaks where the compressor speed changes from high to low speed and vice versa (see Figure 5.3). Using the *simulated* compressor mass flow rate signal and the *measured* EEV mass flow rate signal as inputs, the refrigerant receiver attached to the condenser either rapidly fills or empties of refrigerant, causing the simulation to fail. For this reason, the condenser model was validated using the measured mass flow rate signal as *both* its input and output mass flow rate signal at the -20|50 °F operating condition. While this is not entirely accurate (the mass flow rates should not be equal during transients) the results should nonetheless provide some qualitative information about the predictive capability of the model.

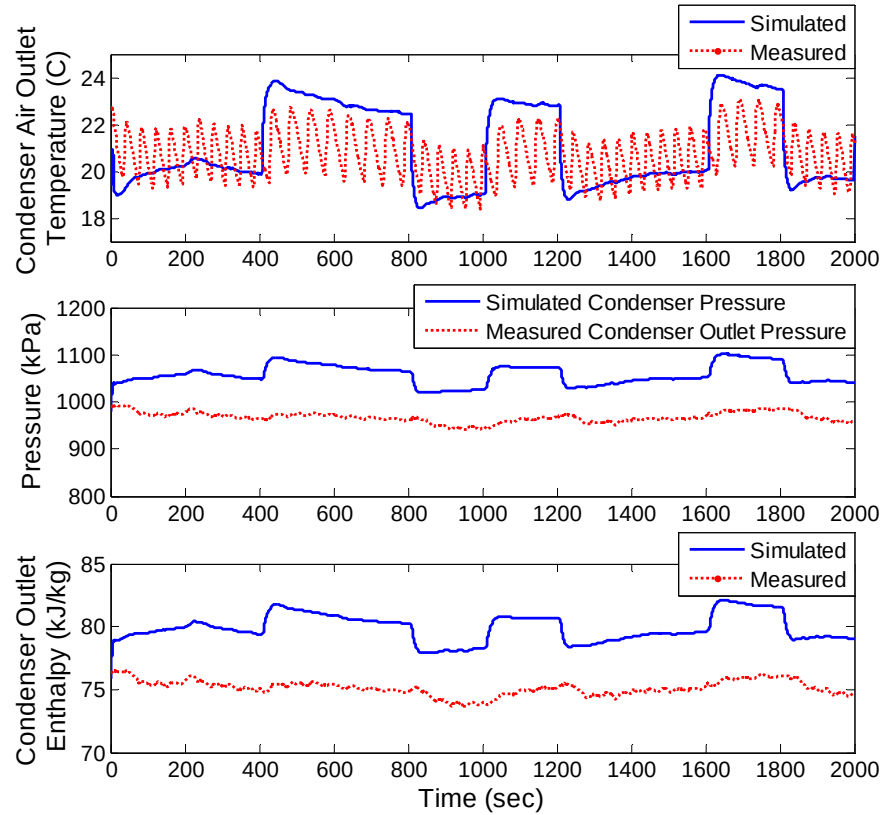


Figure 5.6: Condenser with receiver transient validation results at the -20|50 °F condition for perturbations in compressor speed.

Based on the validation at the 0|100 °F operating condition, the condenser model accurately predicts the dynamics of the actual condenser. Recall from Chapter 3 that the condenser was modeled as a single-circuit condenser albeit its multi-circuit geometry. These results suggest that this was a reasonable modeling approach.

From the validation conducted at 20|50 °F, it is difficult to accurately characterize how well the condenser model can predict the dynamics of the actual condenser because the inlet and outlet mass flow rate signals were equal in the simulation, and a key dynamic in the condenser is a direct function of the difference between the inlet and outlet mass flow rates. However, we can still see that the condenser model accurately predicts the average value of refrigerant properties such as pressure and temperature, as well as outlet air temperature, at -20|50 °F.

Note that in both Figure 5.5 and Figure 5.6, the predicted condenser pressure is compared to the measured condenser *outlet* pressure. In order to simplify the condenser

with receiver model, conservation of momentum was not used in the model derivation. Instead, the condenser with receiver assumes isobaric condensation, and so the simulated condenser pressure signal represents the uniform pressure inside the condenser coil. However, the actual system contains a pressure drop across the condenser coil, and consequently, the pressure was measured at the outlet of the receiver. Table 5.10 shows the normalized RMS error between the simulated and measured condenser output signals at each of the transient testing conditions.

Table 5.10 Normalized RMS Error for Condenser with Receiver Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS Error		
	Tc_{ao}	Pc	Hc_{ro}
0 100	0.0521	0.0819	0.0489
-20 50	0.0815	0.0955	0.0646

The largest normalized RMS error at both operating conditions is between the simulated and measured condenser pressure. This is to be expected given the isobaric assumption of the model.

5.1.3 Electronic Expansion Valve

The electronic expansion valve (EEV) model predicts its outlet mass flow rate. The production refrigeration unit contains a thermostatic expansion valve (TXV) instead of an EEV, primarily due to the significant difference in cost between the two valves. However, the TXV was replaced with an EEV for the purpose of transient data collection and model validation. Therefore an EEV model was developed based on the valve which was used in the experimental test unit. Table 5.11 and Table 5.12 describe the inputs and outputs, respectively, of the EEV model.

Table 5.11 EEV Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{v,ri}$	PV_ri	Pressure at EEV inlet	kPa
$P_{v,ro}$	PV_ro	Pressure at EEV outlet	kPa
$H_{v,ri}$	HV_ri	Enthalpy at EEV inlet	kJ/kg
<i>Stroke</i>	ValveOpen	EEV stroke	% open

Table 5.12 EEV Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_v$	mdot_v	Refrigerant mass flow rate through EEV	kg/s
$H_{v,ro}$	Hv_ro	Enthalpy at EEV outlet	kJ/kg

Figure 5.7 shows the Thermosys representation of the EEV model in Matlab Simulink.

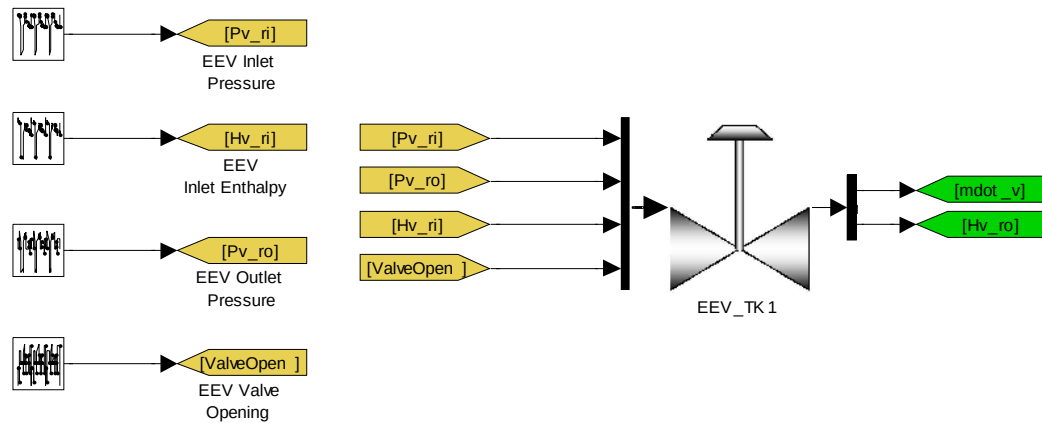


Figure 5.7: Thermosys Simulink model for EEV validation.

Table 5.13 shows the steady state validation results for the EEV. Steady state data was collected only at two operating conditions with the EEV system.

Table 5.13 Steady State Validation Results for EEV

Operating Condition Box/Amb (°F)	Test Number	Liquid Injection Valve	Actual Mass Flow Rate (kg/s)	Simulated Mass Flow Rate (kg/s)	Percent Error (%)	Absolute Error
0/100	TR16172-007-B	OFF	0.0566	0.0408	27.95	0.0158
-20/50	TR16172-009-A	ON	0.0409	0.0329	19.66	0.0080

Figure 5.8 and Figure 5.9 compare the measured outputs with the predicted outputs for the EEV model at different transient test conditions. Note that the EEV was validated with two sets of transient data, both of which were collected with perturbations to the system in EEV stroke.

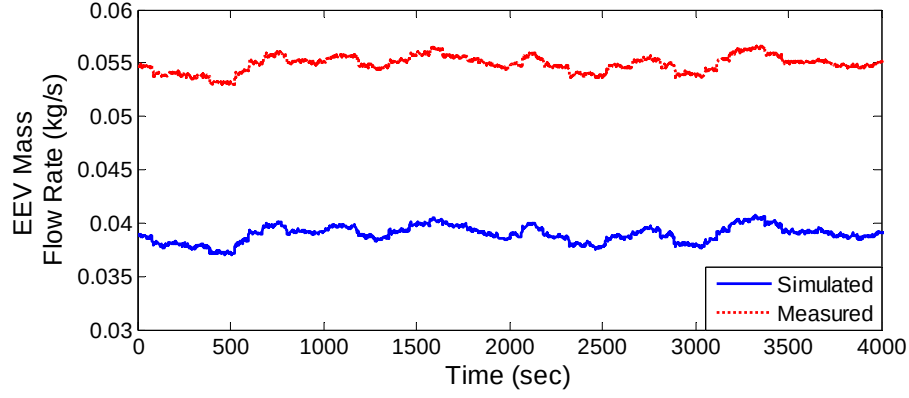


Figure 5.8: EEV transient validation results at 0|100 °F for perturbations in EEV stroke.

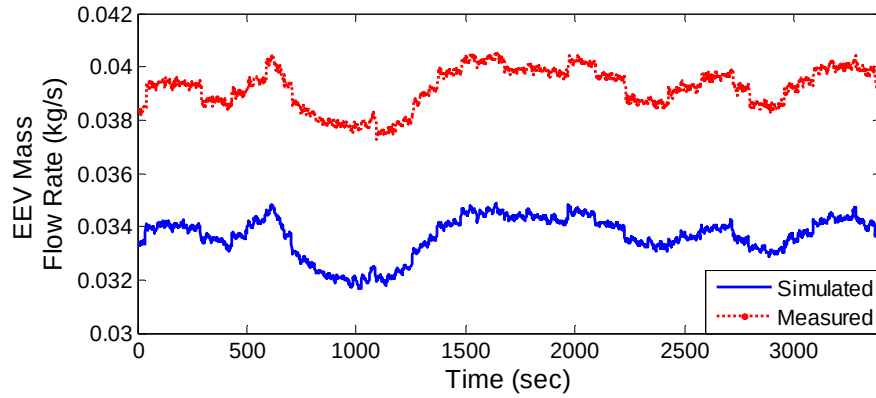


Figure 5.9: EEV transient validation results at -20|50 °F for perturbations in EEV stroke.

Qualitatively, the EEV model accurately predicts the dynamics of the EEV at both the 0|100 °F and -20|50 °F operating conditions. However, there is a clear vertical offset between the measured and predicted signals in both Figure 5.8 and Figure 5.9. Interestingly, this shift is consistent with the absolute error seen in the steady state validation at the same conditions. This suggests that the offset may be a direct result of inaccuracies related to the flow coefficient map that was developed for the EEV model. Recall Figure 3.11 which visually characterized these inaccuracies.

To compensate for this, an additive offset factor, K_v , can be computed for a given operating condition to improve the predictive capability of the model.

$$\dot{m}_{v,adjusted} = \dot{m}_v + K_v \quad (5.2)$$

The value of valve mass flow rate used in simulations is $\dot{m}_{v,adjusted}$. The predicted signal at each operating condition was adjusted using Equation 5.2 (where K_v is equal to the absolute error shown in Table 5.13 for each operating condition), and the new results are shown in Figure 5.10 and Figure 5.11.

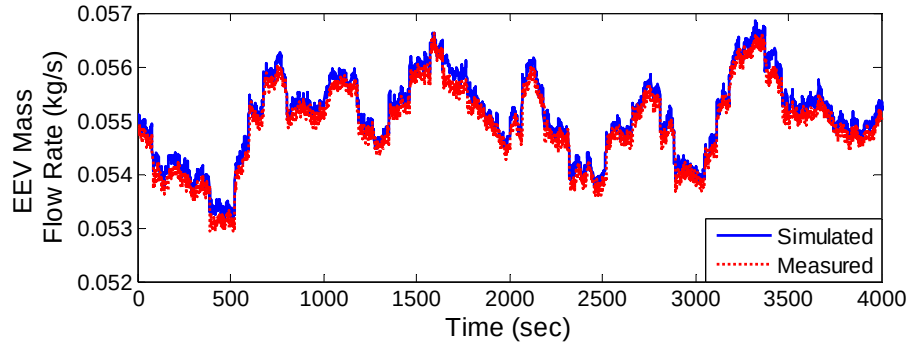


Figure 5.10: EEV transient validation results at 0|100 °F, adjusted by $K_v = 0.0158$.

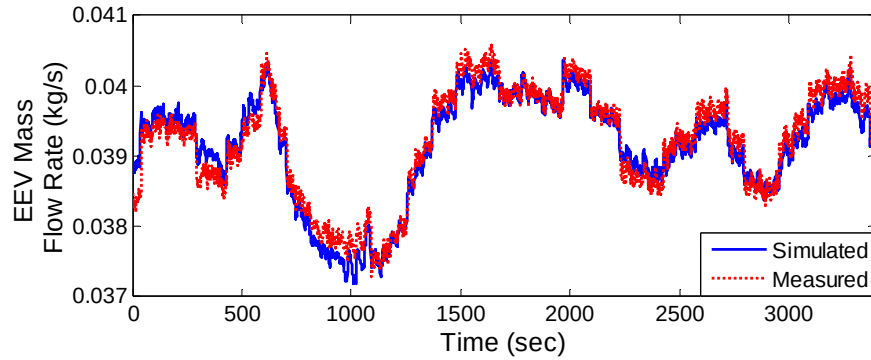


Figure 5.11: EEV transient validation results at 0|100 °F, adjusted by $K_v = 0.008$.

Next, the EEV model was validated using transient data collected with perturbations to the system in compressor speed (see Figure 5.12).

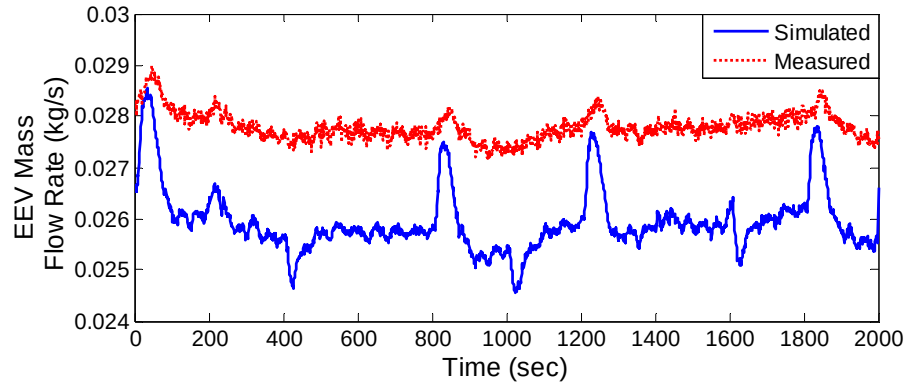


Figure 5.12: EEV transient validation results at -20|50 °F for perturbations in compressor speed.

The measured mass flow rate signal looks like a filtered version of the simulated mass flow rate signal. The filtering effect may be a function of where the mass flow meter measured the mass flow rate on the experimental system. For this reason, it is difficult to determine whether or not the EEV model is over-predicting the increase in mass flow rate for perturbations in compressor speed. Again, the shift between the measured and simulated signals could be the result of inaccuracies in the EEV flow coefficient map.

Table 5.14 shows the normalized RMS error between the simulated and measured EEV output signals at each of the transient testing conditions.

Table 5.14 Normalized RMS Error for EEV Transient Validation

Operating Condition Box/Amb (°F)	Normalized RMS Error $\dot{m}_{dot_v}$
0 100	0.0038
-20 50 (for perturbations in EEV stroke)	0.0051
-20 50 (for perturbations in compressor speed)	0.2061

5.1.4 Distributor

The distributor model predicts the pressure of the refrigerant entering the distributor. Table 5.15 and Table 5.16 describe the inputs and outputs, respectively, of the distributor model.

Table 5.15 Distributor Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_d$	mdot_d	Refrigerant mass flow rate through distributor	kg/s
$P_{d,ro}$	Pd_ro	Pressure at distributor outlet	kPa

Table 5.16 Distributor Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{d,ri}$	Pd_ri	Pressure at distributor inlet	kPa

Figure 5.13 shows the Thermosys representation of the distributor model in Matlab Simulink.

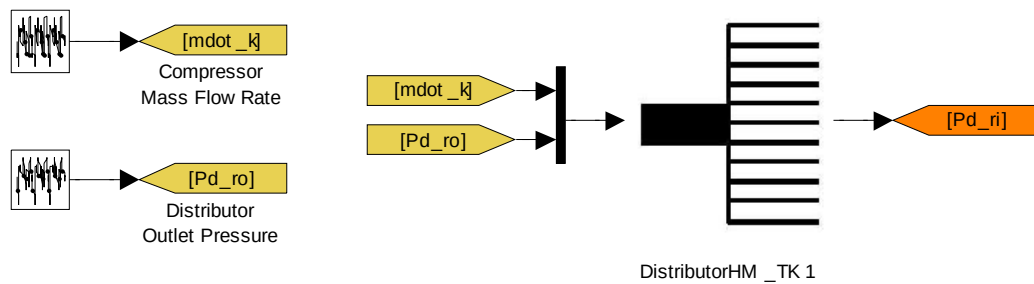
**Figure 5.13: Thermosys Simulink model for distributor validation.**

Table 5.17 shows the results of the steady state validation for the distributor model.

Table 5.17 Steady State Validation Results for Distributor

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual $P_{d,ri}$ (kPa)	Simulated $P_{d,ri}$ (kPa)	Percent Error (%)
35/100	TR16172-001-B	0.0699	948.72	948.72	0
0/100	TR16172-007-B	0.0566	723.26	723.26	0
-20/50	TR16172-009-A	0.0409	385.4	385.42	-0.0052

At steady state, the distributor model predicts the inlet distributor pressure very accurately because the estimates for the detrend constant associated with each input and output signal are based on the steady state data itself. Figure 5.14 and Figure 5.15

compare the measured outputs with the predicted outputs for the distributor model at different transient test conditions.

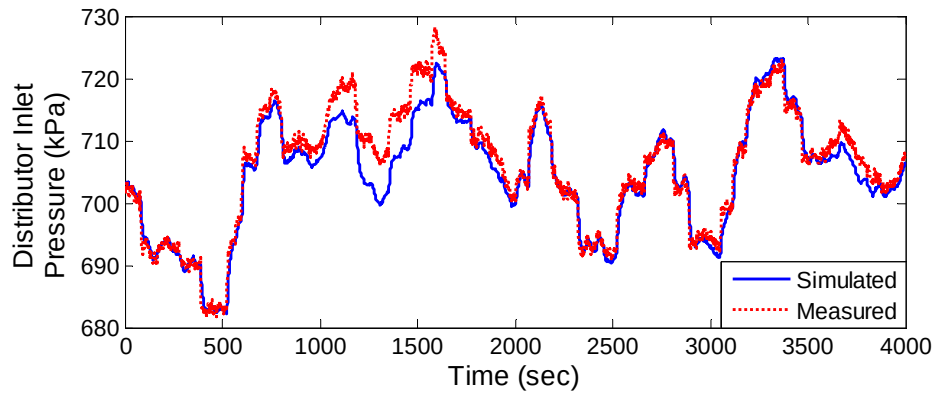


Figure 5.14: Distributor transient validation results at 0|100 °F for perturbations in EEV stroke.

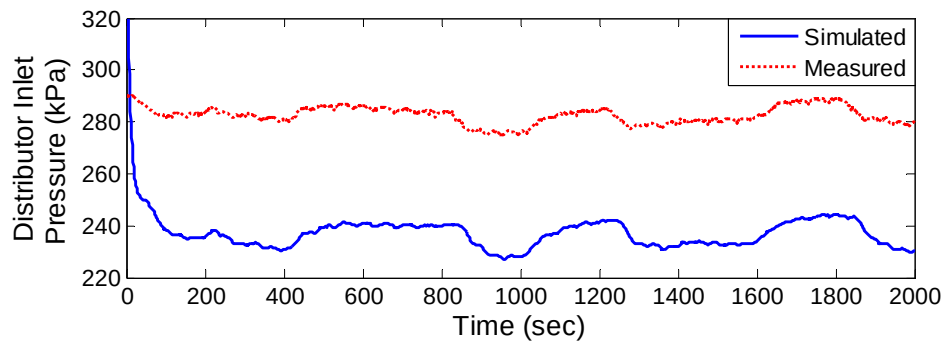


Figure 5.15: Distributor transient validation results at -20|50 °F for perturbations in compressor speed.

Table 5.18 shows the normalized RMS error between the simulated and measured distributor output signals at each of the transient testing conditions

Table 5.18 Normalized RMS Error for Distributor Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS Error
	<i>Pd_ri</i>
0 100	0.0039
-20 50	0.1613

The distributor model accurately predicts the dynamics of the distributor at both operating conditions tested. There is an offset between the measured and predicted output signals at the -20|50 °F operating condition; nevertheless, the offset is within a 15% error margin. Recall that the identification data was detrended before the model was identified; at different operating conditions, the sample mean of each input and output signal changes. The offset at -20|50 °F is most likely due to inaccuracies associated with the estimation of detrend constants for the input and output signals of the model at that condition.

5.1.5 Evaporator

The evaporator model predicts the pressure inside the evaporator coil, the enthalpy of the refrigerant exiting the evaporator, and the temperature of the air as it exits the evaporator. Table 5.19 and Table 5.20 describe the inputs and outputs, respectively, of the evaporator model.

Table 5.19 Evaporator Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_{r,i}$	mdot_i	Refrigerant mass flow rate entering evaporator	kg/s
$\dot{m}_{r,o}$	mdot_o	Refrigerant mass flow rate exiting evaporator	kg/s
$H_{e,ri}$	He_ri	Enthalpy at evaporator inlet	kJ/kg
$T_{e,ai}$	Te_ai	Temperature of air entering evaporator	°C
$\dot{m}_{e,ai}$	me_air	Air mass flow rate through evaporator	kg/s

Table 5.20 Evaporator Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
P_e	Pe	Pressure inside evaporator	kPa
$H_{e,ro}$	He_ro	Enthalpy at evaporator outlet	kJ/kg
$T_{e,ao}$	Te_ao	Temperature of air exiting evaporator	°C

Figure 5.16 shows the Thermosys representation of the evaporator model in Matlab Simulink.

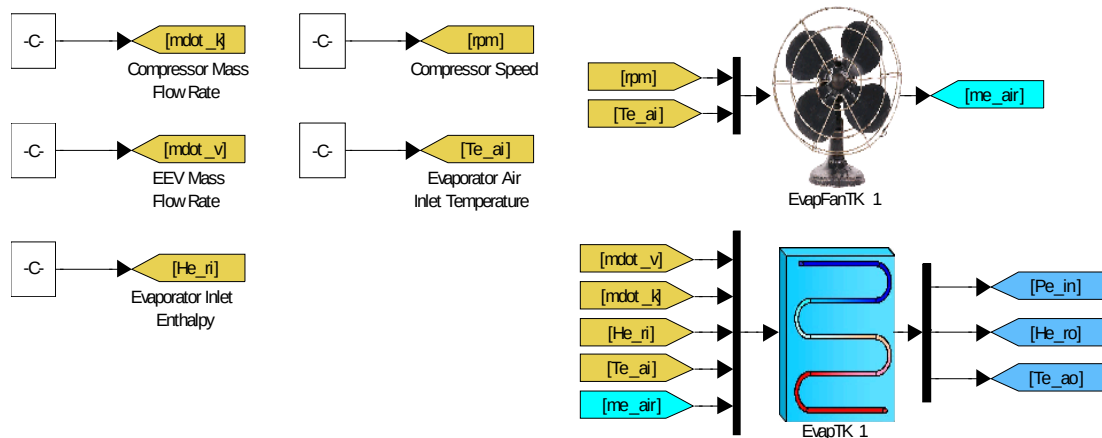


Figure 5.16: Thermosys Simulink model for evaporator validation.

Table 5.21, Table 5.22, and Table 5.23 show the results of the steady state validation for the evaporator model.

Table 5.21 Steady State Validation Results for Evaporator Pressure

Operating Condition Box/Amb (°F)	Test Number	Actual P_e (kPa)	Simulated P_e (kPa)	Percent Error (%)
35/50	TR15640-018-A	373.01	352.99	5.4
35/70	TR15640-014-A	408.86	365.96	10.5
0/100	TR15640-002-C	246.83	229.23	7.1
35/120	TR15640-007-C	447.47	404.96	9.5
70/120	TR15640-008-D	783.93	822.90	-5.0

Table 5.22 Steady State Validation Results for Evaporator Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Actual He_{ro} (kJ/kg)	Simulated He_{ro} (kJ/kg)	Percent Error (%)
35/50	TR15640-018-A	222.02	219.83	1.0
35/70	TR15640-014-A	221.95	219.44	1.1
0/100	TR15640-002-C	208.21	206.67	0.7
35/120	TR15640-007-C	222.21	220.47	0.8
70/120	TR15640-008-D	233.86	232.48	0.6

Table 5.23 Steady State Validation Results for Evaporator Outlet Air Temperature

Operating Condition Box/Amb (°F)	Test Number	Actual $T_{e_{ao}}$ (C)	Simulated $T_{e_{ao}}$ (C)	Percent Error (%)
35/50	TR15640-018-A	-6.11	-8.14	-33.2
35/70	TR15640-014-A	-5.17	-7.64	-47.8
0/100	TR15640-002-C	-21.78	-23.65	-8.6
35/120	TR15640-007-C	-3.72	-5.96	-60.2
70/120	TR15640-008-D	15.94	13.80	13.4

Like the condenser with receiver model, the evaporator model requires *both* its inlet and outlet mass flow rate signals as inputs. However, only one measured mass flow rate signal is available. For the condenser transient model validation, the compressor model was used to provide a *predicted* transient compressor mass flow rate signal for the simulation. Although this predicted signal is necessarily different than the actual signal, a scaling factor was used to have the signals match more closely. Unfortunately, the evaporator model is more sensitive to inaccuracies in inlet and outlet mass flow rate signals than the condenser. This is largely due to the fact that the flow length of the single-circuit evaporator is much smaller than that of the condenser; if the inlet refrigerant mass flow rate is much larger than the outlet flow rate, the superheat region disappears quickly, causing the simulation to crash.

For this reason, the evaporator model was validated using the measured mass flow rate signal as *both* its input and output mass flow rate signal at the 0|100 °F and -20|50 °F operating conditions. While this is not entirely accurate (i.e. the mass flow rates should not be equal during transients), the results should nonetheless provide some qualitative information about the predictive capability of the model.

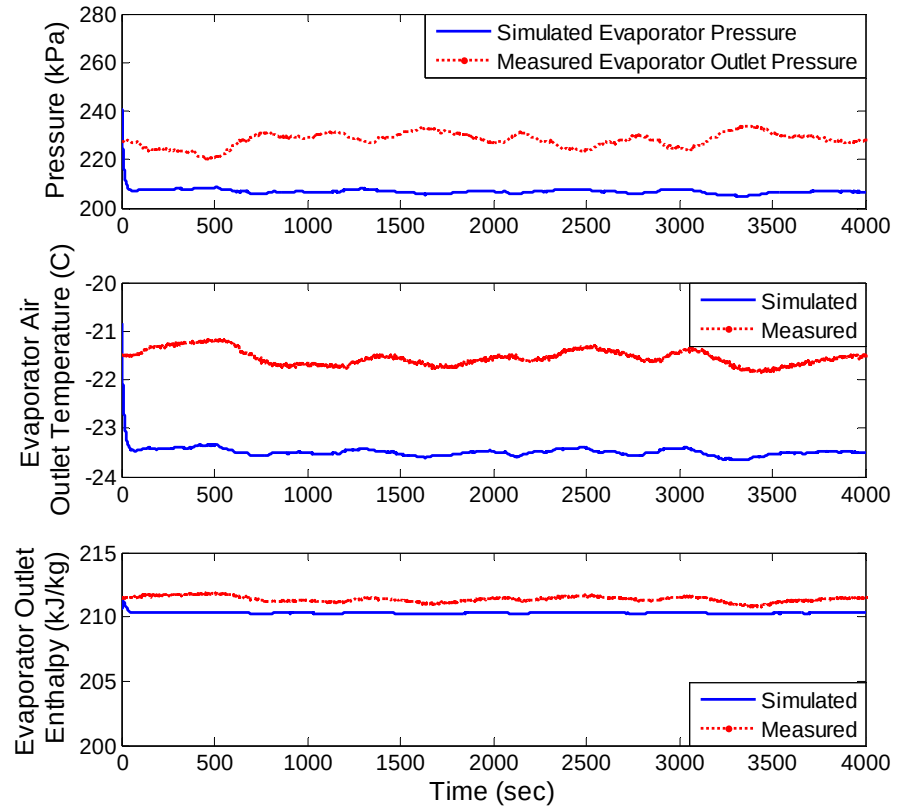


Figure 5.17: Evaporator transient validation results at 0|100 °F for perturbations in EEV stroke.

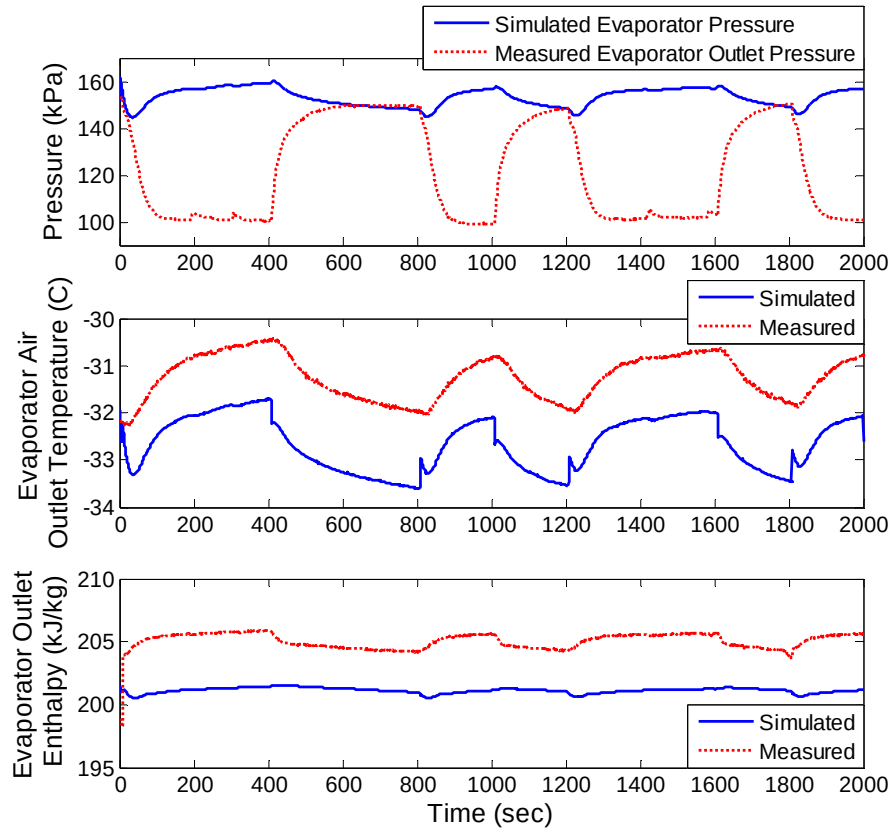


Figure 5.18: Evaporator transient validation results at -20/50 °F for perturbations in compressor speed.

It is difficult to accurately characterize how well the evaporator model can predict the dynamics of the actual evaporator through a simulation where the inlet and outlet mass flow rate signals are equal, primarily because a key dynamic in the evaporator is a direct function of the difference between the inlet and outlet mass flow rates. Nevertheless, we can see that the evaporator model accurately predicts the average value of refrigerant properties such as pressure and temperature, as well as outlet air temperature.

The main discrepancy is seen in Figure 5.18 between the measured and simulated evaporator pressure signals. Again, it is unclear whether this is a result of the model itself or of having sent the model the same inlet and outlet mass flow rate signals. Further testing and validation should be done to better understand the predictive capability of the evaporator model for perturbations in compressor speed.

5.1.6 Suction Line Heat Exchanger and Accumulator

The suction line heat exchanger (SLHX) model predicts the pressure and enthalpy of the refrigerant exiting the vapor and liquid passes of the heat exchanger. The vapor pass contains the superheated refrigerant exiting the evaporator, and the liquid pass contains liquid refrigerant exiting the condenser receiver. Table 5.24 and Table 5.25 list the inputs and outputs, respectively, of the SLHX model.

Table 5.24 SLHX Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$T_{l,i}$	Tliq_i	Liquid inlet temperature	°C
$T_{v,i}$	Tvap_i	Vapor inlet temperature	°C
$P_{l,i}$	Pliq_i	Liquid inlet pressure	kPa
$P_{v,i}$	Pvap_i	Vapor inlet pressure	kPa
$\dot{m}_l$	mdot_l	Refrigerant mass flow rate through liquid pass	kg/s
$\dot{m}_v$	mdot_v	Refrigerant mass flow rate through vapor pass	kg/s

Table 5.25 SLHX Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{v,o}$	Pvap_o	Vapor outlet pressure	kPa
$P_{l,o}$	Pliq_o	Liquid outlet pressure	kPa
$h_{v,o}$	hvp_o	Vapor outlet enthalpy	kJ/kg
$h_{l,o}$	hliq_o	Liquid outlet enthalpy	kJ/kg

The accumulator model predicts the pressure and enthalpy of the refrigerant exiting the accumulator. Table 5.26 and Table 5.27 describe the inputs and outputs, respectively, of the accumulator model.

Table 5.26 Accumulator Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_a$	mdot_i	Mass flow rate through accumulator	kg/s
$P_{a,ri}$	Pa_ri	Pressure at accumulator inlet	kPa
$H_{a,ri}$	Ha_ri	Enthalpy at accumulator inlet	kJ/kg

Table 5.27 Accumulator Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{a,ro}$	Pa_ro	Pressure at accumulator outlet	kPa
$H_{a,ro}$	Ha_ro	Enthalpy at accumulator outlet	kJ/kg

The SLHX model was validated with the pipe models immediately upstream and downstream of it because of the particular pressure and temperature measurements that were available from the experimental system. Furthermore, the accumulator model, which is downstream of the *vapor pass* of the SLHX, was included in the validation because pressure and temperature measurements were available at the accumulator outlet but not at the accumulator inlet. Figure 5.19 shows the Thermosys representation of the SLHX and accumulator model subsystem in Matlab Simulink.

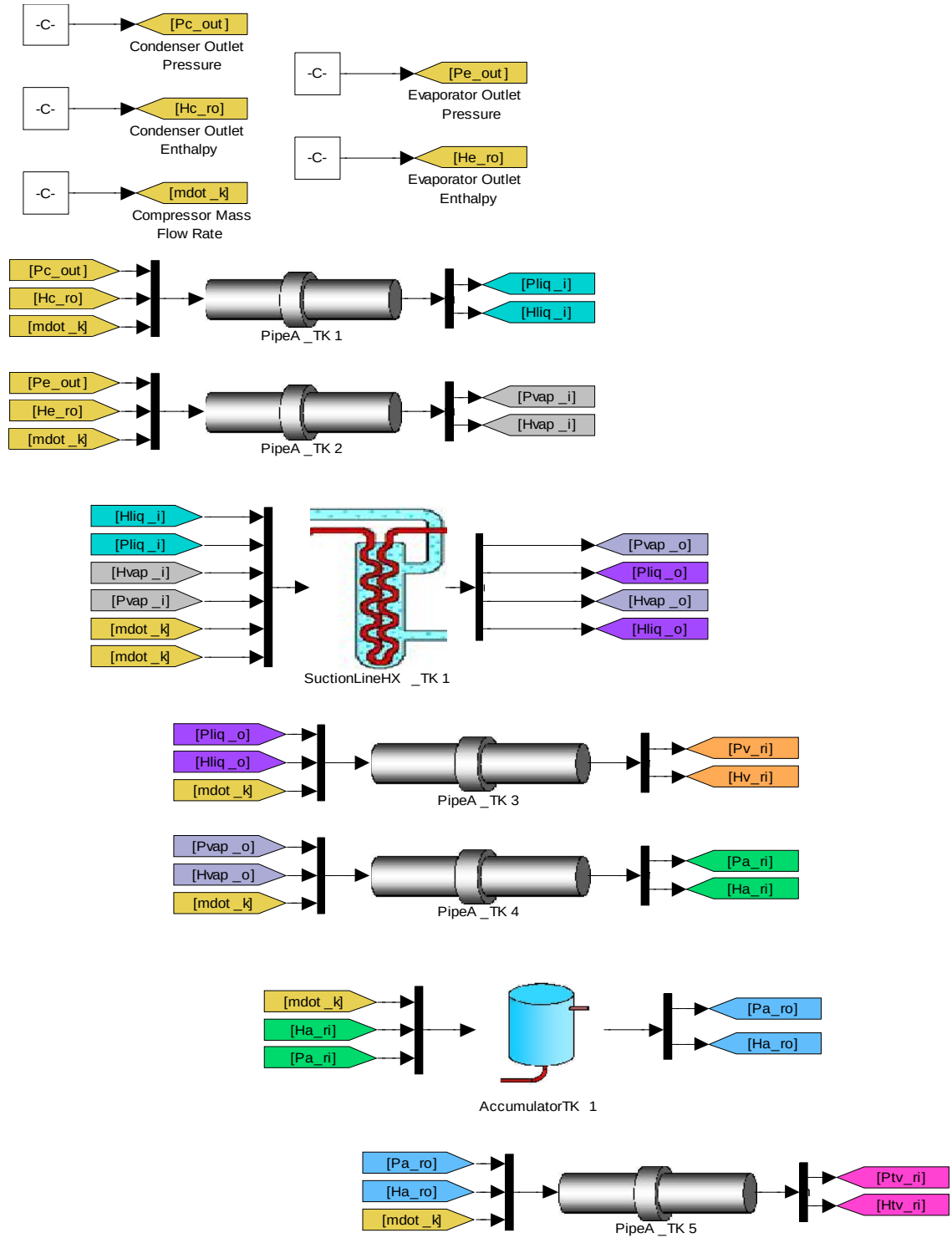


Figure 5.19: Thermosys Simulink model for SLHX and accumulator validation.

Table 5.28 and Table 5.29 show the steady state validation results for the liquid pass of the SLHX. Table 5.30 and Table 5.31 show the steady state validation results for

the vapor pass of the SLHX. The accumulator model is an extension of the vapor pass of the SLHX. Note that Pv_ri and Hv_ri denote the inlet pressure and enthalpy of the EEV, and Ptv_ri and Htv_ri denote the inlet pressure and enthalpy of the throttle valve.

Table 5.28 Steady State Validation Results for SLHX Liquid Side Pressure

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual Pv_ri (kPa)	Simulated Pv_ri (kPa)	Percent Error (%)
35/50	TR15640-018-A	0.0745	1225.89	1227.6	-0.14
35/70	TR15640-014-A	0.0738	1423.8	1425.2	-0.10
0/100	TR15640-002-C	0.0381	2113.9	2108.6	0.25
35/120	TR15640-007-C	0.0684	2133.2	2129.7	0.16
70/120	TR15640-008-D	0.0435	2809.6	2803.7	0.21

Table 5.29 Steady State Validation Results for SLHX Liquid Side Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual Hv_ri (kJ/kg)	Simulated Hv_ri (kJ/kg)	Percent Error (%)
35/50	TR15640-018-A	0.0745	76.45	76.74	-0.38
35/70	TR15640-014-A	0.0738	82.64	83.6	-1.16
0/100	TR15640-002-C	0.0381	88.97	94.8	-6.55
35/120	TR15640-007-C	0.0684	101.86	104.4	-2.49
70/120	TR15640-008-D	0.0435	125.87	128.7	-2.25

Table 5.30 Steady State Validation Results for SLHX Vapor Side Pressure

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual Ptv_ri (kPa)	Simulated Ptv_ri (kPa)	Percent Error (%)
35/50	TR15640-018-A	0.0745	369.65	369.64	0.00
35/70	TR15640-014-A	0.0738	404.44	405.8	0.34
0/100	TR15640-002-C	0.0381	245.08	245.2	0.05
35/120	TR15640-007-C	0.0684	439.58	441.3	0.39
70/120	TR15640-008-D	0.0435	781.67	783.2	0.20

Table 5.31 Steady State Validation Results for SLHX Vapor Side Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual H_{tv_ri} (kJ/kg)	Simulated H_{tv_ri} (kJ/kg)	Percent Error (%)
35/50	TR15640-018-A	0.0745	242.47	242.2	0.11
35/70	TR15640-014-A	0.0738	245.78	245.2	0.24
0/100	TR15640-002-C	0.0381	260.33	254.3	2.32
35/120	TR15640-007-C	0.0684	257.6	256.2	0.54
70/120	TR15640-008-D	0.0435	268.53	270.4	0.70

The following figures show the transient validation results for the liquid and vapor passes of the SLHX model. Recall that the accumulator model is an extension of the vapor pass.

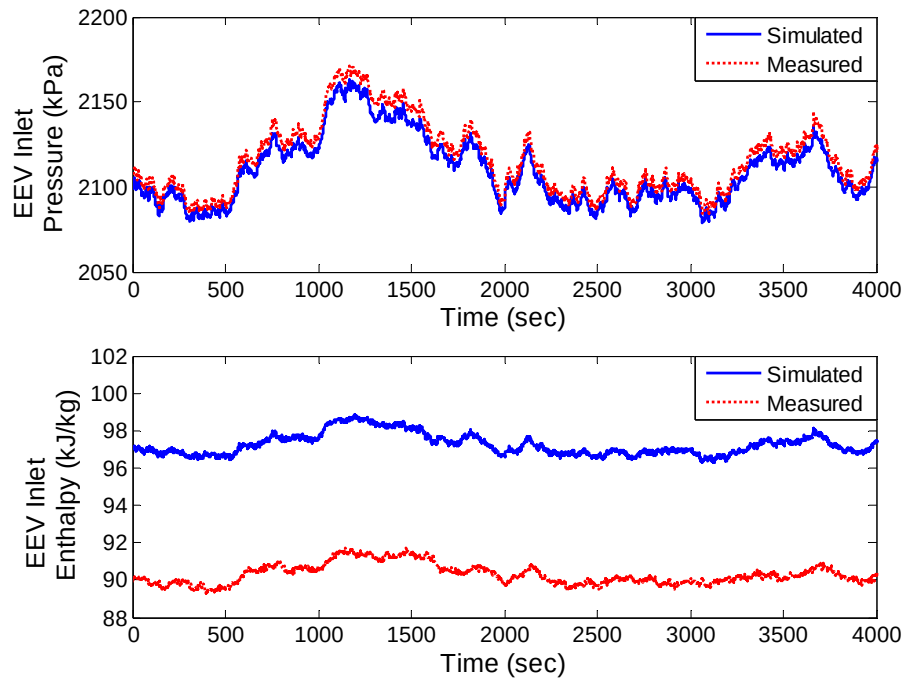


Figure 5.20: Liquid side SLHX transient validation results at 0/100 °F for perturbations in EEV stroke.

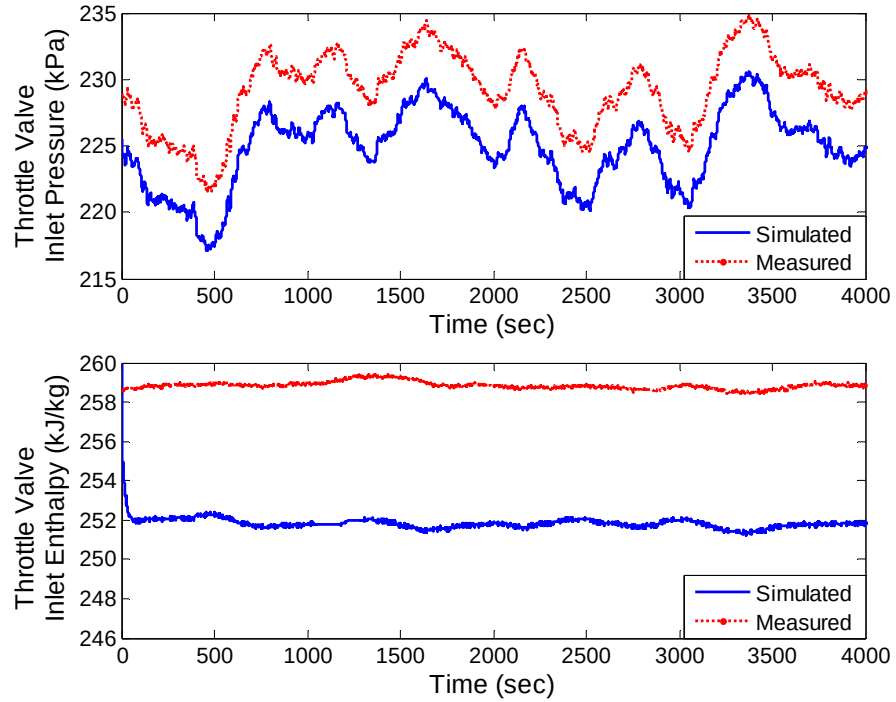


Figure 5.21: Vapor side SLHX transient validation results at 0|100 °F for perturbations in EEV stroke.

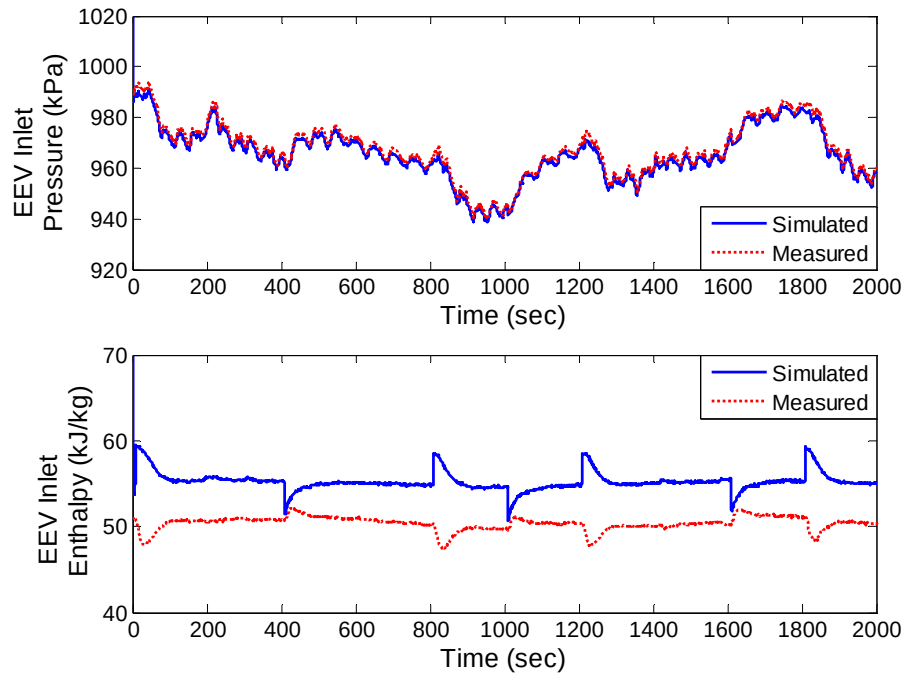


Figure 5.22: Liquid side SLHX transient validation results at -20|50 °F for perturbations in compressor speed.

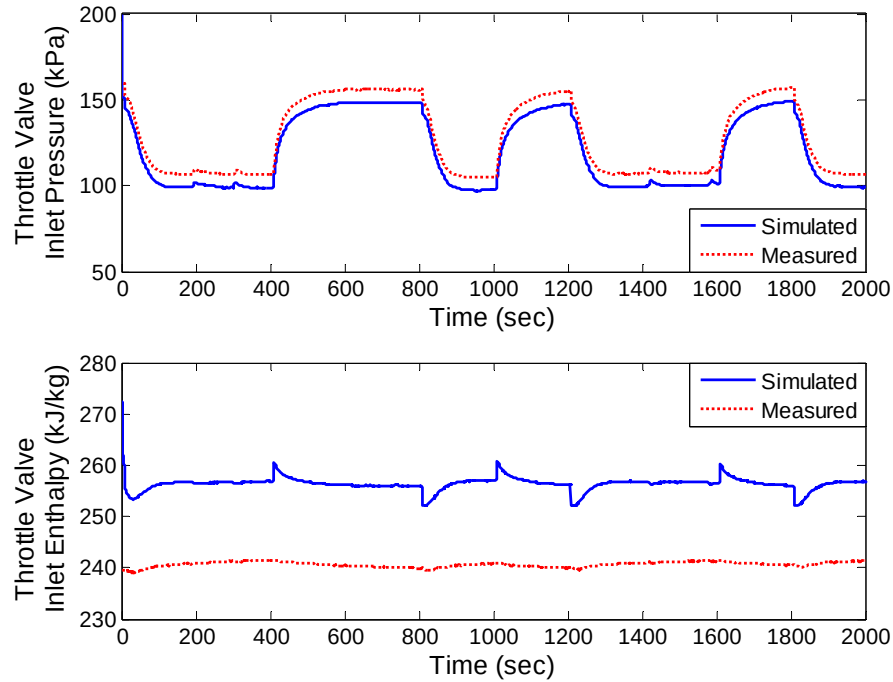


Figure 5.23: Vapor side SLHX transient validation results at -20|50 °F for perturbations in compressor speed.

Table 5.32 Normalized RMS Error for SLHX Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS			
	Pv_{ri}	Hv_{ri}	Ptv_{ri}	Htv_{ri}
0 100	0.0032	0.0769	0.0189	0.0271
-20 50	0.0019	0.1022	0.0594	0.0654

The largest error is seen in Figure 5.22 which compares the measured and predicted inlet EEV enthalpy signals at the -20|50 °F condition. The measured signal shows that when the compressor speed is stepped down to low speed, there is a corresponding drop in inlet EEV enthalpy. This makes sense because for lower compressor speeds, there will be less heat transfer across the condenser, causing the enthalpy of the refrigerant exiting the condenser (and then exiting the liquid pass of the SLHX) to be lower. However, the model predicts a slight increase in enthalpy as a result of drops in compressor speed. Given that the model accurately predicts the enthalpy dynamics at the EEV inlet at a different operating condition (see Figure 5.20), the discrepancy could be a function of operating condition rather than the model itself. The

SLHX model should be tested at additional operating conditions to better understand the cause of the behavior seen in Figure 5.22.

5.1.7 Throttle Valve

The throttle valve model predicts its refrigerant outlet pressure which is the refrigerant pressure entering the compressor. Table 5.33 and Table 5.34 describe the inputs and outputs, respectively, of the throttle valve model.

Table 5.33 Throttle Valve Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_i$	mdot_tv	Refrigerant mass flow rate at throttle valve inlet	kg/s
$P_{tv,ri}$	Ptv_ri	Pressure at throttle valve inlet	kPa
$H_{tv,ri}$	Htv_ri	Enthalpy at throttle valve inlet	kJ/kg

Table 5.34 Throttle Valve Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{tv,ro}$	Ptv_ro	Pressure at throttle valve outlet	kPa
$H_{tv,ro}$	Htv_ro	Enthalpy at throttle valve outlet	kJ/kg

Figure 5.24 shows the Thermosys representation of the throttle valve model in Matlab Simulink. The throttle valve, and the pipe connecting it to the compressor inlet, are validated together because measurements of the pressure and temperature are available at the compressor inlet but not at the throttle valve outlet.

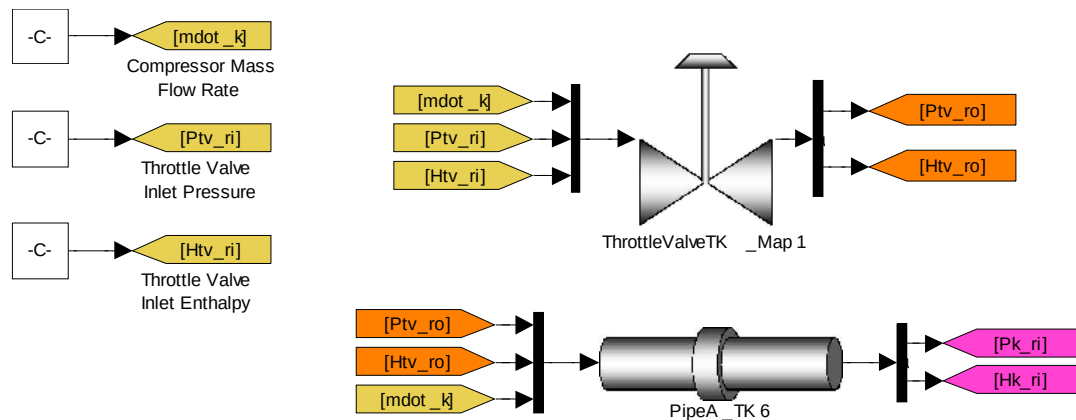


Figure 5.24: Thermosys Simulink model for throttle valve validation.

The steady state validation results are presented in Table 5.35 and Table 5.36. Note that Pk_{ri} and Hk_{ri} denote compressor inlet pressure and inlet enthalpy, respectively.

Table 5.35 Steady State Validation Results for Throttle Valve Outlet Pressure

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual Pk_{ri} (kPa)	Simulated Pk_{ri} (kPa)	Percent Error (%)
35/50	TR15640-018-A	0.0745	283.37	294.46	3.91
35/70	TR15640-014-A	0.0738	286.82	295.62	3.07
0/100	TR15640-002-C	0.0381	246.14	244.38	0.72
35/120	TR15640-007-C	0.0684	287.51	296.37	3.08
70/120	TR15640-008-D	0.0435	303.37	298.43	1.63
-20/50	TR15640-016-A	0.0403	154.44	153.68	0.49

Table 5.36 Steady State Validation Results for Throttle Valve Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Mass Flow Rate (kg/s)	Actual Hk_{ri} (kJ/kg)	Simulated Hk_{ri} (kJ/kg)	Percent Error (%)
35/50	TR15640-018-A	0.0745	241.96	242.47	-0.21
35/70	TR15640-014-A	0.0738	245.19	245.78	-0.24
0/100	TR15640-002-C	0.0381	258.69	260.33	-0.63
35/120	TR15640-007-C	0.0684	256.61	257.60	-0.39
70/120	TR15640-008-D	0.0435	269.08	268.52	0.21
-20/50	TR15640-016-A	0.0403	238.85	240.10	-0.52

Figure 5.25 and Figure 5.26 show the transient validation results for the throttle valve model.

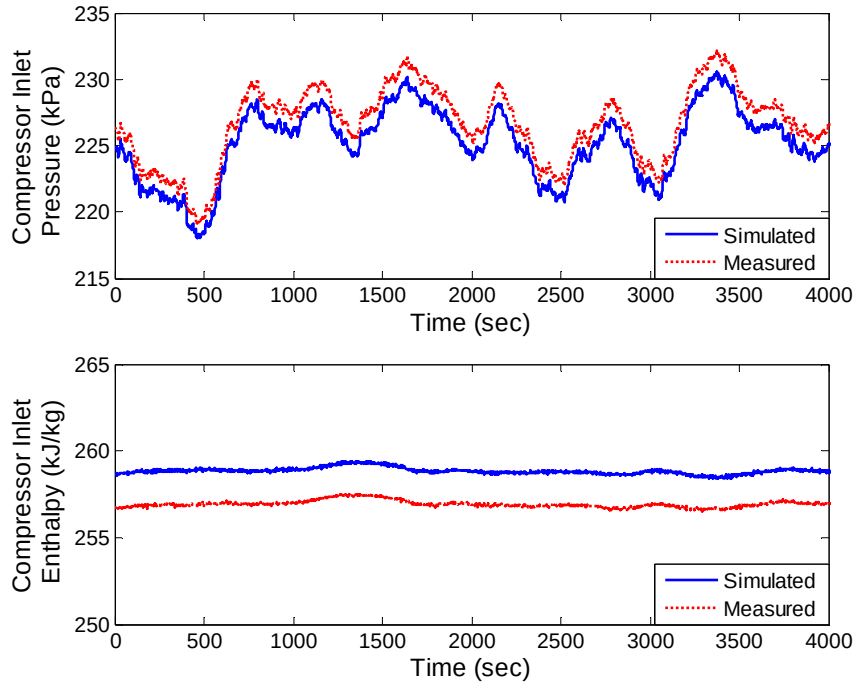


Figure 5.25: Throttle valve transient validation results at 0|100 °F for perturbations in EEV stroke.

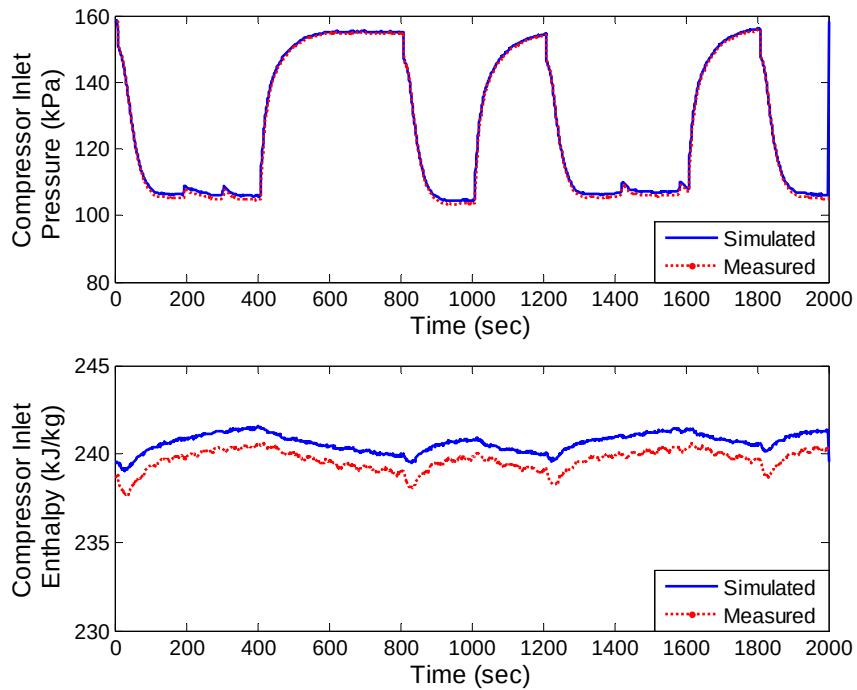


Figure 5.26: Throttle valve transient validation results at -20|50 °F for perturbations in compressor speed.

Table 5.37 shows the normalized RMS error between the simulated and measured throttle valve output signals at each of the transient testing conditions.

Table 5.37 Normalized RMS Error for Throttle Valve Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS Error	
	<i>Pk_ri</i>	<i>Hk_ri</i>
0 100	0.0062	0.0074
-20 50	0.0114	0.0042

The throttle valve model consistently predicts both compressor inlet pressure and inlet enthalpy accurately across both operating conditions.

5.2 Complete System Validation

In this section, the validation of the complete cycle system is presented. This validation was conducted at the 0|100 °F box|ambient operating condition. Table 5.38 shows the inputs for the complete system model.

Table 5.38 Complete System Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
<i>rpm</i>	rpm	Compressor rotational speed	rev/min
<i>stroke</i>	ValveOpen	EEV stroke	% open
<i>T_{c,ai}</i>	Tc_ai	Temperature of air entering condenser	°C
<i>T_{e,ai}</i>	Te_ai	Temperature of air entering evaporator	°C
<i>T_{amb}</i>	T_amb	Temperature of ambient air	°C

Figure 5.27 shows the input signals used in the complete system validation.

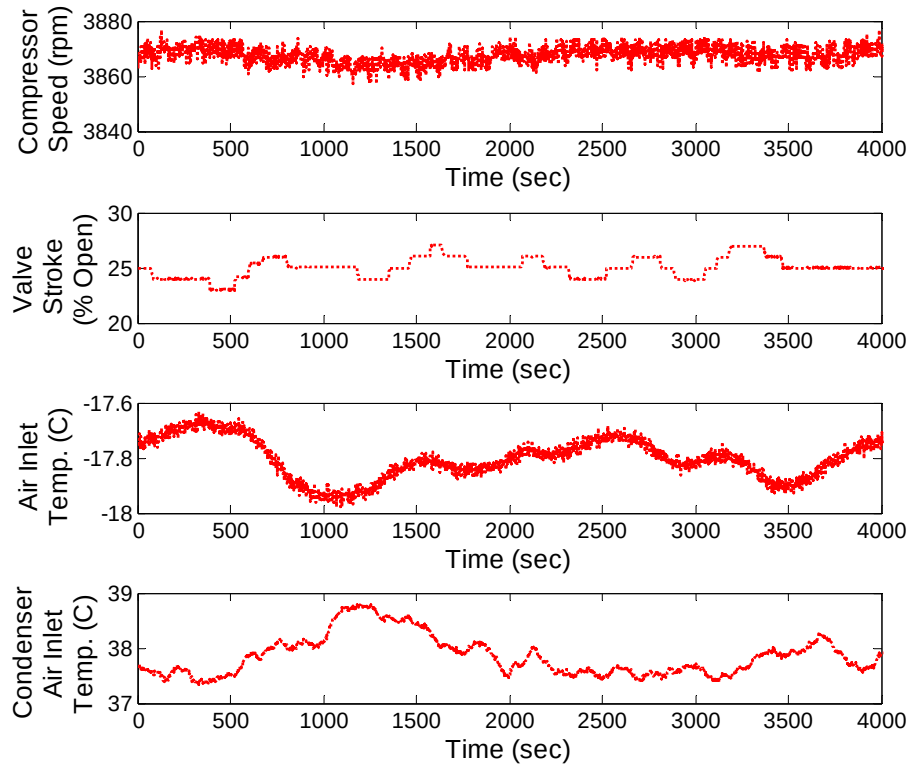


Figure 5.27: Complete cycle system transient inputs at 0/100 °F.

The individual component system responses are presented in the order that they appear in the closed refrigeration cycle.

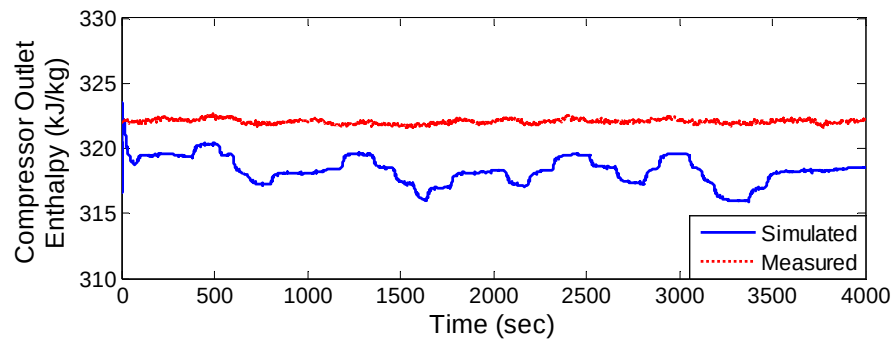


Figure 5.28: Transient validation results for compressor outlet enthalpy.

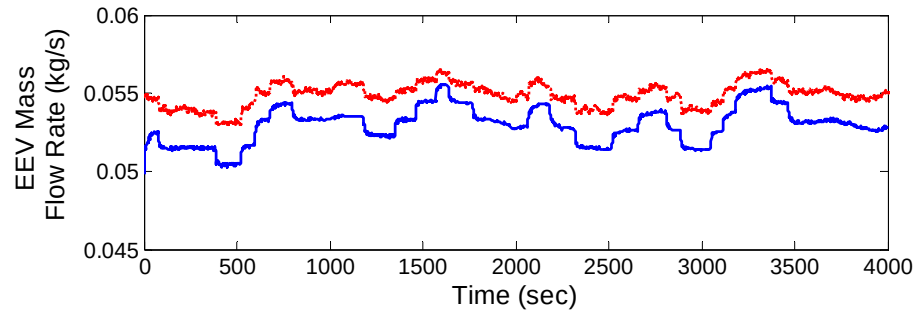


Figure 5.29: Transient validation results for EEV mass flow rate.

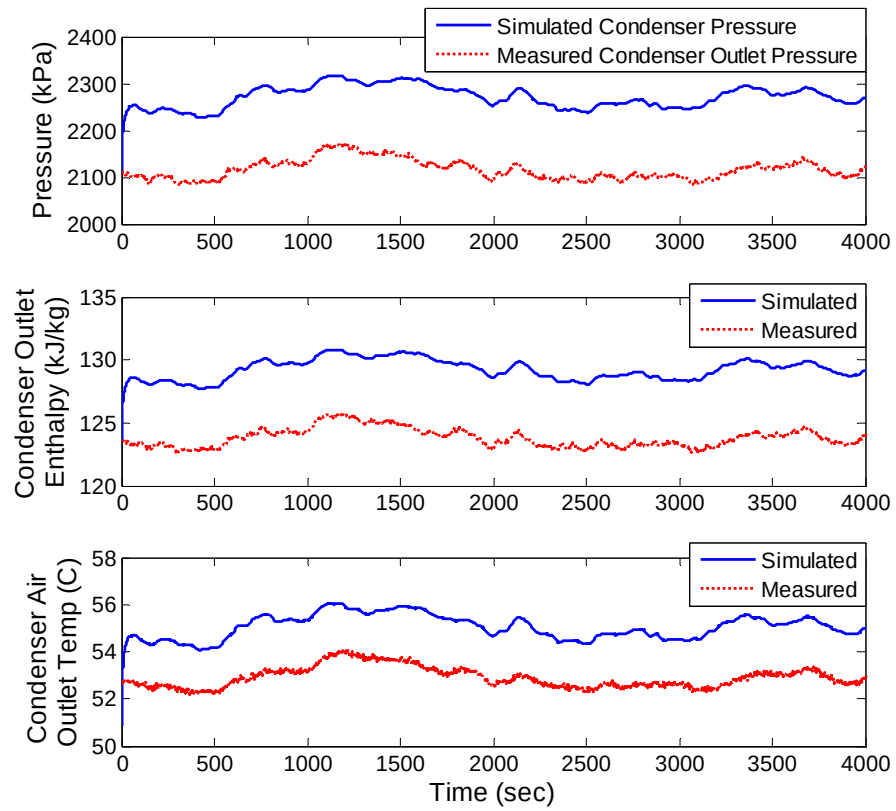


Figure 5.30: Transient validation results for condenser model.

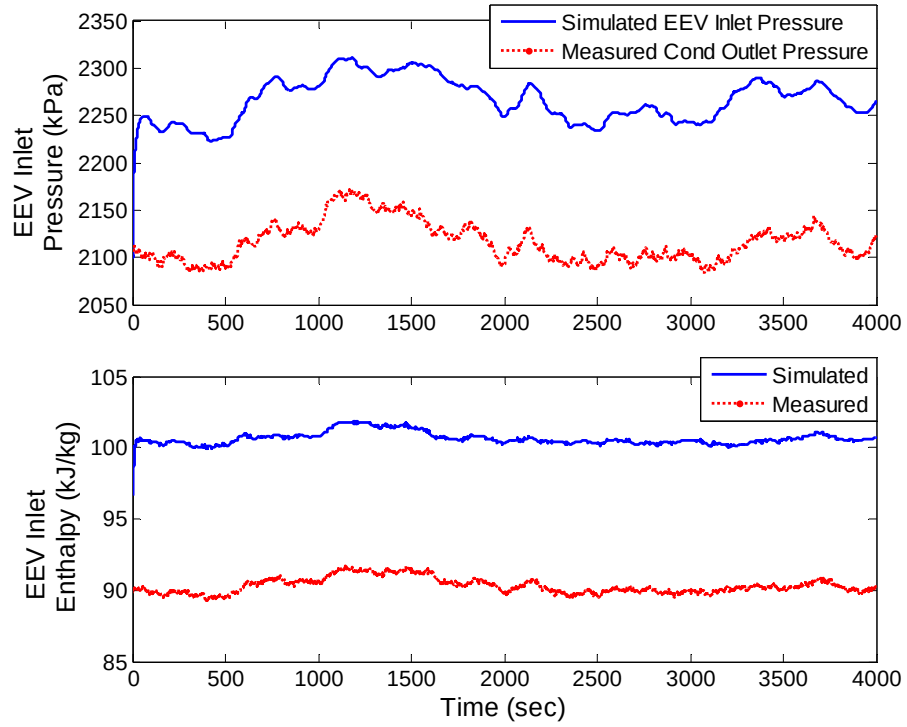


Figure 5.31: Transient validation results for liquid pass of SLHX.

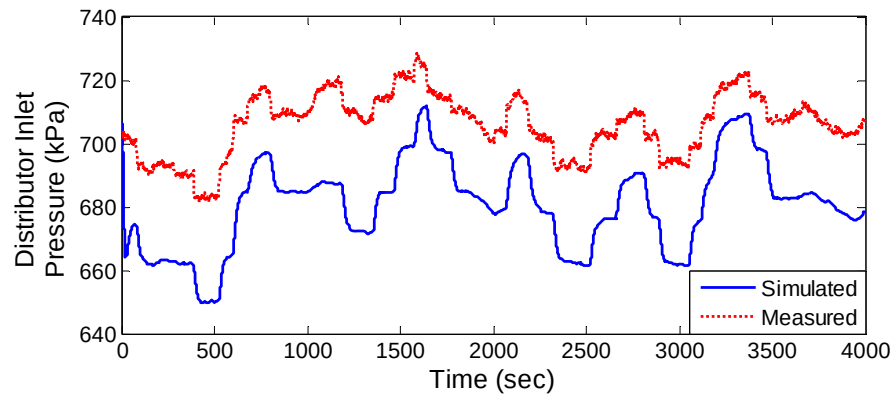


Figure 5.32: Transient validation results for distributor model.

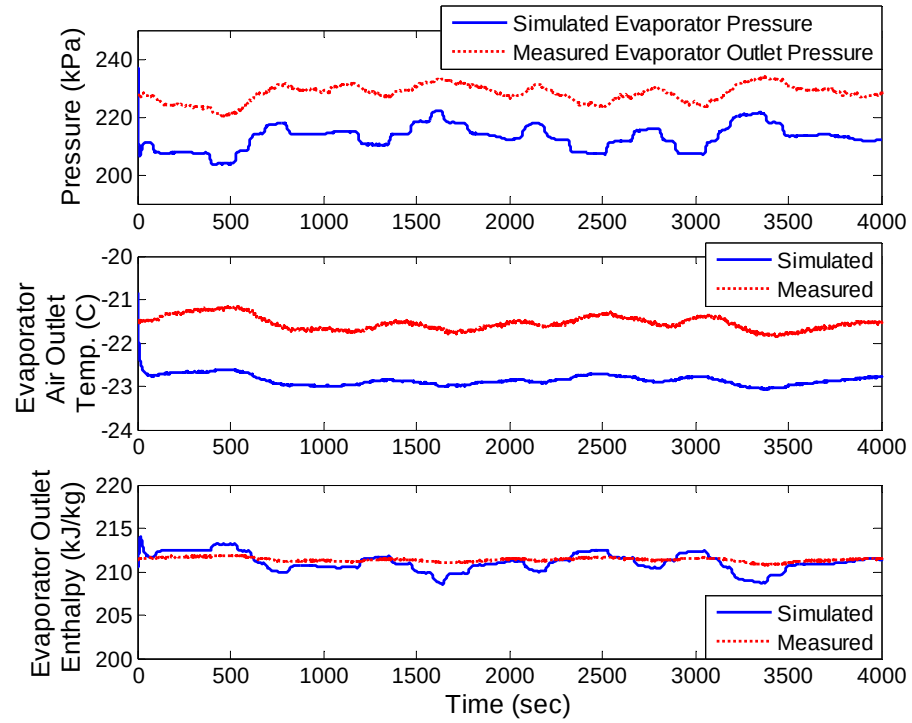


Figure 5.33: Transient validation results for evaporator model.

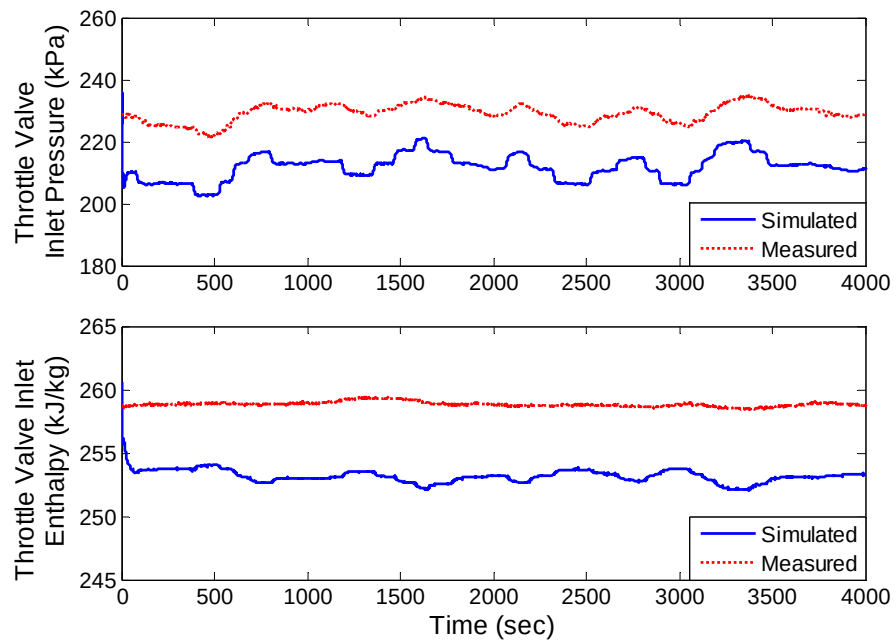


Figure 5.34: Transient validation results for vapor pass of SLHX and accumulator.

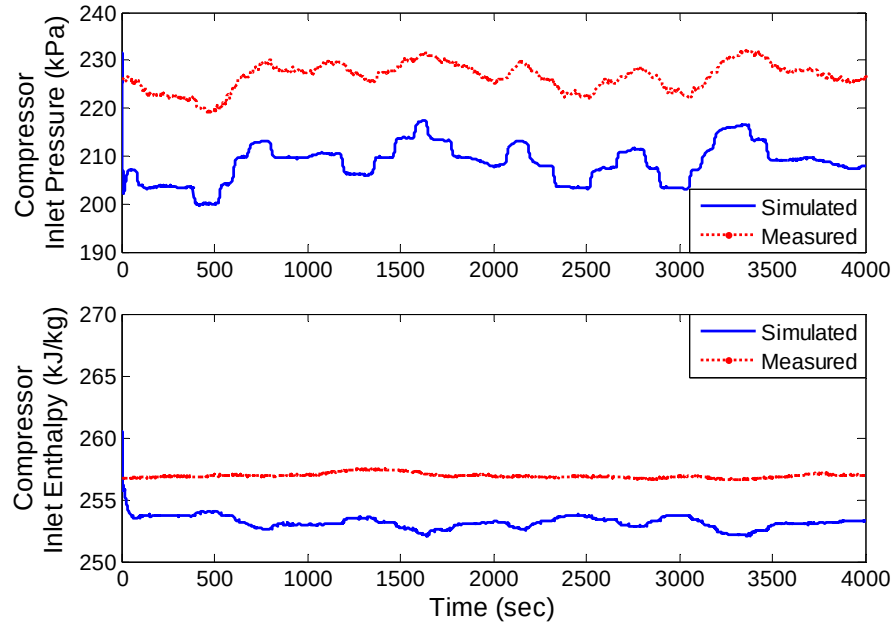


Figure 5.35: Transient validation results for throttle valve model.

Although there is an offset between almost all of the measured and predicted signals, the transient dynamics of the system are accurately captured by the complete cycle model. This model was not validated using a transient data set collected while the compressor speed was perturbed; given that any control algorithm for the actual system is restricted to changing the compressor speed between zero, low, and high speed, an additional validation should be part of future work. That being said, the compressor can be treated as a variable speed compressor in simulation; therefore, initial controller development aimed at better understanding the current limitations of the model can be completed on the model presented here.

Table 5.39 shows the normalized RMS error between the simulated and measured output signals shown in the previous figures.

Table 5.39 Normalized RMS Error for Complete Cool Mode Cycle System

Parameter	Normalized RMS Error
Hk_{ro}	0.0120
P_c	0.0734
Tc_{ao}	0.0406
Hc_{ro}	0.0439
Pv_{ri}	0.0711
Hv_{ri}	0.1144
$\dot{m}_{dot_v}$	0.0359
Pv_{ro}	0.0367
Pe	0.0669
Te_{ao}	-0.0610
He_{ro}	0.0042
Ptv_{ri}	0.0762
Htv_{ri}	0.0220
Pk_{ri}	0.0796
Hk_{ri}	0.0148

While the normalized RMS error does provide a method for quantifying the relative error between the measured and predicted output signals for each model at one operating condition over another, it is difficult to use as an absolute metric. That is, we cannot easily define an upper bound beyond which the model is no longer a good predictor of dynamic behavior. One method that could be applied is the use of a performance metric, J , which is the weighted sum of the individual normalized RMS errors for each signal of interest.

$$J = c_1 e_{RMS, Hk_{ro}} + c_2 e_{RMS, P_c} + c_3 e_{RMS, Tc_{ao}} + \dots \quad (5.3)$$

By first choosing each of the weights, c_i , to equal 1, the user can gain an understanding of how the performance metric measures the effectiveness of the model in predicting system behavior. From there, the weights can be adjusted heuristically so that some errors are more heavily weighted than others. For example, if it is more critical that the evaporator predict the correct outlet air temperature than the correct pressure, then the weight on the evaporator air temperature should be larger. With experience, a person proficient in the model will be able to customize each of the weights in the performance metric as well as determine a reasonable maximum cost, J_{max} , which can be used as an upper bound on the error that can be tolerated.

Chapter 6

HEAT MODE MODEL VALIDATION

6.1 Individual Component Validation

Each component in the Heat Mode model was validated individually using both steady state and transient data. For the steady state case, each model was tested with data collected at a range of operating conditions as per the data collection procedure described in Chapter 2. For the transient case, each model was tested with data collected at three different operating conditions: 70|35 °F box|ambient, 70|0 °F box|ambient, and 35|0 °F box|ambient. Recall that because the EEV is isolated from the refrigeration loop during Heat Mode, the transient data was collected by stepping the compressor speed between the high and low speed settings.

6.1.1 Compressor

The compressor model predicts its outlet mass flow rate and outlet refrigerant enthalpy. The inputs and outputs of this model were described in Table 5.1 and Table 5.2, respectively. During Heat Mode, the compressor mass flow rate, rather than the EEV flow rate, is measured on the experimental test unit. Therefore, we can directly validate the outputs of the compressor model.

Figure 6.1 shows the Thermosys representation of the compressor model in Matlab Simulink. As described in Chapter 5, the yellow tags represent input parameters that were supplied to the model from each of the validation data sets.

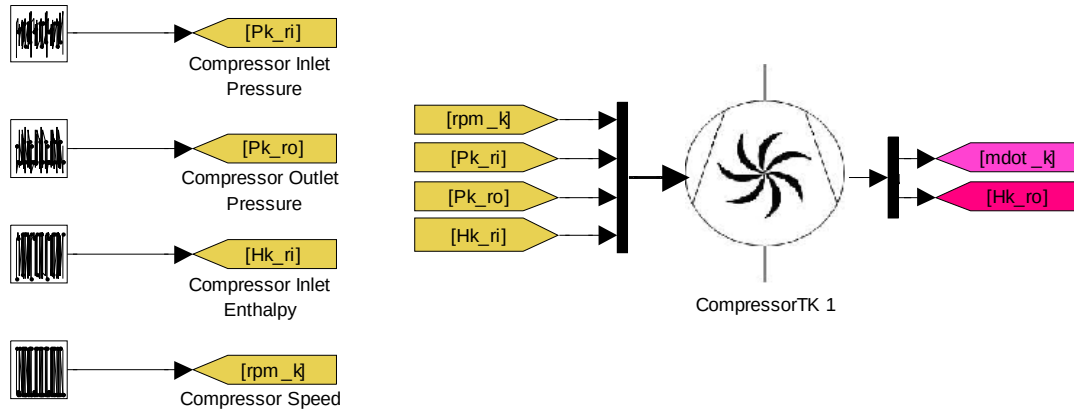


Figure 6.1: Thermosys Simulink model for compressor validation.

Table 6.1 and Table 6.2 show the validation results for the compressor model using steady state data.

Table 6.1 Steady State Validation Results for Compressor Mass Flow Rate

Operating Condition Box/Amb (°F)	Test Number	Measured <i>mdot_k</i> (kPa)	Simulated <i>mdot_k</i> (kPa)	Percent Error (%)
35 0	TR16319-017-B	0.0656	0.0624	1.05
35 -20	TR16319-018-A	0.0992	0.0901	1.10
50 -20	TR16319-022-A	0.0668	0.0592	1.13
70 -20	TR16319-024-A	0.1003	0.0926	1.08
50 0	TR16319-029-A	0.0659	0.0601	1.10
50 35	TR16319-035-A	0.0936	0.0931	1.01

Table 6.2 Steady State Validation Results for Compressor Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Measured <i>Hk_ro</i> (kJ/kg)	Simulated <i>Hk_ro</i> (kJ/kg)	Percent Error (%)
35 0	TR16319-017-B	235.8	234.7	0.47
35 -20	TR16319-018-A	238.7	234.7	1.68
50 -20	TR16319-022-A	236.3	257.8	-9.10
70 -20	TR16319-024-A	248.2	256.2	-3.22
50 0	TR16319-029-A	245.9	259.4	-5.49
50 35	TR16319-035-A	267.9	257.7	3.81

Figure 6.2, Figure 6.3, and Figure 6.4 show the transient validation results for the compressor model. In each figure, the measured output signals are compared with the predicted output signals at a particular operating condition.

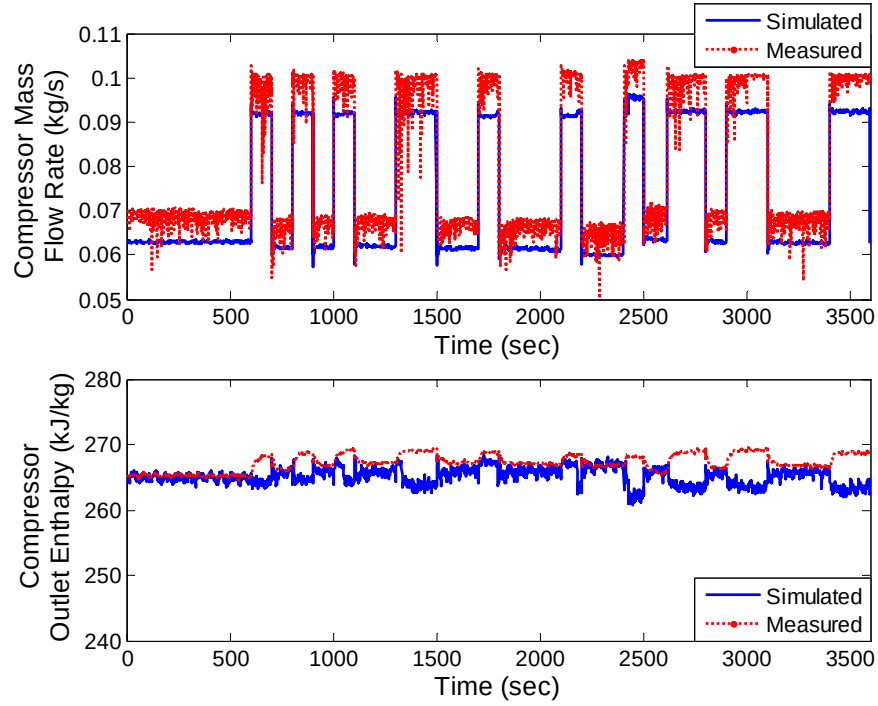


Figure 6.2: Compressor transient validation results at 70|35 °F.

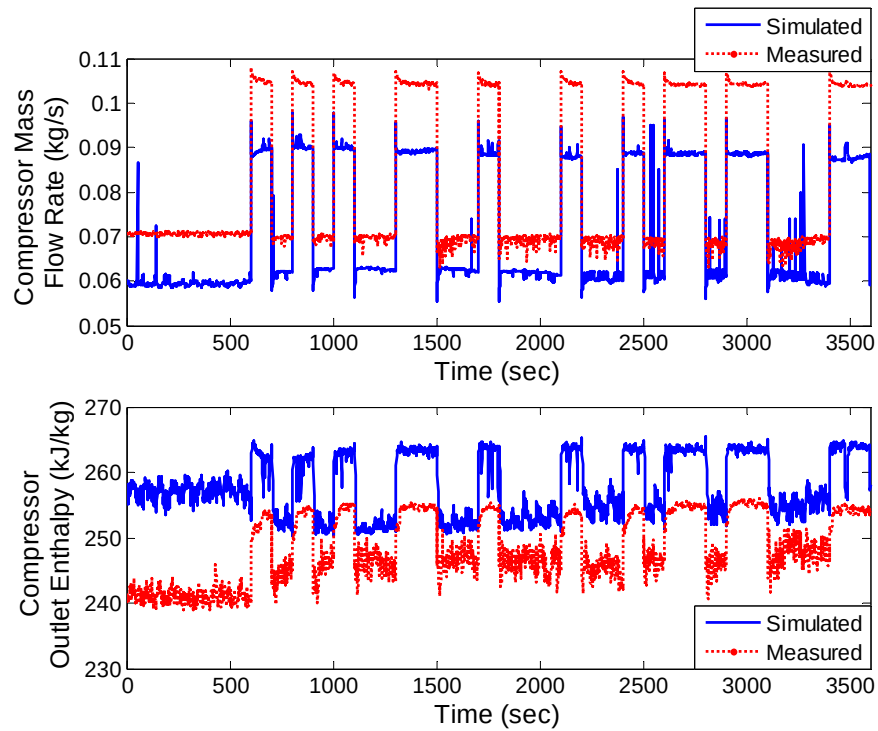


Figure 6.3: Compressor transient validation results at 70|0 °F.

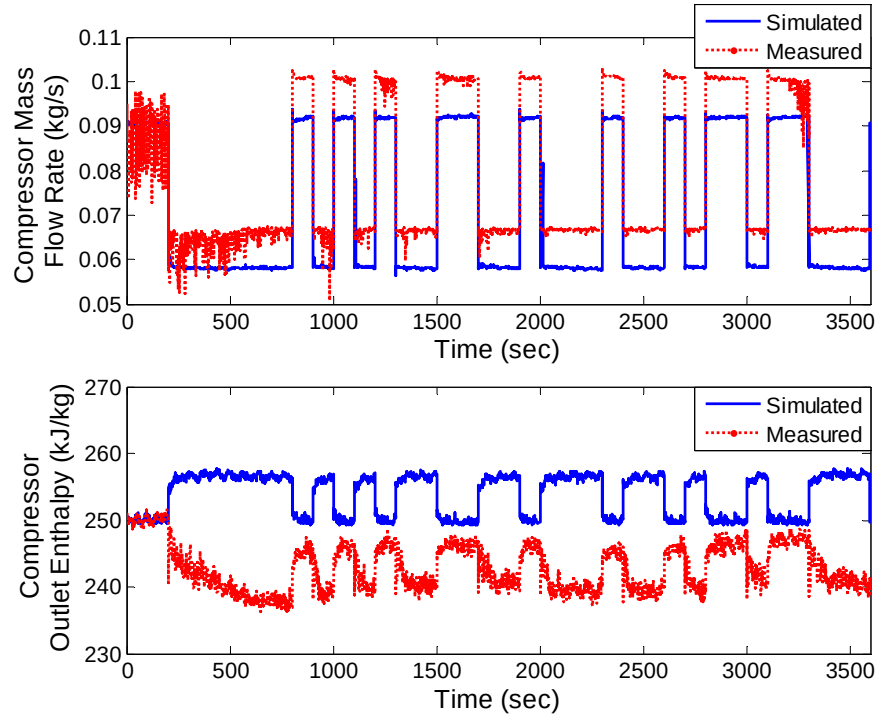


Figure 6.4: Compressor transient validation results at 35°F.

Qualitatively, the compressor model accurately predicts the dynamics of the mass flow rate of refrigerant through the compressor. However, the dynamics of the compressor outlet enthalpy do not match between the simulation and the experimental system. For increases in compressor speed, one would expect an increase in refrigerant outlet enthalpy; this is confirmed by the data. However, the model predicts the reverse; i.e. for step increases in compressor speed, there is a decrease in refrigerant outlet enthalpy at two of the three operating conditions tested. Moreover, the model over-predicts the enthalpy at the compressor outlet at two of the three operating conditions.

There are two possible explanations for the phase and magnitude difference between the measured and simulated enthalpy signals. First, the compressor model assumes isentropic compression which will result in an under-prediction of the outlet refrigerant enthalpy. Second, the compressor model does not account for heat loss through the shell of the compressor which would result in a higher outlet enthalpy than that measured on the experimental system. It is possible that at particular operating conditions, there is a larger over-prediction of refrigerant enthalpy than under-prediction, resulting in a net increase in enthalpy when in fact, the enthalpy decreases. Future work

should include validation at additional operating conditions to better understand the limitations of the model.

Table 6.3 shows the normalized RMS error between the simulated and measured compressor output signals at each of the transient testing conditions.

Table 6.3 Normalized RMS Error for Compressor Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS	
	$\dot{m}_{dot_k}$	Hk_ro
70 35	0.0555	0.0112
70 0	0.1184	0.0462
35 0	0.0811	0.0600

The normalized RMS error is useful in assessing the performance of a model at one operating condition over another. From Table 6.3 it seems that the model is most accurate at the 70|35 °F operating condition. However, qualitatively, the model is most accurate at the 70|0 °F operating condition.

6.1.2 Discharge Pressure Regulator Valve

The discharge pressure regulator (DPR) valve predicts its *inlet* pressure and its *outlet* enthalpy. Table 6.4 and Table 6.5 describe the inputs and outputs, respectively, of the DPR valve model.

Table 6.4 DPR Valve Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$\dot{m}_{dpr}$	mdot_dpr	Refrigerant mass flow rate through DPR valve	kg/s
$P_{dpr,ro}$	Pdpr_ro	Pressure at DPR valve outlet	kPa
$H_{dpr,ri}$	Hdpr_ri	Enthalpy at DPR valve inlet	kJ/kg

Table 6.5 DPR Valve Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{dpr,ri}$	Pdpr_ri	Pressure at DPR valve inlet	kPa
$H_{dpr,ro}$	Hdpr_ro	Enthalpy at DPR valve outlet	kJ/kg

Figure 6.5 shows the Thermosys representation of the DPR valve model in Matlab Simulink.

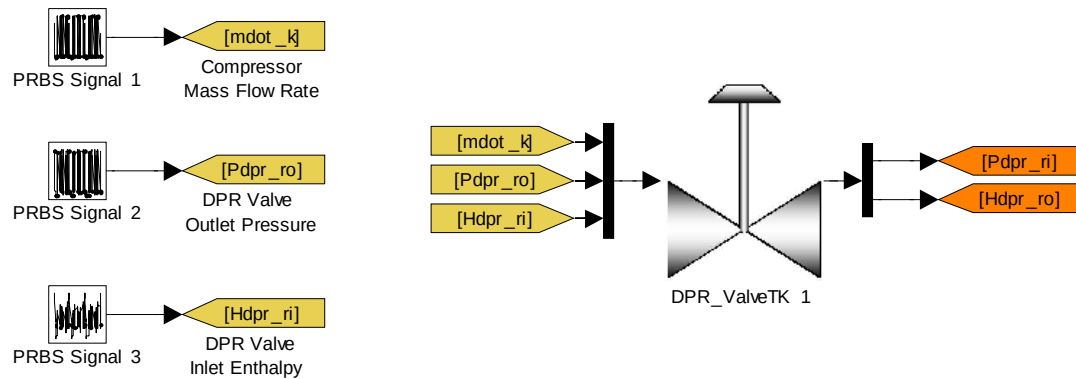


Figure 6.5: Thermosys Simulink model for DPR valve validation.

Table 6.6 and Table 6.7 show the steady state validation results for the DPR valve model.

Table 6.6 Steady State Validation Results for DPR Valve Inlet Pressure

Operating Condition Box/Amb (°F)	Test Number	Measured P_{dpr_ri} (kPa)	Simulated P_{dpr_ri} (kPa)	Percent Error
35 0	TR16319-017-B	2190.331	2386.7	8.97
35 -20	TR16319-018-A	2194.465	2156.1	1.75
50 -20	TR16319-022-A	2173.106	2422	11.45
70 -20	TR16319-024-A	2217.202	2328.1	5.00
50 0	TR16319-029-A	2215.824	2360.5	6.53
50 35	TR16319-035-A	2319.863	2329.6	0.42

Table 6.7 Steady State Validation Results for DPR Valve Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Measured H_{dpr_ro} (kJ/kg)	Simulated H_{dpr_ro} (kJ/kg)	Percent Error
35 0	TR16319-017-B	232.30	248.9	7.15
35 -20	TR16319-018-A	238.70	184.7	22.62
50 -20	TR16319-022-A	230.90	257.8	11.65
70 -20	TR16319-024-A	242.40	256.2	5.69
50 0	TR16319-029-A	239.70	259.4	8.22
50 35	TR16319-035-A	256.97	257.7	0.28

Figure 6.6 and Figure 6.7 compare the measured transient output signals with the predicted transient output signals for the DPR valve model at different operating conditions.

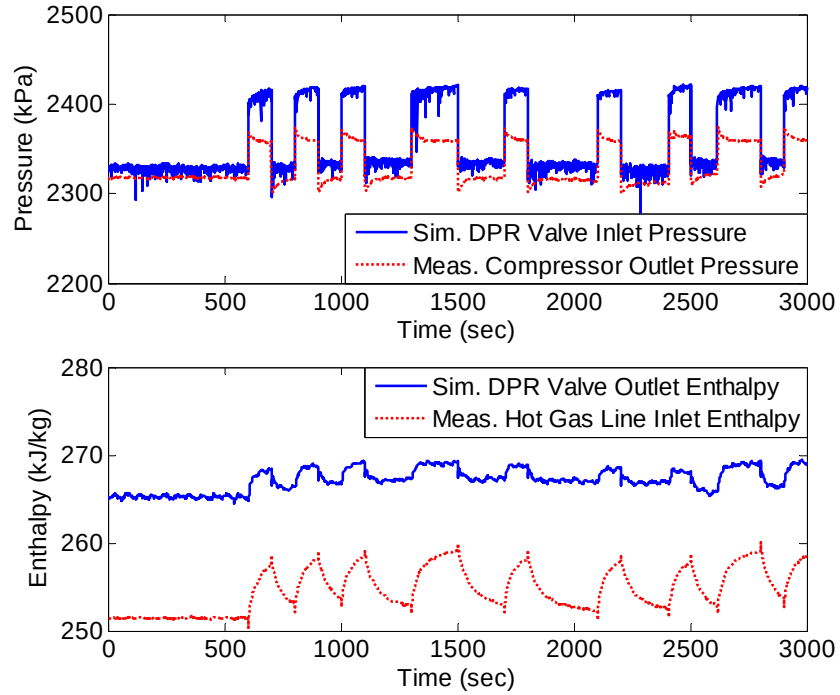


Figure 6.6: DPR valve transient validation results at 70|35 °F.

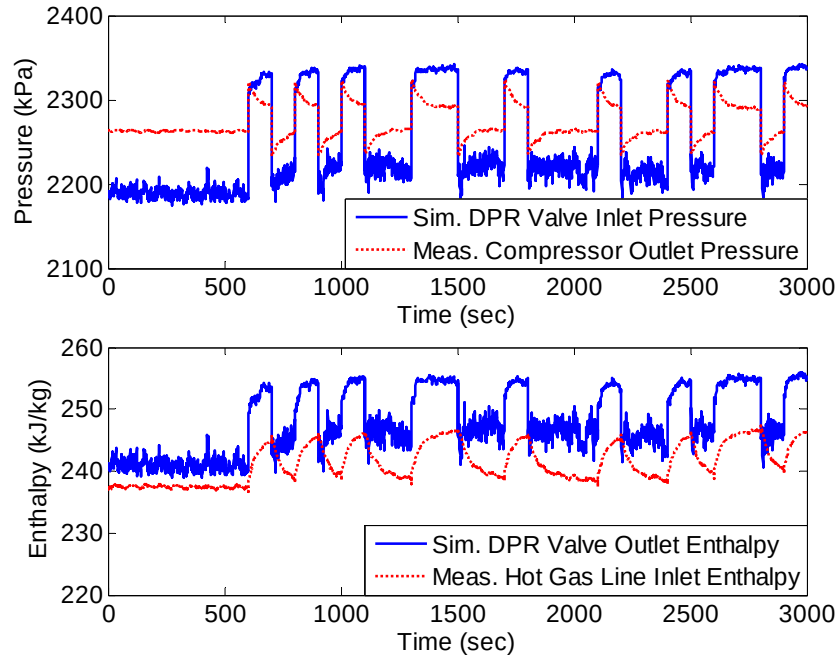


Figure 6.7: DPR valve transient validation results at 70|0 °F.

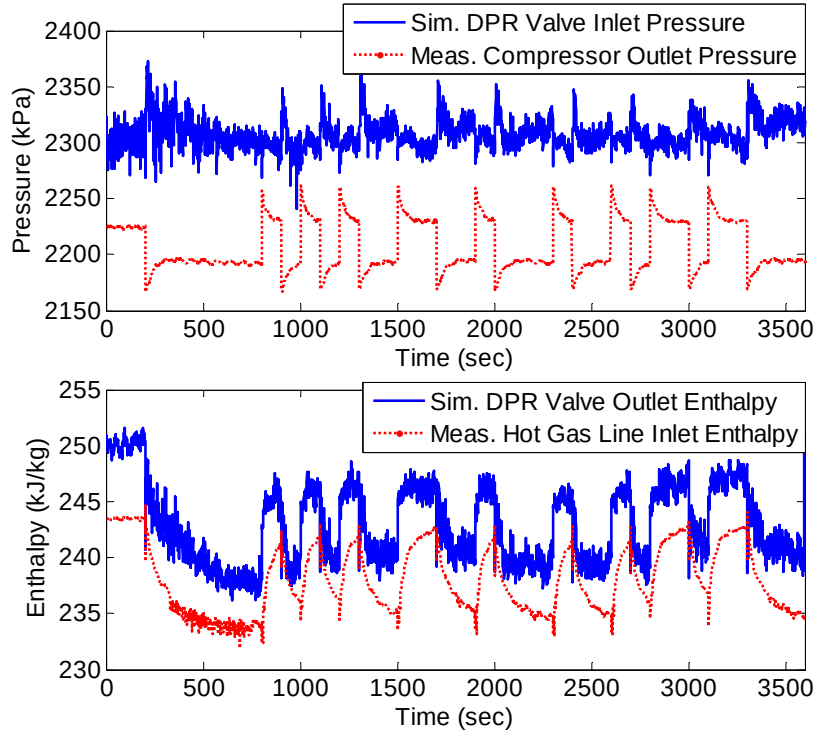


Figure 6.8: DPR valve transient validation results at 35|0 °F.

Table 6.8 shows the normalized RMS error between the simulated and measured DPR valve output signals at each of the transient testing conditions. The current model assumes constant enthalpy across the valve, but the transient data indicates that this is not an accurate assumption. The measured enthalpy signals at each operating condition appear to be filtered versions of the simulated enthalpy signals, suggesting that there is a first-order dynamic that is being neglected. This could be the result of unmodeled heat transfer between the compressor outlet and hot gas line inlet, a first-order dynamic which would cause a change in enthalpy. Although not critical for the purpose of this model validation, future work could improve the transient accuracy of the DPR valve model by identifying and incorporating this dynamic effect.

Table 6.8 Normalized RMS Error for DPR Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS	
	<i>Pk_ro</i>	<i>Hdpr_ro</i>
70 35	0.0147	0.0494
70 0	0.0213	0.0299
35 0	0.0477	0.0215

6.1.3 Hot Gas Line

The hot gas line predicts its *inlet* refrigerant pressure and its *outlet* refrigerant enthalpy. Table 6.9 and Table 6.10 list the inputs and outputs, respectively, of the hot gas line model.

Table 6.9 Hot Gas Line Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{g,ro}$	Pg_ro	Pressure at Hot Gas Line outlet	kPa
$H_{g,ri}$	Hg_ri	Enthalpy at Hot Gas Line inlet	kJ/kg
$\dot{m}_g$	mdot_g	Refrigerant mass flow rate through Hot Gas Line	kg/s
$T_{e,ai}$	Te_ai	Air temperature at evaporator inlet	°C

Table 6.10 Hot Gas Line Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{g,ri}$	Pg_ri	Pressure at Hot Gas Line inlet	kPa
$H_{g,ro}$	Hg_ro	Enthalpy at Hot Gas Line outlet	kJ/kg

Figure 6.9 shows the Thermosys representation of the hot gas line model in Matlab Simulink.

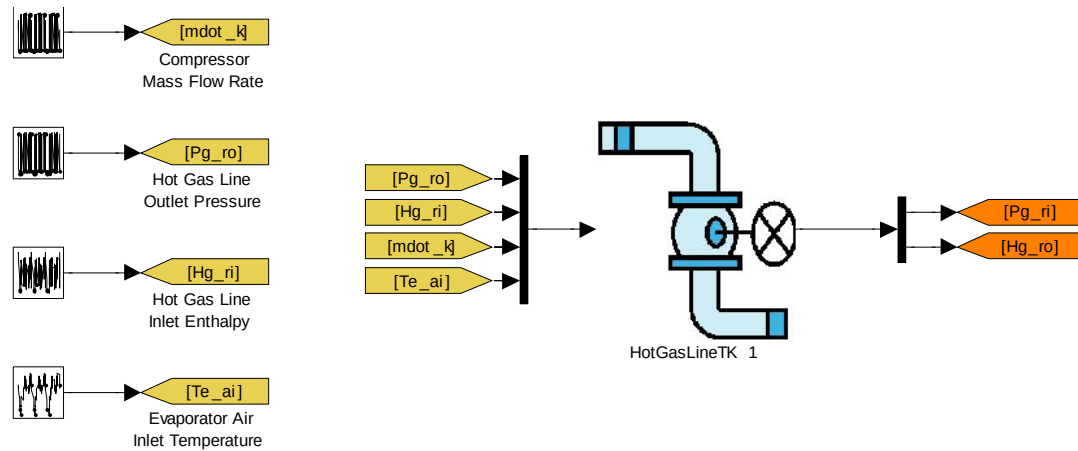


Figure 6.9: Thermosys Simulink model for hot gas line validation.

Table 6.11 and Table 6.12 show the steady state validation results for the hot gas line model.

Table 6.11 Steady State Validation Results for Hot Gas Line Inlet Pressure

Operating Condition Box/Amb (°F)	Test Number	Measured <i>Pg_ri</i> (kPa)	Simulated <i>Pg_ri</i> (kPa)	Percent Error (%)
35 0	TR16319-017-B	824.26	871.02	-5.67
35 -20	TR16319-018-A	1013.79	1081.40	-6.67
50 -20	TR16319-022-A	857.06	915.23	-6.79
70 -20	TR16319-024-A	1216.47	1288.30	-5.90
50 0	TR16319-029-A	943.60	994.70	-5.42
50 35	TR16319-035-A	1188.86	1236.10	-3.97

Table 6.12 Steady State Validation Results for Hot Gas Line Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Measured <i>Hg_ro</i> (kJ/kg)	Simulated <i>Hg_ro</i> (kJ/kg)	Percent Error (%)
35 0	TR16319-017-B	227.62	226.09	0.67
35 -20	TR16319-018-A	231.60	227.23	1.89
50 -20	TR16319-022-A	227.90	225.20	1.18
70 -20	TR16319-024-A	237.79	233.43	1.84
50 0	TR16319-029-A	233.79	230.70	1.32
50 35	TR16319-035-A	248.23	247.90	0.13

Figure 6.10, Figure 6.11, and Figure 6.12 compare the measured transient output signals with the predicted transient output signals for the hot gas line model at different operating conditions.

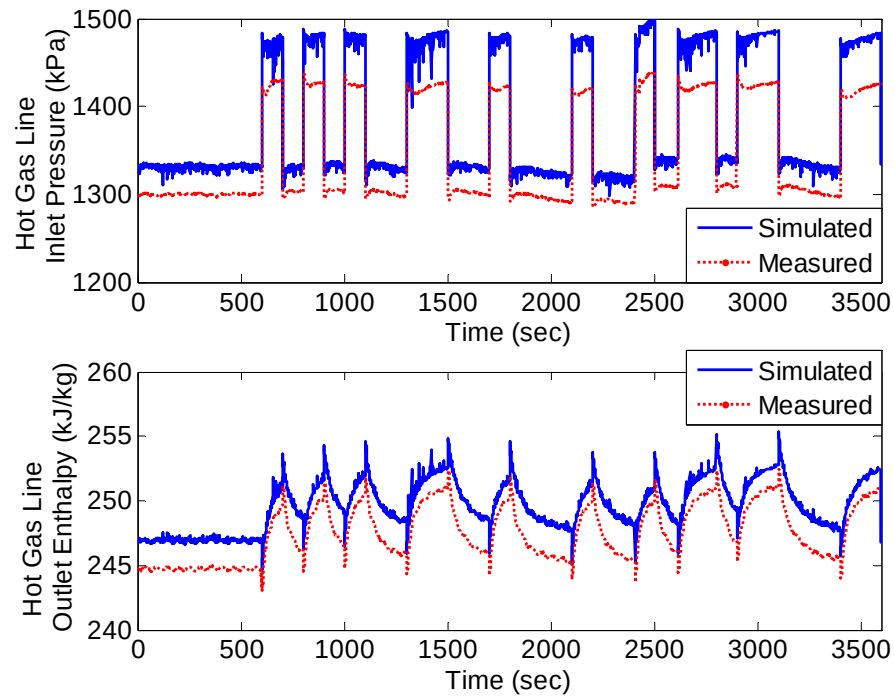


Figure 6.10: Hot Gas Line transient validation results at 70|35 °F.

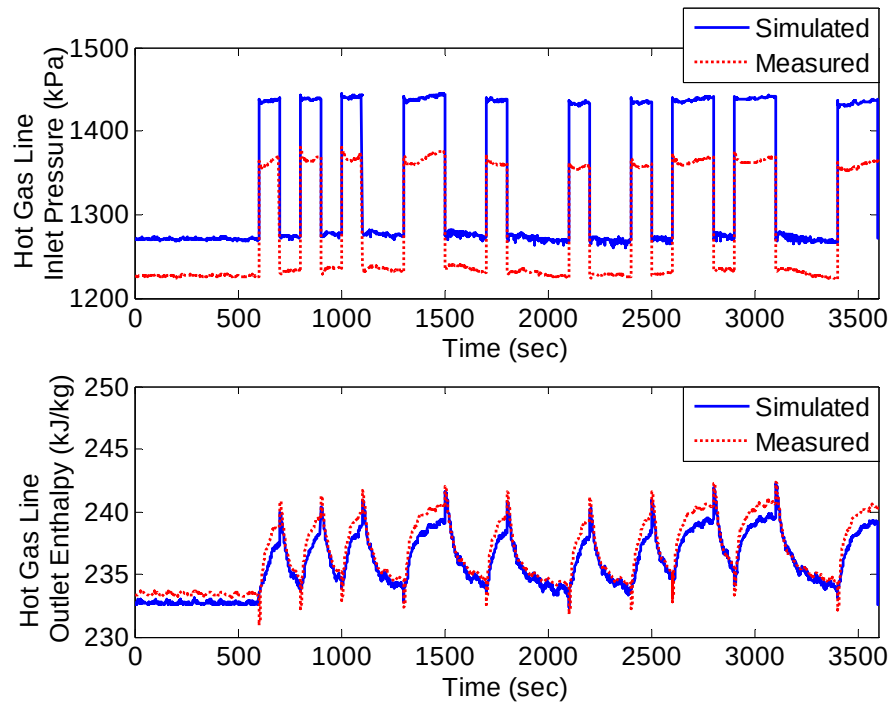


Figure 6.11: Hot Gas Line transient validation results at 70|0 °F.

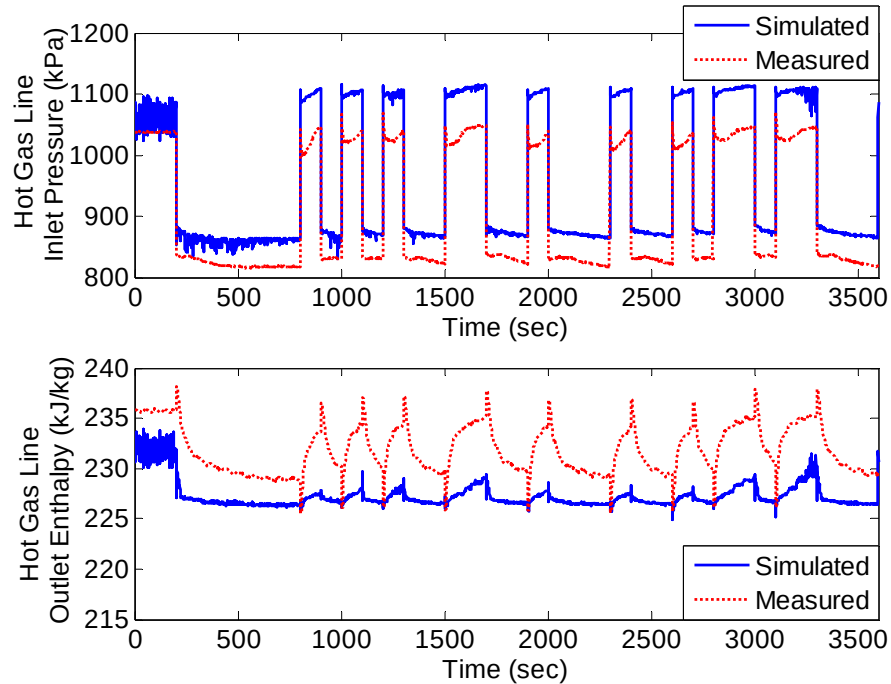


Figure 6.12: Hot Gas Line transient validation results at 35|0 °F.

Qualitatively, the hot gas line model accurately predicts the dynamics of both its inlet refrigerant pressure and outlet refrigerant enthalpy. There appears to be more inaccuracy in the prediction of outlet enthalpy at the 35|0 °F condition than at the other two conditions tested. This could be a result of the way information flows through this model. That is to say, there is an upstream flow of pressure information between the evaporator and compressor model, but a downstream flow of enthalpy information. This is due to the absence of a condenser in the Heat Mode model. For this reason, the hot gas line is required to compute its refrigerant density based on an interpolation using its *inlet* pressure and *outlet* enthalpy, values. At conditions where there is a considerable amount of heat transfer across the hot gas line, this can result in a poor estimate of density.

Table 6.13 shows the normalized RMS error between the simulated and measured hot gas line output signals at each of the transient testing conditions.

Table 6.13 Normalized RMS Error for Hot Gas Line Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS	
	Pg_{ri}	Hg_{ro}
70 35	0.0306	0.0089
70 0	0.0443	0.0040
35 0	0.0605	0.0210

6.1.4 Distributor

The distributor model predicts the pressure of the refrigerant entering the distributor. The inputs and outputs of this model were described in Table 5.15 and Table 5.16, respectively. Figure 6.13 shows the Thermosys representation of the distributor model in Matlab Simulink.

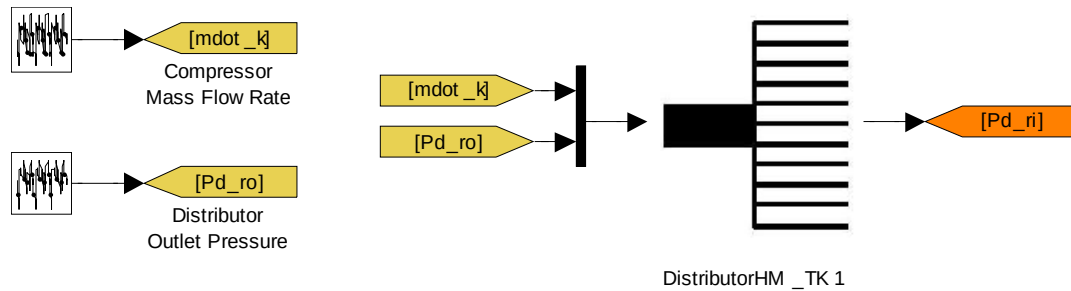
**Figure 6.13: Thermosys Simulink model for distributor validation.**

Table 5.17 shows the results of the steady state validation for the distributor model. Note that the Pg_{ro} denotes hot gas line outlet refrigerant pressure, i.e. the refrigerant pressure at the inlet of the distributor.

Table 6.14 Steady State Validation Results for Distributor

Operating Condition Box/Amb (°F)	Test Number	Measured Pg_{ro} (kPa)	Simulated Pg_{ro} (kPa)	Percent Error
35 0	TR16319-017-B	782.4	781.6	0.10
35 -20	TR16319-018-A	908.1	907.8	0.04
50 -20	TR16319-022-A	830.5	829.0	0.18
70 -20	TR16319-024-A	1149.2	1147.5	0.14
50 0	TR16319-029-A	916.7	916.1	0.07
50 35	TR16319-035-A	1096.8	1097.4	-0.05

Figure 6.14, Figure 6.15, and Figure 6.16 compare the measured transient output signals with the predicted transient output signals for the distributor model at different operating conditions.

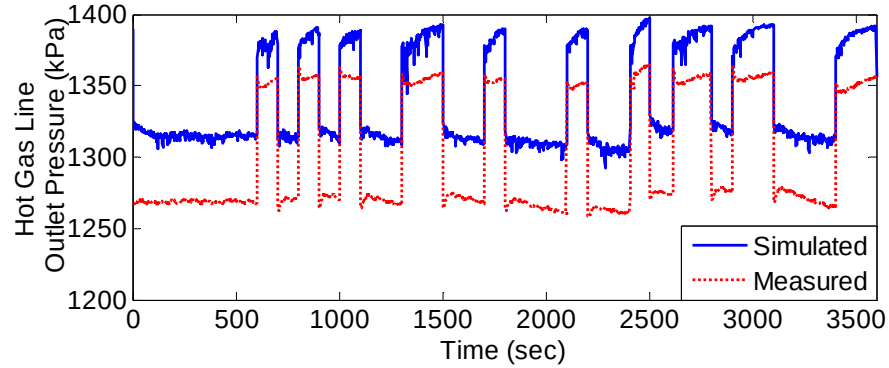


Figure 6.14: Distributor transient validation results at 70|35 °F.

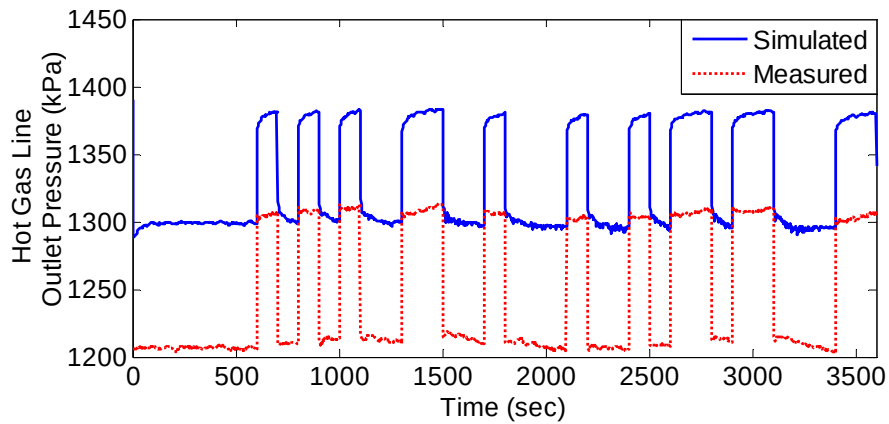


Figure 6.15: Distributor transient validation results at 70|0 °F with detrend constants based on the 70|35 °F operating condition.

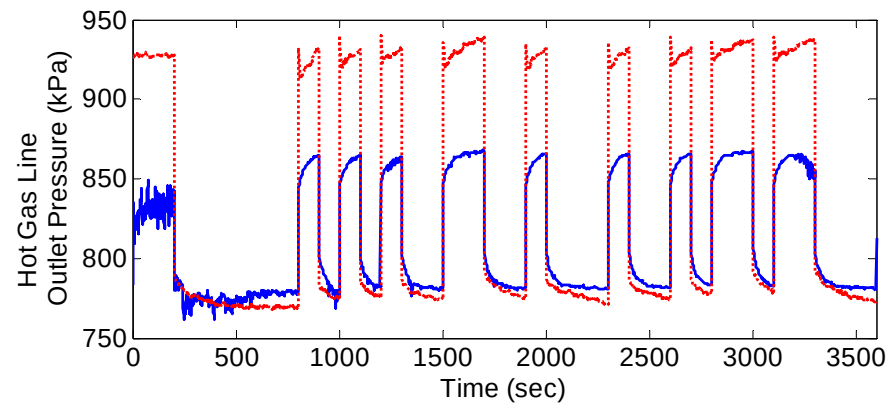


Figure 6.16: Distributor transient validation results at 35|0 °F.

Qualitatively, the distributor model accurately predicts the dynamics of the distributor inlet refrigerant pressure. At 70|35 °F and 70|0 °F, there is a constant offset between the measured and simulated output signals. At the 35|0 °F operating condition, there appears to be a constant scaling factor between the measured and simulated output signals. These discrepancies are a direct result of the *detrend* that was performed on the identification data set. The model has been developed so different *detrend* constants are used at different operating conditions. At steady state this results in a very accurate model. While the transient results are not as accurate, the percentage error between the measured and simulated signals at each of the operating conditions tested remains within 10%.

Table 6.15 shows the normalized RMS error between the simulated and measured throttle valve output signals at each of the transient testing conditions.

Table 6.15 Normalized RMS Error for Distributor Transient Validation

Operating Condition Box/Amb (°F)	Normalized RMS Error
	<i>Pg_ro</i>
70 35	0.0310
70 0	0.0671
35 0	0.0553

Based on Table 6.15, the model is most accurate at the 70|35 °F operating condition. This is expected given that the distributor model was identified using data collected at that operating condition.

6.1.5 Evaporator and Accumulator

As in Cool Mode, the Heat Mode evaporator model predicts the pressure inside the evaporator coil, the enthalpy of the refrigerant exiting the evaporator, and the temperature of the air as it exits the evaporator. Recall that during Heat Mode operation, the evaporator behaves like a condenser, with superheated vapor entering the heat exchanger and then leaving as a two-phase refrigerant. Nevertheless, the inputs and outputs of this model remain the same as those of the Cool Mode evaporator, and were described in Table 5.19 and Table 5.20, respectively.

The Heat Mode accumulator model predicts the pressure and enthalpy of the refrigerant exiting the accumulator. During Heat Mode, the accumulator is used as a separation device and consequently, a different set of inputs and outputs (from those of the Cool Mode accumulator) are needed to run the model. The inputs and outputs of the Heat Mode accumulator model are described in Table 6.16 and Table 6.17, respectively.

Table 6.16 Heat Mode Accumulator Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$H_{a,ri}$	Ha_ri	Enthalpy at accumulator inlet	kJ/kg
$\dot{m}_a$	mdot_i	Mass flow rate at accumulator inlet	kg/s
$\dot{m}_a$	mdot_o	Mass flow rate through accumulator	kg/s
T_{amb}	Tamb	Temperature of air surrounding accumulator	°C

Table 6.17 Heat Mode Accumulator Model Outputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
$P_{a,ri}$	Pa_ro	Pressure at accumulator outlet	kPa
$H_{a,ri}$	Ha_ro	Enthalpy at accumulator outlet	kJ/kg

The evaporator and accumulator models are unique in that they require *both* their inlet and outlet mass flow rates as inputs. However, on the experimental system, only the mass flow rate at the exit of the compressor was measured. At steady state this is not a problem; the inlet and mass flow rates for any component will be equal, and so steady state validation results can be collected for the evaporator and accumulator models individually. However, for transient validation, it is necessary to validate these models together, connected with a hydraulic resistance pipe model which estimates the “missing” mass flow rate measurement.

Figure 6.17 shows the Thermosys representation of the evaporator and accumulator model subsystem in Matlab Simulink.

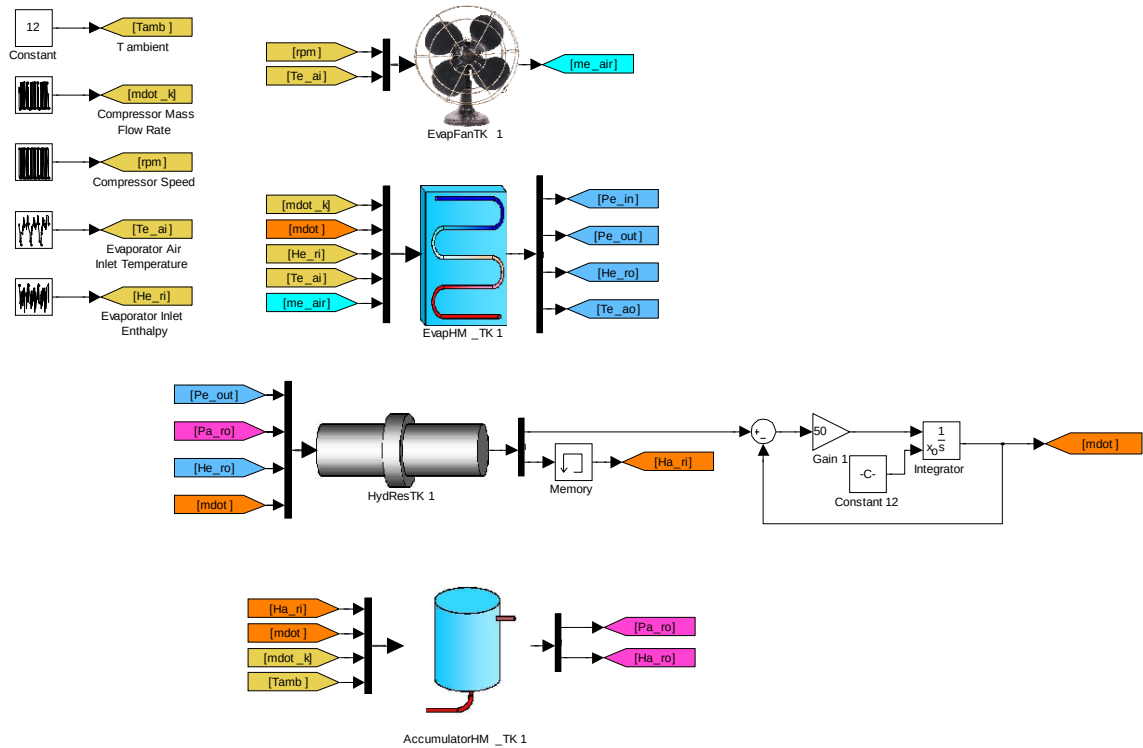


Figure 6.17: Thermosys Simulink model for evaporator and accumulator validation.

Table 6.18 and Table 6.19 show the steady state validation results for the evaporator model. Note that the evaporator model was validated at only three conditions. This is because after the data collection was completed, we noticed that superheated vapor was exiting the evaporator at some Heat Mode operating conditions; although there was a decrease in refrigerant enthalpy across the length of the evaporator, the outlet enthalpy was still higher than the saturated enthalpy at the evaporator pressure. Given that the Heat Mode evaporator model has been designed to have two fluid regions, it cannot run at operating conditions where the evaporator does not condense the superheated vapor refrigerant sufficiently to generate a two-phase region.

Table 6.18 Steady State Validation Results for Evaporator Pressure

Operating Condition Box/Amb (°F)	Test Number	Measured P_e (kPa)	Simulated P_e (kPa)	Percent Error (%)
70 35	TR16319-007-B	1187.28	1182.5	0.40
35 0	TR16319-017-B	667.41	667.2	0.03
50 35	TR16319-035-A	894.94	893.9	0.12

Table 6.19 Steady State Validation Results for Evaporator Outlet Air Temperature

Operating Condition Box/Amb (°F)	Test Number	Measured Te_{ao} (kJ/kg)	Simulated Te_{ao} (kJ/kg)	Percent Error (%)
70 35	TR16319-007-B	23.28	24.2	-3.95
35 0	TR16319-017-B	3.00	3.07	-2.33
50 35	TR16319-035-A	12.69	13.5	-6.38

Table 6.20 and Table 6.21 show the steady state validation results for the accumulator model. The accumulator model is difficult to validate accurately because it is dependent on three unknown values. The separation efficiency of the accumulator, η_{sep} , is a model parameter that is unknown but temporarily assumed to equal 98%. The initial refrigerant mass in the accumulator is an unknown initial condition that cannot be calculated; here we assume that there is initially 0.5 kg of refrigerant in the accumulator. Finally, the engine air temperature (i.e. the temperature of the air surrounding the accumulator tank) is an unknown input to the system; here we assume that it is approximately 30 °C larger than the actual ambient temperature. It is important to note that varying any one, or all three, of these parameters will affect the predictive capability of the model.

Table 6.20 Steady State Validation Results for Accumulator Pressure

Operating Condition Box/Amb (°F)	Test Number	Measured Pa (kPa)	Simulated Pa (kPa)	Percent Error (%)
70 35	TR16319-007-B	1172	1157	1.28
35 0	TR16319-017-B	652	690.4	-5.89
50 35	TR16319-035-A	900.46	835	7.27

Table 6.21 Steady State Validation Results for Refrigerant Temperature

Operating Condition Box/Amb (°F)	Test Number	Measured Ta_{ro} (°C)	Simulated Ta_{ro} (°C)	Percent Error (%)
70 35	TR16319-007-B	23.67	21.80	7.90
35 0	TR16319-017-B	3.67	3.76	-2.45
50 35	TR16319-035-A	13.28	10.10	24.0

Figure 6.18 and Figure 6.19 show the transient validation results for the evaporator and accumulator models, respectively, at the 70|35 °F operating condition. Figure 6.20 and Figure 6.21, respectively, show the transient validation results for the evaporator and accumulator models at the 35|0 °F operating condition.

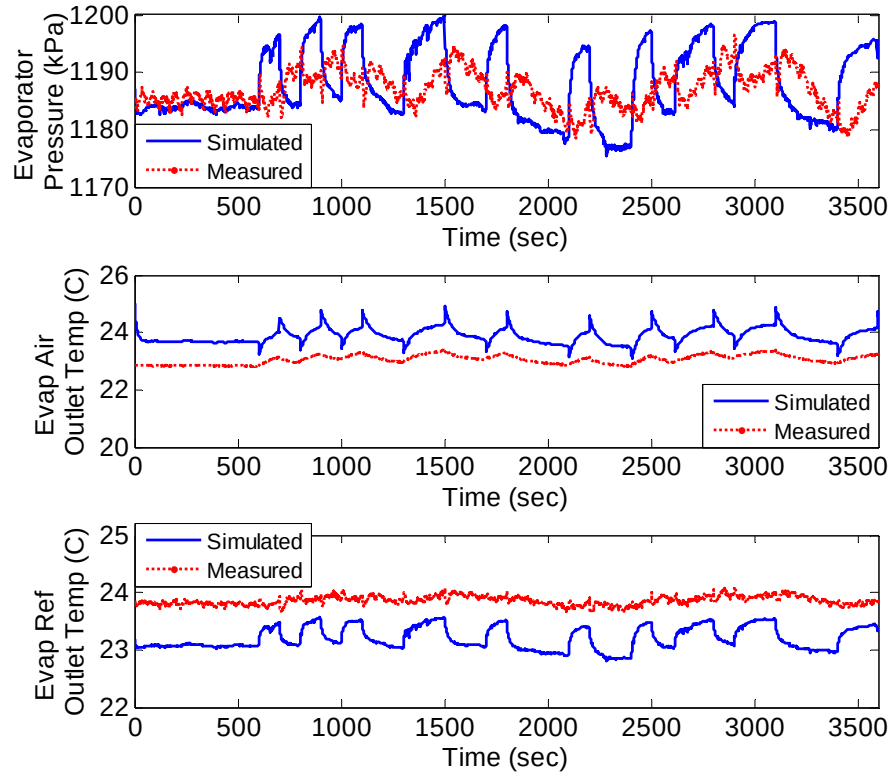


Figure 6.18: Evaporator transient validation results at 70|35 °F.

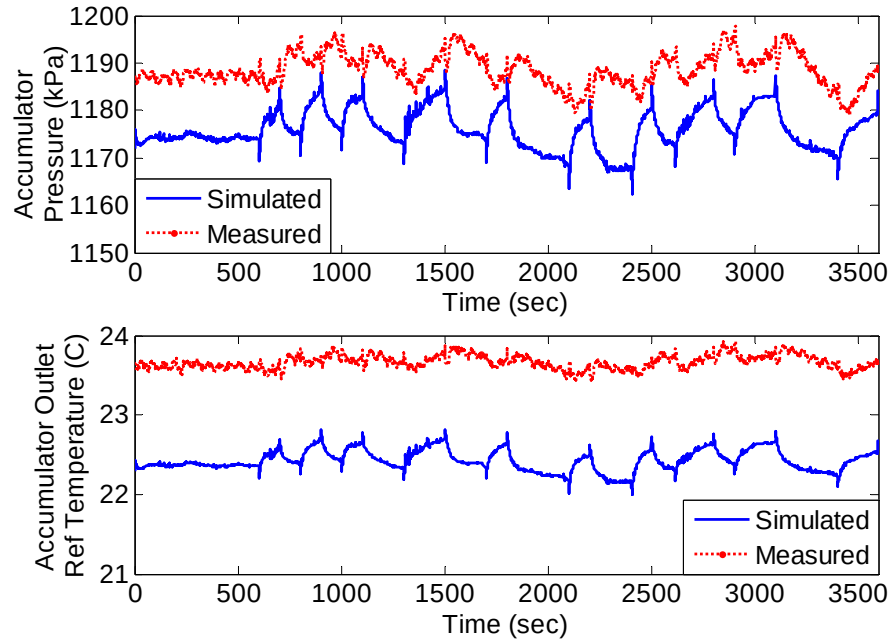


Figure 6.19: Accumulator transient validation results at 70|35 °F.

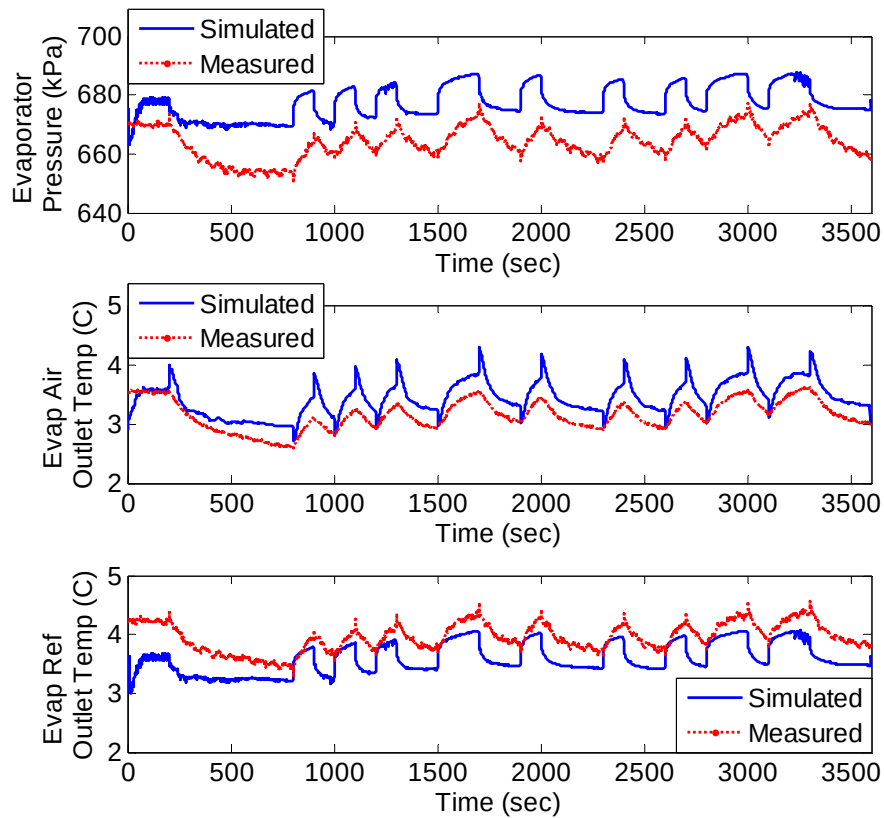


Figure 6.20: Evaporator transient validation results at 35|0 °F.

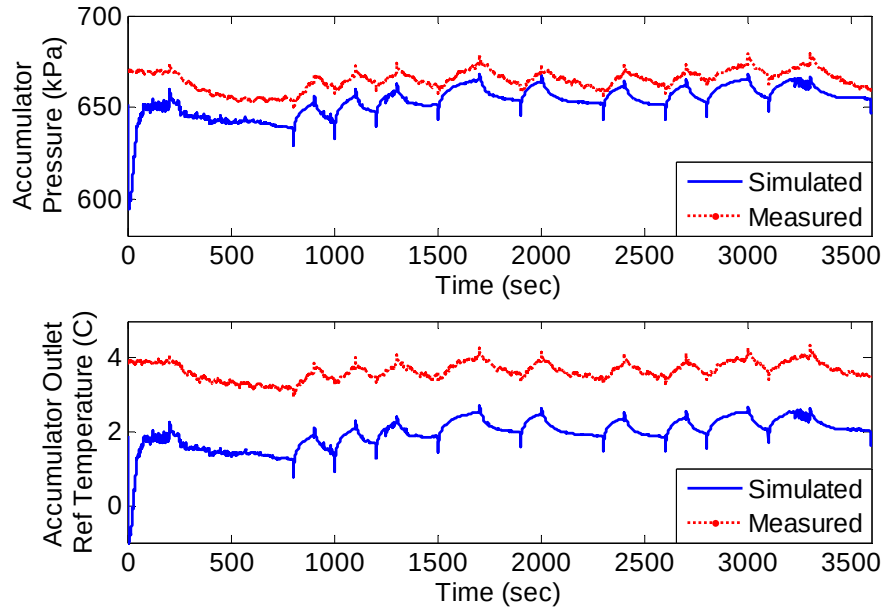


Figure 6.21: Accumulator transient validation results at 35[0 °F.

Qualitatively, the evaporator and accumulator models accurately predict the dynamics of the system. However, the dynamic response of the simulated signals appears to be faster than that of the measured signals. Recall from Chapter 4 that the evaporator was modeled as a single-circuit evaporator albeit its multi-circuit geometry. Modeling the evaporator in this way assumes that the heat transfer across each circuit is uniform, and the validation results suggest that this assumption is a reasonable one. However, decreasing the mass of the evaporator by $\frac{1}{n}$ results in a decrease in thermal inertia that may not be proportional to the decrease in surface area when considering a single-circuit of the evaporator. For example, consider a cube which is cut in half; although its mass has decreased by a factor of $\frac{1}{2}$, the surface area has not. This difference in scaling is one of many possible explanations for why the evaporator dynamics are faster in simulation than what is observed on the experimental system. Future work should continue an investigation of this phenomenon to ultimately improve the accuracy of the prediction of the outlet air temperature.

Table 6.22 shows the normalized RMS error between the simulated and measured evaporator and accumulator output signals at each of the transient testing conditions.

Table 6.22 Normalized RMS Error for Evaporator and Accumulator Transient Validation

Operating Condition Box/Amb (°F)	Normalized RMS Error				
	P_e	T_{e_ro}	T_{e_ao}	P_{a_ro}	T_{a_ro}
70 35	0.0058	0.0288	0.0369	0.0113	0.0528
35 0	0.0209	0.1058	0.1018	0.0204	0.4753

The normalized RMS error indicates that the evaporator and accumulator models are significantly more accurate at the 70|35 °F operating condition than at the 35|0 °F operating condition. However, from the figures above, it is clear that qualitatively, the results at each of the conditions are comparable. Without a performance metric, absolute RMS errors cannot provide any significant information about the performance of the model.

6.1.6 Throttle Valve

The throttle valve model predicts its refrigerant outlet pressure. The inputs and outputs of this model were described in Table 5.33 and Table 5.34, respectively. Figure 6.22 shows the Thermosys representation of the throttle valve model in Matlab Simulink. The throttle valve, and the pipe connecting it to the compressor inlet, are validated together because measurements of the pressure and temperature are available at the compressor inlet but not at the throttle valve outlet

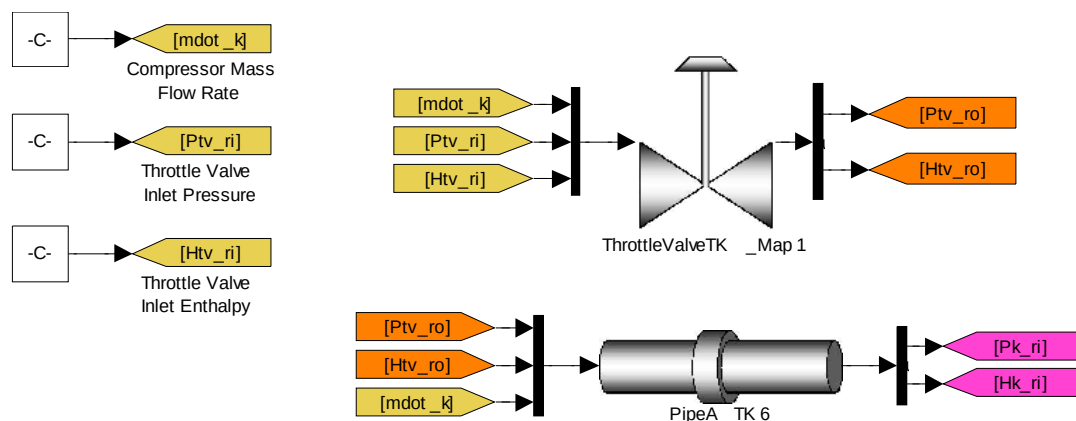


Figure 6.22: Thermosys Simulink model for throttle valve validation.

The steady state validation results are presented in Table 6.23 and Table 6.24. Note that Pk_{ri} and Hk_{ri} denote compressor inlet pressure and inlet enthalpy, respectively.

Table 6.23 Steady State Validation Results of Throttle Valve Outlet Pressure

Operating Condition Box/Amb (°F)	Test Number	Measured Pk_{ri} (kPa)	Simulated Pk_{ri} (kPa)	Percent Error (%)
35 0	TR16319-017-B	287.31	297.60	-3.58
35 -20	TR16319-018-A	267.33	296.00	-10.72
50 -20	TR16319-022-A	285.25	297.80	-4.40
70 -20	TR16319-024-A	287.31	297.70	-3.62
50 0	TR16319-029-A	290.07	298.10	-2.77
50 35	TR16319-035-A	289.38	297.70	-2.88

Table 6.24 Steady State Validation Results of Throttle Valve Outlet Enthalpy

Operating Condition Box/Amb (°F)	Test Number	Measured Hk_{ri} (kJ/kg)	Simulated Hk_{ri} (kJ/kg)	Percent Error (%)
35 0	TR16319-017-B	207.73	220.40	-6.10
35 -20	TR16319-018-A	221.00	219.90	0.50
50 -20	TR16319-022-A	214.49	222.00	-3.50
70 -20	TR16319-024-A	213.05	226.30	-6.22
50 0	TR16319-029-A	215.77	223.90	-3.77
50 35	TR16319-035-A	213.62	224.80	-5.23

Figure 6.23, Figure 6.24, and Figure 6.25 compare the measured transient output signals with the predicted transient output signals for the throttle valve model at different operating conditions.

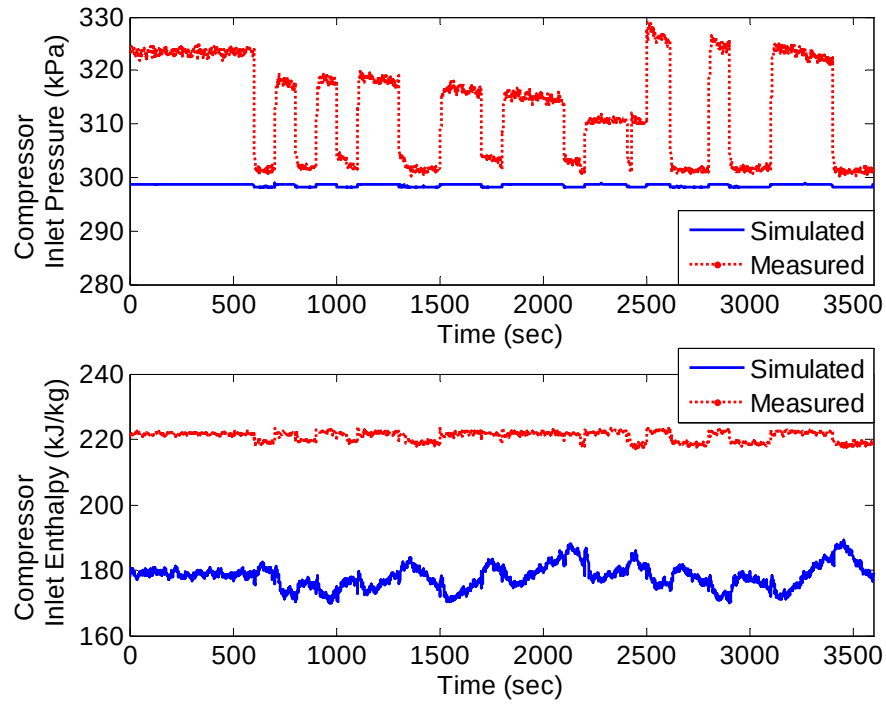


Figure 6.23: Throttle Valve transient validation results at 70|35 °F.

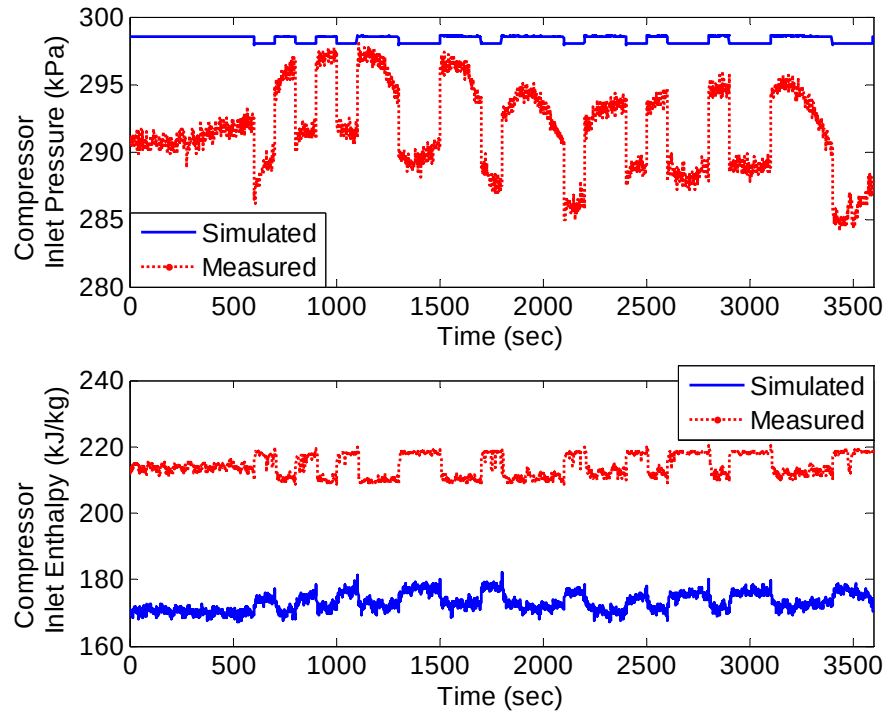


Figure 6.24: Throttle Valve transient validation results at 70|0 °F.

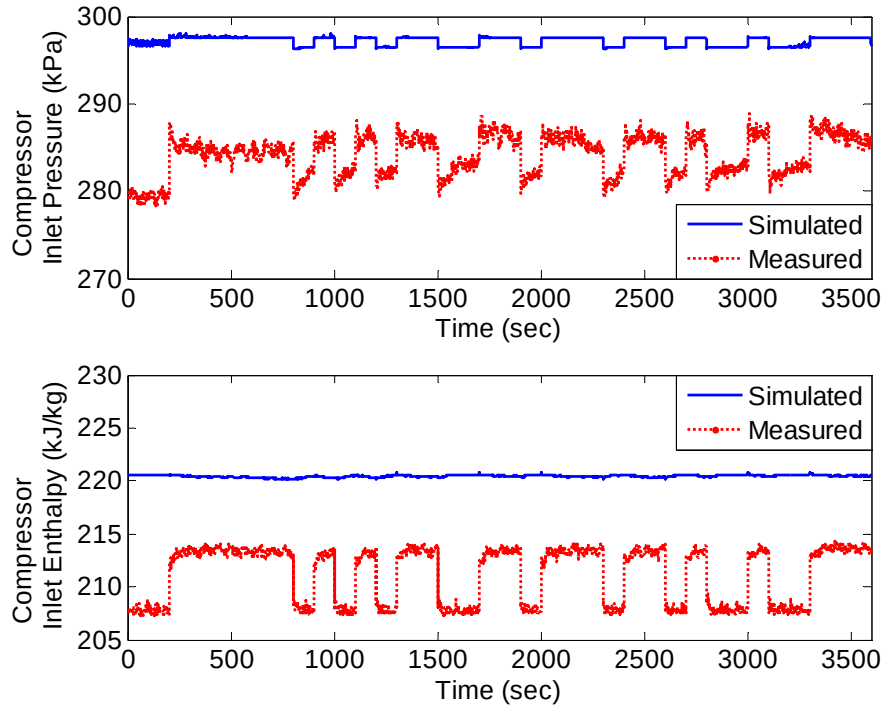


Figure 6.25: Throttle Valve transient validation results at 35|0 °F.

Qualitatively, the pressure dynamics predicted by the throttle valve are more subdued than those of the actual system. That is, for a given change in compressor speed, the actual pressure at the outlet of the throttle valve (i.e. at the inlet of the compressor) changed by at least 5 kPa at two of the three operating conditions and by at least 15 kPa at 70|35 °F. However, the model did not predict as large of a change.

The throttle valve model also assumes constant enthalpy across the valve. Recall that the same empirical throttle valve map is used in both the Cool and Heat Mode models. Based on the Cool Mode model validation presented in Chapter 5, the isenthalpic assumption was a reasonable one. However, during Heat Mode operation, the enthalpy across the valve changes significantly at some operating conditions. At 70|35 °F and 70|0 °F, the dynamics predicted by the model are accurate, but there is a shift between the measured and predicted signals. It is important to note that in the simulation that was conducted to validate the throttle valve model, the inlet enthalpy signal is the *measured* inlet enthalpy signal to the throttle valve. Given that the throttle valve and pipe model both assume constant enthalpy, the *simulated* inlet compressor enthalpy signal is in fact the *measured* inlet throttle valve enthalpy signal. This indicates that the difference

between the measured and simulated enthalpy signals (at each operating condition) is a result of some change in enthalpy that exists on the experimental system itself.

Futhermore, it is possible that some liquid refrigerant is expanding inside the valve; this would certainly affect the valve dynamics. Future work should include the development of a throttle valve map specifically for Heat Mode operating conditions as well as the identification of the heat transfer dynamics across the throttle valve.

Table 6.25 shows the normalized RMS error between the simulated and measured throttle valve output signals at each of the transient testing conditions.

Table 6.25 Normalized RMS Error for Throttle Valve Transient Validation

Operating Condition Box/Amb(°F)	Normalized RMS	
	<i>Pk_ri</i>	<i>Hk_ri</i>
70 35	0.0546	0.1949
70 0	0.0251	0.1919
35 0	0.0463	0.0459

6.2 Complete System Validation

In this section, the validation of the complete cycle system is presented. This validation was conducted at two different transient operating conditions: 70|35 °F and 35|0 °F box|ambient. Table 6.26 shows the inputs for the complete system model.

Table 6.26 Complete System Model Inputs

Parameter Name	Parameter Name in THERMOSYS	Description	Units
<i>rpm</i>	rpm	Compressor rotational speed	rev/min
<i>T_{e,ai}</i>	Te_ai	Temperature of air entering evaporator	°C
<i>T_{amb}</i>	T_amb	Temperature of ambient air	°C

Figure 6.26 shows the input signals used in the complete system validation at the 70|35 °F operating condition. This is followed by a set of figures comparing the simulated and measured transient output signals for each of the components in the Heat Mode model at 70|35 °F.

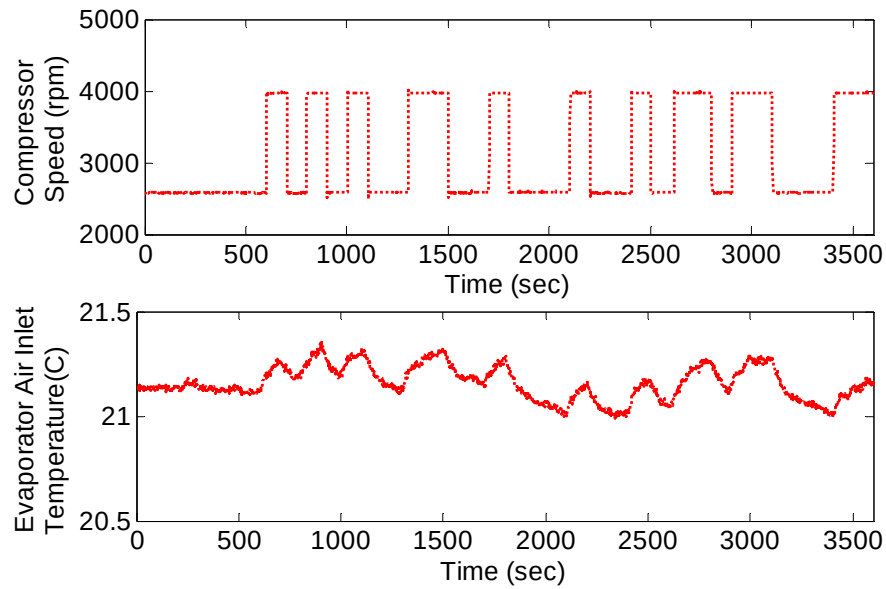


Figure 6.26: Complete cycle system transient inputs at 70|35 °F.

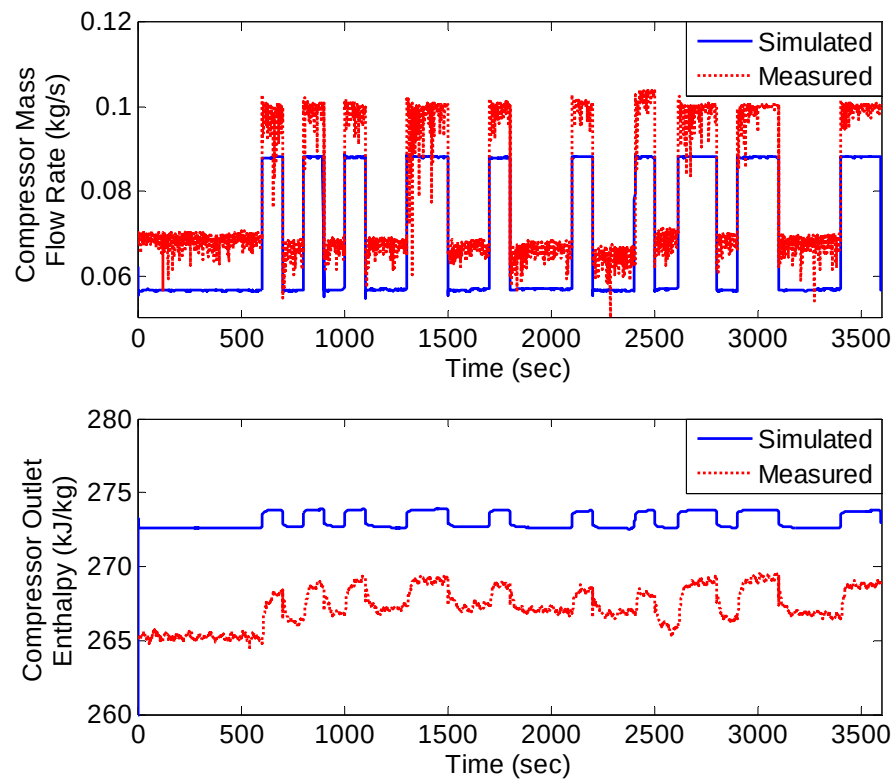


Figure 6.27: Compressor transient validation results at 70|35 °F.

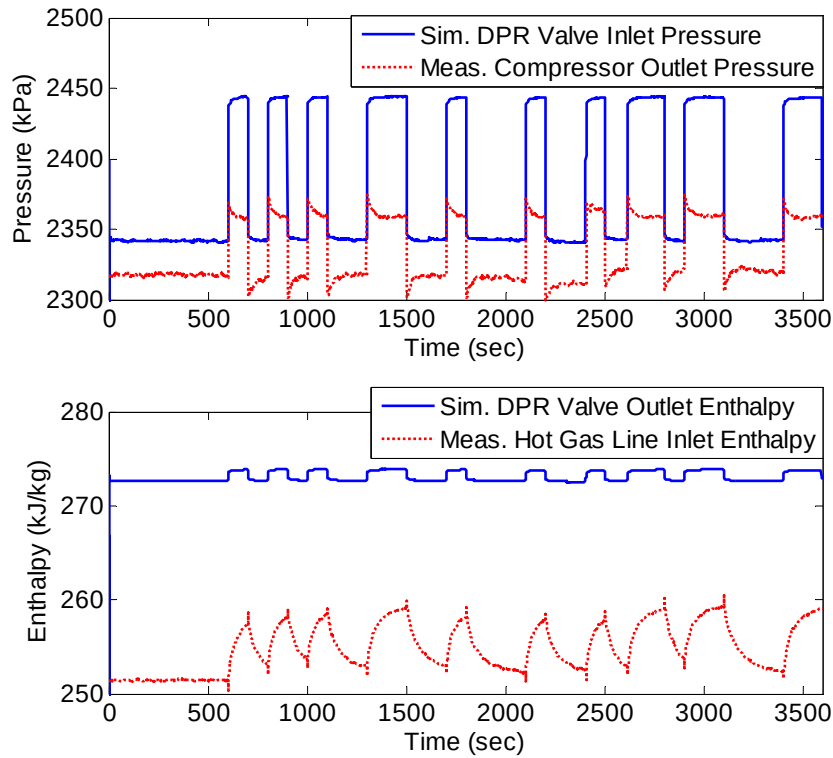


Figure 6.28: DPR valve transient validation results at 70|35 °F.

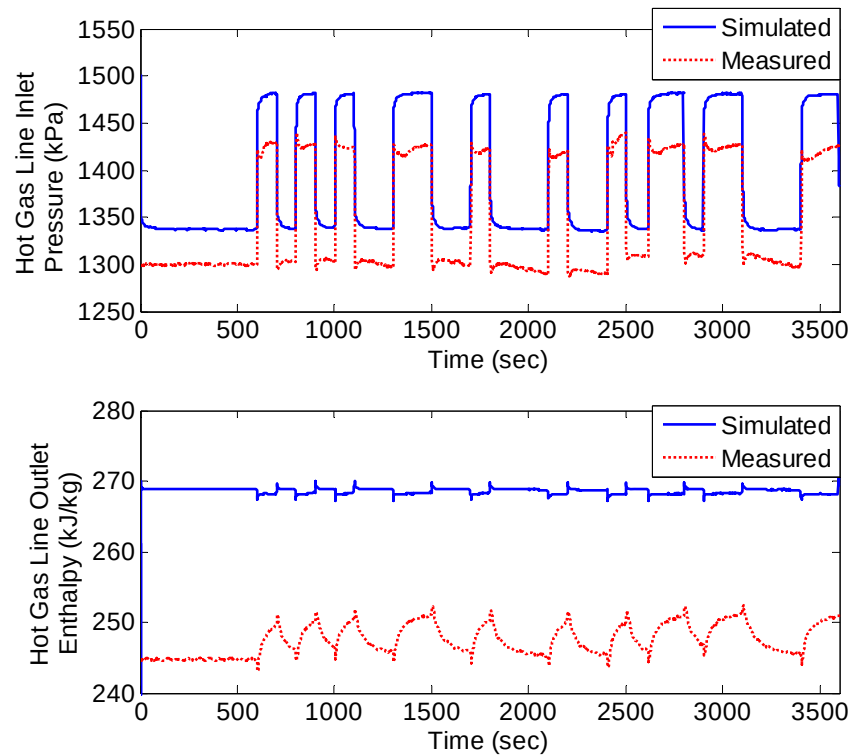


Figure 6.29: Hot gas line transient validation results at 70|35 °F.

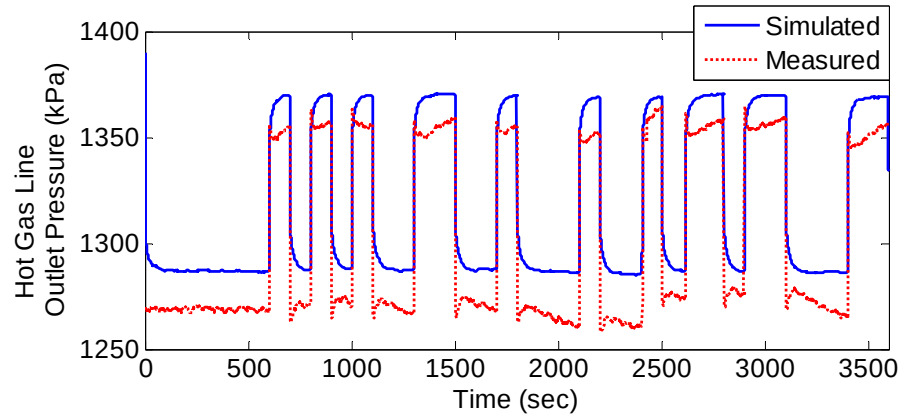


Figure 6.30: Distributor transient validation results at 70|35 °F.

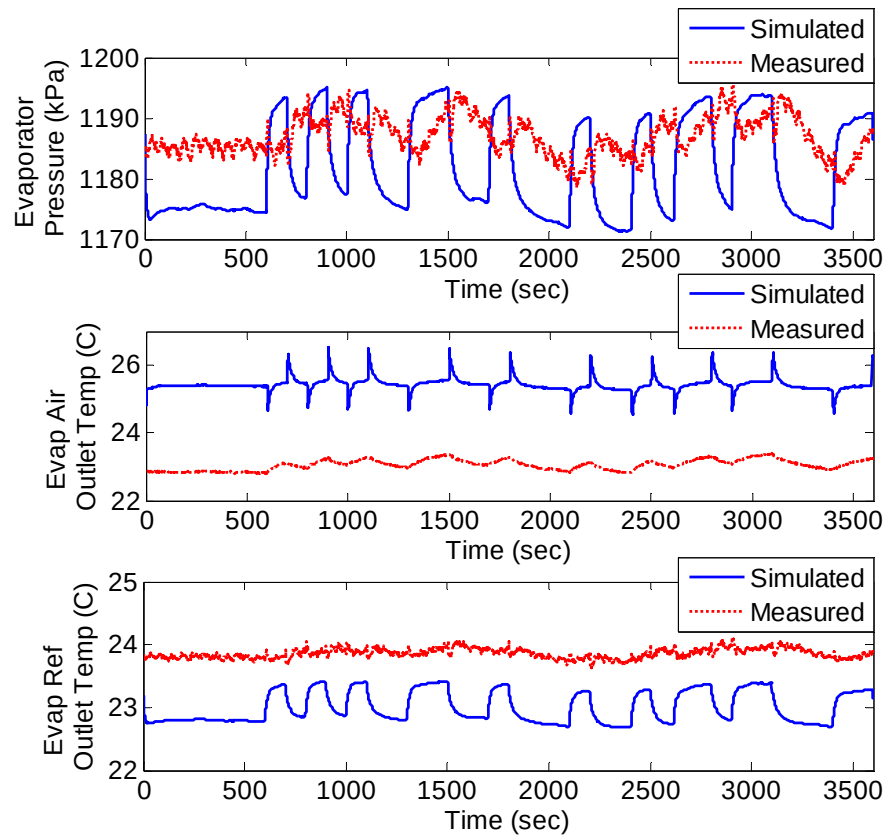


Figure 6.31: Evaporator transient validation results at 70|35 °F.

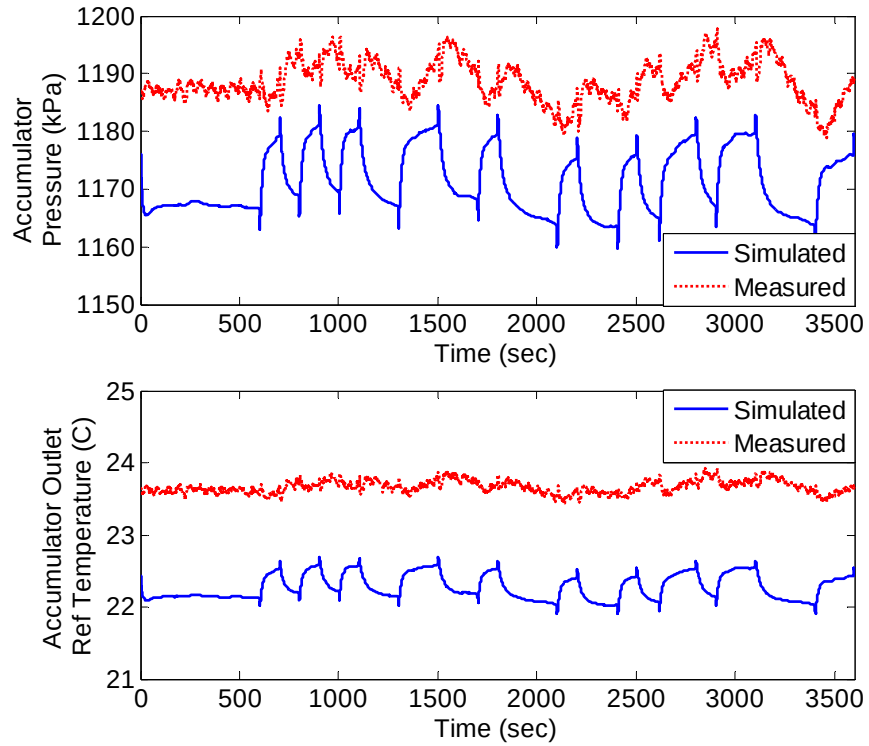


Figure 6.32: Accumulator transient validation results at 70|35 °F.

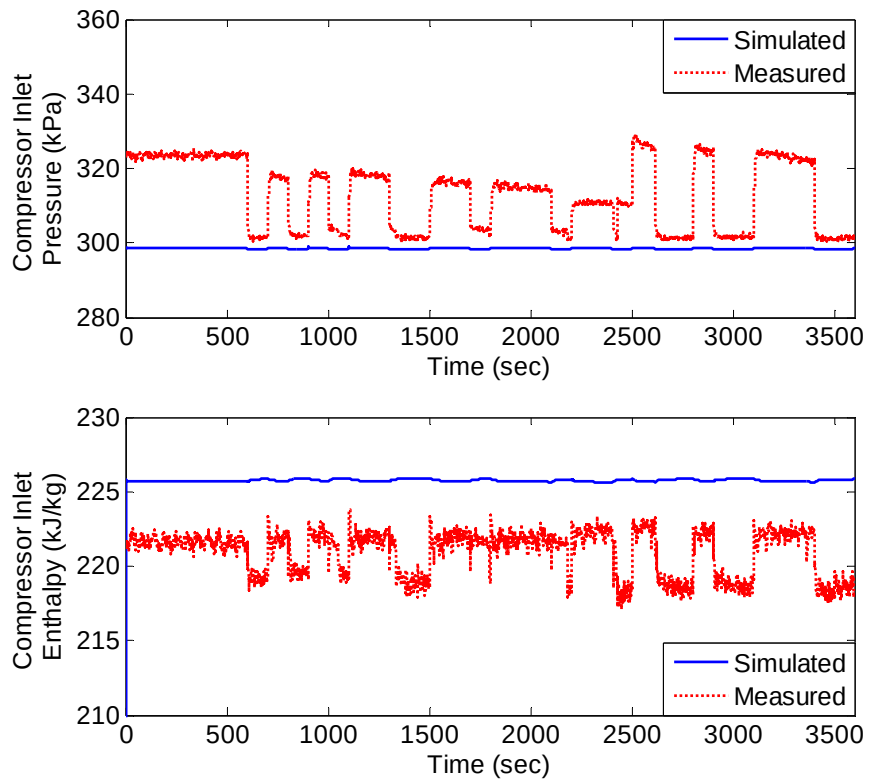


Figure 6.33: Throttle valve transient validation results at 70|35 °F.

Figure 6.34 shows the input signals used in the complete system validation at the 35|0 °F operating condition. This is followed by a set of figures comparing the simulated and measured transient output signals for each of the components in the Heat Mode model at 35|0 °F.

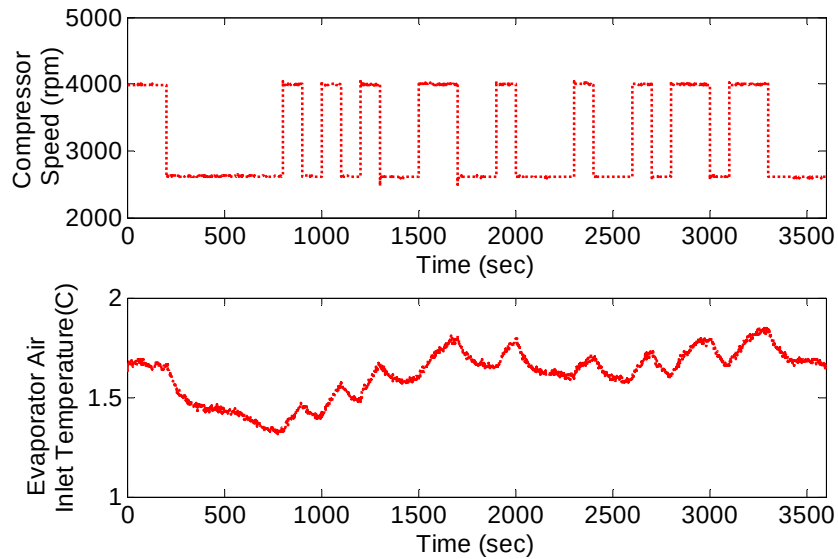


Figure 6.34: Complete cycle system transient inputs at 35|0 °F.

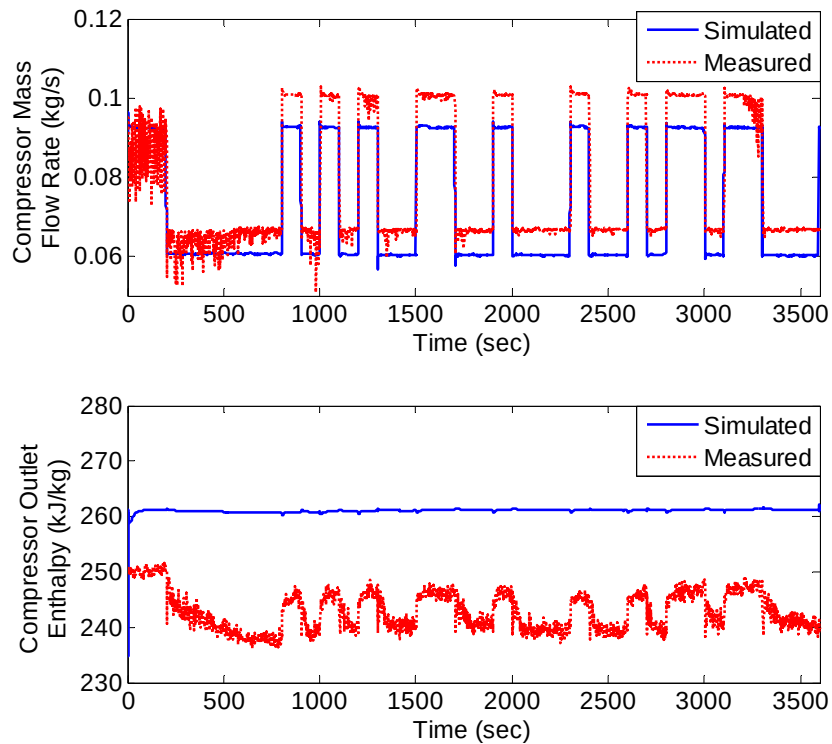


Figure 6.35: Compressor transient validation results at 35|0 °F.

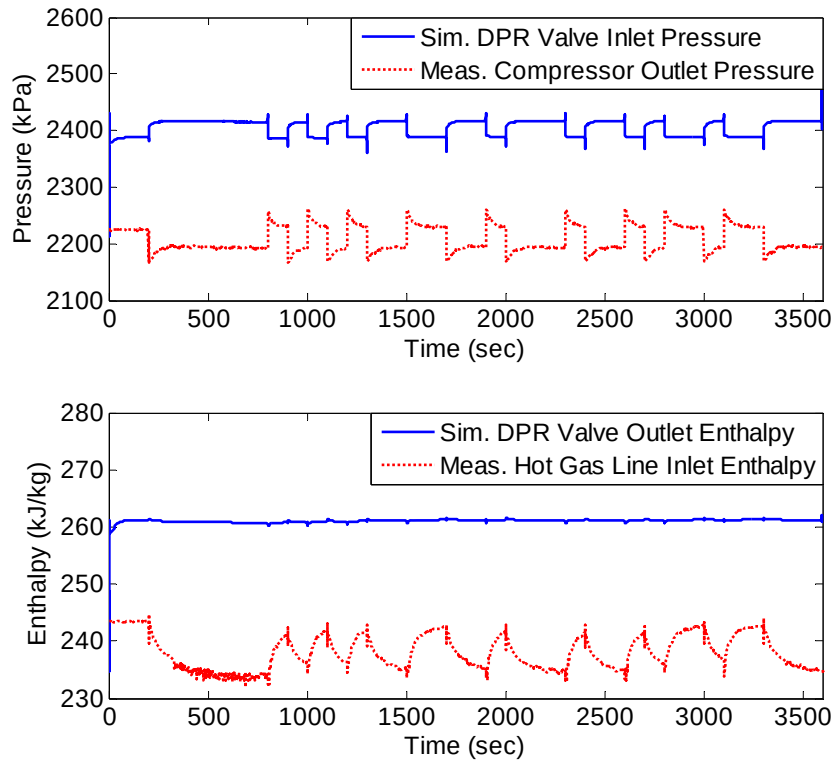


Figure 6.36: DPR valve transient validation results at 350 °F.

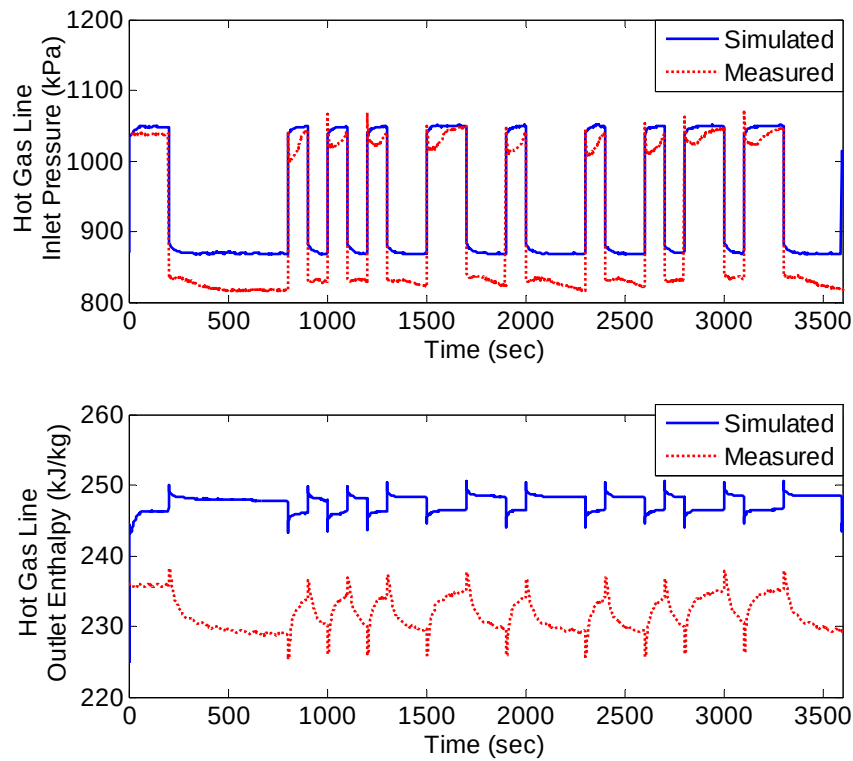


Figure 6.37: Hot gas line transient validation results at 350 °F.

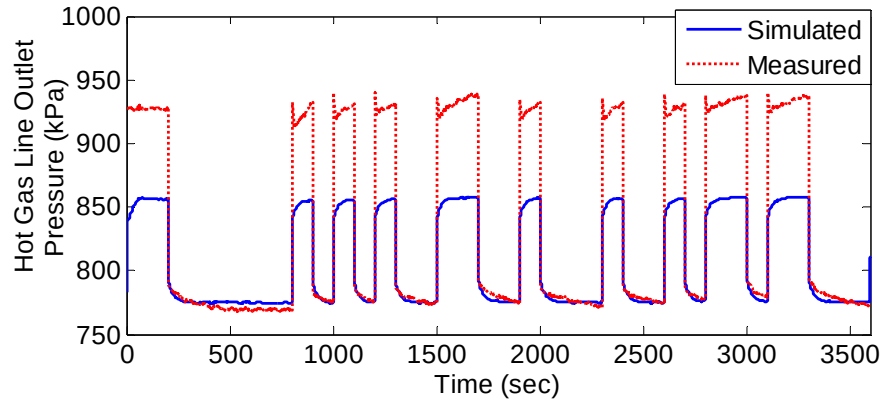


Figure 6.38: Distributor transient validation results at 35°F.

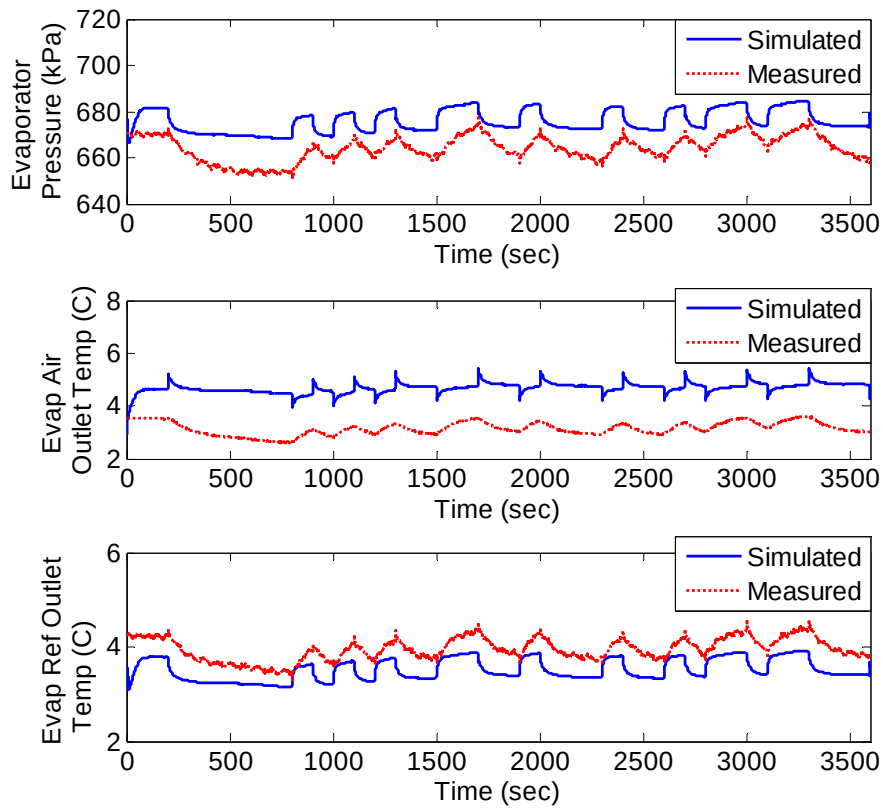


Figure 6.39: Evaporator transient validation results at 35°F.

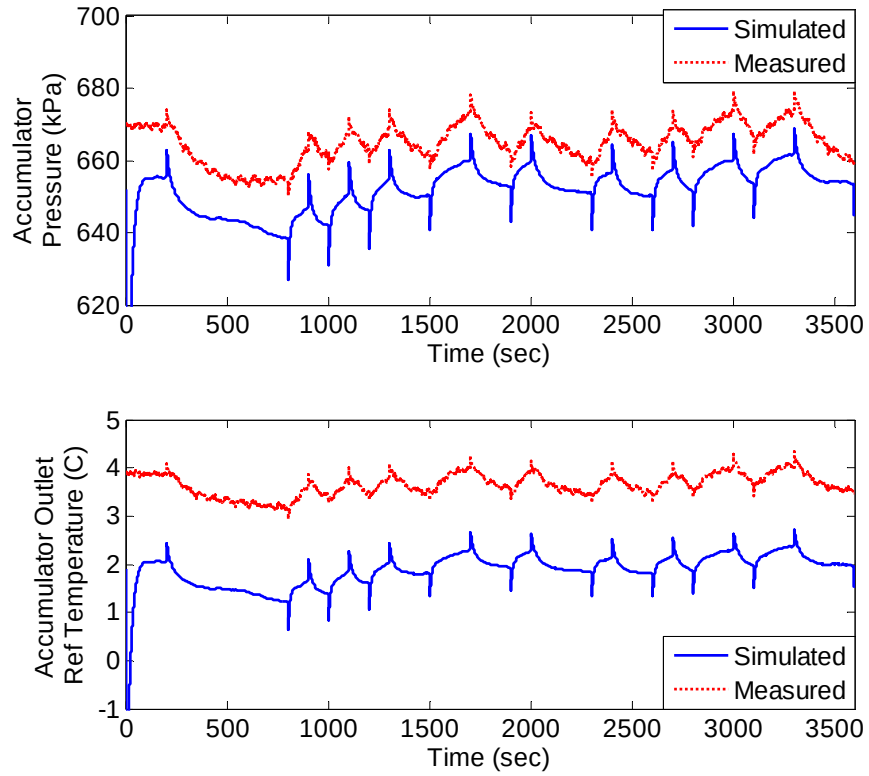


Figure 6.40: Accumulator transient validation results at 35°F.

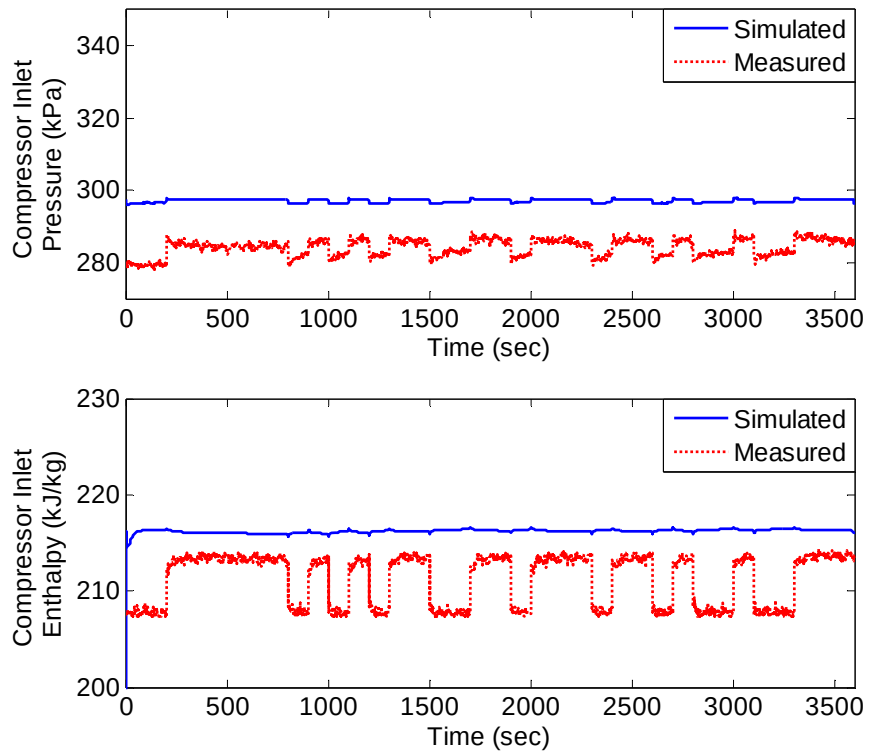


Figure 6.41: Throttle valve transient validation results at 35°F.

After connecting all of the individual Heat Mode component models to form the complete system model, we see that inaccuracies in the individual models propagate through the complete model to cause additional inaccuracies in component models, such as the hot gas line, which had very good transient validation results when validated individually. This suggests that addressing the individual model inaccuracies will effectively improve the closed system model performance as well.

Table 6.27 shows the normalized RMS error between the predicted and measured output signals shown in the previous figures. As discussed in Chapter 5, a stronger performance metric, which incorporates the individual RMS errors, should be designed to better characterize the predictive capability and reliability of the complete system model.

Table 6.27 Normalized RMS Error for Complete Heat Mode Cycle System

Parameter	Normalized RMS Error	
	70 35 °F	35 0 °F
Hk_{ro}	0.0207	0.0749
$\dot{m}_k$	0.3323	0.2954
Pk_{ro}	0.0111	0.0818
Hg_{ri}	0.0702	0.0957
Pg_{ri}	0.0460	0.1890
Hg_{ro}	0.0865	0.0612
Pg_{ro}	0.0347	0.0915
Pe	0.0084	0.0185
Te_{ao}	0.1030	0.4945
Te_{ro}	0.0373	0.1187
Pa_{ro}	0.0154	0.0220
Ta_{ro}	0.0593	0.4944
Pk_{ri}	0.0542	0.0440
Hk_{ri}	0.0228	0.0276

Recall Equation 5.3 which described a possible performance metric; with experience, a person proficient in the model will be able to customize each of the weights in the metric as well as determine a reasonable maximum cost, J_{max} , which can be used as an upper bound on the error that can be tolerated.

The model presented here is suitable for initial controller design development in simulation. The model should then be validated against data collected using the same controller on the experimental system during Heat Mode operation. This will lead to increased user confidence as well as a better understanding of the limitations of the Heat

Mode system model. This knowledge can also be applied to designing the performance metric for evaluating the model as well as the choice of a suitable J_{max} .

Chapter 7

CONCLUSIONS AND FUTURE WORK

7.1 Summary of Research Contributions

In this thesis, a control-oriented dynamic model of a commercially available transport refrigeration unit is developed and validated with test data collected on a fully instrumented unit. A major contribution of this thesis is the development of a control-oriented model of the heat cycle, or Heat Mode, operation of the transport refrigerant unit. Moreover, the Thermosys Toolbox was expanded by the development of new component models specific to commercial transport refrigeration units (e.g. refrigerant distributor, suction line heat exchanger).

7.1.1 Model Developments

One set of models presented in this thesis are adaptations of models which already existed in the Thermosys Toolbox. A second set are models used internally at Thermo King Corporation which have been incorporated into the Thermosys modeling framework. Finally, a third set of components are original models developed specifically for the Cool and Heat Mode system models.

7.1.1.1 Adaptations of Previously Developed Models

The condenser with receiver, Cool Mode evaporator, and Heat Mode evaporator models in this thesis are adaptations of previously developed heat exchanger models. They were improved to accommodate a counter-cross airflow configuration as well as multi-circuit coil geometry. The accumulator model adapted for Heat Mode has an added dynamic state to represent the thermal capacitance of the accumulator tank. Finally, the

electronic expansion valve (EEV) model utilizes a new set of predictors, valve stroke and condenser subcooling, in its flow coefficient map.

A number of the static models were originally developed at Thermo King Corporation and have been incorporated into the Thermosys modeling framework. These include the compressor, throttle valve, suction line heat exchanger (SLHX), and evaporator and condenser fan models.

7.1.1.2 Newly Developed Models

The discharge pressure regulator (DPR) valve and hot gas line models have been developed using semi-empirical approaches for both pressure drop and heat transfer characteristics. The distributor models for both the Cool and Heat Mode system models are derived using system identification techniques to accurately capture the dynamics of the distributor. The identified models have been incorporated into the Thermosys modeling framework.

7.1.2 Model Validation

The models developed in this thesis are validated with data collected on a fully instrumented unit. Validation with both steady-state data and transient data was conducted for each individual component model in both the Cool Mode and Heat Mode model configurations. The complete cycle models for Cool Mode and Heat Mode were validated with transient data. The results showed that the models accurately capture the system dynamics albeit offsets between some measured and predicted output signals. The normalized RMS error provides a method for quantifying the relative error between the measured and predicted output signals for each model at one operating condition over another, but is difficult to use as an absolute metric. The development of a cost function is proposed for future assessment of the predictive capability of each model.

7.2 Future Work

Future work in both modeling and model validation will improve the quality of both the Cool and Heat Mode system models as well as enhance their applicability to controller design and new component selection.

7.2.1 Model Development

There are a number of minor model developments that have been mentioned throughout the thesis that can be pursued to improve the accuracy of the Cool and Heat mode system models. These are summarized here.

- Cool Mode
 - Develop a thermostatic expansion valve model based on the method used for the EEV model development.
 - Incorporate pressure drop across the condenser coil into the condenser with receiver model.
- Heat Mode
 - Incorporate heat transfer dynamics through compressor shell into compressor model.
 - Incorporate heat transfer dynamics into DPR valve model.
 - Incorporate separation efficiency as a variable parameter in the accumulator model.
 - Develop a throttle valve map for the Heat Mode operating envelope.
 - Develop a first-principles model for the refrigerant distributor.
- Cool and Heat Mode
 - Decouple of the effects of the liquid injective valve (LIV) and the compressor by developing a separate LIV model (similar to the EEV model) and a new compressor model.
 - Examine scaling of mass versus coil surface area in heat exchangers when approximating multi-circuit geometry.
 - Develop a look-up table or library of pipe-loss coefficients in Thermosys for both pipe models and the hot gas line model.
 - Develop additional code for the purpose of tracking the total mass and location of refrigerant inside the system at all times.

Possible directions for future research are given in the following list in an order of suggested priority.

- Adapt current condenser and evaporator models with the capability to gain and lose fluid regions using an existing switch-state moving-boundary framework. This is essential for the successful simulation of start-up and shut-down conditions, as well as for the design of control algorithms (in simulation) which require the compressor to cycle on and off.
- Develop a switching scheme between Cool Mode and Heat Mode.

7.2.2 Model Validation

Additional successful model validation results will always increase user confidence in the accuracy of a given model. For the Cool and Heat Mode models, future model validation could focus on validating individual component models at a larger range of operating conditions, and on a new test unit, to better understand specific limitations of each model. Additionally, the complete cycle Cool Mode should be validated with transient data characterizing perturbations in compressor speed. As the complete cycle models are used for controller design, the models should be revalidated against closed-loop test data to understand the accuracy of the model when control loops are incorporated.

List of References

- [1] Ljung, L., *System Identification Toolbox: For Use with Matlab*, rel. 2001. The Math Works Inc. Natick, MA.
- [2] "EASY5 Multiphase Fluid Library,"
http://www.mscsoftware.com/products/products_detail.cfm?PI=492, 2005.
- [3] G. Anand, M. Mahajan, N. Jain, B. Maniam, and T.M. Tumas, "e-Thermal: Automobile Air Conditioning Module," in *Society of Automotive Engineers 2004 World Congress*, Detroit, Michigan, 2004.
- [4] Bendapudi, S. and Braun, J.E., "A Review of Literature on Dynamic Models of Vapor Compression Equipment," ASHRAE Report #4036-5, May 2002.
- [5] B.A. Cullimore and T.J. Hendricks, "Design and Transient Simulation of Vehicle Air Conditioning Systems," in *Society of Automotive Engineers 2004 World Congress*, Detroit, Michigan.
- [6] B.D. Eldredge, "Improving the Accuracy and Scope of Control-Oriented Vapor Compression Cycle System Models," University of Illinois at Urbana-Champaign, Urbana, IL, 2006.
- [7] L.J. Erickson, "Rating Equations for Positive Displacement Compressors with Auxiliary Suction Ports," in *Proceedings of the 1998 International Compressor Engineering Conference at Purdue*, West Lafayette, Indiana.
- [8] E.W. Grald and J.W. MacArthur, "A Moving-Boundary Formulation for Modeling Time-Dependent Two-Phase Flows," *International Journal of Heat & Fluid Flow*, vol. 13, no. 3, pp. 266-272, Sep, 1992.
- [9] X.D. He, S. Liu, and H. Asada, "Modeling of Vapor Compression Cycles for Multivariable Feedback Control of HVAC Systems," *ASME Journal of Dynamic Systems Measurement & Control*, vol. 119, no. 2, pp. 183-191, Jun, 1997.
- [10] X.-D. He, S. Liu, H.H. Asada, and H. Itoh, "Multivariable Control of Vapor Compression Systems," *ASHRAE Transactions*, vol. 105, no. 1, pp. 289, 1999.

- [11] Incropera, F. P. and DeWitt, D. P. Fundamentals of Heat and Mass Transfer, New York: John Wiley & Sons, 1996.
- [12] J.M. Jensen and H. Tummescheit, "Moving Boundary Models for Dynamic Simulations of Two-Phase Flows," in *Proceedings of the 2nd International Modelica Conference*, 2002, pp. 235-244.
- [13] M. Kapadia and C.H. Wolgemuth, "A Dynamic Model of a Condenser in a Closed Rankine Cycle Power Plant," in *Proceedings of the 1984 American Control Conference*, New York, NY, vol. 1, 1984, pp. 79-84.
- [14] M. Keir, B.P. Rasmussen, and A. Alleyne, "Improving Energy Efficiency in Automotive Vapor Compression Cycles through Advanced Control Design," in *SAE 2006 World Congress and Exposition*, Detroit, MI.
- [15] M.C. Keir, "Dynamic Modeling, Control, and Fault Detection in Vapor Compression Systems," University of Illinois at Urbana-Champaign, Urbana, IL, 2006.
- [16] Krasner-Khait, Barbara "The Impact of Refrigeration," <http://www.history-magazine.com/refrig.html>, 2009.
- [17] T.L. McKinley and A.G. Alleyne, "Real-Time Modeling of Liquid Cooling Networks in Vehicle Thermal Management Systems," in *2008 SAE World Congress*, Detroit, MI.
- [18] T.L. McKinley and A.G. Alleyne, "An Advanced Nonlinear Switched Heat Exchanger Model for Vapor Compression Cycles Using the Moving Boundary Method," *International Journal of Refrigeration*, vol. 31, pp. 1253-1264, 2008.
- [19] Miller, D. S. Internal Flow Systems, Britain: BHRA Fluid Engineering, 1978.
- [20] Mills, A. F. Basic Heat and Mass Transfer, New Jersey: Prentice Hall, 1999.
- [21] Ott, R. L. and Longnecker, M. An Introduction to Statistical Methods and Data Analysis, United States: Duxbury, 2001.
- [22] N.B.O.L. Pettit, M. Willatzen, and L. Ploug-Sorensen, "General Dynamic Simulation Model for Evaporators and Condensers in Refrigeration. Part II: Simulation and Control of an Evaporator," *International Journal of Refrigeration*, vol. 21, no. 5, pp. 404-414, Aug, 1998.
- [23] Popper, K. R. Logik der Forschung, Vienna, Austria: Julius Springer, 1935.
- [24] B.P. Rasmussen, "Dynamic Modeling and Advanced Control of Air Conditioning and Refrigeration Systems," Dept. of Mechanical and Industrial Engineering, University of Illinois, Urbana, IL, Dec. 2005.

- [25] B.P. Rasmussen and A. Alleyne, "Control-Oriented Modeling of Transcritical Vapor Compression Systems," *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 126, no. 1, pp. 54-64, Mar, 2004.
- [26] R. Shah, "Dynamic Modeling and Control of Single and Multi-Evaporator Subcritical Vapor Compression Systems," University of Illinois at Urbana-Champaign, Urbana, IL, July 2003.
- [27] Srichai, R. , Mohs, B., and Dykes, D. *Heat Exchanger Performance Testing Trailer Products*, Internal Report, Thermo King Corp., 2002. (UnPub)
- [28] Stoecker, W. F. and Jones, J. W. Refrigeration and Air Conditioning, New York: McGraw-Hill Book Company, 1982.
- [29] S.A. Tassou , G. De-Lille, and Y.T. Ge, "Food transport refrigeration - Approaches to reduce energy consumption and environmental impacts of road transport," *Applied Thermal Engineering*, vol. 2008.
- [30] Thermo King "About Us...History of Thermo King," <http://www.thermoking.com/tk/index.asp>, 2009.
- [31] H. Tummescheit and J. Eborn, "Design of a Thermo-Hydraulic Model Library in Modelica™," in *Simulation: Past, Present and Future. 12th European Simulation Multiconference*, San Diego, CA, pp. 132-6.
- [32] C. Wang, K. Chi, and C. Chang, "Heat Transfer and Friction Characteristics of Plain Fin-,"
- [33] Whalley, P. B. Two-Phase Flow and Heat Transfer, New York: Oxford University Press, 1996.
- [34] White, F. M. Fluid Mechanics, Boston: McGraw-Hill, Inc., 2003.
- [35] M. Willatzen, N.B.O.L. Pettit, and L. Ploug-Sorensen, "General Dynamic Simulation Model for Evaporators and Condensers in Refrigeration. Part I: Moving-Boundary Formulation of Two-Phase Flows With Heat Exchange," *International Journal of Refrigeration*, vol. 21, no. 5, pp. 398-403, Aug, 1998.

Appendix A

INPUT SIGNAL GENERATION

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% This m-file generates inputs for a time domain system ID
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% The first block of code simultaneously generates two PRBS signals

% 1st row controls valve steps, 2nd row controls compressor steps
u_1 = [ zeros(1,300) -1*ones(1,300) zeros(1,300) -2*ones(1,300) zeros(1,1500);...
        zeros(1,1500) 1*ones(1,300) zeros(1,300) 2*ones(1,300) zeros(1,300)];

% Generate Random ID input signals to append to the end of the u_1 vector
u_2 = round(idinput([4200 2], 'prbs', [0 0.01], [-0.8 0.8]));

% Shift valve command to force the valve to avoid opening the valve large
% amounts
u_2(1,:) = u_2(1, :)-1;

% Combine u_1 and u_2 to form ID input signals
u = [u_1 u_2 zeros(2,800)]';

% Plot the ID input signals
figure;
plot(u)

IDValve = u(:,1);
IDComp = u(:,2);

% The second block of code generates a single RGS signal for the EEV

u_EEV = round(idinput([4200 1], 'rgs', [0 0.01], [-0.8 0.8]));

% Shift valve command to force the valve to avoid opening the valve large
% amounts
u_EEV = u_EEV-1;

% Plot the ID EEV input signal
figure;
plot(u_EEV)
```

Appendix B

DISTRIBUTOR SYSTEM ID

B.1 Cool Mode

```
%Distributor Model Identification for Cool Mode
%Created: 11 Feb '08 by Neera Jain, University of Illinois at Urbana-Champaign
%Last Updated: 28 April '08

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%   Load Identification Data
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

clear all; close all; clc;

load PRBS_EEV_042808_8A

%Data used for the system ID
PVRI = PRBS.Pc(1:1001);
PVR0 = PRBS.Pv_ro(1:1001);
PDR0 = PRBS.Pd_ro(1:1001);
MDOT = PRBS.mdot(1:1001);
PE = PRBS.Pe(1:1001);

%Data used to cross-validate the identification
PVRI2 = PRBS.Pc(2000:3000);
PVR02 = PRBS.Pv_ro(2000:3000);
PDR02 = PRBS.Pd_ro(2000:3000);
MDOT2 = PRBS.mdot(2000:3000);
PE2 = PRBS.Pe(2000:3000);
DP2 = PVR02 - PDR02;

u1 = MDOT;
u2 = PE;
y1 = PVR0;
y2 = PVR0 - PDR0;

%Input/Output Labels
InputLabels = {'mdot', 'Pe'};
OutputLabels = {'Pv_ro', 'dP_dist'};
InputUnits = {'[kg/s]', '[kPa]'};
OutputUnits = {'[kPa]', '[kPa]'};

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%   Identify a Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Inputs_all = [u1 u2];
Outputs_all = [y1 y2];
IDData_all = detrend(iddata(Outputs_all, Inputs_all, 1, 'InputName', InputLabels, 'OutputName', OutputLabels, 'InputUnit', InputUnits, 'OutputUnit', OutputUnits), 1);
```



```

PHRO = PH_R0(start:finish);
PDRI = PHRO;
PDR0 = PD_R0(start:finish);
MDOT = M_DOT(start:finish);

u1 = MDOT;
u2 = PDR0;
y1 = PDRI;

%Input/Output Labels
InputLabels = {'mdot', 'Pdri'};
OutputLabels = {'Pdri'};
InputUnits = {'[kg/s]', '[kPa]'};
OutputUnits = {'[kPa]'};

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Identify a Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u1_detrend = u1 - mean(u1);
u2_detrend = u2 - mean(u2);
y1_detrend = y1 - mean(y1);

Inputs_all = [u1_detrend/0.01 u2_detrend/3.5];
Outputs_all = [y1_detrend/43];

IDData_all = iddata(Outputs_all,Inputs_all,1, 'InputName', InputLabels, 'OutputName',
OutputLabels, 'InputUnit', InputUnits, 'OutputUnit', OutputUnits);

%Perform model ID - using a 2nd order model was found to be
%appropriate
Sys = n4sid(IDData_all,2);
%Plot results of Sys ID
figure;
compare(IDData_all,Sys);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Scale the model back to initial units
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%Convert system to continuous time

%G_ContinuousScaled
G_cs = minreal(d2c(Sys));

%Scale system back to match physical system

Nx = eye(2);
Nu = [1/0.01 0;0 1/3.5];
Ny = [43];
%G_ContinuousUnScaled
G_cus = ss(Nx*G_cs.A*inv(Nx),Nx*G_cs.B*Nu,Ny*G_cs.C*inv(Nx),Ny*G_cs.D*Nu);

```

Appendix C

MAP GENERATING FILES

C.1 Mean Void Fraction Lookup Table

```
%This .m file is used to generate a look up table mapping mu_s and change
%in quality to mean void fraction.

clear all;close all;clc;

load FluidProp_R404a

Slip_ratio = 4;

%   determine number of points along the vapor pressure curve

numvalues = length( FluidProp.Psat );

%   calculate the mean density and mean enthalpy from the given void fraction correlation

for i = 1:numvalues
    Rho_g = FluidProp.Rhog(i);
    Rho_f = FluidProp.Rhof(i);
    H_g = FluidProp.Hg(i);
    H_f = FluidProp.Hf(i);
    U_g = FluidProp.Ug(i);
    U_f = FluidProp.Uf(i);
    mus = (Rho_g / Rho_f) * Slip_ratio;
    Gamma_bar = 1.0 / (1.0 - mus) + mus / (1.0 - mus)^2 * ...
        log( mus / (mus + (1 - mus)) );
    TwoPhaseProp.Rho_bar(i) = Gamma_bar * Rho_g + (1.0 - Gamma_bar) * Rho_f;
    X_bar = Gamma_bar * Rho_g / TwoPhaseProp.Rho_bar(i);
    TwoPhaseProp.H_bar(i) = X_bar * H_g + (1.0 - X_bar) * H_f;
    TwoPhaseProp.U_bar(i) = X_bar * U_g + (1.0 - X_bar) * U_f;
    TwoPhaseProp.Gamma_Total(i) = Gamma_bar;
end

%   Set up the interpolation table for outlet quality vs. condenser mean void fraction
%   COORDINATE. The mean void fraction COORDINATE is defined by:
%
%   Gammac_cood = (1.0 - Gamma) / (1.0 - Gamma_Total)
%
%   Note that when the outlet vapor quality is 1.0, the condenser mean void
%   fraction coordinate is 0.0. Conversely, when the outlet vapor quality
%   is 0.0, the condenser mean void fraction coordinate is one. This
%   transformation is desirable so that we can use a fast (evenly spaced)
%   interpolation scheme to find the outlet vapor quality.

TwoPhaseProp.Gamma_Coord = linspace(0.0, 1.0, 50);
numgamma = length(TwoPhaseProp.Gamma_Coord);
Xvector = linspace(0.0, 1.0, 100);
```

```

numX = length(Xvector);

for iprs = 1:numvalues
    Rho_g = FluidProp.Rhog(iprs);
    Rho_f = FluidProp.Rhof(iprs);
    mus = (Rho_g / Rho_f) * Slip_ratio;
    % create the table of gamma (mean void fraction) vs uniformly spaced
    % outlet quality
    for ix = 1:numX-1
        Xout = Xvector(ix);
        del_X = Xout - 1.0;
        Gamma = 1.0 / (1.0 - mus) + mus / (del_X * (1.0 - mus)^2) * ...
            log( 1.0 / (1.0 + del_X*(1.0 - mus)) );
        Gamma_check(ix) = Gamma;
        Gamma_ctab(ix) = (1.0 - Gamma) / (1.0 - TwoPhaseProp.Gamma_Total(iprs));
    end
    Gamma_ctab(numX) = 0.0;
    % reinterpolate outlet quality vs uniformly spaced gamma (mean void fraction)
    for ig = 1:numgamma-1
        g = TwoPhaseProp.Gamma_Coord(ig);
        TwoPhaseProp.Xeout(iprs,ig) = interp1( Gamma_ctab, Xvector, g);
    end
    TwoPhaseProp.Xeout(iprs,numgamma) = 0.0;
end

disp( [ 'Void fraction table completed.' ] );
disp( [ ' ' ] );

```

C.2 EEV Flow Coefficient Map

% This file creates a EEV flow coefficient performance map with inputs of
% condenser subcooling, and valve command signal.

```

clear all;
clc;

load FluidProp_R404a

%Load data from Excel file
XVDP = xlsread('H:\My Documents\Briefcase\Research Files\Validation
Testing\DataFormattingMacro','3D Data','a2:a3395');
EEVV = xlsread('H:\My Documents\Briefcase\Research Files\Validation
Testing\DataFormattingMacro','3D Data','b2:b3395');
RFL0 = xlsread('H:\My Documents\Briefcase\Research Files\Validation
Testing\DataFormattingMacro','3D Data','c2:c3395');
XLIT = xlsread('H:\My Documents\Briefcase\Research Files\Validation
Testing\DataFormattingMacro','3D Data','d2:d3395');
RLOP = xlsread('H:\My Documents\Briefcase\Research Files\Validation
Testing\DataFormattingMacro','3D Data','e2:e3395');
XVOP = xlsread('H:\My Documents\Briefcase\Research Files\Validation
Testing\DataFormattingMacro','3D Data','f2:f3395');

%Convert data to SI units
for i=1:length(EEVV)
    EEV(i,1) = (EEVV(i))*10;
    TVRI(i,1) = (XLIT(i)-32)*(5/9);
    DP(i,1) = (XVDP(i))*6.89;
    PVRI(i,1) = (RLOP(i))*6.89;
    PVRO(i,1) = (XVOP(i))*6.89;
    MDOT(i,1) = (RFL0(i))*0.45359237/3600;
end

% Calculate the inlet fluid density, assuming the inlet fluid is
% saturated liquid

RHO_V_SAT = interp1(FluidProp.Psat,FluidProp.Rhof,PVRI);
RHO_V_SUB = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,TVRI,PVRI); %
Subcooled liquid inlet density
RHO_V = RHO_V_SAT;

```

```

% Calculate the saturated refrigerant temperature and the corresponding
% subcooling out of the condenser coil based on TVRI
TSAT = interp1(FluidProp.Psat,FluidProp.Tsat,PVRI);
T_SC = TSAT - TVRI;
% Calculate the flow coefficient
CF = MDOT./(sqrt(RHO_V.*DP));
% Form input matrix X, assuming a model structure
X = [EEV T_SC EEV.*T_SC EEV.^2 T_SC.^2];
% Use 'stepwisefit' to determine which predictors in X are best
[b,se,pval,inmodel,stats,nextstep,history] = stepwisefit(X,CF)

X2 = [ones(size(MDOT)) EEV T_SC EEV.*T_SC EEV.^2 T_SC.^2];
[b2,bint,r,rint,stats] = regress(CF,X2);
CF_model = X2*b2;

figure; rcoplot(r,rint);
figure; plot(CF,CF_model,'*'); grid on;
xlabel('Kv Measured'); ylabel('Kv Predicted');

% Define input variable vectors
tsc = 10:1:40;
eev = 1:2:100;
% Generate the lookup table using a for loop for each input, equation
% should match the model structure chosen for the X matrix
for c1 = 1:length(eev)
    for c2 = 1:length(tsc)
        Cf_map(c1,c2) = b2(1) + b2(2)*eev(c1) + b2(3)*tsc(c2) + b2(4)*eev(c1)*tsc(c2) +
b2(5)*eev(c1)^2 + b2(6)*tsc(c2)^2;
    end
end

% Store the input vectors and lookup table array in a structure
[TSCmg,EEVmg] = meshgrid(tsc, eev);
ValveProp.u = EEVmg;
ValveProp.Tsc= TSCmg;
ValveProp.Kv = Cf_map;
figure;
surf(ValveProp.u, ValveProp.Tsc,ValveProp.Kv); hold on;
plot3(EEV, T_SC, CF);
xlabel('Valve Stroke (% Open)');
ylabel('Condenser Subcooling (C)');
zlabel('Flow Coefficient');

```

C.3 Moist Air Fluid Properties Table

```

% This is a script for adding the properties of moist air to THERMOSYS
% Author: T.L. McKinley - 09 Oct '06
%
% This script was modified to include the specific heat of moist air
% Author: Neera Jain - 30 March '07
%
% This script was modified to include the density of moist air
% Author: Neera Jain - 21 May '07
%
%
global MoistAirProp;
%
% Generate water vapor pressure curve, -100 to 0 deg C
% Ref: 2001 ASHRAE Handbook - Fundamentals, pp. 6.2, equation (5)
%
% Generate dry air enthalpy curve assuming constant specific heat
% Ref: 2005 ASHRAE Handbook - Fundamentals, pp. 6.9, equation (30)
%
% Generate water vapor enthalpy curve assuming constant specific heat
% Ref: 2005 ASHRAE Handbook - Fundamentals, pp. 6.9, equation (31)
%
% Generate saturated liquid water enthalpy curve assuming constant
% specific heat

```

```

% Ref: 2005 ASHRAE Handbook - Fundamentals, pp. 6.9, equation (34)
%
C1 = -5.6745359e+3;
C2 = 6.3925247e+0;
C3 = -9.6778430e-3;
C4 = 6.2215701e-7;
C5 = 2.0747825e-9;
C6 = -9.4840240e-13;
C7 = 4.1635019e+0;

for i=1:101
    T_degC = -100.0 + (i-1);
    MoistAirProp.Tsat(i) = T_degC;
    T = T_degC + 273.15;
    lnP = C1/T + C2 + C3*T + C4*T^2 + C5*T^3 + C6*T^4 + C7*log(T);
    MoistAirProp.Psat(i) = exp( lnP ) * 1.0e-3;
    MoistAirProp.Hair_t(i) = 1.006 * T_degC;
    MoistAirProp.Hg_t(i) = 2501.0 + 1.86 * T_degC;
    MoistAirProp.Hf_t(i) = 4.186 * T_degC;
end

%
% Generate water vapor pressure curve, 0 to 200 deg C
% Ref: 2001 ASHRAE Handbook: Fundamentals, pp. 6.2, equation (6)
%
% Generate dry air enthalpy curve assuming constant specific heat
% Ref: 2005 ASHRAE Handbook: Fundamentals, pp. 6.9, equation (30)
%
% Generate saturated water vapor enthalpy curve assuming constant specific heat
% Ref: 2005 ASHRAE Handbook - Fundamentals, pp. 6.9, equation (31)
%
% Generate saturated liquid water enthalpy curve assuming constant
% specific heat
% Ref: 2005 ASHRAE Handbook - Fundamentals, pp. 6.9, equation (34)
%
C8 = -5.8002206e+3;
C9 = 1.3914493e+0;
C10 = -4.8640239e-2;
C11 = 4.1764768e-5;
C12 = -1.4452093e-8;
C13 = 6.5459673e+0;

for i=102:301
    T_degC = -100 + (i-1);
    MoistAirProp.Tsat(i) = T_degC;
    T = T_degC + 273.15;
    lnP = C8/T + C9 + C10*T + C11*T^2 + C12*T^3 + C13*log(T);
    MoistAirProp.Psat(i) = exp( lnP ) * 1.0e-3;
    MoistAirProp.Hair_t(i) = 1.006 * T_degC;
    MoistAirProp.Hg_t(i) = 2501.0 + 1.86 * T_degC;
    MoistAirProp.Hf_t(i) = 4.186 * T_degC;
end

%
% Generate viscosity (Pa-sec), thermal conductivity (kW/m-K), and specific heat (kJ/kg-
% K), and density (kg/m^3) tables from Incropera and
% Dewitt, Fundamentals of Heat and Mass Transfer, Table A.4
%
MoistAirProp.T = ...
    [ 100.00  150.00  200.00  250.00  300.00  350.00  400.00  ...
      450.00  500.00  550.00  600.00  650.00  700.00  750.00  ...
      800.00  850.00  900.00  950.00  1000.00  1100.00  1200.00  ...
      1300.00  1400.00  1500.00  1600.00  1700.00  1800.00  1900.00  ...
      2000.00  2100.00  2200.00  2300.00  2400.00  2500.00  3000.00 ];
MoistAirProp.mu_T = ...
    [ 71.1e-7  103.4e-7  132.5e-7  159.6e-7  184.6e-7  208.2e-7  230.1e-7  ...
      250.7e-7  270.1e-7  288.4e-7  305.8e-7  322.5e-7  338.8e-7  354.6e-7  ...
      369.8e-7  384.3e-7  398.1e-7  411.3e-7  424.4e-7  449.0e-7  473.0e-7  ...
      496.0e-7  530.0e-7  557.0e-7  584.0e-7  611.0e-7  637.0e-7  663.0e-7  ...
      689.0e-7  715.0e-7  740.0e-7  766.0e-7  792.0e-7  818.0e-7  955.0e-7 ];
MoistAirProp.k_T = ...
    [ 9.34e-6  13.8e-6  18.1e-6  22.3e-6  26.3e-6  30.0e-6  33.8e-6  ...
      37.3e-6  40.7e-6  43.9e-6  46.9e-6  49.7e-6  52.4e-6  54.9e-6  ...

```



```

57.3e-6 59.6e-6 62.0e-6 64.3e-6 66.7e-6 71.5e-6 76.3e-6 ...
82.0e-6 91.0e-6 100.0e-6 106.0e-6 113.0e-6 120.0e-6 128.0e-6 ...
137.0e-6 147.0e-6 160.0e-6 175.0e-6 196.0e-6 222.0e-6 486.0e-6 ];

MoistAirProp.cp_T = ...
[ 1.032 1.012 1.007 1.006 1.007 1.009 1.014 ...
  1.021 1.030 1.040 1.051 1.063 1.075 1.087 ...
  1.099 1.110 1.121 1.131 1.141 1.159 1.175 ...
  1.189 1.207 1.230 1.248 1.267 1.286 1.307 ...
  1.337 1.372 1.417 1.478 1.558 1.665 2.726 ];

MoistAirProp.rho_T = ...
[ 3.5562 2.3364 1.7458 1.3947 1.1614 0.9950 0.8711 ...
  0.7740 0.6964 0.6329 0.5804 0.5356 0.4975 0.4643 ...
  0.4354 0.4097 0.3868 0.3666 0.3482 0.3166 0.2902 ...
  0.2679 0.2488 0.2322 0.2177 0.2049 0.1935 0.1833 ...
  0.1741 0.1658 0.1582 0.1513 0.1448 0.1389 0.1135 ];

MoistAirProp.Pr_T = ...
[ 0.786 0.758 0.737 0.720 0.707 0.700 0.690 ...
  0.686 0.684 0.683 0.685 0.690 0.695 0.702 ...
  0.709 0.716 0.720 0.723 0.726 0.728 0.728 ...
  0.719 0.703 0.685 0.688 0.685 0.683 0.677 ...
  0.672 0.667 0.655 0.647 0.630 0.613 0.536 ];

%
% Convert T from Kelvin to deg C
%
MoistAirProp.T = MoistAirProp.T - 273.15;
%
% Clean up workspace
%
clear C1 C2 C3 C4 C5 C6 C7;
clear C8 C9 C10 C11 C12 C13;
clear T T_degC lnP i

```

C.4 Condenser J Factor

```

% This file generates the experimental system j factor information to be
% stored in the data structure HX for the Thermo King TS-500 condenser.
% The information includes Reynolds number data points, j factor data
% points, the constriction ratio (sigma) and the air-side hydraulic
% diameter (Dh).
%
% The j factor correlation is from the following article:
% International Journal of Heat and Mass Transfer Volume 43, Issue 15,
% Pages 2641-2823 (1 August 2000)
%
% This code is adapted from the following thesis:
% Eldredge, B.D., "Improving the Accuracy and Scope of Control-Oriented
% Vapor Compression Cycle System Models," Dept. of Mechanical and
% Industrial Engineering, University of Illinois, Urbana, IL, Oct. 2002.

% Create a Reynolds number vector (stored in HX structure)
Re = [100 200 300 400 500 600 700 800 900 1000 ...
      2000 3000 4000 5000 6000 7000 8000 9000 10000];

% Number of tube rows in the condenser
N = 4;

% Fin pitch (distance between fins) in inches
Fp = .0683;

% Tube outside diameter in inches
Do = .375;

% Fin thickness in inches
Ft = .0075;

% Fin collar outside diameter in inches (fin collar is defined as tube
% diameter plus twice the fin thickness)
Dc = Do + 2*Ft;

```

```

% Longitudinal tube pitch (distance between tube centers in direction of
% air flow) in inches
Pl = .866;

% Flow length in inches (flow length is explained in Chapter 3)
L = 4*.866;

% Flow cross-sectional area, also known as the minimum free flow area, in
% square inches
% Flow cross-sectional area = Frontal Area - Tube Diameter*Number of
% Tubes at cross section*Tube_Length - Fin thickness*Fin height*Number of
% Fins
Ac = 310.975 - .375*N*35.2635 - .0075*8.5*490;

% Total heat transfer area in square inches, provided by manufacturer
A = 25964.2;

% Air-side hydraulic diameter in inches as defined by Kays and London
Dh = 4*L*Ac/A;

% Transverse tube pitch in inches (distance between tube centers in
% direction transverse to air flow)
Pt = 1;

% Powers used in the j factor correlation
P3 = -.361 - (.042*N./log(Re)) + .158*log(N*(Fp/Dc)^.41);
P4 = -1.224 - (.076*(Pl/Dh)^1.42)./(log(Re));
P5 = -.083 + (.058*N)./(log(Re));
P6 = -5.735 + 1.21*log(Re/N);

% Colburn j factor correlation (stored in HX structure)
j_cond = .086.*Re.^P3.*N.^P4.*(Fp./Dc).^P5.*(Fp/Dh).^P6*(Fp./Pt).^-.93;

% Calculate the frontal area in square inches
Afr = 310.975;

% Calculate the constriction ratio (stored in HX structure)
sigma = Ac/Afr;

% Convert Dh from inches to meters (stored in HX structure)
DhSI = Dh*0.0254;

```

C.5 Evaporator J Factor

```

% This file generates the experimental system j factor information to be
% stored in the data structure HX for the Thermo King TS-500 evaporator.
% The information includes Reynolds number data points, j factor data
% points, the constriction ratio (sigma) and the air-side hydraulic
% diameter (Dh).
%
% The j factor correlation is from the following article:
% International Journal of Heat and Mass Transfer Volume 43, Issue 15,
% Pages 2641-2823 (1 August 2000)
%
% This code is adapted from the following thesis:
% Eldredge, B.D., "Improving the Accuracy and Scope of Control-Oriented
% Vapor Compression Cycle System Models," Dept. of Mechanical and
% Industrial Engineering, University of Illinois, Urbana, IL, Oct. 2002.

% Create a Reynolds number vector (stored in HX structure)
Re = [100 200 300 400 500 600 700 800 900 1000 ...
      2000 3000 4000 5000 6000 7000 8000 9000 10000];

% Number of tube rows in the evaporator
N = 3;

% Fin pitch (distance between fins) in inches
Fp = 0.1361;

```

```

% Tube outside diameter in inches
Do = .375;

% Fin thickness in inches
Ft = .008;

% Fin collar outside diameter in inches (fin collar is defined as tube
% diameter plus twice the fin thickness)
Dc = Do + 2*Ft;

% Longitudinal tube pitch (distance between tube centers in direction of
% air flow) in inches
Pl = 1.358;

% Flow length in inches (flow length is explained in Chapter 3)
L = 6.93;

% Flow cross-sectional area, also known as the minimum free flow area, in
% square inches
% Flow cross-sectional area = Frontal Area - Tube Diameter*Number of
% Tubes at cross section*Tube_Length - Fin thickness*Fin height*Number of
% Fins
Ac = 483.84 - 0.375*11*44.254 - Ft*11*308;

% Total heat transfer area in square inches, provided by manufacturer
A = 41908.7;

% Air-side hydraulic diameter in inches as defined by Kays and London
Dh = 4*L*Ac/A;

% Transverse tube pitch in inches (distance between tube centers in
% direction transverse to air flow)
Pt = 1;

% Powers used in the j factor correlation
P3 = -.361 - (.042*N./log(Re)) + .158*log(N*(Fp/Dc)^.41);
P4 = -1.224 - (.076*(Pl/Dh)^1.42)./(log(Re));
P5 = -.083 + (.058*N)./(log(Re));
P6 = -5.735 + 1.21*log(Re/N);

% Colburn j factor correlation (stored in HX structure)
j_cond = .086.*Re.^P3.*N.^P4.*(Fp./Dc).^P5.*(Fp/Dh).^P6*(Fp./Pt).^-.93;

% Calculate the frontal area in square inches
Afr = (10.656*13.062);

% Calculate the constriction ratio (stored in HX structure)
sigma = Ac/Afr;

% Convert Dh from inches to meters (stored in HX structure)
DhSI = Dh*0.0254;

```

Appendix D

COOL MODE MODEL FILES

D.1 Compressor

```
function [output] = CompressorTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global OPTIONS

persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u      = U(1:4);
Time   = U(end-2);
Num     = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Compressor Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1    %Only run this code once at the start of the simulation

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.1 Set Default Physical Parameters of the Compressor
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %Main Volumetric Efficiency Coefficients
    A0 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A0'));
    A1 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A1'));
    A2 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A2'));
    A3 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A3'));
    A4 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A4'));
    A5 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A5'));
    A6 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A6'));
    A7 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A7'));
    A8 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A8'));
    A9 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A9'));
    A10 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A10'));
    A11 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.A11'));
```

```

%Auxiliary Volumetric Efficiency Coefficients (N/A for our compressor model but
%included for completeness)
B0 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B0'));
B1 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B1'));
B2 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B2'));
B3 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B3'));
B4 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B4'));
B5 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B5'));
B6 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B6'));
B7 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B7'));
B8 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B8'));
B9 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B9'));
B10 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B10'));
B11 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.B11'));

%Adiabatic Efficiency Coefficients
C0 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C0'));
C1 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C1'));
C2 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C2'));
C3 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C3'));
C4 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C4'));
C5 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C5'));
C6 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C6'));
C7 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C7'));
C8 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C8'));
C9 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C9'));
C10 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C10'));
C11 = eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.C11'));

%Large Truck compressor displacement in units of in^3/rev
Disp= eval(strcat('Physical_Parameters.CompressorTK',num2str(Num),'.Disp'));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.2 Define Default Initial Operating Conditions for the Compressor
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Pk_ri      - compressor inlet pressure, kPa
% Pk_ro      - compressor outlet pressure, kPa
% Pk_re      - compressor economizer pressure - always zero, kPa
% Hk_ri      - compressor inlet enthalpy, kJ/kg
% rpm_k      - compressor speed, rev/min

Pk_ri = eval(strcat('Operating_Conditions.CompressorTK',num2str(Num),'.Pk_ri'));
Pk_ro = eval(strcat('Operating_Conditions.CompressorTK',num2str(Num),'.Pk_ro'));
Pk_re = eval(strcat('Operating_Conditions.CompressorTK',num2str(Num),'.Pk_re'));
Hk_ri = eval(strcat('Operating_Conditions.CompressorTK',num2str(Num),'.Hk_ri'));
rpm_k = eval(strcat('Operating_Conditions.CompressorTK',num2str(Num),'.rpm_k'));

ModParam = [A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 A11 ...
            B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 B10 B11 ...
            C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 C10 C11 ...
            Disp Pk_re];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Compressor Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

rpm_k = u(1);
Pk_ri = u(2);
Pk_ro = u(3);
Hk_ri = u(4);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

A0      = ModParam(1);
A1      = ModParam(2);
A2      = ModParam(3);
A3      = ModParam(4);
A4      = ModParam(5);
A5      = ModParam(6);
A6      = ModParam(7);
A7      = ModParam(8);
A8      = ModParam(9);
A9      = ModParam(10);
A10     = ModParam(11);
A11     = ModParam(12);

B0      = ModParam(13);
B1      = ModParam(14);
B2      = ModParam(15);
B3      = ModParam(16);
B4      = ModParam(17);
B5      = ModParam(18);
B6      = ModParam(19);
B7      = ModParam(20);
B8      = ModParam(21);
B9      = ModParam(22);
B10     = ModParam(23);
B11     = ModParam(24);

C0      = ModParam(25);
C1      = ModParam(26);
C2      = ModParam(27);
C3      = ModParam(28);
C4      = ModParam(29);
C5      = ModParam(30);
C6      = ModParam(31);
C7      = ModParam(32);
C8      = ModParam(33);
C9      = ModParam(34);
C10     = ModParam(35);
C11     = ModParam(36);

Disp    = ModParam(37);
Pk_re   = ModParam(38);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Model Calculations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%Calculating relevant refrigerant properties
Tk_ri   = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hk_ri,Pk_ri);
Sk_ri   = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.S_ph,Hk_ri,Pk_ri);
Tk_ro   = qinterp2(FluidProp.S_mg_ps,FluidProp.P_mg_ps,FluidProp.T_ps,Sk_ri,Pk_ro);
Hk_ro   = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.H_pt,Tk_ro,Pk_ro);

Rho_ri  = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,Tk_ri,Pk_ri);
Rho_ro  = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,Tk_ro,Pk_ro);
Rhog_i  = qinterp1(FluidProp.Psat, FluidProp.Rhog, Pk_ri);
Rhog_o  = qinterp1(FluidProp.Psat, FluidProp.Rhog, Pk_ro);

%Converting inlet and outlet refrigerant pressure from kPa to psia (for
%correlation)
Pk_ri_Eng = Pk_ri*0.145037738;
Pk_ro_Eng = Pk_ro*0.145037738;

%Converting inlet and outlet refrigerant density from kg/m^3 to lbm/ft^3
Rho_ri_Eng = Rho_ri*0.028316847/0.45359237;
Rho_ro_Eng = Rho_ro*0.028316847/0.45359237;

phi_main = Rho_ro/Rho_ri;
phi_main_sat= Rhog_o/Rhog_i;

%Determining if there is an interpolation error; if so, use saturated
%phi_main
if phi_main_sat - phi_main > 4

```

```

        phi_main    = phi_main_sat;
        Rho_ri_Eng  = Rhog_i*0.028316847/0.45359237;
        Rho_ro_Eng  = Rhog_o*0.028316847/0.45359237;
    end

    Teis           = 0; %Always 0 for our compressor but included for completeness
    phi_aux        = 0; %Always 0 for our compressor but included for completeness

    %Calculating main volumetric and auxiliary volumetric efficiency

    eta_vol        = (A0 + A1*rpm_k + A2*rpm_k^2 + A3*phi_main + A4 + A5*rpm_k*phi_main +
A6*phi_aux ...
                    + A7*phi_main*phi_aux + A8*rpm_k*phi_aux + A11*Pk_ro_Eng);

    eta_aux        = (B6*phi_aux + B7*phi_main*phi_aux + B8*rpm_k*phi_aux +
B9*rpm_k*phi_aux^2 + ...
                    B10*rpm_k^2*phi_aux);

    %Calculating adiabatic efficiency

    eta_adb        = (C0 + C1*rpm_k + C2*rpm_k^2 + C3*phi_main + C4*phi_main^2 +
C5*rpm_k*phi_main + ...
                    C6*phi_aux + C7*phi_main*phi_aux + C8*rpm_k*phi_aux + C11*Pk_ro_Eng);

    %Calculating refrigerant mass flow rate through the compressor in units of lbm/hr

    mdot_main_Eng  = eta_vol*Disp*rpm_k*60/(1728*100/Rho_ri_Eng);

    % %This is zero for our compressor but is included for completeness
    % mdot_aux_Eng  = eta_aux* Disp * rpm_k * 60/(1728*100*VOLUME(refrig$, P
    % = Pk_re, T = Teis));

    mdot_aux_Eng   = 0;
    mdot_k_Eng     = mdot_main_Eng + mdot_aux_Eng;

    %Converting refrigerant mass flow rate from lbm/hr to kg/s

    mdot_main      = mdot_main_Eng*0.45359237/3600;
    mdot_aux       = mdot_aux_Eng*0.45359237/3600;
    mdot_k         = mdot_k_Eng*0.45359237/3600;

    %Calculating power in units of kW
    power = mdot_main*(Hk_ro - Hk_ri)/(eta_adb/100);

    % If using a compressor with auxiliary port, include "flowax *
    % (ENTHALPY(refrig$, P = Pk_ro, T = Tdis) - ENTHALPY(refrig$, P = Pe, T =
    % Teis)) / (effis / 100)" in the calculation for Power

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1) = mdot_k;
y(2) = Hk_ro;
y(3) = Tk_ro;
y(4) = power;
y(5) = eta_vol;
y(6) = eta_adb;

yout = [y(1) y(2)]; %Output mdot_k, Hk_ro
output = [yout y];

return

```

D.2 Condenser with Receiver

```
function [output] = CondwRecTK(U)
```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global OPTIONS
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x      = U(1:6);
u      = U(7:12);
Time   = U(end-2);
HX_num = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Condenser with Receiver Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

    % Set Default Parameters of the Condenser

    Mass_c      =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Mass_c'));
    Cpw_c      =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Cpw_c'));
    Ac_o       = eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Ac_o'));
    Diameter_c =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Diameter_c'));
    Lc_total   =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Lc_total'));
    Ac_i       = eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Ac_i'));
    Ac_cs      =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Ac_cs'));
    Ac_fr      =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.Ac_fr'));
    V_rec      =
eval(strcat('Physical_Parameters.CondwRecTK',num2str(HX_num),'.V_rec'));
    Slip_c     =
eval(strcat('Operating_Conditions.CondwRecTK',num2str(HX_num),'.Slip_c'));

    UA_rec     =
eval(strcat('Operating_Conditions.CondwRecTK',num2str(HX_num),'.UA_rec'));

    Cpc_air    =
eval(strcat('Initial_Conditions.CondwRecTK',num2str(HX_num),'.Cpc_air'));
    mc_air     = eval(strcat('Initial_Conditions.CondFanTK',num2str(HX_num),'.mc_air'));

    ModParam = [Lc_total Ac_cs Ac_i Ac_o Mass_c Cpw_c Cpc_air V_rec UA_rec Slip_c Ac_fr
Diameter_c];
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Nonlinear Condenser with Receiver Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 States and Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

mdot_i_total= u(1);    % Mass flow rate entering the heat exchanger
mdot_o_total= u(2);    % Mass flow rate exiting the receiver
Hc_ri       = u(3);    % Enthalpy of entering refrigerant
Tc_ai       = u(4);    % Inlet air temperature

```



```

mc_air_total= u(5);      % Mass flow rate of air across the heat exchanger
Tamb         = u(6);      % Ambient air temperature

L1           = x(1);      % Length of the first region
Pc           = x(2);      % Condenser pressure
Gamma        = x(3);      % Value of the mean void fraction in 2-phase flow region
m_rec        = x(4);      % Mass of refrigerant in the receiver
Tw1          = x(5);      % Average tube wall temperature in the first region
Tw2          = x(6);      % Average tube wall temperature in the second region

%Inlet and Outlet refrigerant mass flow rate, and air mass flow rate,
%adjusted for the 2 parallel circuits inside the heat exchanger
mdot_i       = mdot_i_total/2;
mdot_o       = mdot_o_total/2;
mc_air       = mc_air_total/2;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Lc_total     = ModParam(1);
% Total length of heat exchanger "tubes" irrespective of the geometry,
% the heat exchanger is assumed to be a single tube)
Ac_cs        = ModParam(2);      % Total cross sectional area for fluid flow
Ac_i         = ModParam(3);      % Total internal surface area
Ac_o         = ModParam(4);      % Total exterior surface area
m            = ModParam(5);      % Mass of heat exchanger
Cpw          = ModParam(6);      % Specific heat of the heat exchanger
Cpc_air      = ModParam(7);      % Specific heat of the external fluid
Vol_rec      = ModParam(8);      % Internal volume of the receiver
UA_rec       = ModParam(9);      % Lumped heat transfer parameter for receiver
Slip_c       = ModParam(10);     % Two-phase region void fraction slip ratio
Ac_fr        = ModParam(11);     % Frontal Area
Diameter_c   = ModParam(12);     % Diameter

L2           = Lc_total-L1;      % Length of second region

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Calculate Refrigerant Properties
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Density, Enthalpy, and Internal Energy

% Saturated liquid density
% Saturated vapor density
% Saturated liquid enthalpy
% Saturated vapor enthalpy

Rho_F        = qinterp1(FluidProp.Psat,FluidProp.Rhof,Pc);
Rho_G        = qinterp1(FluidProp.Psat,FluidProp.Rhog,Pc);
Hc_f         = qinterp1(FluidProp.Psat,FluidProp.Hf,Pc);
Hc_g         = qinterp1(FluidProp.Psat,FluidProp.Hg,Pc);

% Average enthalpy of the refrigerant in the first region
% Saturated liquid internal energy
% Saturated vapor internal energy
% Average density of refrigerant in the first region

Hc_r1        = (Hc_ri+Hc_g)/2;
uc_f         = qinterp1(FluidProp.Psat,FluidProp.Uf,Pc);
uc_g         = qinterp1(FluidProp.Psat,FluidProp.Ug,Pc);
Rho_1        = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.Rho_ph,Hc_r1,Pc);

% Partial Derivatives

% Partial derivative of the saturated liquid density with respect to pressure
% Partial derivative of the saturated vapor density with respect to pressure
% Partial derivative of the average density in the first region with respect to
pressure

```

```

% Partial derivative of the average density in the first region with respect to
enthalpy
% Partial derivative of the saturated vapor enthalpy with respect to pressure
% Partial derivative of the saturated liquid enthalpy with respect to pressure
% Partial derivative of the saturated liquid internal energy with respect to pressure
% Partial derivative of the saturated vapor internal energy with respect to pressure

dRhoF_dP = qinterp1(FluidProp.Psat,FluidProp.dRhof_dP,Pc);
dRhoG_dP = qinterp1(FluidProp.Psat,FluidProp.dRhog_dP,Pc);
dRho1_dP =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.dRho_dP_H_ph,Hc_r1,Pc);
dRho1_dh =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.dRho_dH_P_ph,Hc_r1,Pc);
dhG_dP = qinterp1(FluidProp.Psat,FluidProp.dHg_dP,Pc);
dhF_dP = qinterp1(FluidProp.Psat,FluidProp.dHf_dP,Pc);
duF_dP = qinterp1(FluidProp.Psat,FluidProp.dUf_dP,Pc);
duG_dP = qinterp1(FluidProp.Psat,FluidProp.dUg_dP,Pc);

% Fluid Temperatures and Specific Heats

% Average refrigerant temperature in the first region
% Average refrigerant temperature in the second region

Tc_r1 = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hc_r1,Pc);
Tc_r2 = qinterp1(FluidProp.Psat,FluidProp.Tsat,Pc);
Tc_ri = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hc_ri,Pc);
Tcsh = Tc_ri-Tc_r2;

% Average refrigerant viscosity in the first region
% Average refrigerant specific heat in the first region
% Average refrigerant conductivity in the first region

mu_r1 = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Tc_r1,Pc);
Cp_r1 = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Cp_pt,Tc_r1,Pc);
k_r1 = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.k_pt,Tc_r1,Pc);

% Solve for condenser outlet quality and enthalpy
Hc_ro = (UA_rec*(Tc_r2-Tamb))/mdot_o + Hc_f;
Xc_ro = (Hc_ro-Hc_f)/(Hc_g-Hc_f);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.4 Determine heat transfer coefficients
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Qc_1 = mdot_i*(Hc_ri-Hc_g);
Qc_2 = mdot_i*(Hc_g-Hc_ro);
Qc_total = Qc_1+Qc_2;
G_r = mdot_i/Ac_cs;
Rc_r1 = G_r*Diameter_c/mu_r1;
Relrough_c = 1e-6/Diameter_c;

ff_r1 = F_1PH_CHURCHILL(Rc_r1,Relrough_c);

% refrigerant to HX wall heat transfer coefficient in the single phase region
% refrigerant to HX wall heat transfer coefficient in the two phase region
% heat transfer coefficient between the HX wall and the external fluid

alpha_c_i1 = (H_1PH_GNIELINSKI(Diameter_c, G_r, ff_r1, Cp_r1, mu_r1, k_r1));
alpha_c_i2 = (H_COND_DOBSON(Diameter_c, G_r, Pc, 0.5*(Xc_ro+1)));
alpha_c_o = H_COLBURN_FACTOR(26, mc_air, Ac_fr, Tc_ai);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.5 Calculate Air Temperatures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Tc_a2 = ((2*mc_air*Cpc_air*Tc_ai) + (alpha_c_o*Ac_o*(L2/Lc_total)*Tw2))/...
(2*mc_air*Cpc_air + alpha_c_o*Ac_o*(L2/Lc_total));
Tc_am = 2*Tc_a2 - Tc_ai;
Tc_a1 = ((2*mc_air*Cpc_air*Tc_am) + (alpha_c_o*Ac_o*(L1/Lc_total)*Tw1))/...
(2*mc_air*Cpc_air + alpha_c_o*Ac_o*(L1/Lc_total));
Tc_a0 = 2*Tc_a1 - Tc_am;

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.6 Other Calculations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

A_mvf      = (Rho_G/Rho_F)*Slip_c;
B_mvf      = 1 - A_mvf;
x_int2     = 1/(B_mvf*exp((B_mvf-Gamma*B_mvf^2)/A_mvf)) - A_mvf/B_mvf;
% Enthalpy of refrigerant exiting the heat exchanger and entering the
% receiver
hint       = Hc_f*(1-x_int2) + Hc_g*(x_int2);

% Calculated inputs

% The temperature of the refrigerant in the receiver
% Enthalpy of exiting refrigerant
% Average density in the 2-phase region

Trec       = Tc_r2;
Hc_ro      = Hc_f;
Rho_2      = Rho_F*(1-Gamma)+Rho_G*Gamma;

% Receiver equation terms
VG_rec     = (Vol_rec*Rho_F-m_rec)/(Rho_F-Rho_G);
VF_rec     = Vol_rec-VG_rec;
mG_rec     = VG_rec*Rho_G;
mF_rec     = VF_rec*Rho_F;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.7 Calculate Nonlinear State Space System, derivatives, and outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Z matrix
Z = zeros(6,6);
Z(1,1)=(Rho_1*Hc_r1-Rho_1*Hc_g)*Ac_cs;
Z(1,2)=((dRho1_dP+0.5*dRho1_dh*dhG_dP)*(Hc_r1-Hc_g)+(0.5*dhG_dP)*Rho_1-1)*Ac_cs*L1;
Z(2,1)=(Rho_F*Hc_g-Rho_F*Hc_f)*(1-Gamma)*Ac_cs;
Z(2,2)=((dRhoF_dP*Hc_f + dhF_dP*Rho_F - dRhoF_dP*Hc_g)*(1-Gamma)+...
(dRhoG_dP*Hc_g + dhG_dP*Rho_G - dRhoG_dP*Hc_g)*(Gamma)-1)*Ac_cs*L2;
Z(2,3)=(Rho_F*Hc_g-Rho_F*Hc_f)*Ac_cs*L2;
Z(2,4)=hint-Hc_g;
Z(3,1)=(Rho_1-Rho_F*(1-Gamma)-Rho_G*(Gamma))*Ac_cs;
Z(3,2)=((dRhoF_dP*(1-
Gamma)+dRhoG_dP*(Gamma))*L2+(dRho1_dP+0.5*dRho1_dh*dhG_dP)*L1)*Ac_cs;
Z(3,3)=(Rho_G-Rho_F)*Ac_cs*L2;
Z(3,4)=1;
Z(4,2)=dRhoG_dP*VG_rec*uc_g+dRhoF_dP*VF_rec*uc_f+mG_rec*duG_dP+mF_rec*duF_dP-...
((Rho_G*uc_g-Rho_F*uc_f)/(Rho_G-Rho_F))*(dRhoG_dP*VG_rec+dRhoF_dP*VF_rec);
Z(4,4)=(Rho_G*uc_g-Rho_F*uc_f)/(Rho_G-Rho_F)-hint;
Z(5,1)=Cpw*m*(Tw1-Tw2)/L1;
Z(5,5)=Cpw*m;
Z(6,6)=Cpw*m;

% f vector
f(1,1)=mdot_i_total*(Hc_ri-Hc_g)+ alpha_c_i1*Ac_i*(L1/Lc_total)*(Tw1-Tc_r1);
f(2,1)=mdot_o_total*(Hc_g-hint)+ alpha_c_i2*Ac_i*(L2/Lc_total)*(Tw2-Tc_r2);
f(3,1)=mdot_i_total-mdot_o_total;
f(4,1)=mdot_o_total*(hint-Hc_f)-0*(Trec-Tamb);
f(5,1)=alpha_c_i1*Ac_i*(Tc_r1-Tw1)+alpha_c_o*Ac_o*(Tc_a1-Tw1);
%adjusted for counterflow configuration (Tc_a --> Tc_a1)
f(6,1)=alpha_c_i2*Ac_i*(Tc_r2-Tw2)+alpha_c_o*Ac_o*(Tc_a2-Tw2);
%adjusted for counterflow configuration (Tc_a --> Tc_a2)

% Calculate Derivatives
x_dot = Z\f;

% Refrigerant Mass Inventory
Mass = Ac_cs*L1*Rho_1 + Ac_cs*L2*Rho_2;
% Outlet Refrigerant Temperature
Tc_ro = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hc_ro,Pc);

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Check for violations of modeling assumptions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

m_rec_max = Rho_F*Vol_rec;
m_rec_min = Rho_G*Vol_rec;

if x(1) <= 0,
    disp('Modeling Assumptions Violated :-(')
    disp('Length of superheat region has gone to zero.')
    % set_param(gcs,'SimulationCommand','stop')
end

if (x(1)/Lc_total) >= 1,
    disp('Modeling Assumptions Violated :-(')
    disp('Length of two-phase region has gone to zero.')
    % set_param(gcs,'SimulationCommand','stop')
end

if u(1) < 0,
    disp('Modeling Assumptions Violated :-(')
    disp('Negative mass flow rate into heat exchanger is not allowed.')
    % set_param(gcs,'SimulationCommand','stop')
end

if x(4) > m_rec_max,
    disp('Modeling Assumptions Violated :-(')
    disp('Refrigerant mass in the receiver has exceeded the maximal allowable value.')
    % set_param(gcs,'SimulationCommand','stop')
end

if x(4) < m_rec_min,
    disp('Modeling Assumptions Violated :-(')
    disp('Refrigerant mass in the receiver has decreased below the minimal allowable value.')
    % set_param(gcs,'SimulationCommand','stop')
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 6.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1)=L1;
y(2)=Pc;
y(3)=Gamma;
y(4)=Hc_ro;
y(5)=Tw1;
y(6)=Tw2;
y(7)=Tc_ao;
y(8)=Tc_ro;
y(9)=Mass;
y(10)=m_rec;

yout = [y(2) y(2) y(4) y(7)]; %Output Pin, Pout, Hout, Tc_ao
output = [x_dot' yout y];

return

```

D.3 Suction Line Heat Exchanger (SLHX)

```
function [output] = SuctionLineHX_TK(U)
```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

global Initial_Conditions
global Physical_Parameters

```

```

global Operating_Conditions
global FluidProp
global OPTIONS

persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u      = U(1:6);
Time   = U(end-2);
Num     = U(end-1);
flag    = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Suction Line HX Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1
    %Only run this code once at the start of the simulation

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.1 Set Default Physical Parameters of the Suction Line HX
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %Correlation Coefficients
    A = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.A'));
    B = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.B'));
    C = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.C'));
    %Tube and Shell Diameters and Lengths
    D1 = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.D1'));
    D2 = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.D2'));
    D3 = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.D3'));
    D4 = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.D4'));
    L_l = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.L_l'));
    L_v = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.L_v'));
    D_l = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.D_l'));
    %Total Surface Area
    A_l = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.A_l'));
    A_v = eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Num),'.A_v'));

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.2 Define Default Initial Operating Conditions for the Suction Line
    % HX
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %Calculation of additional physical parameters
    D_v = ((D1^2)-((D2^2)/2)+((D3^2)/2)-(D4^2))/(D1+(D2/2)+(D3/2)+D4);

    ModParam = [A B C D1 D2 D3 D4 L_l L_v D_l D_v A_l A_v];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Suction Line HX Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Hliq_i = u(1);
Pliq_i = u(2);
Hvap_i = u(3);
Pvap_i = u(4);
mdot_l = u(5);
mdot_v = u(6);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
A      = ModParam(1);
B      = ModParam(2);
C      = ModParam(3);
D1     = ModParam(4);
D2     = ModParam(5);
D3     = ModParam(6);
D4     = ModParam(7);
L_l    = ModParam(8);
L_v    = ModParam(9);
D_l    = ModParam(10);
D_v    = ModParam(11);
A_l    = ModParam(12);
A_v    = ModParam(13);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

% 4.3 Model Calculations

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%Calculating relevant refrigerant properties
Tliq_i = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hliq_i,Pliq_i);
Tvap_i = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hvap_i,Pvap_i);
rho_l  =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,Tliq_i,Pliq_i);
rho_v  =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,Tvap_i,Pvap_i);
mu_l   =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Tliq_i,Pliq_i);
mu_v   =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Tvap_i,Pvap_i);
k_l    = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.k_pt,Tliq_i,Pliq_i);
k_v    = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.k_pt,Tvap_i,Pvap_i);
cp_l   =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Cp_pt,Tliq_i,Pliq_i);
cp_v   =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Cp_pt,Tvap_i,Pvap_i);
```

```
%Calculating Mass Flux
Acs_l  = pi/4*(D_l^2);
Acs_v  = pi/4*(D_v^2);
G_l    = mdot_l/Acs_l;
G_v    = mdot_v/Acs_v;
```

```
%Calculating Velocity
V_l    = (mdot_l/rho_l)/Acs_l;
V_v    = (mdot_v/rho_v)/Acs_v;
```

```
%Calculating Reynolds number
Re_l   = G_l*D_l/mu_l;
Re_v   = G_v*D_v/mu_v;
```

```
%Calculating Prandtl number
Pr_l   = mu_l*cp_l/k_l;
Pr_v   = mu_v*cp_v/k_v;
```

```
%Determining pressure drop across the liquid side of the HX (kPa)
deltaP_l = A*(Re_l^0.2)*(L_l/D_l)*(rho_l*(V_l^2)/2)/1000;
%Determining pressure drop across the vapor side of the HX (kPa)
deltaP_v = B*(Re_v^0.2)*(L_v/D_v)*(rho_v*(V_v^2)/2)/1000;
%Determining the UA product for the HX
UA      = C*((Re_v^0.8)*(Pr_v^(1/3))*k_v*A_v*(Re_l^0.8)*...
            (Pr_l^(1/3))*k_l*A_l)/(((Re_v^0.8)*(Pr_v^(1/3))*...
            k_v*A_v*D_l) + ((Re_l^0.8)*(Pr_l^(1/3))*k_l*A_l*D_v));
```

```
%Calculating outlet pressures
Pvap_o  = Pvap_i - deltaP_v;
Pliq_o  = Pliq_i - deltaP_l;
```

```
%Solving iteratively for outlet liquid and vapor temperatures
```

```

i = 1;
j = 1;
for Tvp_o = [Tliq_i:-0.2:Tvp_i]
    Qvap = mdot_v*cp_v*(Tvp_o-Tvp_i);
    Qliq = Qvap;
    Tliq_o = Tliq_i - (Qliq/(mdot_l*cp_l));
    dT1 = Tliq_i - Tvp_i;
    dT2 = Tliq_o - Tvp_o;
    dTlmt = (dT2-dT1)/(log(dT2/dT1));
    if imag(dTlmt)~= 0
        extraneous(j) = dTlmt;
        j = j + 1;
    else
        Qtot = UA*dTlmt;
        Error(i) = (Qtot - Qvap)^2;
        soln(i,:) = [Tliq_o Tvp_o];
        i = i + 1;
    end
end

[value, index] = min(Error);
solution = soln(index,:);
Tliq_o = solution(1);
Tvp_o = solution(2);

%Interpolating for outlet liquid and vapor enthalpies
Hvap_o = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.H_pt,Tvp_o,Pvap_o);
Hliq_o = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.H_pt,Tliq_o,Pliq_o);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1) = Pvap_o;
y(2) = Pliq_o;
y(3) = Hvap_o;
y(4) = Hliq_o;
y(5) = Tvp_o;
y(6) = Tliq_o;

yout = [y(1) y(2) y(3) y(4)]; %Output Pvap_o, Pliq_o, Hvap_o, Hliq_o
output = [yout y];

return

```

D.4 Electronic Expansion Valve (EEV)

```

function [output] = EEV_TK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global ValveProp
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u = U(1:4);
Time = U(end-2);
Num = U(end-1);

```

```

flag      = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Electronic Expansion Valve Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.1 Define Default Physical Parameters for the EEV
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

A0 = eval(strcat('Physical_Parameters.EEV_TK',num2str(Num),'.A0'));
A1 = eval(strcat('Physical_Parameters.EEV_TK',num2str(Num),'.A1'));
A2 = eval(strcat('Physical_Parameters.EEV_TK',num2str(Num),'.A2'));
A3 = eval(strcat('Physical_Parameters.EEV_TK',num2str(Num),'.A3'));
A4 = eval(strcat('Physical_Parameters.EEV_TK',num2str(Num),'.A4'));
A5 = eval(strcat('Physical_Parameters.EEV_TK',num2str(Num),'.A5'));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.2 Define Default Initial Operating Conditions for the EEV
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Pv_ri;      %valve inlet pressure
Pv_ro;      %valve outlet pressure
Hv_ri;      %valve inlet enthalpy
ValveCommand; %valve stroke command

Pv_ri      = eval(strcat('Operating_Conditions.EEV_TK',num2str(Num),'.Pv_ri'));
Pv_ro      = eval(strcat('Operating_Conditions.EEV_TK',num2str(Num),'.Pv_ro'));
Hv_ri      = eval(strcat('Operating_Conditions.EEV_TK',num2str(Num),'.Hv_ri'));
ValveOpen  =
eval(strcat('Operating_Conditions.EEV_TK',num2str(Num),'.ValveOpen'));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.3 Initial Conditions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Load necessary initial conditions

Kv;      %correction factor between user-specified mass flow
%        initial condition and performance map mass flow initial condition
Kv      = eval(strcat('Initial_Conditions.EEV_TK',num2str(Num),'.Kv'));

% Define Model Parameters
ModParam = [A0 A1 A2 A3 A4 A5 Kv];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Electronic Expansion Valve Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 States and Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Pv_ri      = u(1);
Pv_ro      = u(2);
Hv_ri      = u(3);
ValveOpen  = u(4);

```



```

A0      = ModParam(1);
A1      = ModParam(2);
A2      = ModParam(3);
A3      = ModParam(4);
A4      = ModParam(5);
A5      = ModParam(6);
Kv      = ModParam(7);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Refrigerant Properties
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Inlet refrigerant temperature
% Saturation temperature based on inlet pressure
% Subcooling at EEV inlet
% Saturated liquid inlet density
% Subcooled liquid inlet density
% Choose the maximum of saturated and subcooled densities
% Differential pressure across EEV

Tv_ri   = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hv_ri,Pv_ri);
Tsatsat = qinterp1(FluidProp.Psat,FluidProp.Tsat,Pv_ri);
Tsc      = Tsatsat - Tv_ri;
rho_v_sat = qinterp1(FluidProp.Psat,FluidProp.Rhof,Pv_ri);
rho_v_sub =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.Rho_ph,Hv_ri,Pv_ri);
rho_v     = max([rho_v_sat rho_v_sub]);
dP        = Pv_ri - Pv_ro;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Mass Flow
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Ka       = A0 + A1*ValveOpen + A2*Tsc + A3*ValveOpen*Tsc + A4*ValveOpen^2 +
A5*Tsc^2;
mdot_v_pred = Ka*sqrt(rho_v*dP);
mdot_v      = mdot_v_pred + Kv;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.4 Outlet Enthalpy, Temperature
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Assume isenthalpic process
Hv_ro     = Hv_ri;
% Outlet refrigerant temperature
Tv_ro     =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hv_ri,Pv_ro);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1)= mdot_v;
y(2)= Tv_ro;
y(3)= Hv_ro;
y(4)= Ka;
y(5)= Tsc;

yout = [y(1) y(3)]; %Output mdot_v, Hv_ro
output = [yout y];

return

```

D.5 Distributor – Cool Mode

```
function [output] = DistributorTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global OPTIONS

persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x      = U(1:2);
u      = U(3:4);
Time   = U(end-2);
Num    = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Distributor Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

    %Set Default Physical Parameters of the Distributor

    u1_detrend =
eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.u1_detrend'));
    u2_detrend =
eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.u2_detrend'));
    y1_detrend =
eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.y1_detrend'));
    y2_detrend =
eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.y2_detrend'));

    ModParam = [u1_detrend u2_detrend y1_detrend y2_detrend];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Distributor Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 States and Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x1      = x(1);
x2      = x(2);

mdot_d  = u(1);
Pd_ro   = u(2);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u1_detrend = ModParam(1);
u2_detrend = ModParam(2);
y1_detrend = ModParam(3);
y2_detrend = ModParam(4);
```

```

A      = eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.A'));
B      = eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.B'));
C      = eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.C'));
D      = eval(strcat('Physical_Parameters.DistributorTK',num2str(Num),'.D'));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Calculations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%Detrend inputs
u1  = mdot_d - u1_detrend;
u2  = Pd_ro - u2_detrend;

Ud  = [u1; u2];
Xd  = [x1; x2];

Y      = C*Xd + D*Ud;
x_dot  = A*Xd + B*Ud;

Pd_ri  = Y(1) + y1_detrend;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1)= Pd_ri;

x_dot  = x_dot';
yout   = [Pd_ri];
output = [x_dot yout y];

return

```

D.6 Evaporator – Cool Mode

```

function [output] = EvapTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x      = U(1:5);
u      = U(6:10);
Time   = U(end-2);
HX_num = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Evaporator Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.1 Set Default Parameters of the Evaporator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Mass_e      = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Mass_e'));

```

```

Cpw_e      = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Cpw_e'));
Ae_o       = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Ae_o'));
Diameter_e =
eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Diameter_e'));
Le_total   = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Le_total'));
Ae_i       = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Ae_i'));
Ae_cs      = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Ae_cs'));
Ae_fr      = eval(strcat('Physical_Parameters.EvapTK',num2str(HX_num),'.Ae_fr'));

Slip_e     = eval(strcat('Operating_Conditions.EvapTK',num2str(HX_num),'.Slip_e'));
Cpe_air    = eval(strcat('Operating_Conditions.EvapTK',num2str(HX_num),'.Cpe_air'));

% Te_a_mu   = eval(strcat('Initial_Conditions.EvapTK',num2str(HX_num),'.Te_a_mu'));
% me_air_total=
eval(strcat('Initial_Conditions.EvapFanTK',num2str(HX_num),'.me_air'));

ModParam   = [Le_total Ae_cs Ae_i Ae_o Mass_e Cpw_e Cpe_air Slip_e Ae_fr
Diameter_e];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Nonlinear Evaporator Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 States and Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Le_1       = x(1);           % Length of the first region
Pe         = x(2);           % Evaporator Pressure
He_ro      = x(3);           % Outlet enthalpy
Te_w1      = x(4);           % Average tube wall temperature in the first region
Te_w2      = x(5);           % Average tube wall temperature in the second region

mdot_i_total= u(1);          % Mass flow rate entering the heat exchanger
mdot_o_total= u(2);          % Mass flow rate exiting the heat exchanger
He_ri      = u(3);           % Inlet Enthalpy
Te_ai      = u(4);           % Inlet air temperature
me_air_total= u(5);          % Mass flow rate of air across the heat exchanger

%Inlet and Outlet refrigerant mass flow rate, and air mass flow rate,
%adjusted for the 11 parallel circuits inside the heat exchanger
mdot_i      = mdot_i_total/11;
mdot_o      = mdot_o_total/11;
me_air      = me_air_total/11;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Le_total    = ModParam(1);
% Total length of heat exchanger "tubes" irrespective of the geometry,
% the heat exchanger is assumed to be a single tube)
Ae_cs       = ModParam(2); % Total cross sectional area for fluid flow
Ae_i        = ModParam(3); % Total internal surface area
Ae_o        = ModParam(4); % Total exterior surface area
Mass_e      = ModParam(5); % Mass of heat exchanger
Cpw_e       = ModParam(6); % Specific heat of the heat exchanger
Cpe_air     = ModParam(7); % Specific heat of the external fluid
Slip_e      = ModParam(8); % Slip ratio for void fraction
Ae_fr       = ModParam(9); % Heat exchanger frontal area
Diameter_e  = ModParam(10); % Hydarulic diameter of heat exchanger tubes

Le_2        = Le_total-Le_1;% Length of second region

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Calculate Refrigerant Properties
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Saturated liquid enthalpy

```

```

% Saturated vapor enthalpy
% Enthalpy of vaporization
% Average enthalpy in the second region
He_f      = qinterp1(FluidProp.Psat,FluidProp.Hf,Pe);
He_g      = qinterp1(FluidProp.Psat,FluidProp.Hg,Pe);
He_fg     = He_g-He_f;
He_r2     = (He_g+He_ro)/2;

% Saturated liquid density
% Saturated vapor density
% Average density of refrigerant in the second region
Rho_F     = qinterp1(FluidProp.Psat,FluidProp.Rhof,Pe);
Rho_G     = qinterp1(FluidProp.Psat,FluidProp.Rhog,Pe);
Rho_2     =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.Rho_ph,He_r2,Pe);

% Average refrigerant temperature in the first region
% Average refrigerant temperature in the second region
% Average refrigerant temperature in the second region
% Average refrigerant specific heat in the second region
% Average refrigerant viscosity in the second region
% Average refrigerant conductivity in the second region
% Inlet Temperature
% Superheat Temperature
Te_r1     = qinterp1(FluidProp.Psat,FluidProp.Tsat,Pe);
Te_r2     = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,He_r2,Pe);
Te_ro     = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,He_ro,Pe);
Cp_r2     = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Cp_pt,Te_r2,Pe);
mu_r2     = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Te_r2,Pe);
k_r2      = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.k_pt,Te_r2,Pe);
Te_ri     = Te_r1;
Te_sh     = Te_ro-Te_ri;

% Partial derivative of the saturated liquid enthalpy with respect to pressure
% Partial derivative of the saturated vapor enthalpy with respect to pressure
% Partial derivative of the saturated liquid density with respect to pressure
% Partial derivative of the saturated vapor density with respect to enthalpy
% Partial derivative of the product of saturated liquid density and
% enthalpy with respect to pressure
% Partial derivative of the product of saturated vapor density and
% enthalpy with respect to pressure
% Partial derivative of the average density in the second region with respect to
pressure
% Partial derivative of the average density in the second region with respect to
enthalpy
dhF_dP    = qinterp1(FluidProp.Psat,FluidProp.dHf_dP,Pe);
dhG_dP    = qinterp1(FluidProp.Psat,FluidProp.dHg_dP,Pe);
dRhoF_dP  = qinterp1(FluidProp.Psat,FluidProp.dRhof_dP,Pe);
dRhoG_dP  = qinterp1(FluidProp.Psat,FluidProp.dRhog_dP,Pe);
dRhoFhF_dP = dRhoF_dP*He_f+Rho_F*dhF_dP;
dRhoGhG_dP = dRhoG_dP*He_g+Rho_G*dhG_dP;
dRho2_dP  =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.dRho_dP_H_ph,He_r2,Pe);
dRho2_dh  =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.dRho_dH_P_ph,He_r2,Pe);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.4 Determine heat transfer coefficients
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Qe_1      = mdot_i*(He_g-He_ri);
Xe_ri     = (He_ri-He_f)/(He_g-He_f);
Ae_i1     = (Le_1/Le_total)*Ae_i;
G_ri      = mdot_i/Ae_cs;
G_ro      = mdot_o/Ae_cs;
Re_r2     = G_ro*Diameter_e/mu_r2;
Relrough_e = 1e-6/Diameter_e;
ff_r      = F_1PH_CHURCHILL(Re_r2,Relrough_e);

alpha_e_i1 = H_EVAP_WATTLET(Diameter_e, G_ri, Pe, 0.5*(Xe_ri+1), Qe_1/Ae_i1);
alpha_e_i2 = H_1PH_GNIELINSKI(Diameter_e, G_ro, ff_r, Cp_r2, mu_r2, k_r2);

```

```

alpha_e_o = H_COLBURN_FACTOR(27, me_air, Ae_fr, Te_ai);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.5 Calculate Air Temperatures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Te_a2 = ((2*me_air*Cpe_air*Te_ai) + (alpha_e_o*Ae_o*(Le_2/Le_total)*Te_w2))/...
        (2*me_air*Cpe_air + alpha_e_o*Ae_o*(Le_2/Le_total));
Te_am = 2*Te_a2 - Te_ai;
Te_a1 = ((2*me_air*Cpe_air*Te_am) + (alpha_e_o*Ae_o*(Le_1/Le_total)*Te_w1))/...
        (2*me_air*Cpe_air + alpha_e_o*Ae_o*(Le_1/Le_total));
Te_ao = 2*Te_a1 - Te_am;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.6 Determine Mean Void Fraction
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if He_ri<=He_f
    x1 = 0;
else
    x1 = (He_ri-He_f)/He_fg;
end

if He_ro>=He_g
    x2 = 1;
else
    x2 = (He_ro-He_f)/He_fg;
end

A_mvf = (Rho_G/Rho_F)*Slip_e;
B_mvf = 1-A_mvf;
% Value of the Mean Void Fraction in the 2-phase region
Gamma = 1/B_mvf+(1/(x2-
x1))*(A_mvf/(B_mvf^2))*log((B_mvf*x1+A_mvf)/(B_mvf*x2+A_mvf)));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.7 Calculate Nonlinear State Space System, derivatives, and outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Z matrix
Z = zeros(5,5);
Z(1,1) = Rho_F*(He_f-He_g)*(1-Gamma)*Ae_cs;
Z(1,2) = ((dRhoFhF_dP-dRhoF_dP*He_g)*(1-Gamma)+...
        (dRhoGhG_dP-dRhoG_dP*He_g)*(Gamma)-1)*Ae_cs*Le_1;
Z(2,1) = 0.5*Rho_2*(He_g-He_ro)*Ae_cs;
Z(2,2) = ((dRho2_dP+0.5*dRho2_dh*dhG_dP)*(0.5*(He_ro-He_g))+...
        0.5*Rho_2*dhG_dP-1)*Ae_cs*Le_2;
Z(2,3) = (0.5*dRho2_dh*(0.5*(He_ro-He_g))+0.5*Rho_2)*Ae_cs*Le_2;
Z(3,1) = ((Rho_G-Rho_2)+(Rho_F-Rho_G)*(1-Gamma))*Ae_cs;
Z(3,2) = ((dRho2_dP+0.5*dRho2_dh*dhG_dP)*Le_2+((dRhoF_dP)*...
        (1-Gamma)+(dRhoG_dP)*(Gamma))*Le_1)*Ae_cs;
Z(3,3) = 0.5*dRho2_dh*Ae_cs*Le_2;
Z(4,4) = Cpw_e*Mass_e;
Z(5,1) = Cpw_e*Mass_e*(Te_w1-Te_w2)/Le_2;
Z(5,5) = Cpw_e*Mass_e;

% F vector
f(1,1) = mdot_i*He_ri-mdot_i*He_g+alpha_e_i1*(Ae_i/Le_total)*Le_1*(Te_w1-Te_r1);
f(2,1) = mdot_o*He_g-mdot_o*He_ro+alpha_e_i2*(Ae_i/Le_total)*Le_2*(Te_w2-Te_r2);
f(3,1) = mdot_i-mdot_o;
f(4,1) = alpha_e_i1*Ae_i*(Te_r1-Te_w1)+alpha_e_o*Ae_o*(Te_a1-Te_w1);
f(5,1) = alpha_e_i2*Ae_i*(Te_r2-Te_w2)+alpha_e_o*Ae_o*(Te_a2-Te_w2);

% Calculate Derivatives
x_dot = Z\f;

% Refrigerant Mass Inventory

% Average density in the 2-phase region

```

```

Rho_1 = Rho_F*(1-Gamma)+Rho_G*Gamma;
minv_e = Ae_cs*Le_1*Rho_1 + Ae_cs*Le_2*Rho_2;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Check for violations of modeling assumptions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if (x(1)/ModParam(1)) >= 0.99,
    disp('Modeling Assumptions Violated :-(')
    disp('Length of superheat region has gone to zero.')
    % set_param(gcs,'SimulationCommand','stop')
end

if x(1) <= 0.01,
    disp('Modeling Assumptions Violated :-(')
    disp('Length of two-phase region has gone to zero.')
    % set_param(gcs,'SimulationCommand','stop')
end

if u(1) < 0,
    disp('Modeling Assumptions Violated :-(')
    disp('Negative mass flow rate into heat exchanger is not allowed.')
    % set_param(gcs,'SimulationCommand','stop')
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 6.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

y(1) = x(1);
y(2) = x(2);
y(3) = x(3);
y(4) = x(4);
y(5) = x(5);
y(6) = Te_ao;
y(7) = Te_ro;
y(8) = Te_sh;
y(9) = minv_e;
y(10) = Gamma;

yout = [y(2) y(2) y(3) y(6)]; %Output Pin, Pout, He_ro, Te_ao
output = [x_dot' yout y];

return

```

D.7 Accumulator – Cool Mode

```

function [output] = AccumulatorTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global MoistAirProp
global OPTIONS
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x = U(1:2);
u = U(3:5);

```

```

Time      = U(end-2);
Num       = U(end-1);
flag      = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Accumulator Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.1 Set Default Physical Parameters of the Accumulator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   d_i      - inlet pipe diameter
%   d_o      - outlet pipe diameter
%   d_accum  - accumulator diameter
%   Cp_steel - specific heat of steel
%   L_tank   - height of accumulator tank
%   M_tank   - mass of accumulator tank

d_i      = eval(strcat('Physical_Parameters.AccumulatorTK',num2str(Num),'.d_i'));
d_o      = eval(strcat('Physical_Parameters.AccumulatorTK',num2str(Num),'.d_o'));
d_tank   =
eval(strcat('Physical_Parameters.AccumulatorTK',num2str(Num),'.d_tank'));
Cp_steel =
eval(strcat('Physical_Parameters.AccumulatorTK',num2str(Num),'.Cp_steel'));
L_tank   =
eval(strcat('Physical_Parameters.AccumulatorTK',num2str(Num),'.L_tank'));
M_tank   =
eval(strcat('Physical_Parameters.AccumulatorTK',num2str(Num),'.M_tank'));

K_i      = (1 - ((d_i^2)/(d_tank^2)))^2;
K_o      = 0.42*(1 - ((d_i^2)/(d_tank^2)));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.2 Define Default Initial Operating Conditions for the Accumulator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   mdot_a    - accumulator inlet mass flow rate
%   Pa_ri     - accumulator inlet pressure
%   Ha_ri     - accumulator inlet enthalpy
%   T_engine  - temperature of air surrounding accumulator tank
%   v_air     - velocity of air passing across accumulator tank
%   gamma     - weighting factor used to determine average
%               refrigerant temperature

mdot_a    =
eval(strcat('Operating_Conditions.AccumulatorTK',num2str(Num),'.mdot_a'));
Pa_ri     =
eval(strcat('Operating_Conditions.AccumulatorTK',num2str(Num),'.Pa_ri'));
Ha_ri     =
eval(strcat('Operating_Conditions.AccumulatorTK',num2str(Num),'.Ha_ri'));
T_engine  =
eval(strcat('Operating_Conditions.AccumulatorTK',num2str(Num),'.T_engine'));
v_air     =
eval(strcat('Operating_Conditions.AccumulatorTK',num2str(Num),'.v_air'));
gamma     =
eval(strcat('Operating_Conditions.AccumulatorTK',num2str(Num),'.gamma'));

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.3 Load Initial Conditions for the Accumulator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   beta      - weighting factor on convection coefficient
%   alpha_ref  - refrigerant side heat transfer coefficient

beta      = eval(strcat('Initial_Conditions.AccumulatorTK',num2str(Num),'.beta'));

```



```

        alpha_ref =
eval(strcat('Initial_Conditions.AccumulatorTK',num2str(Num),'.alpha_ref'));

        ModParam = [d_i d_o d_tank Cp_steel L_tank M_tank K_i K_o T_engine ...
                    v_air gamma beta alpha_ref];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Accumulator Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

T_surface = x(1);
T_ref     = x(2);

mdot_a    = u(1);
Ha_ri     = u(2);
Pa_ri     = u(3);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

d_i       = ModParam(1);
d_o       = ModParam(2);
d_tank    = ModParam(3);
Cp_steel  = ModParam(4);
L_tank    = ModParam(5);
M_tank    = ModParam(6);
K_i       = ModParam(7);
K_o       = ModParam(8);
T_engine  = ModParam(9);
v_air     = ModParam(10);
gamma     = ModParam(11);
beta      = ModParam(12);
alpha_ref = ModParam(13);

%Refrigerant properties

Ta_ri     = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph, FluidProp.T_ph, Ha_ri,
Pa_ri);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Pressure Drop
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%Calculating pressure drop due to sudden expansion of refrigerant
rho_i     = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph, FluidProp.Rho_ph, Ha_ri,
Pa_ri);
mu_i      = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt, FluidProp.mu_pt, Ta_ri,
Pa_ri);
k_i       = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt, FluidProp.k_pt, Ta_ri,
Pa_ri);
Cp_i      = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt, FluidProp.Cp_pt, Ta_ri,
Pa_ri);
Vdot_i    = mdot_a/rho_i;
v_i       = Vdot_i/(pi/4*(d_i^2));
deltaP_in = K_i*(1/2)*rho_i*(v_i^2)/1000;
P_tank    = Pa_ri-deltaP_in;

%Calculating pressure drop due to sudden contraction of refrigerant
rho_o     = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt, FluidProp.Rho_pt, T_ref,
P_tank);
Vdot_o    = mdot_a/rho_o;
v_o       = Vdot_o/(pi/4*(d_o^2));
deltaP_out = K_o*(1/2)*rho_o*(v_o^2)/1000;

```

```

%Calculating total pressure drop across accumulator
deltaP_total= deltaP_in + deltaP_out;
Pa_ro       = Pa_ri - deltaP_total;

%Calculating outlet refrigerant temperature and enthalpy
Ta_ro       = (T_ref - gamma*Ta_ri)/(1-gamma);
Ha_ro       =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.H_pt,Ta_ro,Pa_ro);

%Calculating refrigerant-side heat transfer coefficient
alpha_ref   = 7.5447*mdot_a - 0.0385;

mu_surface  = interp1(MoistAirProp.T,MoistAirProp.mu_T,T_surface);
rho_surface = interp1(MoistAirProp.T,MoistAirProp.rho_T,T_surface);
k_surface   = interp1(MoistAirProp.T,MoistAirProp.k_T,T_surface);
Pr_surface  = interp1(MoistAirProp.T,MoistAirProp.Pr_T,T_surface);

Re_surface  = (rho_surface*v_air*d_tank)/mu_surface;

A_surf      = pi*d_tank*L_tank;

Nu          = 0.3 + [ (0.62 * (Re_surface^(1/2)) * (Pr_surface^(1/3))) / ...
    ( 1 + ((0.4/Pr_surface)^(2/3))^(1/4)) ] * ...
    [ 1 + (Re_surface/282000)^(1/2) ];

h_conv      = Nu*k_surface/d_tank;

T_surf_dot  = (10*h_conv*A_surf*(T_engine-T_surface)-alpha_ref*A_surf*...
    (T_surface-T_ref))/(Cp_steel*M_tank);

%Estimate thermal capacitance of refrigerant
Vol_tank    = ((pi*d_tank^2)/4)*L_tank;
Mass_ref    = Vol_tank*rho_i;
T_ref_dot   = (mdot_a*Ha_ri - mdot_a*Ha_ro + alpha_ref*A_surf*...
    (T_surface-T_ref))/(Mass_ref*Cp_i);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   Outputs
y(1)= Pa_ro;
y(2)= Ha_ro;
y(3)= Ta_ro;
y(4)= T_surface;
y(5)= T_ref;

x_dot      = [T_surf_dot T_ref_dot];
yout       = [y(1) y(2)]; %Output Pa_ro, Ha_ro
output     = [x_dot yout y];

return

```

D.8 Throttle Valve

```

function [output] = ThrottleValve_MapTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global OPTIONS
global ValvePropThrottle

```

persistent ModParam

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 2.0 Break Input Vector, U, into Proper Components  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
u      = U(1:3);  
Time   = U(end-2);  
Num     = U(end-1);  
flag    = U(end);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 3.0 Throttle Valve Model Initialization  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
if flag == 1 %Only run this code once at the start of the simulation
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 3.1 Set Default Physical Parameters of the Throttle Valve  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% m_dot - a vector of mass flow rates  
% P_in - a vector of inlet pressure values  
% P_out - a matrix of outlet pressure values
```

```
mdot_matrix      = ValvePropThrottle.m_dot;  
P_in_matrix      = ValvePropThrottle.P_in;  
P_out_matrix     = ValvePropThrottle.P_out;
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 3.2 Define Default Initial Operating Conditions for the Throttle Valve  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
% mdot_tv;      %valve inlet mass flow rate  
% Ptv_ri;       %valve inlet pressure
```

```
mdot_tv      =  
eval(strcat('Operating_Conditions.ThrottleValveTK',num2str(Num),'.mdot_tv'));  
Ptv_ri      =  
eval(strcat('Operating_Conditions.ThrottleValveTK',num2str(Num),'.Ptv_ri'));  
end
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 4.0 Throttle Valve Model  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 4.1 Inputs  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
mdot_tv = u(1);  
Ptv_ri  = u(2);  
Htv_ri  = u(3);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 4.2 Model Parameters  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
mdot_matrix      = ValvePropThrottle.m_dot;  
P_in_matrix      = ValvePropThrottle.P_in;  
P_out_matrix     = ValvePropThrottle.P_out;
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
% 4.3 Outlet Pressure  
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

```
Ptv_ro = interp2(mdot_matrix, P_in_matrix, P_out_matrix, mdot_tv, Ptv_ri, 'spline');  
Ttv_ro = qinterp2(FluidProp.H_mg_ph, FluidProp.P_mg_ph, FluidProp.T_ph, Htv_ri, Ptv_ro);  
Htv_ro = Htv_ri;
```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   Outputs
y(1)= Ptv_ro;
y(2)= Ttv_ro;
y(3)= Htv_ro;

yout = [y(1) y(3)];    %Output Ptv_ro, Htv_ro
output = [yout y];

return

```

D.9 Evaporator Fan

```

function [output] = EvapFanTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global MoistAirProp
global OPTIONS

persistent ModParam AF_vector SP_vector HP_vector

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u      = U(1:2);
Time   = U(end-2);
Num    = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Evaporator Fan Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1
    %Only run this code once at the start of the simulation

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.1 Set Default Physical Parameters of the Fan
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %Static Pressure Curve Coefficients
    A0 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.A0'));
    A1 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.A1'));
    A2 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.A2'));
    A3 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.A3'));
    A4 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.A4'));

    %Horse Power Curve Coefficients
    B0 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.B0'));
    B1 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.B1'));
    B2 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.B2'));

```

```

%Resistance Curve Coefficients
C1 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.C1'));
C2 = eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.C2'));

%num_fan - number of fans drawing air across evaporator
%rpm_base - fan rpm base value, rev/min
%speed_ratio- gear ratio between compressor speed and fan speed
%AF_vector - air flow test range, CFM

num_fan =
eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.num_fan'));
rpm_base =
eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.rpm_base'));
speed_ratio =
eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.speed_ratio'));
AF_vector =
eval(strcat('Physical_Parameters.EvapFanTK',num2str(Num),'.AF_vector'));

%Calculating the base fan curves.
SP_vector = A4.*(AF_vector.^4) + A3.*(AF_vector.^3) + A2.*(AF_vector.^2) + ...
            A1.*(AF_vector) + A0;
HP_vector = B2.*(AF_vector.^2) + B1.*(AF_vector) + B0;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.2 Define Default Initial Operating Conditions for the Fan
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%rpm_k - compressor speed, rev/min
%Tf_ai - evaporator fan air inlet temperature, C

rpm_k = eval(strcat('Operating_Conditions.EvapFanTK',num2str(Num),'.rpm_k'));
Te_ai = eval(strcat('Operating_Conditions.EvapFanTK',num2str(Num),'.Te_ai'));

%Generate ModParam vector.

ModParam = [A0 A1 A2 A3 A4 B0 B1 B2 C1 C2 num_fan rpm_base speed_ratio];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Evaporator Fan Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

rpm_k = u(1);
Te_ai = u(2);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
A0 = ModParam(1);
A1 = ModParam(2);
A2 = ModParam(3);
A3 = ModParam(4);
A4 = ModParam(5);
B0 = ModParam(6);
B1 = ModParam(7);
B2 = ModParam(8);
C1 = ModParam(9);
C2 = ModParam(10);

```

```

num_fan      = ModParam(11);
rpm_base     = ModParam(12);
speed_ratio  = ModParam(13);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Model Calculations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%Converting the input compressor speed into the equivalent fan speed.

rpm_f = rpm_k*speed_ratio;

%Calculating the fan curves as a function of the current fan speed.

AF_act      = AF_vector.*(rpm_f/rpm_base*num_fan);
SP_act      = SP_vector.*((rpm_f/rpm_base)^2);
HP_act      = HP_vector.*(((rpm_f/rpm_base)^3)*num_fan);
Resistance  = C2.*(AF_act.^2) + C1.*AF_act;

%Calculating equations for fan and resistance curves
[p1,S1,mu_f] = polyfit(AF_act,SP_act,3);
Fan_curve    = p1;
mu_f1        = mu_f(1);
mu_f2        = mu_f(2);
[p2,S2,mu_r] = polyfit(AF_act,Resistance,3);
Resis_curve  = p2;
mu_r1        = mu_r(1);
mu_r2        = mu_r(2);

%Re-naming coefficients of each polynomial fit

%Coefficients for Fan curve
P3           = Fan_curve(1);
P2           = Fan_curve(2);
P1           = Fan_curve(3);
P0           = Fan_curve(4);
%Coefficients for Resistance curve
Q3           = Resis_curve(1);
Q2           = Resis_curve(2);
Q1           = Resis_curve(3);
Q0           = Resis_curve(4);

%Finding actual coefficients of each polynomial fit, adjusted for mu1 and mu2

%Coefficients for Fan curve
F3           = P3/(mu_f2^3);
F2           = P3/(mu_f2^3)*(-3)*mu_f1 + P2/(mu_f2^2);
F1           = P3/(mu_f2^3)*3*(mu_f1^2) + P2/(mu_f2^2)*(-2)*mu_f1 + P1/mu_f2;
F0           = -P3/(mu_f2^3)*(mu_f1^3) + P2/(mu_f2^2)*(mu_f1^2) + P1/mu_f2*(-mu_f1) +
P0;

%Coefficients for Resistance curve
R3           = Q3/(mu_r2^3);
R2           = Q3/(mu_r2^3)*(-3)*mu_r1 + Q2/(mu_r2^2);
R1           = Q3/(mu_r2^3)*3*(mu_r1^2) + Q2/(mu_r2^2)*(-2)*mu_r1 + Q1/mu_r2;
R0           = -Q3/(mu_r2^3)*(mu_r1^3) + Q2/(mu_r2^2)*(mu_r1^2) + Q1/mu_r2*(-mu_r1) +
Q0;

%Finding the intersection point between the two curves

soln = roots([(F3-R3) (F2-R2) (F1-R1) (F0-R0)]);

%Check each root to find the desired solution

```

```

j = 1;
k = 1;
for i = [1:1:3]
    root_i = soln(i);
    if imag(root_i)~= 0
        extraneous(j) = root_i;
        j = j+1;
    else possible(k) = root_i;
        k = k+1;
    end
end

airflow_rate = possible(1);

%Convert air volumetric flow rate into air mass flow rate (CFM to kg/s)

rho_air      = interp1(MoistAirProp.T,MoistAirProp.rho_T,Te_ai);
me_air       = airflow_rate/(60*35.314666572)*rho_air;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   Outputs
y(1) = me_air;

yout = [y(1)];    %Output mdot_f
output = [yout y];

return

```

D.10 Condenser Fan

```

function [output] = CondFanTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global MoistAirProp
global OPTIONS

persistent ModParam AF_vector SP_vector HP_vector

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u      = U(1:2);
Time   = U(end-2);
Num     = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Condenser Fan Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

if flag == 1
    %Only run this code once at the start of the simulation

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.1 Set Default Physical Parameters of the Fan
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %Static Pressure Curve Coefficients
    A0 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.A0'));
    A1 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.A1'));
    A2 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.A2'));
    A3 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.A3'));
    A4 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.A4'));

    %Horse Power Curve Coefficients
    B0 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.B0'));
    B1 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.B1'));
    B2 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.B2'));

    %Resistance Curve Coefficients
    C1 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.C1'));
    C2 = eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.C2'));

    %num_fan    - number of fans drawing air across evaporator
    %rpm_base   - fan rpm base value, rev/min
    %speed_ratio- gear ratio between compressor speed and fan speed
    %AF_vector  - air flow test range, CFM

    num_fan    =
    eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.num_fan'));
    rpm_base    =
    eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.rpm_base'));
    speed_ratio =
    eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.speed_ratio'));
    AF_vector   =
    eval(strcat('Physical_Parameters.CondFanTK',num2str(Num),'.AF_vector'));

    %Calculating the base fan curves.
    SP_vector = A4.*(AF_vector.^4) + A3.*(AF_vector.^3) + A2.*(AF_vector.^2) + ...
                A1.*(AF_vector) + A0;
    HP_vector = B2.*(AF_vector.^2) + B1.*(AF_vector) + B0;

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.2 Define Default Initial Operating Conditions for the Condenser Fan
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %rpm_k      - compressor speed, rev/min
    %Tf_ai      - evaporator fan air inlet temperature, C

    rpm_k       = eval(strcat('Operating_Conditions.CondFanTK',num2str(Num),'.rpm_k'));
    Tc_ai       = eval(strcat('Operating_Conditions.CondFanTK',num2str(Num),'.Tc_ai'));

    %Generate ModParam vector.

    ModParam = [A0 A1 A2 A3 A4 B0 B1 B2 C1 C2 num_fan rpm_base speed_ratio];

end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Condenser Fan Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```



```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

rpm_k          = u(1);
Tc_ai          = u(2);

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

A0              = ModParam(1);
A1              = ModParam(2);
A2              = ModParam(3);
A3              = ModParam(4);
A4              = ModParam(5);
B0              = ModParam(6);
B1              = ModParam(7);
B2              = ModParam(8);
C1              = ModParam(9);
C2              = ModParam(10);
num_fan         = ModParam(11);
rpm_base        = ModParam(12);
speed_ratio     = ModParam(13);

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Model Calculations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

%Converting the input compressor speed into the equivalent fan speed.

```

```

rpm_f = rpm_k*speed_ratio;

```

```

%Calculating the fan curves as a function of the current fan speed.

```

```

AF_act         = AF_vector.*(rpm_f/rpm_base*num_fan);
SP_act         = SP_vector.*((rpm_f/rpm_base)^2);
HP_act         = HP_vector.*(((rpm_f/rpm_base)^3)*num_fan);
Resistance     = C2.*(AF_act.^2) + C1.*AF_act;

```

```

%Calculating equations for fan and resistance curves

```

```

[p1,S1,mu_f]   = polyfit(AF_act,SP_act,3);
Fan_curve      = p1;
mu_f1          = mu_f(1);
mu_f2          = mu_f(2);
[p2,S2,mu_r]   = polyfit(AF_act,Resistance,3);
Resis_curve    = p2;
mu_r1          = mu_r(1);
mu_r2          = mu_r(2);

```

```

%Re-naming coefficients of each polynomial fit

```

```

%Coefficients for Fan curve
P3              = Fan_curve(1);
P2              = Fan_curve(2);
P1              = Fan_curve(3);
P0              = Fan_curve(4);
%Coefficients for Resistance curve
Q3              = Resis_curve(1);
Q2              = Resis_curve(2);
Q1              = Resis_curve(3);
Q0              = Resis_curve(4);

```

```

%Finding actual coefficients of each polynomial fit, adjusted for mu1 and mu2

```

```

%Coefficients for Fan curve
F3 = P3/(mu_f2^3);
F2 = P3/(mu_f2^3)*(-3)*mu_f1 + P2/(mu_f2^2);
F1 = P3/(mu_f2^3)*3*(mu_f1^2) + P2/(mu_f2^2)*(-2)*mu_f1 + P1/mu_f2;
F0 = -P3/(mu_f2^3)*(mu_f1^3) + P2/(mu_f2^2)*(mu_f1^2) + P1/mu_f2*(-mu_f1) + P0;

%Coefficients for Resistance curve
R3 = Q3/(mu_r2^3);
R2 = Q3/(mu_r2^3)*(-3)*mu_r1 + Q2/(mu_r2^2);
R1 = Q3/(mu_r2^3)*3*(mu_r1^2) + Q2/(mu_r2^2)*(-2)*mu_r1 + Q1/mu_r2;
R0 = -Q3/(mu_r2^3)*(mu_r1^3) + Q2/(mu_r2^2)*(mu_r1^2) + Q1/mu_r2*(-mu_r1) + Q0;

%Finding the intersection point between the two curves

soln = roots([(F3-R3) (F2-R2) (F1-R1) (F0-R0)]);

%Check each root to find the desired solution

j = 1;
k = 1;
for i = 1:1:3
    root_i = soln(i);
    if root_i > 0
        if root_i < 3500
            possible(k) = root_i;
            k = k+1;
        else
            extraneous(j) = root_i;
            j = j+1;
        end
    end
end

airflow_rate = possible(1);

%Convert air volumetric flow rate into air mass flow rate (CFM to kg/s)

rho_air = interp1(MoistAirProp.T, MoistAirProp.rho_T, Tc_ai);
mc_air = airflow_rate/(60*35.314666572)*rho_air;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1) = mc_air;

yout = [y(1)]; %Output mdot_f
output = [yout y];

return

```

Appendix E

HEAT MODE MODEL FILES

E.1 DPR Valve

```
function [output] = DPR_ValveTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u      = U(1:3);
Time   = U(end-2);
Num     = U(end-1);
flag    = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Throttle Valve Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.1 Define Default Initial Operating Conditions for the DPR Valve
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    % mdot_dpr;          %valve inlet mass flow rate
    % Pdpr_ro;           %valve outlet pressure
    % Hdpr_ri;           %valve inlet enthalpy

    mdot_dpr      =
    eval(strcat('Operating_Conditions.DPR_ValveTK',num2str(Num),'.mdot_dpr'));
    Pdpr_ro       =
    eval(strcat('Operating_Conditions.DPR_ValveTK',num2str(Num),'.Pdpr_ro'));
    Hdpr_ri       =
    eval(strcat('Operating_Conditions.CompressorTK',num2str(Num),'.Hk_ro'));
```

end

```
% 4.0 DPR Valve Model

% 4.1 Inputs
mdot_dpr = u(1);
Pdpr_ro = u(2);
Hdpr_ri = u(3);

% 4.2 Outlet Pressure

%Calculate refrigerant properties
Tdpr_ro =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hdpr_ri,Pdpr_ro);
Rho_ro =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,Tdpr_ro,Pdpr_ro);
Rho_g = qinterp1(FluidProp.Psat,FluidProp.Rhog,Pdpr_ro);

%Converting outlet refrigerant pressure from kPa to psia (for
%correlation)
Pdpr_ro_ENG = Pdpr_ro*0.145037738;

%Converting refrigerant mass flow rate from kg/s to lbm/hr
mdot_ENG = mdot_dpr/0.45359237*3600;

%Converting inlet and outlet refrigerant density from kg/m^3 to lbm/ft^3
Rho_ro_ENG = Rho_ro*0.028316847/0.45359237;
Rhog_ENG = Rho_g*0.028316847/0.45359237;

%Check to see if saturated vapor density needs to be used
if Rho_ro_ENG - Rhog_ENG > 10
    Rho_ro_ENG = Rhog_ENG;
end

%Calculate flow coefficient, Ka
Ka_ENG = 0.0407*mdot_ENG + 1.9063;

%Calculate inlet pressure, Pdpr_ri
deltaP = (1/Rho_ro_ENG)*((mdot_ENG/Ka_ENG)^2);
Pdpr_ri_ENG = Pdpr_ro_ENG + deltaP;

%Converting inlet refrigerant pressure from psia to kPa
Pdpr_ri = Pdpr_ri_ENG/0.145037738;

%Calculating inlet enthalpy and temperature
Tdpr_ri =
qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hdpr_ri,Pdpr_ri);
Hdpr_ro = Hdpr_ri;

% 5.0 Model Output

% Outputs
y(1)= Pdpr_ri;
y(2)= Tdpr_ri;
y(3)= Hdpr_ro;

yout = [y(1) y(3)]; %Output Pdpr_ri, Hdpr_ro
output = [yout y];

return
```

E.2 Hot Gas Line

```
function [output] = HotGasLineTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    global Physical_Parameters
    global Operating_Conditions
    global FluidProp
    persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    u      = U(1:4);
    time   = U(end-2);
    Pipe_num = U(end-1);
    flag    = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Pipe Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    if flag == 1 %Only run this code once at the start of the simulation

        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
        % 3.1 Set Default Physical Parameters of the Hot Gas Line Model
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

            a0      =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.a0'));
            a1      =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.a1'));
            Diameter =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Diameter'));
            L_total =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.L_total'));
            Kb_1    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_1'));
            Kb_2    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_2'));
            Kb_3    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_3'));
            Kb_4    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_4'));
            Kb_5    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_5'));
            Kb_6    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_6'));
            Kb_7    =
            eval(strcat('Physical_Parameters.HotGasLineTK',num2str(Pipe_num),'.Kb_7'));

            %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
            % 3.2 Define Default Initial Operating Conditions for the Hot Gas Line Model
            %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

                Pg_ro      =
            eval(strcat('Operating_Conditions.HotGasLineTK',num2str(Pipe_num),'.Pg_ro'));
                Hg_ri      =
            eval(strcat('Operating_Conditions.HotGasLineTK',num2str(Pipe_num),'.Hg_ri'));
                mdot_g      =
            eval(strcat('Operating_Conditions.HotGasLineTK',num2str(Pipe_num),'.mdot_g'));
                Tg_a        =
            eval(strcat('Operating_Conditions.EvapHM_TK',num2str(Pipe_num),'.Te_ai'));

            ModParam = [a0 a1 Diameter L_total Kb_1 Kb_2 Kb_3 Kb_4 Kb_5 Kb_6 Kb_7];
```

end

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Hot Gas Line model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Pg_ro = u(1);
Hg_ri = u(2);
mdot_g = u(3);
Te_ai = u(4);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

a0      = ModParam(1);
a1      = ModParam(2);
Diameter = ModParam(3);
L_total = ModParam(4);
Kb_1    = ModParam(5);
Kb_2    = ModParam(6);
Kb_3    = ModParam(7);
Kb_4    = ModParam(8);
Kb_5    = ModParam(9);
Kb_6    = ModParam(10);
Kb_7    = ModParam(11);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Model Calculations
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Tg_r = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hg_ri,Pg_ro);
% Refrigerant temperature
Rho_r = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.Rho_ph,Hg_ri,Pg_ro);
% Density of refrigerant

mu = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Tg_r,Pg_ro);
% Viscosity of refrigerant
K = L_total / Diameter;
A = pi * (Diameter^2) / 4;
Re = mdot_g * Diameter / (A * mu);
Relrough = 1.5*10^(-6) / Diameter;
f = F_1PH_CHURCHILL(Re,Relrough);
Kf = K*f;
deltaP = (((Kf + Kb_1 + Kb_2 + Kb_3 + Kb_4 + Kb_5 + Kb_6 +
Kb_7)*(mdot_g^2))/(2*(A^2)*Rho_r))/1000;

Pg_ri = Pg_ro + deltaP;
Tg_ri = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hg_ri,Pg_ri);

%Solving iteratively for outlet refrigerant enthalpy

i = 1;
for Tg_ro = [Tg_ri:-0.2:Te_ai]
    UA = a1*mdot_g + a0;
    Q = UA*(Tg_ri - Tg_ro);
    Hg_ro = ((mdot_g*Hg_ri)-Q)/mdot_g;

    Hg_ro_check =
qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.H_pt,Tg_ro,Pg_ro);

    Error(i) = (Hg_ro - Hg_ro_check)^2;
    soln(i,:) = [Tg_ro Hg_ro];
    i = i + 1;
end

[value, index] = min(Error);
```

```

        solution      = soln(index,:);
        Tg_ro         = solution(1);
        Hg_ro         = solution(2);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Calculate output vectors
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   Outputs
y(1)= Pg_ri;
y(2)= Hg_ro;
y(3)= Tg_ro;

yout = [y(1) y(2)];
output = [yout y];

return

```

E.3 Distributor – Heat Mode

```

function [output] = DistributorHM_TK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global OPTIONS

persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x      = U(1:2);
u      = U(3:4);
Time   = U(end-2);
Num    = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Distributor Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.1 Set Default Physical Parameters of the Distributor
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    u1_detrend =
eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.u1_detrend'));
    u2_detrend =
eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.u2_detrend'));
    y1_detrend =
eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.y1_detrend'));

    ModParam = [u1_detrend u2_detrend y1_detrend];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Distributor Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x1      = x(1);
x2      = x(2);

mdot_d  = u(1);
Pd_ro   = u(2);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u1_detrend = ModParam(1);
u2_detrend = ModParam(2);
y1_detrend = ModParam(3);

A        = eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.A'));
B        = eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.B'));
C        = eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.C'));
D        = eval(strcat('Physical_Parameters.DistributorHM_TK',num2str(Num),'.D'));

%Detrend inputs
u1 = mdot_d - u1_detrend;
u2 = Pd_ro - u2_detrend;
%Place u and x into vectors
Ud = [u1; u2];
Xd = [x1; x2];
%Compute output, Y, and state derivatives, X_dot
Y   = C*Xd + D*Ud;
X_dot = A*Xd + B*Ud;

Pd_ri = Y(1) + y1_detrend;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%   Outputs
y(1)= Pd_ri;

x_dot = X_dot';
yout  = [Pd_ri];
output = [x_dot yout y];

return

```

E.4 Evaporator – Heat Mode

```

function [output] = EvapHM_TK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp TwoPhaseProp
global OPTIONS
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```



```

x      = U(1:5);
u      = U(6:10);
Time   = U(end-2);
HX_num = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Condenser Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.1 Set Default Parameters of the Heat Mode Evaporator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Mass_e   = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Mass_e'));
Cpw_e    = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Cpw_e'));
Ae_o     = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Ae_o'));
Diameter_e = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Diameter_e'));
Le_total = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Le_total'));
Ae_i     = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Ae_i'));
Ae_cs    = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Ae_cs'));
Ae_fr    = eval(strcat('Physical_Parameters.EvapHM_TK',num2str(HX_num),'.Ae_fr'));
Slip_e   = eval(strcat('Operating_Conditions.EvapHM_TK',num2str(HX_num),'.Slip_e'));
Cpe_air  = eval(strcat('Operating_Conditions.EvapHM_TK',num2str(HX_num),'.Cpe_air'));
me_air   = eval(strcat('Initial_Conditions.EvapFanTK',num2str(HX_num),'.me_air'));

ModParam = [Le_total Ae_cs Ae_i Ae_o Mass_e Cpw_e Cpe_air Slip_e Ae_fr Diameter_e];
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Heat Mode Evaporator (i.e. 2-Zone Condenser) Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 States and Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

mdot_i_total= u(1); % Mass flow rate entering the heat exchanger
mdot_o_total= u(2); % Mass flow rate exiting the receiver
He_ri       = u(3); % Enthalpy of entering refrigerant
Te_ai       = u(4); % Inlet air temperature
me_air_total= u(5); % Mass flow rate of air across the heat exchanger

Le_1        = x(1); % Length of the first region
Pe          = x(2); % Evaporator pressure
Gamma       = x(3); % Value of the mean void fraction in the 2-phase flow
region
Te_w1       = x(4); % Average tube wall temperature in the first region
Te_w2       = x(5); % Average tube wall temperature in the second region

%Inlet and Outlet refrigerant mass flow rate, and air mass flow rate,
%adjusted for the 11 parallel circuits inside the heat exchanger
mdot_i      = mdot_i_total/11;
mdot_o      = mdot_o_total/11;
me_air      = me_air_total/11;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Le_total    = ModParam(1);
% Total length of heat exchanger "tubes" (irrespective of the geometry,

```

```

% the heat exchanger is assumed to be a single tube)
Ae_cs      = ModParam(2); % Total cross sectional area for fluid flow
Ae_i       = ModParam(3); % Total internal surface area
Ae_o       = ModParam(4); % Total exterior surface area
Mass_e     = ModParam(5); % Mass of heat exchanger
Cpw_e     = ModParam(6); % Specific heat of the heat exchanger
Cpe_air    = ModParam(7); % Specific heat of the external fluid
Slip_e     = ModParam(8); % Slip ratio for void fraction
Ae_fr      = ModParam(9); % Heat exchanger frontal area
Diameter_e = ModParam(10); % Hydarulic diameter of heat exchanger tubes
Le_2       = Le_total-Le_1;% Length of second region

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.3 Calculate Refrigerant Properties
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Density, Enthalpy, and Internal Energy

% Saturated liquid density
% Saturated vapor density
% Saturated liquid enthalpy
% Saturated vapor enthalpy

Rho_F      = qinterp1(FluidProp.Psat,FluidProp.Rhof,Pe);
Rho_G      = qinterp1(FluidProp.Psat,FluidProp.Rhog,Pe);
He_f       = qinterp1(FluidProp.Psat,FluidProp.Hf,Pe);
He_g       = qinterp1(FluidProp.Psat,FluidProp.Hg,Pe);

% Average enthalpy of the refrigerant in the first region
% Saturated liquid internal energy
% Saturated vapor internal energy
% Average density of refrigerant in the first region

He_r1      = (He_ri+He_g)/2;
ue_f       = qinterp1(FluidProp.Psat,FluidProp.Uf,Pe);
ue_g       = qinterp1(FluidProp.Psat,FluidProp.Ug,Pe);
Rho_1      = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.Rho_ph,He_r1,Pe);

% Partial Derivatives

% Partial derivative of the saturated liquid density with respect to pressure
% Partial derivative of the saturated vapor density with respect to pressure
% Partial derivative of the average density in the first region with respect to
pressure
% Partial derivative of the average density in the first region with respect to
enthalpy
% Partial derivative of the saturated vapor enthalpy with respect to pressure
% Partial derivative of the saturated liquid enthalpy with respect to pressure
% Partial derivative of the saturated liquid internal energy with respect to pressure
% Partial derivative of the saturated vapor internal energy with respect to pressure

dRhoF_dP   = qinterp1(FluidProp.Psat,FluidProp.dRhof_dP,Pe);
dRhoG_dP   = qinterp1(FluidProp.Psat,FluidProp.dRhog_dP,Pe);
dRho1_dP   = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,...
    FluidProp.dRho_dP_H_ph,He_r1,Pe);
dRho1_dh   = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,...
    FluidProp.dRho_dH_P_ph,He_r1,Pe);
dhG_dP     = qinterp1(FluidProp.Psat,FluidProp.dHg_dP,Pe);
dhF_dP     = qinterp1(FluidProp.Psat,FluidProp.dHf_dP,Pe);
duF_dP     = qinterp1(FluidProp.Psat,FluidProp.dUf_dP,Pe);
duG_dP     = qinterp1(FluidProp.Psat,FluidProp.dUg_dP,Pe);

% Fluid Temperatures and Specific Heats

% Average refrigerant temperature in the first region

```

```

% Average refrigerant temperature in the second region

Te_r1      = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,He_r1,Pe);
Te_r2      = qinterp1(FluidProp.Psat,FluidProp.Tsat,Pe);
Te_ri      = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,He_ri,Pe);
Tesh       = Te_ri-Te_r2;

% Average refrigerant viscosity in the first region
% Average refrigerant specific heat in the first region
% Average refrigerant conductivity in the first region

mu_r1      = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Te_r1,Pe);
Cp_r1      = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Cp_pt,Te_r1,Pe);
k_r1       = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.k_pt,Te_r1,Pe);

% Calculate Gamma_coordinate
Gamma_Total = interp1(FluidProp.Psat, TwoPhaseProp.Gamma_Total, Pe);
Gamma_coord = (1.0 - Gamma) / (1.0 - Gamma_Total);
% Interpolate to find outlet quality
Xe_ro      = interp2(TwoPhaseProp.Gamma_Coord, FluidProp.Psat, ...
    TwoPhaseProp.Xeout, Gamma_coord, Pe);
He_ro      = Xe_ro*He_g + (1-Xe_ro)*He_f;
Rho_2      = Rho_F*(1-Gamma)+Rho_G*Gamma;
% Refrigerant Mass Inventory
Mass       = Ae_cs*Le_1*Rho_1 + Ae_cs*Le_2*Rho_2;
% Outlet Refrigerant Temperature
Te_ro      = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,He_ro,Pe);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.4 Determine heat transfer coefficients
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Qe_1       = mdot_i*(He_ri-He_g);
Qe_2       = mdot_i*(He_g-He_ro);
Qe_total   = Qe_1+Qe_2;
G_r        = mdot_i/Ae_cs;
Re_r1      = G_r*Diameter_e/mu_r1;
Relrough_e = 1e-6/Diameter_e;
ff_r1      = F_1PH_CHURCHILL(Re_r1,Relrough_e);

% refrigerant to HX wall heat transfer coefficient in the single phase region
% refrigerant to HX wall heat transfer coefficient in the two phase region
% heat transfer coefficient between the HX wall and the external fluid

alpha_e_i1 = (H_1PH_GNIELINSKI(Diameter_e, G_r, ff_r1, Cp_r1, mu_r1, k_r1));
alpha_e_i2 = (H_COND_DOBSON(Diameter_e, G_r, Pe, 0.5*(Xe_ro+1)));
alpha_e_o  = H_COLBURN_FACTOR(27, me_air, Ae_fr, Te_ai);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.5 Calculate Air Temperatures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Te_a2 = ((2*me_air*Cpe_air*Te_ai) + (alpha_e_o*Ae_o*(Le_2/Le_total)*Te_w2))...
    /(2*me_air*Cpe_air + alpha_e_o*Ae_o*(Le_2/Le_total));
Te_am = 2*Te_a2 - Te_ai;
Te_a1 = ((2*me_air*Cpe_air*Te_am) + (alpha_e_o*Ae_o*(Le_1/Le_total)*Te_w1))...
    /(2*me_air*Cpe_air + alpha_e_o*Ae_o*(Le_1/Le_total));
Te_ao = 2*Te_a1 - Te_am;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.6 Calculate Nonlinear State Space System, derivatives, and outputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Z matrix
Z = zeros(5,5);
Z(1,1)=(Rho_1*He_r1-Rho_1*He_g)*Ae_cs;

```

```

Z(1,2)=((dRho1_dP+0.5*dRho1_dh*dhG_dP)*(He_r1-He_g)+(0.5*dhG_dP)*Rho_1-1)...
    *Ae_cs*Le_1;
Z(2,1)=(Rho_F*He_g-Rho_F*He_f)*(1-Gamma)*Ae_cs;
Z(2,2)=((dRhoF_dP*He_f + dhF_dP*Rho_F - dRhoF_dP*He_g)*(1-Gamma)+...
    (dRhoG_dP*He_g + dhG_dP*Rho_G - dRhoG_dP*He_g)*(Gamma)-1)*...
    Ae_cs*Le_2;
Z(2,3)=(Rho_F*He_g-Rho_F*He_f)*Ae_cs*Le_2;
Z(3,1)=(Rho_1-Rho_F*(1-Gamma)-Rho_G*(Gamma))*Ae_cs;
Z(3,2)=((dRhoF_dP*(1-Gamma)+dRhoG_dP*(Gamma))*Le_2+(dRho1_dP+0.5*dRho1_dh*dhG_dP)*...
    Le_1)*Ae_cs;
Z(3,3)=(Rho_G-Rho_F)*Ae_cs*Le_2;
Z(4,1)=Cpw_e*Mass_e*(Te_w1-Te_w2)/Le_1;
Z(4,4)=Cpw_e*Mass_e;
Z(5,5)=Cpw_e*Mass_e;

% f vector
f(1,1) = mdot_i*He_ri-mdot_i*He_g+alpha_e_i1*(Ae_i/Le_total)*Le_1*(Te_w1-Te_r1);
f(2,1) = mdot_o*He_g-mdot_o*He_ro+alpha_e_i2*(Ae_i/Le_total)*Le_2*(Te_w2-Te_r2);
f(3,1) = mdot_i-mdot_o;
f(4,1) = alpha_e_i1*Ae_i*(Te_r1-Te_w1)+alpha_e_o*Ae_o*(Te_a1-Te_w1);
%adjusted for counterflow configuration (Te_a --> Te_a1)
f(5,1) = alpha_e_i2*Ae_i*(Te_r2-Te_w2)+alpha_e_o*Ae_o*(Te_a2-Te_w2);
%adjusted for counterflow configuration (Te_a --> Te_a2)

% Calculate Derivatives
x_dot = Z\f;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Check for violations of modeling assumptions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if x(1) <= 0,
    disp('Modeling Assumptions Violated :-(')
    disp('Length of superheat region has gone to zero.')
    % set_param(gcs,'SimulationCommand','stop')
end

if (x(1)/Le_total) >= 1,
    disp('Modeling Assumptions Violated :-(')
    disp('Length of two-phase region has gone to zero.')
    % set_param(gcs,'SimulationCommand','stop')
end

if u(1) < 0,
    disp('Modeling Assumptions Violated :-(')
    disp('Negative mass flow rate into heat exchanger is not allowed.')
    % set_param(gcs,'SimulationCommand','stop')
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 6.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Outputs
y(1) = x(1);
y(2) = x(2);
y(3) = x(3);
y(4) = x(4);
y(5) = x(5);
y(6) = Te_ao;
y(7) = Te_ro;
y(8) = Te_ro;
y(9) = He_ro;
y(10) = Mass;

yout = [y(2) y(2) y(9) y(6)]; %Output Pin, Pout, Hout, Te_ao
output = [x_dot' yout y];
return

```

E.5 Hydraulic Resistance

```
function [output] = HydResTK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Physical_Parameters
global Operating_Conditions
global FluidProp
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

u      = U(1:4);
time   = U(end-2);
Pipe_num = U(end-1);
flag    = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Hydraulic Resistance Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

if flag == 1 %Only run this code once at the start of the simulation

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.1 Define Default Initial Operating Conditions for the Hydraulic Resistance Model
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    %Physical Parameters for Pipe 1: Connecting the Evaporator Coil to the SLHX
    Diameter_1 =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Diameter_1'));
    L1_total   =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.L1_total'));
    Kb1_1      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb1_1'));
    Kb1_2      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb1_2'));
    Kb1_3      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb1_3'));
    Kb1_4      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb1_4'));
    Kb1_5      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb1_5'));
    Kb1_6      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb1_6'));
    K1_total   = Kb1_1 + Kb1_2 + Kb1_3 + Kb1_4 + Kb1_5 + Kb1_6;
    Acs_1      = pi*(Diameter_1^2)/4;

    %Physical Parameters for Pipe 2: The outer shell of the SLHX
    D1 = eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.D1'));
    D2 = eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.D2'));
    D3 = eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.D3'));
    D4 = eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.D4'));
    L2_total =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.L2_total'));
    B =
    eval(strcat('Physical_Parameters.SuctionLineHX_TK',num2str(Pipe_num),'.B'));
    Diameter_2 = ((D1^2)-((D2^2)/2)+((D3^2)/2)-(D4^2))/(D1+(D2/2)+(D3/2)+D4);
    Acs_2      = pi*(Diameter_2^2)/4;

    %Physical Parameters for Pipe 3: Connecting the SLHX to the Accumulator
```

```

    Diameter_3      =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Diameter_3'));
    L3_total        =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.L3_total'));
    Kb3_1           =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb3_1'));
    Kb3_2           =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb3_2'));
    Kb3_3           =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb3_3'));
    Kb3_4           =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb3_4'));
    Kb3_5           =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb3_5'));
    Kb3_6           =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.Kb3_6'));
    K3_total        = Kb3_1 + Kb3_2 + Kb3_3 + Kb3_4 + Kb3_5 + Kb3_6 + 1;
    Acs_3           = pi*(Diameter_3^2)/4;

    %Physical Parameters for additional Hydraulic Resistance
    HydResR         =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.HydResR'));
    HydResExp       =
    eval(strcat('Physical_Parameters.HydResTK',num2str(Pipe_num),'.HydResExp'));

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % 3.2 Define Default Initial Operating Conditions for the Hydraulic Resistance Model
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

    ModParam = [Diameter_1 L1_total K1_total Acs_1 ...
                Diameter_3 L3_total K3_total Acs_3 ...
                Diameter_2 L2_total Acs_2 B ...
                HydResR HydResExp];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Calculate Mass Flow Rate
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Pin          = u(1);
Pout         = u(2);
Hin          = u(3);
mdot_initial = u(4);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Diameter_1   = ModParam(1);
L1_total     = ModParam(2);
K1_total     = ModParam(3);
Acs_1        = ModParam(4);
Diameter_3   = ModParam(5);
L3_total     = ModParam(6);
K3_total     = ModParam(7);
Acs_3        = ModParam(8);
Diameter_2   = ModParam(9);
L2_total     = ModParam(10);
Acs_2        = ModParam(11);
B            = ModParam(12);
HydResR      = ModParam(13);
HydResExp    = ModParam(14);

```

```

%Calculate Refrigerant Properties
Tin      = qinterp2(FluidProp.H_mg_ph,FluidProp.P_mg_ph,FluidProp.T_ph,Hin,Pin);
Rho      = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.Rho_pt,Tin,Pin);
mu       = qinterp2(FluidProp.T_mg_pt,FluidProp.P_mg_pt,FluidProp.mu_pt,Tin,Pin);

%Check for reverse flow

if Pout >= Pin
    mdot_calc = 0.0;

%Otherwise, full iterative solution for mass flow rate

else

%Set DP target value, starting flow rate, and error tolerances

mdot_tol = 1.0e-5;
DP_tol = 1.0e-4;
DeltaP_target = Pin - Pout;
mdot = max( [ mdot_initial mdot_tol ] );
maxiter = 100;

    for iter = 1:maxiter

        %Calculate the pressure drop at this flow rate and compare the pressure
        %drop to the desired value

        %Pipe 1:
        V1_ref      = mdot/(Rho*Acs_1);
        P1_int      = 1.0e-3*0.5*Rho*V1_ref^2;
        Re1         = Rho*V1_ref*Diameter_1/mu;
        Relrough1    = 1.5*10^(-6) / Diameter_1;
        ff1         = F_1PH_CHURCHILL(Re1,Relrough1);
        DeltaP1_f    = P1_int*ff1*L1_total/Diameter_1;
        DeltaP1_k    = K1_total*P1_int;

        %Pipe 2:
        V2_ref      = mdot/(Rho*Acs_2);
        P2_int      = 1.0e-3*0.5*Rho*V2_ref^2;
        Re2         = Rho*V2_ref*Diameter_2/mu;
        Relrough2    = 1.5*10^(-6) / Diameter_2;
        ff2         = F_1PH_CHURCHILL(Re2,Relrough2);
        DeltaP2_f    = P2_int*ff2*L2_total/Diameter_2;

        %Pipe 3:
        V3_ref      = mdot/(Rho*Acs_3);
        P3_int      = 1.0e-3*0.5*Rho*V3_ref^2;
        Re3         = Rho*V3_ref*Diameter_3/mu;
        Relrough3    = 1.5*10^(-6) / Diameter_3;
        ff3         = F_1PH_CHURCHILL(Re3,Relrough3);
        DeltaP3_f    = P3_int*ff3*L3_total/Diameter_3;
        DeltaP3_k    = K3_total*P3_int;

        %Hydraulic Resistance
        Qfluid      = abs( mdot )/Rho;
        DeltaP_r     = HydResR * Qfluid^HydResExp;

        DeltaP_calc = DeltaP1_f + DeltaP1_k + DeltaP2_f + ...
                        DeltaP3_f + DeltaP3_k + DeltaP_r;

        DeltaP_error = 1.0 - DeltaP_calc / DeltaP_target;
        if( abs( DeltaP_error ) <= DP_tol )

```

```

        break
    end

    %Choose the next flow rate, assuming that pressure drop is roughly
    %proportional to the square of flow rate

    mdot_new = mdot * ( DeltaP_target / DeltaP_calc ) ^ 0.50;
    mdot_error = abs( mdot_new - mdot );
    if( abs( mdot_error ) <= mdot_tol )
        break
    end
    mdot = mdot_new;
end
if( iter > maxiter )
    disp( [ 'WARNING: Iteration maxed out in find_mdot_HydRes' ] );
end
mdot_calc = mdot;

if ( mdot_calc < 0.0001)
    mdot_calc = 0.0;
end
end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Calculate output vectors
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Outputs
y(1)= mdot_calc;
y(2)= Hin;

yout = [y(1) y(2)];    %Output mdot, Hin
output = [yout y];
return

```

E.6 Accumulator – Heat Mode

```

function [output] = AccumulatorHM_TK(U)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 1.0 Load Global Parameter Structures
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

global Initial_Conditions
global Physical_Parameters
global Operating_Conditions
global FluidProp
global MoistAirProp
global OPTIONS
persistent ModParam

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 2.0 Break Input Vector, U, into Proper Components
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

x      = U(1:3);
u      = U(4:7);
Time   = U(end-2);
Num    = U(end-1);
flag   = U(end);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.0 Accumulator Model Initialization
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```



```

if flag == 1 %Only run this code once at the start of the simulation

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.1 Set Default Physical Parameters of the Accumulator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% d_i      - inlet pipe diameter
% d_o      - outlet pipe diameter
% d_tank    - accumulator diameter
% Cp_steel  - specific heat of steel
% L_tank    - height of accumulator tank
% M_tank    - mass of accumulator tank
% eta_sep   - separation efficiency of accumulator tank
% a0        - Zeroth order coefficient for refrigerat heat transfer
%            coefficient correlation
% a1        - First order coefficient for refrigerat heat transfer
%            coefficient correlation

d_i      =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.d_i'));
d_o      =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.d_o'));
d_tank    =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.d_tank'));
Cp_steel  =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.Cp_steel'));
L_tank    =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.L_tank'));
M_tank    =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.M_tank'));
eta_sep   =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.eta_sep'));
a0        =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.alpha_a0'));
a1        =
eval(strcat('Physical_Parameters.AccumulatorHM_TK',num2str(Num),'.alpha_a1'));
V_tank    = pi*((d_tank / 2)^2)*L_tank;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 3.2 Define Default Initial Operating Conditions for the Accumulator
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Ha_ri     - accumulator inlet enthalpy
% mdot_i    - accumulator inlet mass flow rate
% mdot_o    - accumulator outlet mass flow rate
% T_amb     - ambient air temperature
% v_air     - velocity of ambient air surrounding accumulator tank

Ha_ri     = eval(strcat('Initial_Conditions.EvapHM_TK',num2str(Num),'.He_ro'));
mdot_o    = eval(strcat('Initial_Conditions.CompressorTK',num2str(Num),'.mdot_k'));
mdot_i    = mdot_o;
Tamb      =
eval(strcat('Operating_Conditions.AccumulatorHM_TK',num2str(Num),'.Tamb'));
v_air     =
eval(strcat('Operating_Conditions.AccumulatorHM_TK',num2str(Num),'.v_air'));

ModParam = [V_tank a0 a1 eta_sep d_tank L_tank Cp_steel M_tank v_air];

end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.0 Accumulator Model
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.1 Inputs
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Pa          = x(1);
Ha          = x(2);
T_surface   = x(3);

Ha_ri       = u(1);
mdot_i      = u(2);
mdot_o      = u(3);
Tamb        = u(4);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 4.2 Model Parameters
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

V_tank      = ModParam(1);
a0          = ModParam(2);
a1          = ModParam(3);
eta_sep     = ModParam(4);
d_tank      = ModParam(5);
L_tank      = ModParam(6);
Cp_steel    = ModParam(7);
M_tank      = ModParam(8);
v_air       = ModParam(9);

%Refrigerant Properties

Tr_sat      = interp1(FluidProp.Psat,FluidProp.Tsat,Pa);
alpha_ref   = a1*mdot_i + a0;
A_surf      = pi*d_tank*L_tank;

%Calculating change in enthalpy of refrigerant due to
%heat transfer from engine block
mu_surface  = interp1(MoistAirProp.T,MoistAirProp.mu_T,T_surface);
rho_surface = interp1(MoistAirProp.T,MoistAirProp.rho_T,T_surface);
k_surface   = interp1(MoistAirProp.T,MoistAirProp.k_T,T_surface);
Pr_surface  = interp1(MoistAirProp.T,MoistAirProp.Pr_T,T_surface);

Re_surface  = (rho_surface*v_air*d_tank)/mu_surface;

Nu          = 0.3 + [(0.62 * (Re_surface^(1/2)) * (Pr_surface^(1/3)))...
/ ( (1 + ((0.4/Pr_surface)^(2/3)))^(1/4))] * ...
([ 1 + (Re_surface/282000)^(5/8) ]^(4/5));
h_conv      = Nu*k_surface/d_tank;

T_surf_dot  = (10*h_conv*A_surf*(Tamb-T_surface)-alpha_ref*A_surf * ...
(T_surface-Tr_sat))/(Cp_steel*M_tank);

Qin         = alpha_ref*pi*d_tank*L_tank*(T_surface - Tr_sat);

Hf          = interp1(FluidProp.Psat, FluidProp.Hf, Pa);
Hg          = interp1(FluidProp.Psat, FluidProp.Hg, Pa);
Rhof        = interp1(FluidProp.Psat, FluidProp.Rhof, Pa);
Rhog        = interp1(FluidProp.Psat, FluidProp.Rhog, Pa);

x_vapor     = (Ha - Hf) / (Hg - Hf);
Rho         = ( x_vapor / Rhog + (1.0 - x_vapor) / Rhof )^(-1);
Mass        = Rho*V_tank;
Gamma       = x_vapor * Rho / Rhog;

x_o         = eta_sep;
Ha_ro       = x_o*Hg + (1-x_o)*Hf;

```

```

%Compute partial derivative of density with respect to pressure

delta = -0.0005;

Pstar = Pa + delta*Pa;

Rhof_star = interp1(FluidProp.Psat, FluidProp.Rhof, Pstar);
Rhog_star = interp1(FluidProp.Psat, FluidProp.Rhog, Pstar);
Hf_star   = interp1(FluidProp.Psat, FluidProp.Hf, Pstar);
Hg_star   = interp1(FluidProp.Psat, FluidProp.Hg, Pstar);

xstar      = (Ha - Hf_star) / (Hg_star - Hf_star);
Rhostar    = ( xstar / Rhog_star + (1.0 - xstar) / Rhof_star )^(-1);
Rho_del    = Rho - Rhostar;
dRho_dP    = Rho_del / (-delta*Pa);

%Compute partial derivative of density with respect to enthalpy

if( x_vapor <= 0.5 )
    delta = +0.0002;
else
    delta = -0.0002;
end

Hstar      = Ha + delta*Ha;

xstar      = (Hstar - Hf) / (Hg - Hf);
Rhostar    = ( xstar / Rhog + (1.0 - xstar) / Rhof )^(-1);
Rho_del    = Rho - Rhostar;
dRho_dH    = Rho_del / (-delta*Ha);

%Use governing equations to compute time derivatives of pressure and
%enthalpy

Aterm = (mdot_i - mdot_o) / (Rho*V_tank);
Bterm = ( mdot_i*(Ha_ri - Ha) - mdot_o*(Ha_ro - Ha) + Qin ) / (Rho*V_tank);
Cterm = (dRho_dP / Rho) + (dRho_dH / Rho^2);
P_dot = ( Aterm - (dRho_dH * Bterm / Rho) ) / Cterm;
H_dot = Bterm + P_dot / Rho;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% 5.0 Model Output
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%   Outputs
y(1)= Pa;
y(2)= Ha;
y(3)= Tr_sat;
y(4)= Ha_ro;
y(5)= T_surface;
y(6)= Mass;
y(7)= Gamma;

x_dot = [P_dot H_dot T_surf_dot];
yout = [Pa Ha_ro];
output = [x_dot yout y];

return

```

Appendix F

ADDITIONAL FIGURES

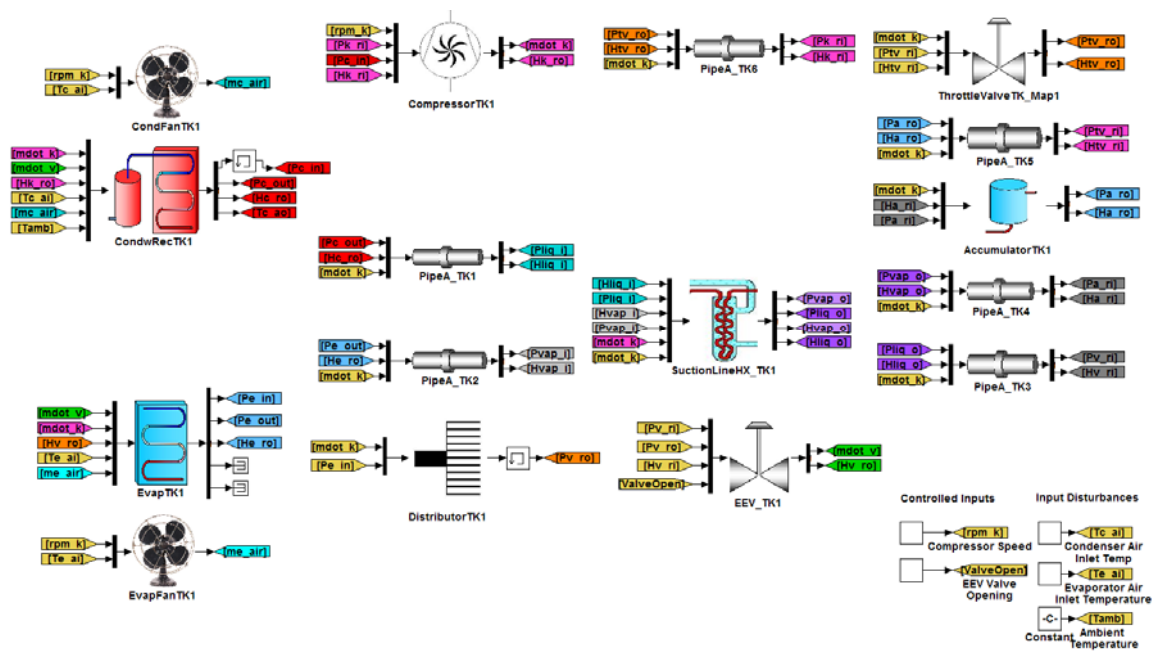


Figure F.1: Cool Mode complete system Simulink diagram.

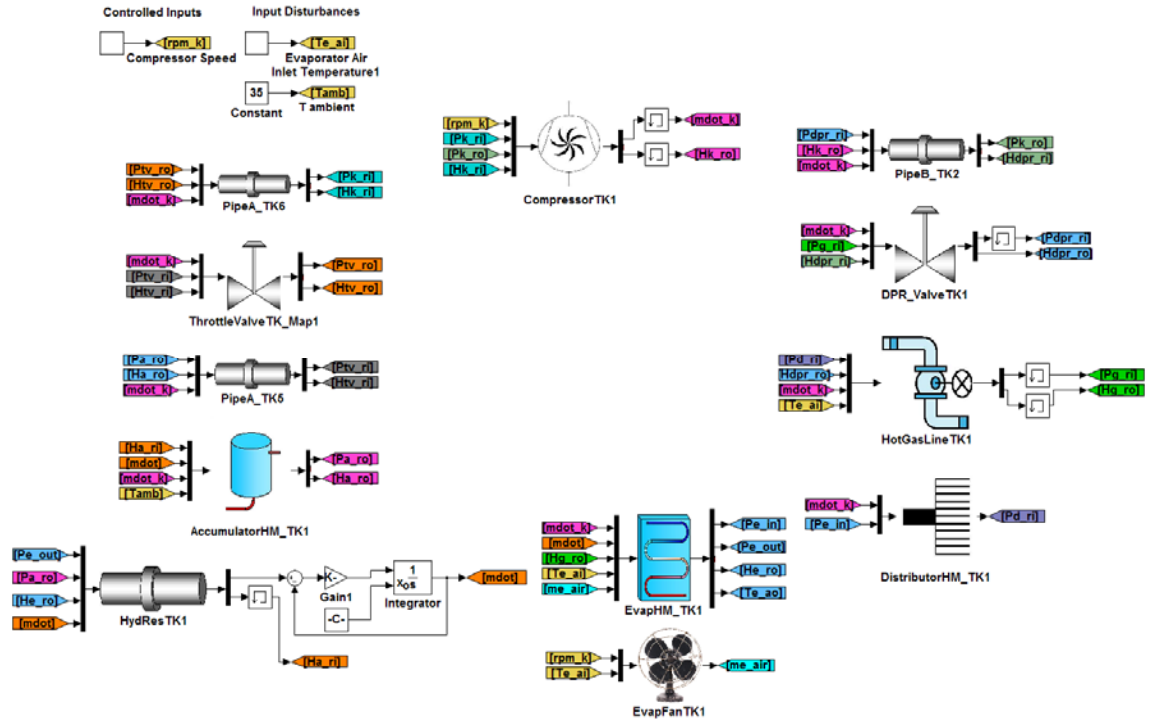


Figure F.2: Heat Mode complete system Simulink diagram.